



Operations Research

01 - Introduction to Operations Research

Mihai Suciu

Faculty of Mathematics and Computer Science

Structure

- Courses 2 h/week
- Seminars 1 h/week
- course web page: <https://www.cs.ubbcluj.ro/~mihai-suciu>
- materials will be posted on MsTeams (access code 8pld2cd)
- activities will take place according to
<https://www.cs.ubbcluj.ro/files/orar/2025-1/disc/MME8215.html>

Evaluation

- seminar:
 - projects (implement/solve optimization problems using the learned models)
- bonus for extra work (0,5p)
- written exam
- final grade:

$$\text{Grade} = 0.6 * E + 0.4 * S + B$$

where:

E = written exam

S = seminar

B = bonus

Minimum standard of performance

$$\text{Grade} = 0.6 * E + 0.4 * S + B$$

- Grade: minimum 5 at each grading activity (except B).

Course content

Tentative schedule, courses

1. Introduction to Operations Research
2. Introduction to Foundation mathematics and statistics
3. Linear Programming
4. Maximization Then Minimization problems.
5. Graphical LP Minimization solution, Introduction, Simplex method definition, formulating the Simplex model
6. Linear Programming – Simplex Method for Maximizing.
7. Transportation model
8. Network model
9. Advanced linear programming, Integer linear programming
10. Metaheuristics Optimization Algorithms
11. Multi-objective optimization
12. Optimization with engineering Applications, as for example Routing and traffic management, Network design, Network connectivity and reliability, Energy consumption consideration in network optimization.
13. Use cases extracted from research papers.
14. Use cases extracted from research papers.

Course content

Tentative schedule, seminar

1. Problem definition, unconstrained optimization, constrained optimization
2. Modeling combinatorial optimization problems
3. Linear programming, Integer and mixed-integer linear programming.
4. Optimization with engineering applications
5. Metaheuristics
6. Multiobjective optimization
7. Different use cases

Let's begin

- 01 Introduction to Operations Research
 - What is OR?

Intro to OR

- The OR technique was initiated in England during II world war to solve strategic and tactical military problems.
- Then British and American military also started the same technique for effective utilization of military resources.
- Specialists in engineering, mathematics, economics, psychology, statistics and physical science were formed as group to do research on military problems.
- The same technique was spread to United states, France and Canada.
- After the world war these teams were shifted towards industry and business sector to solve complex managerial problems.

Intro to OR

- colleges and universities introduced OR as a subject at degree level (~1950)
- OR club was formed in England, later its name was changed as OR society of U.K (~1948)
- International federation of OR society was established with headquarters in London (~1957)
- Today OR techniques are being used in various fields such as hospitals, libraries, city planning, transportation systems, crime investigation and industries.

Definitions

- “OR utilizes the planned approach and an interdisciplinary team in order to represent complex functional relationships as mathematical models for the purpose of providing a quantitative analysis”. - Thierauf and Klekamp
- “OR is a scientific approach to problem solving for executive management”. - H.M.Wagner
- “OR is a scientific knowledge through interdisciplinary team effort for the purpose of determining the best utilization of limited resources. - H.A.Taha
- “OR is the art of winning wars without actually fighting them.” - Auther Clark
- “OR is the art of finding bad answers to problems to which otherwise worse answers are given.” - T.L.Saaty
- “OR is essentially a collection of mathematical techniques and tools which in conjunction with system approach, are applied to solve practical decision problems of an economic or engineering nature”. - Daellenbach and George

What is OR

- In general, the organization goal is to optimize the use of available resources.
- OR (Management science) is a discipline that adapts the scientific approach to problem solving to decision making.

What is OR

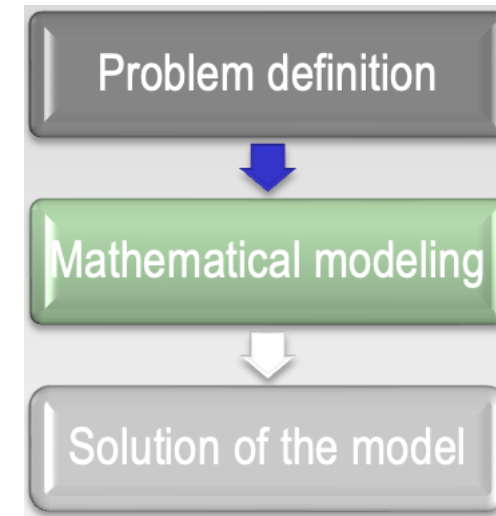
- Application of quantitative mathematical methods and logic to find solutions to managerial problems
- Concerned with means of dealing with:
 - Resource constraints
 - Assignment and maximization of values
 - Appraisal of alternatives in an uncertain, dynamic environment

Benefits of OR

- Ability to deal in a quantitative manner with conditions of uncertainty
- Use of objective methods to sort out in complex situations
- Capacity to illustrate the likely outcomes of alternative courses
- Assist managers in understanding many inter-related factors
- Provision of various logical approaches to make decisions
- Ability to handle masses of data with help of computer

Methodology

1. Formulate the problem.
2. Construct the model.
3. Acquire input data.
4. Solve the model.
5. Test the solution.
6. Implement the solution.
7. Modifying the model.
8. Establishing control over the solution.



1. Defining the problem

- OR is Applied When:
 - Designing and implementing new operations.
 - Evaluating ongoing Operations and Procedures.
 - Determining and recommending corrective actions

1. Defining the problem

- How to Start and How to Proceed
 - Identify the problem.
 - Observe the problem from various points of view.
 - Keep things simple.
 - Identify constraints.
 - Work with management, get feedback

2. Building a Mathematical Model

- Identify Decision Variables
 - Which factors are controllable?
- Quantify the Objective and Constraints
 - Formulate the function to be optimized (profit, cost).
 - Formulate the requirements and/or restrictions.
- Construct a Model Shell
 - Help focus on the exact data required.
- Gather Data
 - Consider time / cost issues.

2. Building a Mathematical Model

- Mathematical modeling formulation: The general OR model can be organized in the following general format:-

Maximize or minimize *Objective Function*

subject to

Constraints

3. Solving a Mathematical Model

- Choose an Appropriate Solution Technique
 - An optimization algorithm.
 - A heuristic algorithm.
- Generate Model Solutions
- Test / Validate Model Results
 - Is the solution reasonable?
 - Are radical changes needed?
 - Does it fit present and future plans?
 - Unacceptable results? Return to modeling.
- Perform “What--If” Analyses

Examples

1. Wilson company produces three products: desk, chairs, and molded steel.

It is trying to decide on the number of desks (D), chair (C), and pounds of molded steel (M).

Assumption :

1- If the nets \$ 50 profit on each desk produced, \$30 on each chair produced, and \$ 6 per pound of modeled steel produced. **What the total profit?**

The answer :

- Total profit for production run can be modeled by the expression

$$50 D + 30 C + 6 M$$

2 - if 7 pounds of raw steel are needed to manufacture a desk, 3 pounds to manufacture a chair, and 1.15 pounds to produce a pound of model steel.

What the amount of raw steel used during the production?

The answer:

- The amount of raw steel used during the production run is modeled by the expression:

$$7D + 3C + 1.5 M$$

3 - Wilson has only 2000 pounds of raw steel available. And at least 100 desk must be produced to satisfy contract commitment.

The answer:

- The function constraint can be modeled as :

$$7 D + 3C + 1.5 M \leq 2000 \text{ (raw steel)}$$

$$D \geq 100 \text{ (contract)}$$

The **Mathematical model** for this problem is formulated as :

MAXIMIZE

$$50 D + 30 C + 6 M$$

SUBJECT TO

$$7D + 3C + 1.5M \leq 2000 \text{ (Raw steel)}$$

$$D \geq 100 \text{ (contract)}$$

Example

2. Consider the following **tickets purchasing problem**.

A businessperson has a 5-week commitment traveling between Fayetteville (FYV) and Denver (DEN). Weekly departure from Fayetteville occurs on Mondays for return on Wednesdays. A regular roundtrip ticket costs \$400, but a 20% discount is granted if the roundtrip dates span a weekend. A one-way ticket in either direction costs 75% of the regular price.

How should the tickets be bought for the 5-week period?

Decision making problem, the solution requires answering three questions:

1. What are the decision **alternatives**?
2. Under what **restrictions** is the decision made?
3. What is an appropriate **objective criterion** for evaluating the alternatives?

- Three **plausible alternatives**:
 1. Buy five regular FYV-DEN-FYV for departure on Monday and return on Wednesday of the same week.
 2. Buy one FYV-DEN, four DEN-FYV-DEN that span weekends, and one DEN-FYV.
 3. Buy one FYV-DEN-FYV to cover Monday of the first week and Wednesday of the last week and four DEN-FYV-DEN to cover the remaining legs. All tickets in this alternative span at least one weekend.

The **restriction** on these options is that the businessperson should be able to leave FYV on Monday and return on Wednesday of the same week.

- An obvious **objective criterion** for evaluating the proposed alternatives is the **price** of the tickets.
- The alternative that yields the smallest cost is the best.
- We have:
 - Alternative 1 cost= $5 * \$400 = \2000
 - Alternative 2 cost= $0.75 * \$400 + 4 * (0.8 * \$400) + 0.75 * \$400 = \1880
 - Alternative 3 cost= $5 * (0.8 * \$400) = \1600

Alternative 3 is the cheapest

- Based on the preceding two examples, the general OR model can be organized in the following general format:

Maximize / minimize

Objective Function

subject to

Constraints

A solution is **feasible** if it satisfies all the constraints. It is **optimal** if, in addition to being feasible, it yields the best (maximum or minimum) value of the objective function.

- In the ticket purchasing problem, the problem considers three feasible alternatives, with the third alternative being optimal.
- Though OR models are designed to optimize a specific objective criterion subject to a set of constraints, the quality of the resulting solution depends on the degree of completeness of the model in representing the real system.
- In the ticket purchasing problem: if all the dominant alternatives for purchasing the tickets are not identified, then the resulting solution is optimum only relative to the alternatives represented in the model, e.g. leave out option 3 -> **suboptimal** solution.

Methods in OR

- Statistical techniques
 - Time trends and forecasting
 - Decision theory and tree
- Activity analysis
 - Time motion studies
 - Work sampling and activity analysis
- Mathematical techniques
 - Simulation study/models
 - System analysis
 - Linear programming
 - Inventory control
 - Network analysis

Methods in OR

- Financial techniques
 - Monitoring expenditure
 - Cost accounting and analysis
 - DALY
 - Cost benefit analysis
 - Cost effectiveness analysis
 - Zero base budgeting
 - Input output analysis
 - Outcome budget
- Miscellaneous
 - Management by exception
 - Situational analysis
 - Current state assessment
 - SWOT analysis
 - Log frame analysis

Some other techniques used in O.R

1. Linear programming.
2. Dynamic programming.
3. Queuing theory.
4. Inventory theory.
5. CPM and PERT techniques.
6. Game theory.
7. Replacement models.
8. Sequencing models.
9. Decision analysis models.
10. Simulation.

Characteristics or significant features of OR

- **Decision making tool**: Optimal solution to the problem.
- **Scientific approach**: Scientific methods, techniques and tools for analyzing and solving the problems.
- **Interdisciplinary team approach**: Members of an interdisciplinary team analyze the cause-and-effect relationship between various parameters of the problem.
- **System approach or wholistic approach**: Indirect effect on all other parts is considered.
- **Objective**: Find the best or optimal solutions to the problem.

General methods for solving OR models

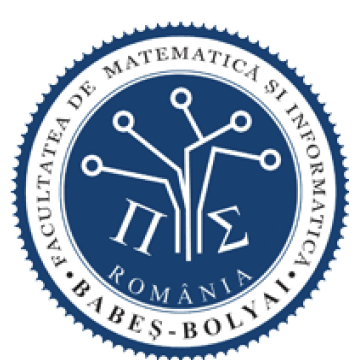
- **Analytical Method**: Mathematical tools like calculus, matrix algebra, probability, graphs etc. are used for solving OR problems.
- **Numerical or Iterative method**: when analytical methods fail to solve complex problems numerical methods are used. Numerical methods involve a trial and error procedure. An iterative process is one in which successive trials tend to approach an optimal solution.
- **Monte-Carlo simulation**: use of probability and random sampling concepts.

Scope

- A decision maker can conduct the experiments on the model by changing various conditions to examine the effect of these changes and determine the best solution.
- The experiments can be performed on the model, avoiding causing any damage to an existing system.
- Computer simulations can be used to solve complex problems.

Shortcomings

- Building complex OR models is economically feasible only for repetitive problems. Limitations on resources like money, time etc.
- OR models may not represent the realistic situations in which decisions have to be taken.
- The data is subject to frequent changes which makes the process of updating the models quite expensive.
- The decision maker may not be fully aware of the limitations of OR model being used.
- The inability of OR to convey and communicate results to business executives who need to make decisions.



Operations Research

02 - Linear Programming

Mihai Suci

Faculty of Mathematics and Computer Science

LINEAR PROGRAMMING (LP)

- In mathematics, linear programming (LP) is a technique for optimization of a linear objective function, subject to linear equality and linear inequality constraints.
- Linear programming determines the way to achieve the best outcome (such as maximum profit or lowest cost) in a given mathematical model and given some list of requirements represented as linear equations.

Mathematical formulation of Linear Programming model:

- Step 1
 - Study the given situation
 - Find the key decision to be made
 - Identify the decision variables of the problem
- Step 2
 - Formulate the objective function to be optimized
- Step 3
 - Formulate the constraints of the problem
- Step 4
 - Add non-negativity restrictions or constraints

The objective function , the set of constraints and the non-negativity

- restrictions together form an LP model.

TWO-VARIABLE LP MODEL

- EXAMPLE:
- “THE GALAXY INDUSTRY PRODUCTION”
- Galaxy manufactures two toy models:
 - Space Ray.
 - Zapper.
- Resources are limited to
 - 1200 pounds of special plastic.
 - 40 hours of production time per week.

- Marketing requirement
 - Total production cannot exceed 800 dozens.
 - Number of dozens of Space Rays cannot exceed number of dozens of Zappers by more than 450.
- Technological input
 - Space Rays requires 2 pounds of plastic and 3 minutes of labor per dozen.
 - Zappers requires 1 pound of plastic and 4 minutes of labor per dozen.

- Current production plan calls for:
 - Producing as much as possible of the more profitable product, Space Ray (\$8 profit per dozen).
 - Use resources left over to produce Zappers (\$5 profit per dozen).
- The current production plan consists of:
 - Space Rays = 550 dozens
 - Zapper = 100 dozens
 - Profit = 4900 dollars per week

A Linear Programming Model can provide an intelligent solution to this problem

Solution

- Decisions variables:
 - X_1 = Production level of Space Rays (in dozens per week).
 - X_2 = Production level of Zappers (in dozens per week).
- Objective Function:
 - Weekly profit, to be maximized

- The Linear Programming Model

$$\text{Max } 8X_1 + 5X_2 \quad (\text{Weekly profit})$$

subject to

$$2X_1 + 1X_2 \leq 1200 \quad (\text{Plastic})$$

$$3X_1 + 4X_2 \leq 2400 \quad (\text{Production Time})$$

$$X_1 + X_2 \leq 800 \quad (\text{Total production})$$

$$X_1 - X_2 \leq 450 \quad (\text{Mix})$$

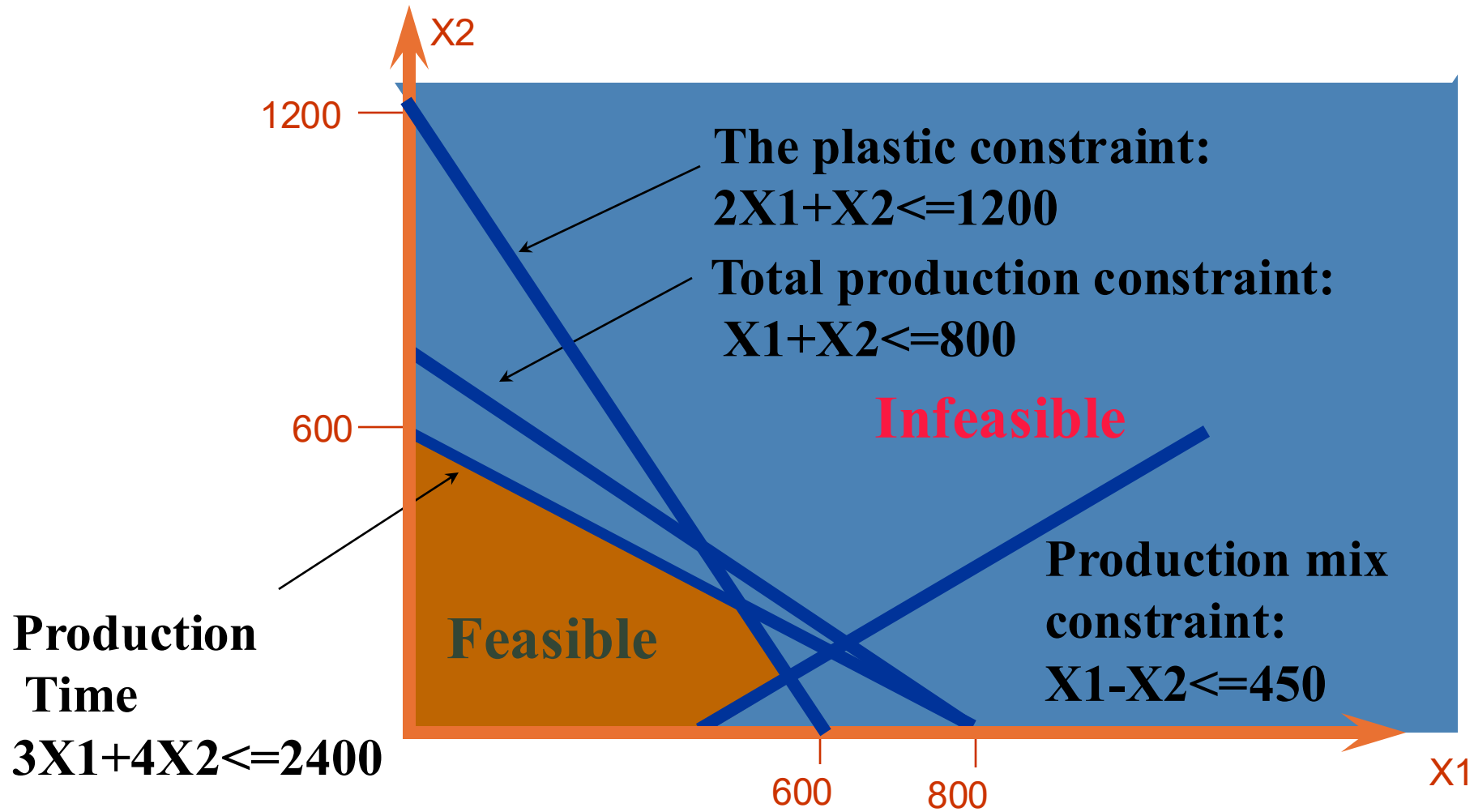
$$X_j \geq 0, \quad j = 1, 2 \quad (\text{Nonnegativity})$$

Feasible Solutions for Linear Programs

- The set of all points that satisfy all the constraints of the model is called

**FEASIBLE
REGION**

- Using a graphical presentation we can represent all the constraints, the objective function, and the three types of feasible points.



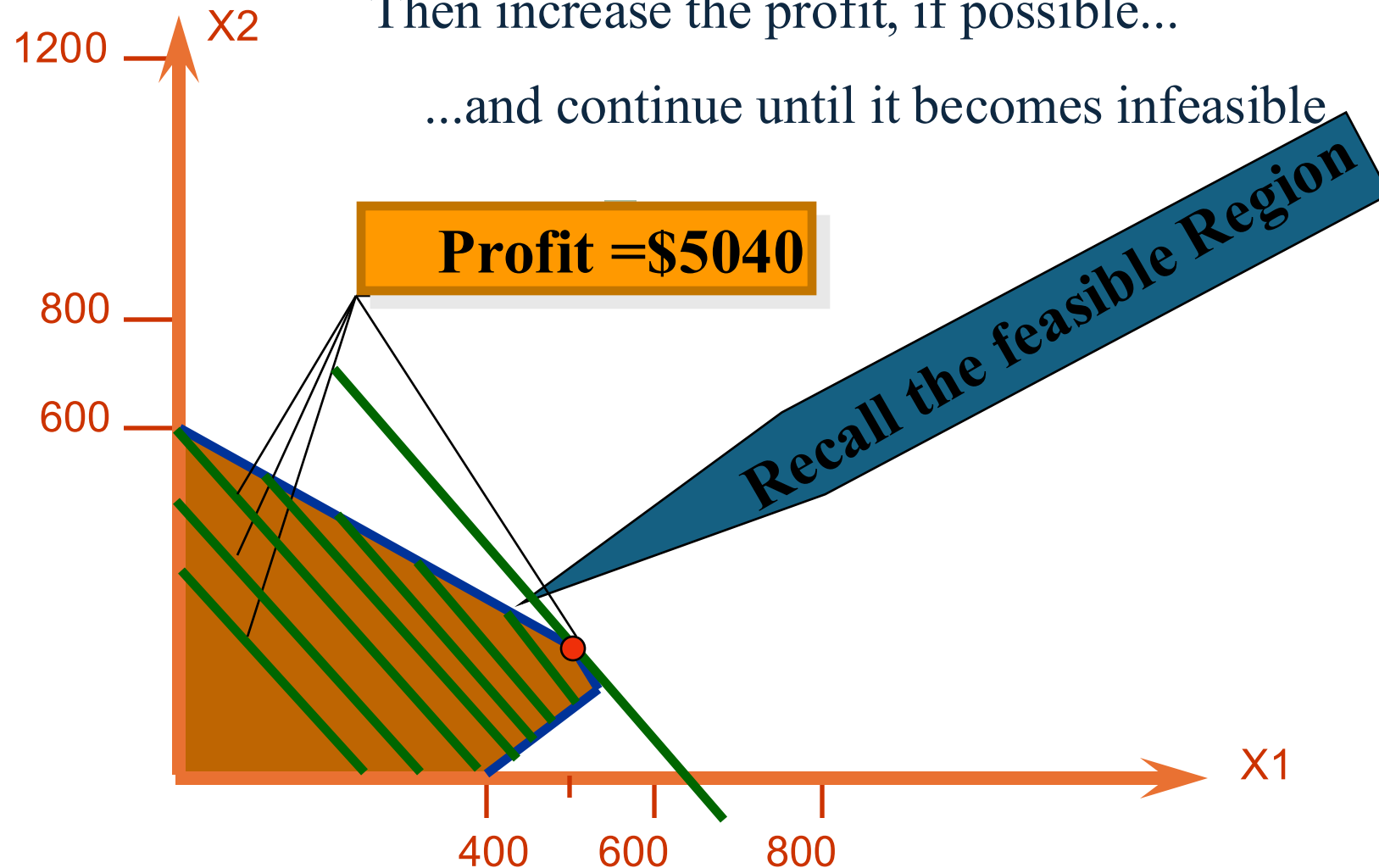
Solving Graphically for an Optimal Solution

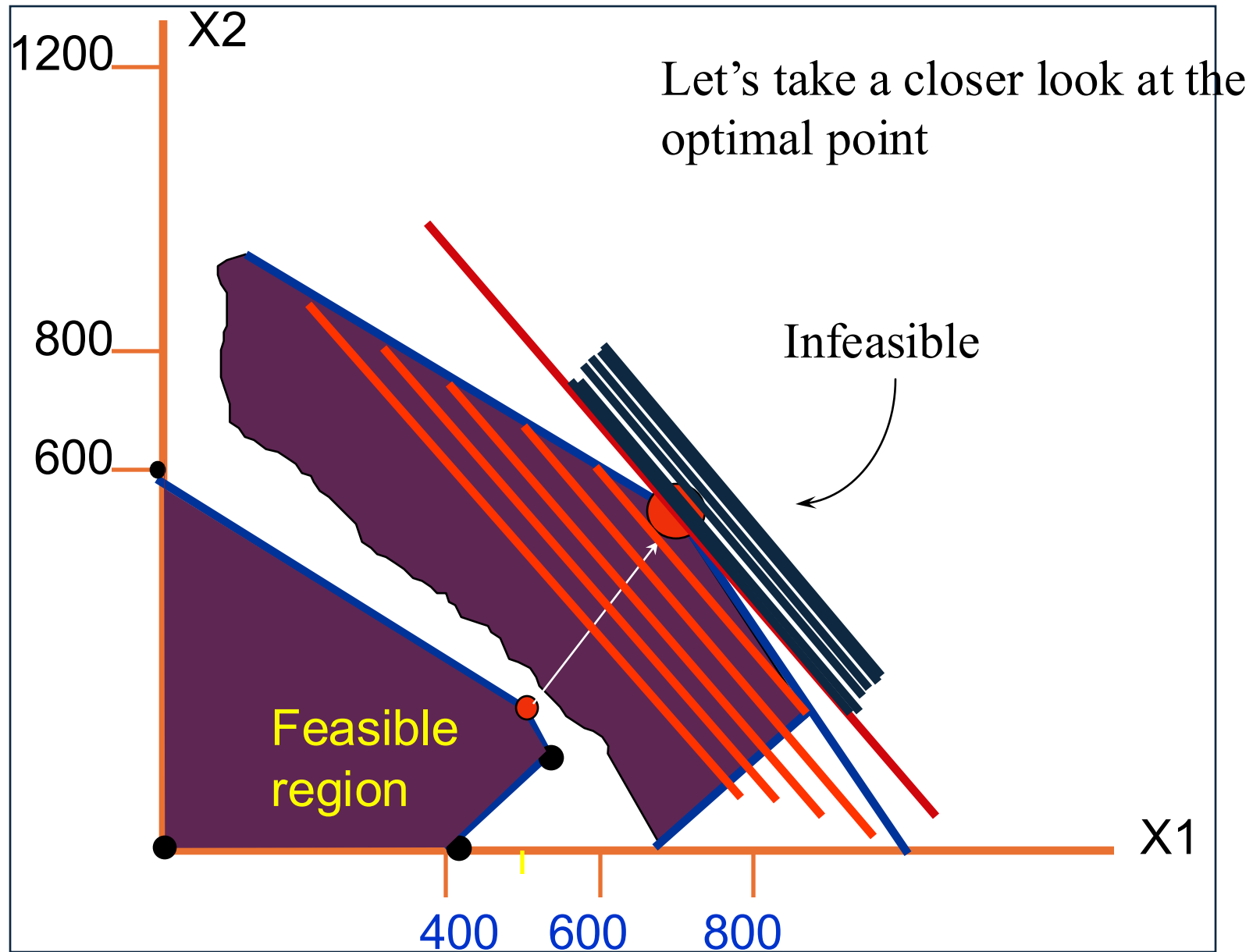
We now demonstrate the search for an optimal solution

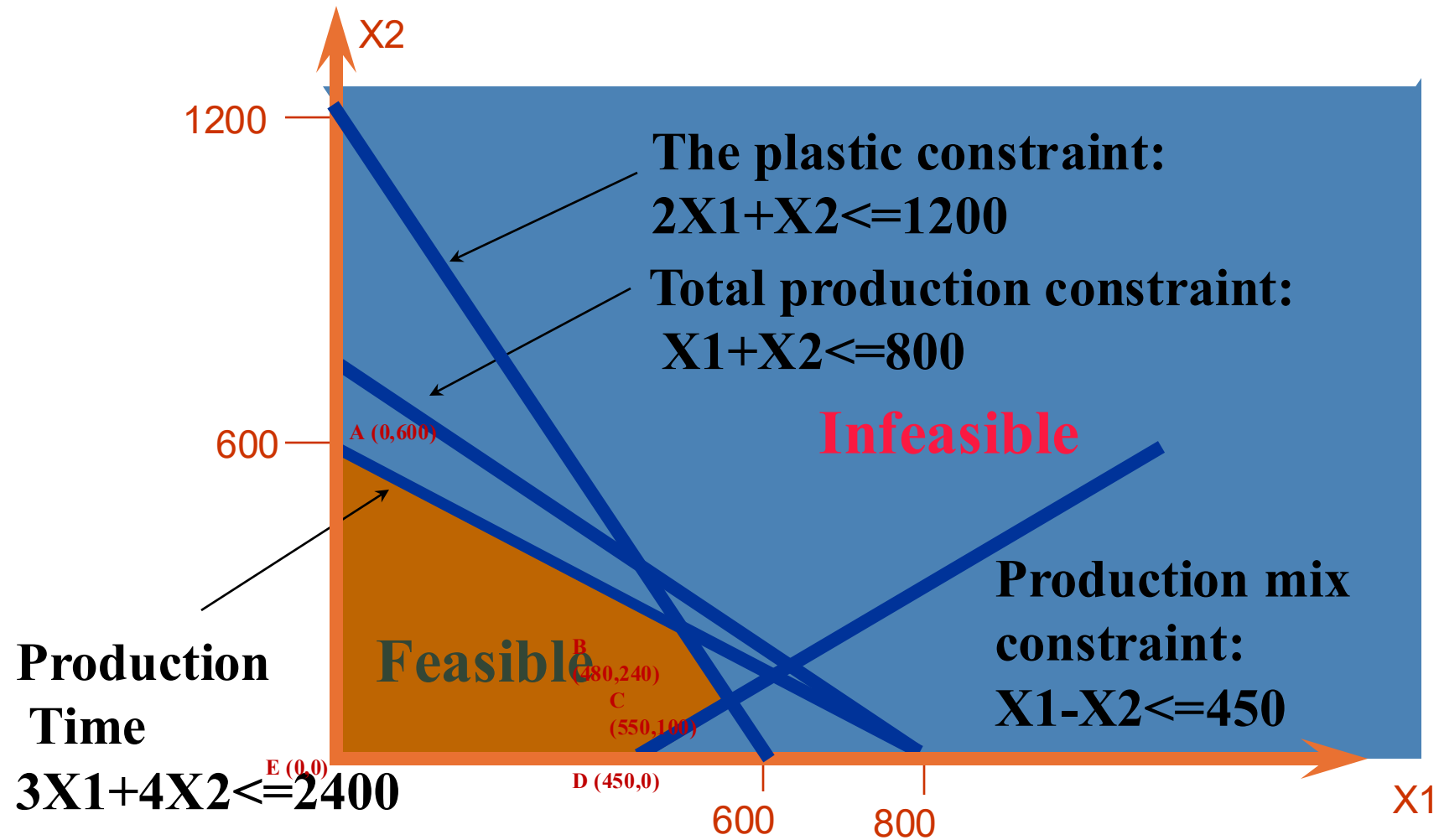
Start at some arbitrary profit, say profit = \$2,000...

Then increase the profit, if possible...

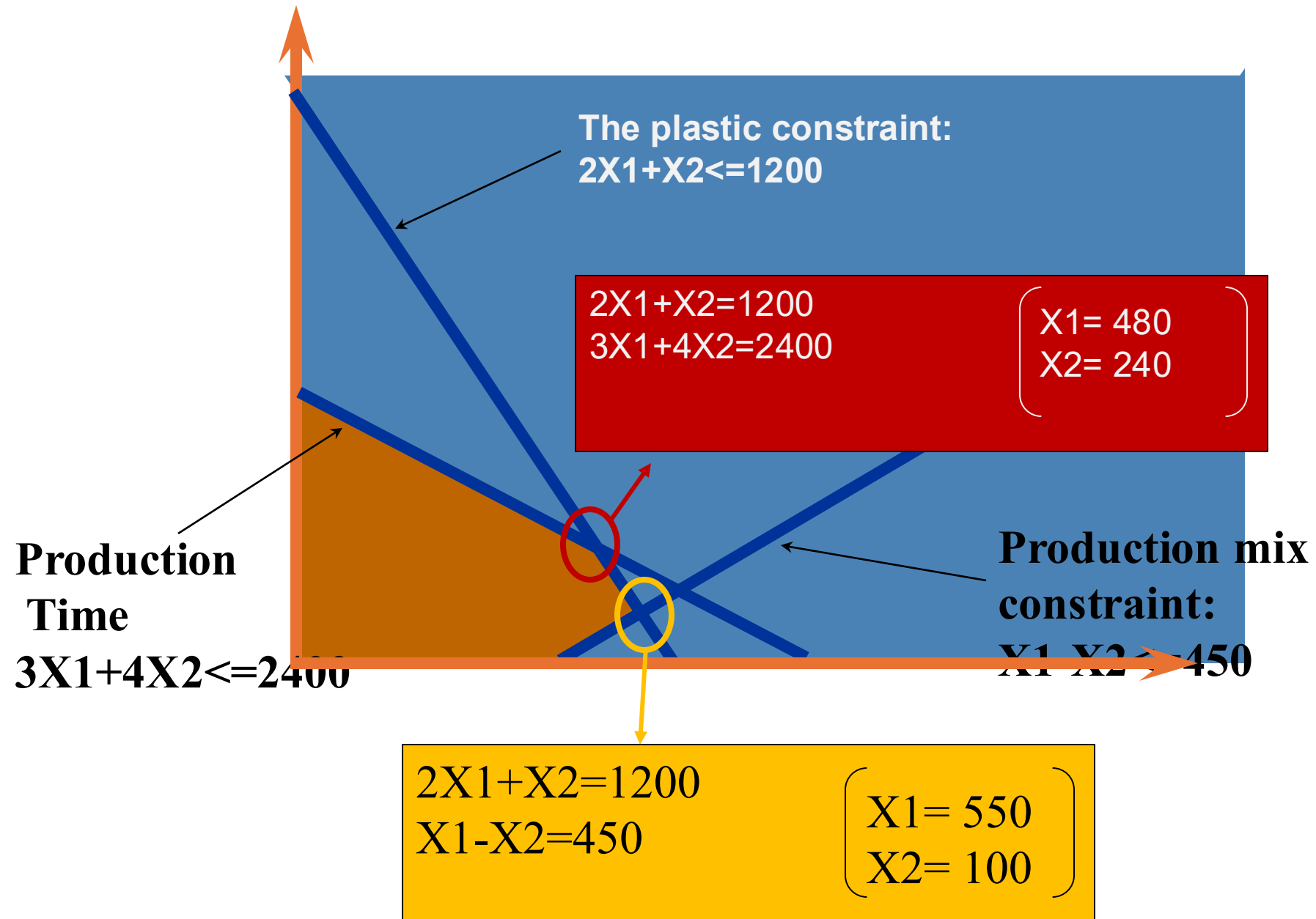
...and continue until it becomes infeasible







- To determine the value for X_1 and X_2 at the optimal point, the two equations of the binding constraint must be solved.



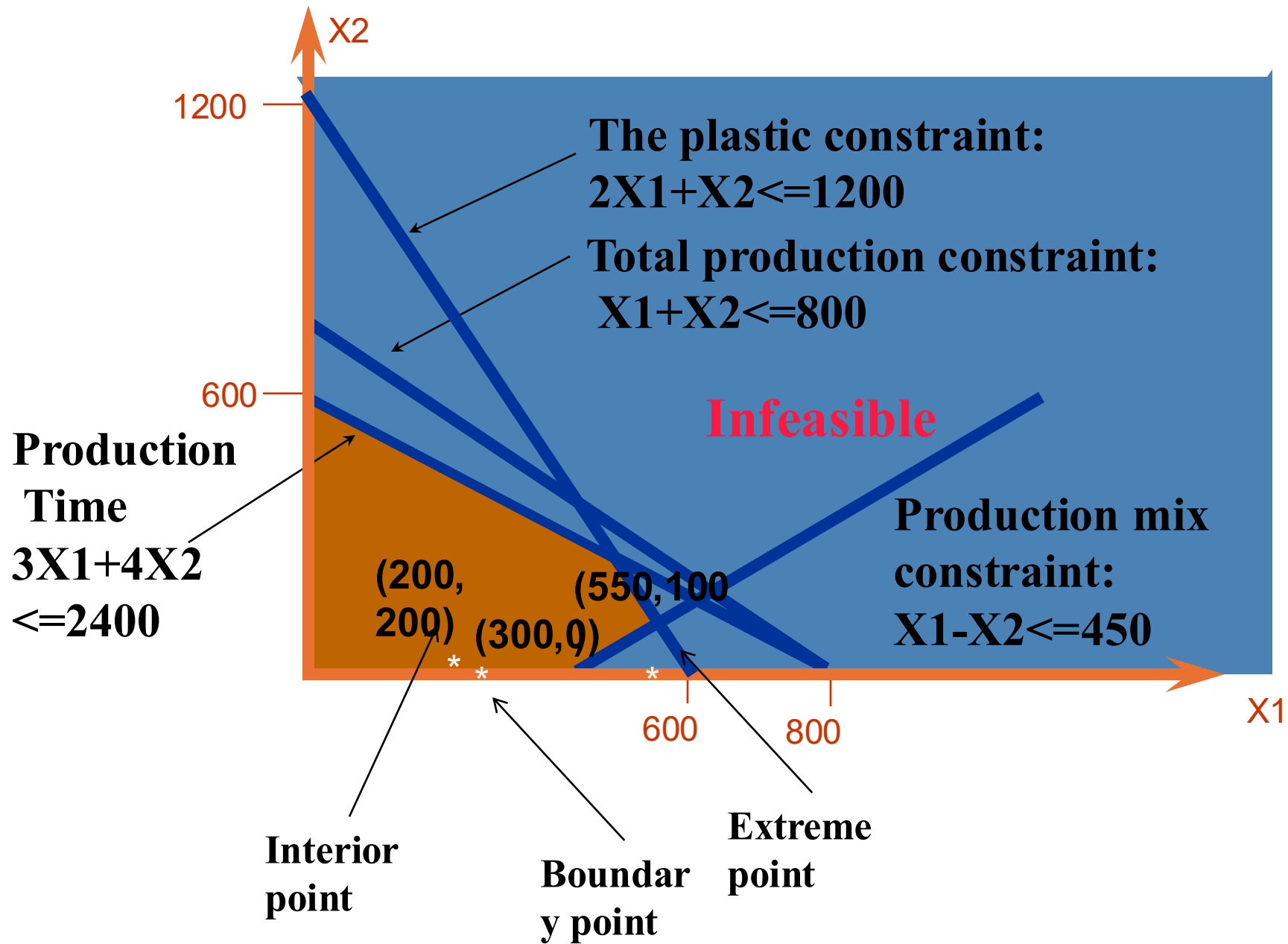
By Compensation on :
Max $8X_1 + 5X_2$

(X1, X2)	Objective fn
(0,0)	0
(450,0)	3600
(480,240)	5040
(550,100)	4900
(0,600)	3000

The maximum profit (5040) will be by producing:
Space Rays = 480 dozens, Zappers = 240 dozens

Type of feasible points

- Interior point: satisfies all constraint but non with equality.
- Boundary points: satisfies all constraints, at least one with equality
- Extreme point: satisfies all constraints, two with equality.



- If a linear programming has an optimal solution, an extreme point is optimal.

Summery of graphical solution procedure

- 1- graph constraint to find the feasible point
- 2- set objective function equal to an arbitrary value so that line passes through the feasible region.
- 3- move the objective function line parallel to itself until it touches the last point of the feasible region .
- 4- solve for X_1 and X_2 by solving the two equation that intersect to determine this point
- 5- substitute these value into objective function to determine its optimal solution.

- MORE EXAMPLES

Example 1 - The Reddy Mikks Company

- Reddy Mikks produces both interior and exterior paints from two raw materials M1 and M2

Tons of raw material per ton of

	Exterior paint	Interior paint	Maximum daily availability (tons)
Raw material M1	6	4	24
Raw material M2	1	2	6
Profit per ton (\$1000)	5	4	

Daily demand for interior paint cannot exceed that of exterior paint by more than 1 ton

Maximum daily demand of interior paint is 2 tons

Reddy Mikks wants to determine the optimum product mix of interior and exterior paints that maximizes the total daily profit

Solution:

Let x_1 = tons produced daily of exterior paint

x_2 = tons produced daily of interior paint

Let z represent the total daily profit (in thousands of dollars)

Objective:

$$\text{Maximize } z = 5x_1 + 4x_2$$

(Usage of a raw material by both paints) < (Maximum raw material availability)

$$\text{Usage of raw material M1 per day} = 6x_1 + 4x_2 \text{ tons}$$

$$\text{Usage of raw material M2 per day} = 1x_1 + 2x_2 \text{ tons}$$

- daily availability of raw material M1 is 24 tons

- daily availability of raw material M2 is 6 tons

Restrictions

$$6x_1 + 4x_2 \leq 24 \quad (\text{raw material M1})$$

$$x_1 + 2x_2 \leq 6 \quad (\text{raw material M2})$$

- Difference between daily demand of interior (x_2) and exterior (x_1) paints does not exceed 1 ton,

$$\text{so} \quad x_2 - x_1 \leq 1$$

- Maximum daily demand of interior paint is 2 tons,

$$\text{so} \quad x_2 \leq 2$$

- Variables x_1 and x_2 cannot assume negative values,

$$\text{so} \quad x_1 \geq 0, \quad x_2 \geq 0$$

Complete Reddy Mikks model

Maximize $z = 5x_1 + 4x_2$ (total daily profit)

subject to

$$6x_1 + 4x_2 \leq 24 \quad (\text{raw material M1})$$

$$x_1 + 2x_2 \leq 6 \quad (\text{raw material M2})$$

$$x_2 - x_1 \leq 1$$

$$x_2 \leq 2$$

$$x_1 \geq 0$$

$$x_2 \geq 0$$

- Objective and the constraints are all linear functions in this example.

Properties of the LP model:

Linearity implies that the LP must satisfy three basic properties:

1) Proportionality:

- contribution of each decision variable in both the objective function and constraints to be directly proportional to the value of the variable

2) Additivity:

- total contribution of all the variables in the objective function and in the constraints to be the direct sum of the individual contributions of each variable

3) Certainty:

- All the objective and constraint coefficients of the LP model are deterministic (known constants)
- LP coefficients are average-value approximations of the probabilistic distributions
- If standard deviations of these distributions are sufficiently small, then the approximation is acceptable
- Large standard deviations can be accounted for directly by using stochastic LP algorithms or indirectly by applying sensitivity analysis to the optimum solution

Example 2 - Problem Mix Model

- Two machines X and Y
- X is designed for 5-ounce bottles
- Y is designed for 10-ounce bottles
- X can also produce 10-ounce bottles with some loss of efficiency
- Y can also produce 5-ounce bottles with some loss of efficiency

Machine	5-ounce bottles	10-ounce bottles
X	80/min	30/min
Y	40/min	50/min

- X and Y machines can run 8 hours per day for 5 days a week
- Profit on 5-ounce bottle is 20 paise
- Profit on 10-ounce bottle is 30 paise
- Weekly production of the drink cannot exceed 500,000 ounces
- Market can utilize 30,000 (5-ounce) bottles and 8000 (10-ounce) bottles per week
- To maximize the profit

Solution:

Let X_1 = number of 5-ounce bottles to be produced per week
 X_2 = number of 10-ounce bottles to be produced per week

Objective:

Maximize profit $z = \text{Rs } (0.20x_1 + 0.30x_2)$

Constraints:

- Time constraint on machine X,
$$(x_1/80) + (x_2/30) \leq 8 \times 60 \times 5 = 2400 \text{ minutes}$$
- Time constraint on machine Y,
$$(x_1/40) + (x_2/50) \leq 8 \times 60 \times 5 = 2400 \text{ minutes}$$
- Weekly production of the drink cannot exceed 500,000 ounces,
$$5x_1 + 10x_2 \leq 500,000 \text{ ounces}$$
- Market demand per week,
$$x_1 \geq 30,000 \text{ (5-ounce bottles)}$$
$$x_2 \geq 8,000 \text{ (10-ounce bottles)}$$

Example 3 – Production allocation model

- Two types of products A and B
- Profit of Rs.4 on type A
- Profit of Rs.5 on type B
- Both A and B are produced by X and Y machines

	Machine	Machine
Products	X	Y
A	2 minutes	3 minutes
B	2 minutes	2 minutes

- Machine X is available for maximum 5 hours and 30 minutes during any working day
- Machine Y is available for maximum 8 hours during any working day
- Formulate the problem as a LP problem.

Solution:

Let x_1 = number of products of type A

x_2 = number of products of type B

Objective:

- Profit of Rs.4 on type A , therefore $4x_1$ will be the profit on selling x_1 units of type A
- Profit of Rs.5 on type B, therefore $5x_2$ will be the profit on selling x_2 units of type B

Total profit,

$$Z = 4x_1 + 5x_2$$

Constraints:

- Time constraint on machine X,

$$2x_1 + 2x_2 \leq 330 \text{ minutes}$$

- Time constraint on machine Y,

$$3x_1 + 2x_2 \leq 480 \text{ minutes}$$

- Non-negativity restrictions are,

$$x_1 \geq 0 \text{ and } x_2 \geq 0$$

The Complete LP model

$$\text{Maximize } z = 4x_1 + 5x_2$$

subject to

$$2x_1 + 2x_2 \leq 330 \text{ minutes}$$

$$3x_1 + 2x_2 \leq 480 \text{ minutes}$$

$$x_1 \geq 0$$

$$x_2 \geq 0$$

GRAPHICAL LP SOLUTION

The graphical procedure includes two steps:

1. Determination of the feasible solution space.
2. Determination of the optimum solution from among all the feasible points in the solution space.

Solution of a Maximization model (Reddy Mikks model)

Step 1:

1) Determine the feasible solution space:

- Find the coordinates for all the 6 equations of the restrictions (only take the equality sign)

$$6x_1 + 4x_2 \leq 24 \quad (1)$$

$$x_1 + 2x_2 \leq 6 \quad (2)$$

$$x_2 - x_1 \leq 1 \quad (3)$$

$$x_2 \leq 2 \quad (4)$$

$$x_1 \geq 0 \quad (5)$$

$$x_2 \geq 0 \quad (6)$$

Change all equations to equality signs

$$6x_1 + 4x_2 = 24$$

$$x_1 + 2x_2 = 6$$

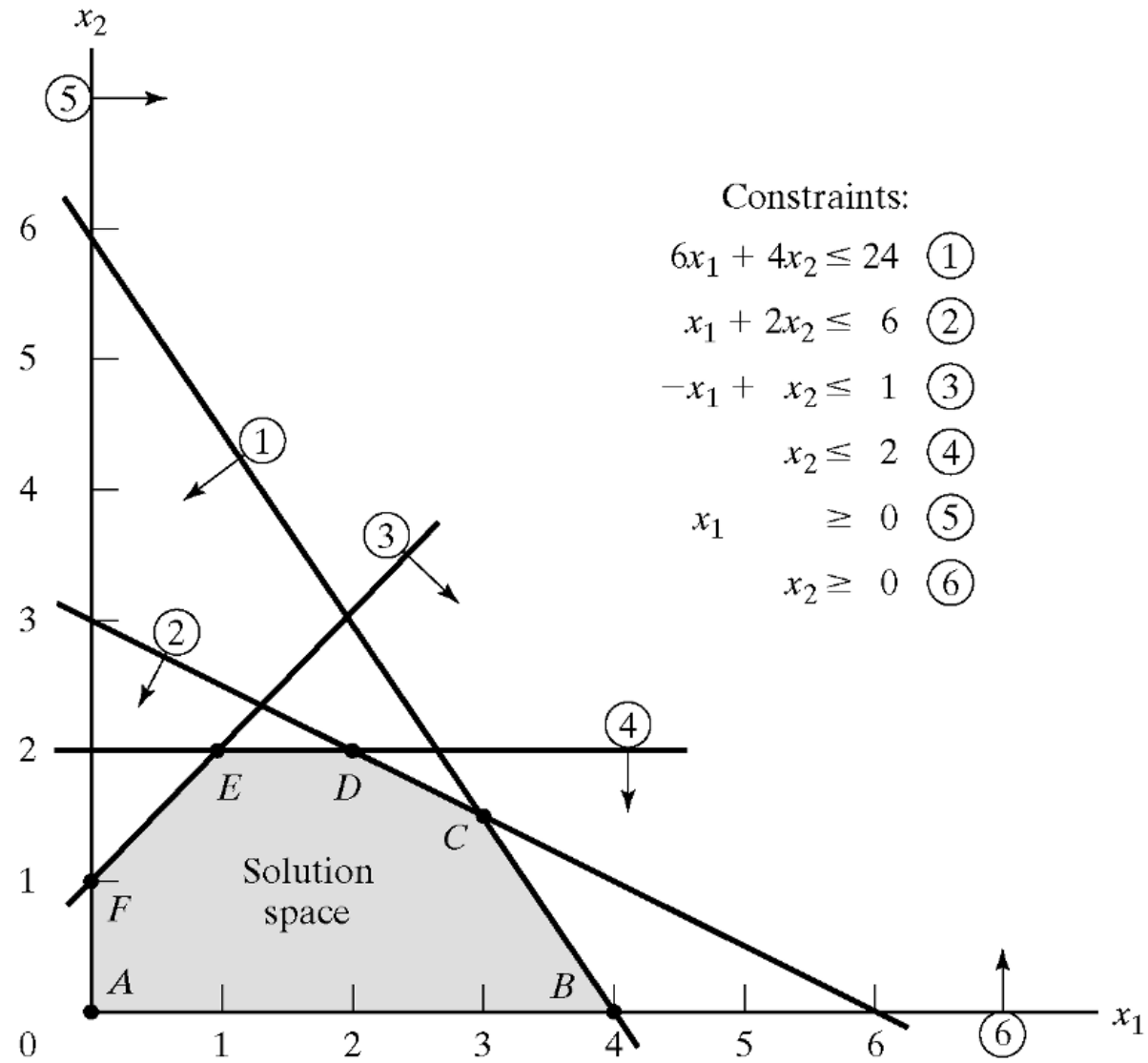
$$x_2 - x_1 = 1$$

$$x_2 = 2$$

$$x_1 = 0$$

$$x_2 = 0$$

- Plot graphs of $x_1 = 0$ and $x_2 = 0$
- Plot graph of $6x_1 + 4x_2 = 24$ by using the coordinates of the equation
- Plot graph of $x_1 + 2x_2 = 6$ by using the coordinates of the equation
- Plot graph of $x_2 - x_1 = 1$ by using the coordinates of the equation
- Plot graph of $x_2 = 2$ by using the coordinates of the equation



- Now include the inequality of all the 6 equations
- Inequality divides the (x_1, x_2) plane into two half spaces , one on each side of the graphed line
- Only one of these two halves satisfies the inequality
- To determine the correct side , choose $(0,0)$ as a reference point
- If $(0,0)$ coordinate satisfies the inequality, then the side in which $(0,0)$ coordinate lies is the feasible half-space , otherwise the other side is
- If the graph line happens to pass through the origin $(0,0)$, then any other point can be used to find the feasible half-space

Step 2:

2) Determination of the optimum solution from among all the feasible points in the solution space:

- After finding out all the feasible half-spaces of all the 6 equations, feasible space is obtained by the line segments joining all the corner points A, B, C, D, E and F

- Any point within or on the boundary of the solution space ABCDEF is feasible as it satisfies all the constraints

- Feasible space ABCDEF consists of infinite number of feasible points

- To find optimum solution identify the direction in which the maximum profit increases , that is $z = 5x_1 + 4x_2$
- Assign random increasing values to z , $z = 10$ and $z = 15$

$$5x_1 + 4x_2 = 10$$

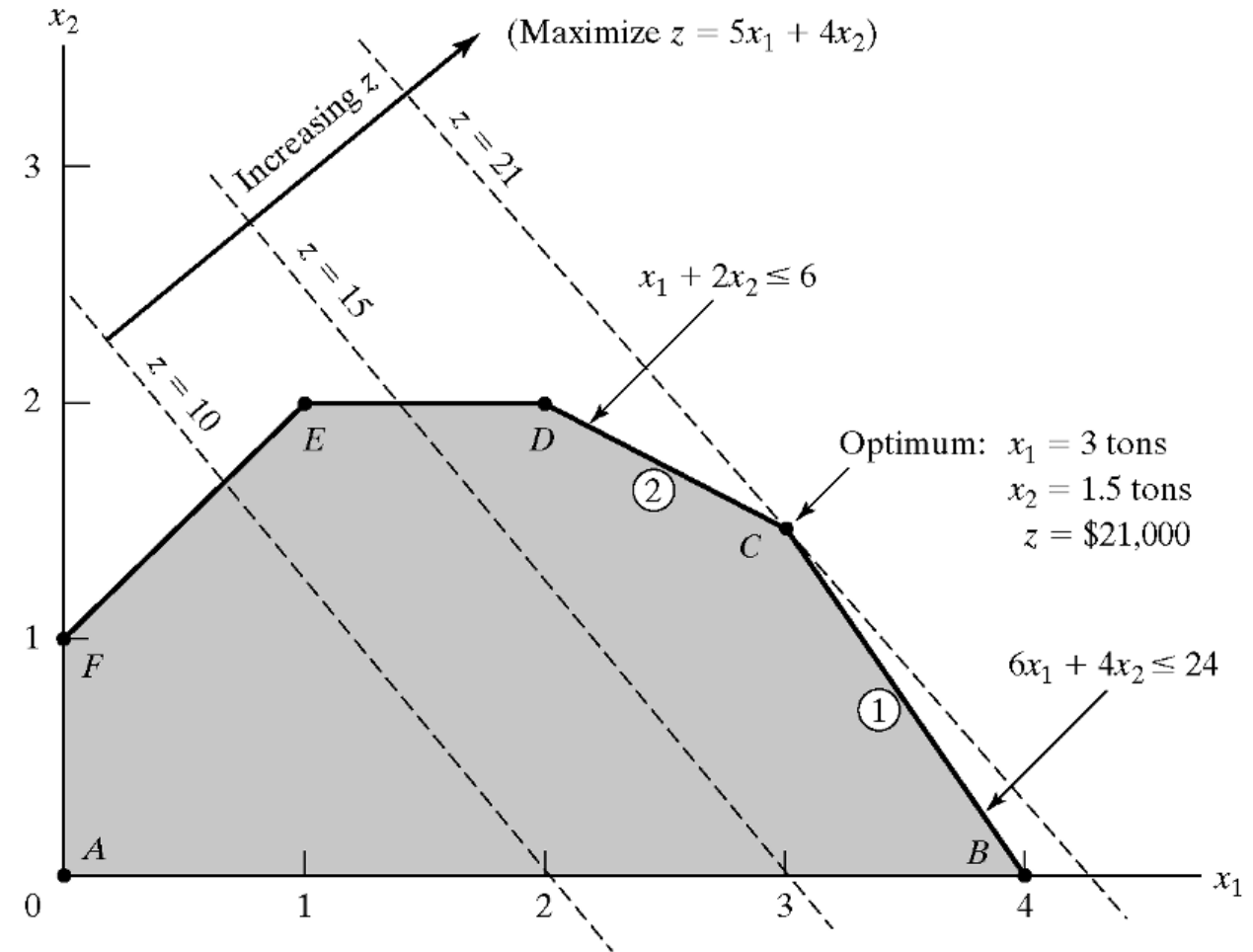
$$5x_1 + 4x_2 = 15$$
- Plot graphs of above two equations
- Thus in this way the optimum solution occurs at corner point C which is the point in the solution space
- Any further increase in z that is beyond corner point C will put points outside the boundaries of ABCDEF feasible space
- Values of X_1 and X_2 associated with optimum corner point C are determined by solving the equations (1) and (2)

$$6x_1 + 4x_2 = 24 \quad (1)$$

$$x_1 + 2x_2 = 6 \quad (2)$$

- $X_1 = 3$ and $X_2 = 1.5$ with $z = 5 \times 3 + 4 \times 1.5 = 21$
- So daily product mix of 3 tons of exterior paint and 1.5 tons of interior paint produces the daily profit of \$21,000 .

- optimum solution



- Important characteristic of the optimum LP solution is that it is always associated with a corner point of the solution space (where two lines intersect)
- This is even true if the objective function happens to be parallel to a constraint
- For example if the objective function is,
$$z = 6x_1 + 4x_2$$
- The above equation is parallel to constraint of equation (1)
- So optimum occurs at either corner point B or corner point C when parallel
- Actually any point on the line segment BC will be an alternative optimum
- Line segment BC is totally defined by the corner points B and C

- Since optimum LP solution is always associated with a corner point of the solution space, so optimum solution can be found by enumerating all the corner points as below:

Corner point	(X_1, X_2)	z
A	(0,0)	0
B	(4,0)	20
C	(3,1.5)	21 (optimum solution)
D	(2,2)	18
E	(1,2)	13
F	(0,1)	4

- As number of constraints and variables increases , the number of corner points also increases

Solution of a Minimization model

- Firm or industry has two bottling plants
- One plant located at Coimbatore and other plant located at Chennai
- Each plant produces three types of drinks Coca-cola , Fanta and Thumps-up

Number of bottles produced per day
by plant at

	Coimbatore	Chennai
Coca-cola	15,000	15,000
Fanta	30,000	10,000
Thumps-up	20,000	50,000
Cost per day (in any unit)	600	400

- Market survey indicates that during the month of April there will be a demand of 200,000 bottles of Coca-cola , 400,000 bottles of Fanta , and 440,000 bottles of Thumps-up
- For how many days each plant be run in April so as to minimize the production cost , while still meeting the market demand?

Solution:

Let X_1 = number of days to produce all the three types of bottles by plant
at Coimbatore

X_2 = number of days to produce all the three types of bottles by plant
at Chennai

Objective:

$$\text{Minimize } z = 600 X_1 + 400 X_2$$

Constraint:

$$15,000 X_1 + 15,000 X_2 \geq 200,000 \quad (1)$$

$$30,000 X_1 + 10,000 X_2 \geq 400,000 \quad (2)$$

$$20,000 X_1 + 50,000 X_2 \geq 440,000 \quad (3)$$

$$X_1 \geq 0 \quad (4)$$

$$X_2 \geq 0 \quad (5)$$

Corner points	(x_1, x_2)	$Z = 600 x_1 + 400 x_2$
A	(0, 40)	16000
B	(12, 4)	8800
C	(22, 0)	13200

- In 12 days all the three types of bottles (Coca-cola, Fanta, Thumps-up) are produced by plant at Coimbatore
- In 4 days all the three types of bottles (Coca-cola, Fanta, Thumps-up) are produced by plant at Chennai
- So minimum production cost is 8800 units to meet the market demand of all the three types of bottles (Coca-cola, Fanta, Thumps-up) to be produced in April

Sensitivity Analysis

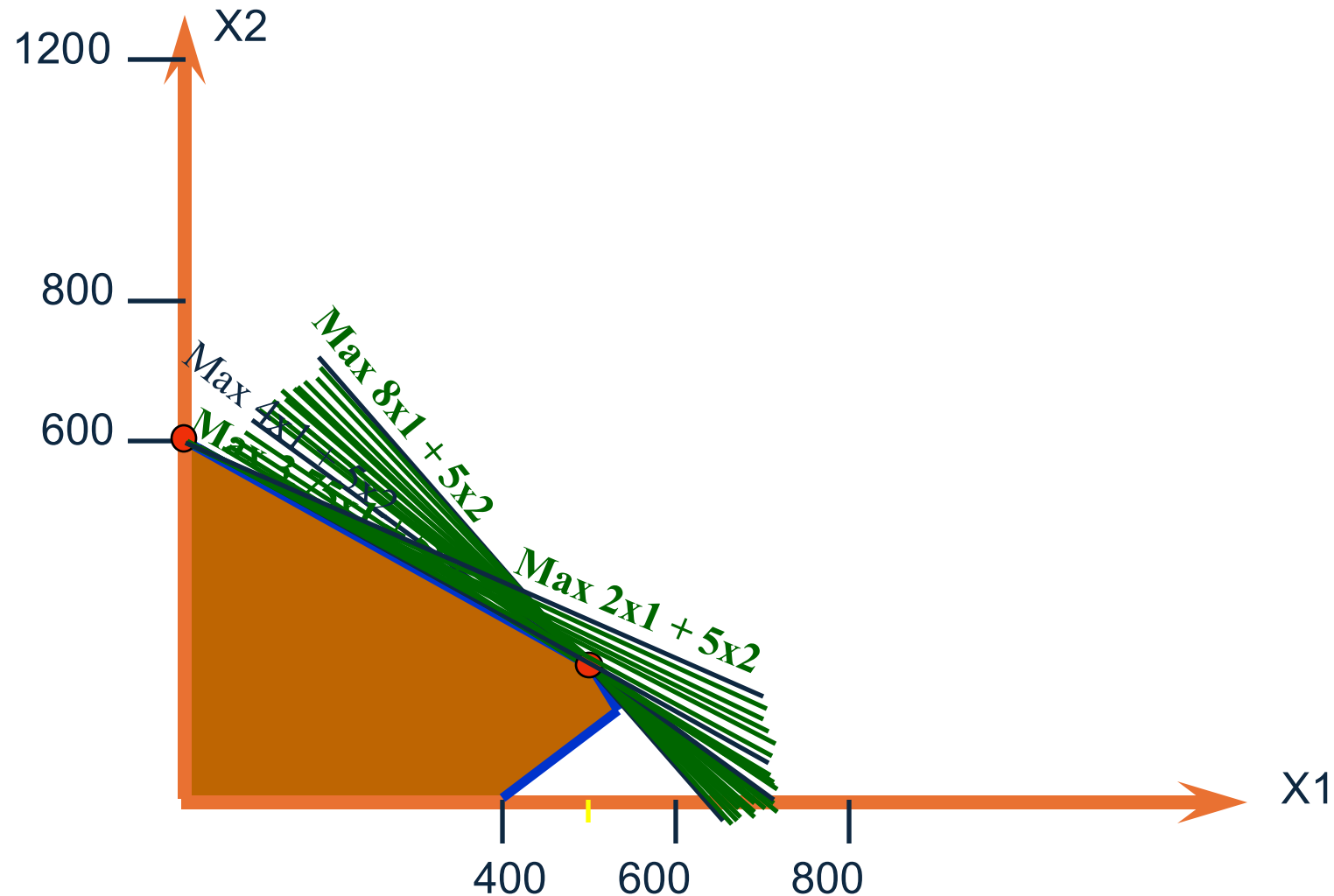
- The Role of Sensitivity Analysis of the Optimal Solution
 - Is the optimal solution sensitive to changes in input parameters?

The effective of this change is known as “sensitivity”

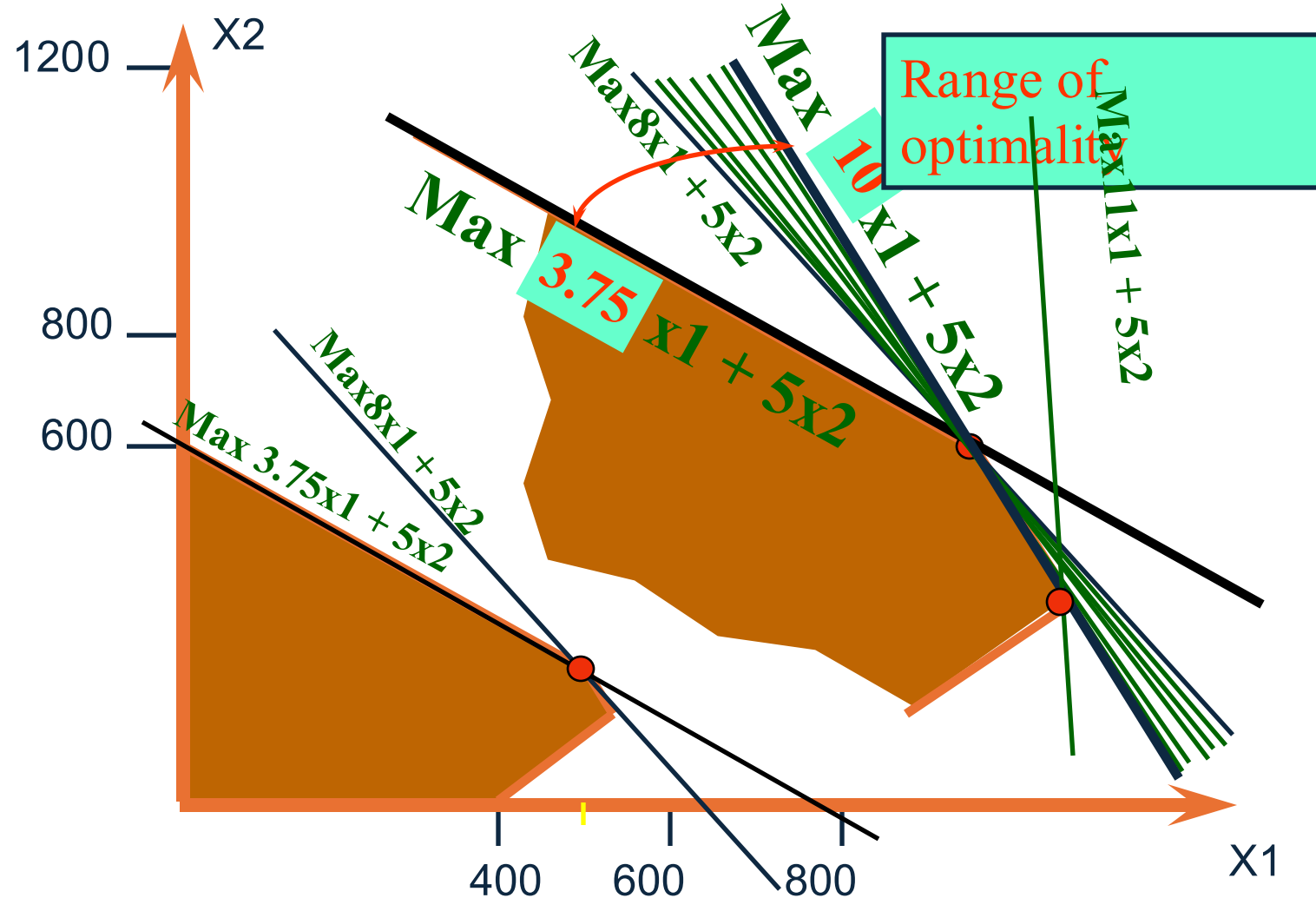
Sensitivity Analysis of Objective Function Coefficients

- Range of Optimality
 - The optimal solution will remain unchanged as long as
 - An objective function coefficient lies within its range of optimality
 - There are no changes in any other input parameters

The effects of changes in an objective function coefficient on the optimal solution



The effects of changes in an objective function coefficients on the optimal solution



- It could be finding the range of optimality for an objectives function coefficient by determining the range of values that gives a slope of the objective function line between the slopes of the binding constraints.

- The binding constraints are:
 - $2X_1 + X_2 = 1200$
 - $3X_1 + 4X_2 = 2400$
- The slopes are: $-2/1$, and $-3/4$ respectively.

- To find range optimality for Space Rays, and coefficient per dozen Zappers is $C_2 = 5$
- Thus the slope of the objective function line can be expressed as $-C_1/5$

- Range of optimality for C1 is found by sloving the following for C1:

$$-2/1 \leq -C1/5 \leq -3/4$$

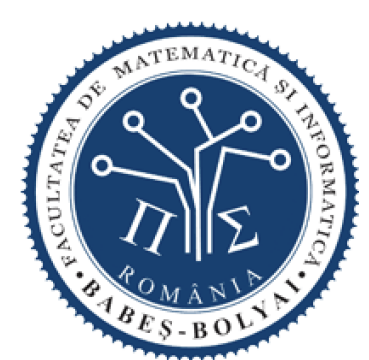
$$3.75 \leq C1 \leq 10$$

- Range optimality for Zapper, and coefficient per dozen space rays is $C_1 = 8$
- Thus the slope of the objective function line can be expressed as $-8/C_2$

- Range of optimality for C2 is found by solving the following for C2:

$$-2/1 \leq -8/C2 \leq -3/4$$

$$4 \leq C2 \leq 10.667$$



Operations Research

03 – Simplex method

Mihai Suci

Faculty of Mathematics and Computer Science

Simplex method when some constraints are not “ $\leq$ ” constraints

- We employ a mathematical “trick” to jumpstart the problem by adding **artificial variables** to the equations.

Simplex method when some constraints are not “ $\leq$ ” constraints (cont.)

Example:

$$\text{Max } 16x_1 + 15x_2 + 20x_3 - 18x_4$$

ST

$$2x_1 + x_2 + 3x_3 \leq 3000 \quad [1]$$

$$3x_1 + 4x_2 + 5x_3 - 60x_4 \leq 2400 \quad [2]$$

$$x_4 \leq 32 \quad [3]$$

$$x_2 \geq 200 \quad [4]$$

$$x_1 + x_2 + x_3 \geq 800 \quad [5]$$

$$x_1 - x_2 - x_3 = 0 \quad [6]$$

$$x_j \geq 0 \text{ for all } j$$

Simplex method when some constraints are not “ $\leq$ ” constraints (cont.)

Example:

$$\text{Max } 16x_1 + 15x_2 + 20x_3 - 18x_4$$

ST

$$2x_1 + x_2 + 3x_3 \leq 3000 \quad [1]$$

$$3x_1 + 4x_2 + 5x_3 - 60x_4 \leq 2400 \quad [2]$$

$$x_4 \leq 32 \quad [3]$$

$$x_2 \geq 200 \quad [4]$$

$$x_1 + x_2 + x_3 \geq 800 \quad [5]$$

$$x_1 - x_2 - x_3 = 0 \quad [6]$$

$$x_j \geq 0 \text{ for all } j$$

We assign a very large negative objective function coefficient, **-M**, (**+M** for minimization problem) to each artificial variable

We add artificial : **R4, R5, R6**, respectively to the fourth, fifth, and sixth equations.

Simplex method when some constraints are not “ $\leq$ ” constraints (cont.)

The solution

$$\text{Max } 16x_1 + 15x_2 + 20x_3 - 18x_4 - \text{MR4} - \text{MR5} - \text{MR6}$$

ST

$$2x_1 + x_2 + 3x_3 + s_1 = 3000 \quad [1]$$

$$3x_1 + 4x_2 + 5x_3 - 60x_4 + s_2 = 2400 \quad [2]$$

$$x_4 + s_3 = 32 \quad [3]$$

$$x_2 - s_4 + R_4 = 200 \quad [4]$$

$$x_1 + x_2 + x_3 - s_5 + R_5 = 800 \quad [5]$$

$$x_1 - x_2 - x_3 + R_6 = 0 \quad [6]$$

$$x_j \geq 0, s_j \geq 0, R_j \geq 0 \text{ for all } j$$

The simplex algorithm can then be used to solve this problem

Solving For the optimal solution of **[Maximization]** when there are artificial variables

Example # 1:

$$\text{MAX } 2x_1 + 5x_2$$

ST

$$x_1 \geq 4$$

$$x_1 + 4x_2 \leq 32$$

$$3x_1 + 2x_2 = 24$$

Solving For the optimal solution of [Maximization] when there are artificial variables (cont.)

The Solution

- By adding the appropriate slack, surplus, and artificial variables, we obtain the following:

$$\text{MAX } 2x_1 + 5x_2 - MR_1 - MR_3$$

ST

$$X_1 - s_1 + R_1 = 4$$

$$X_1 + 4x_2 + s_2 = 32$$

$$3x_1 + 2x_2 + R_3 = 24$$

$$X_1, x_2, s_1, s_2, R_1, R_3 \geq 0$$

Solving For the optimal solution of [Maximization] when there are artificial variables (cont.)

The initial table :

Basis	X1	X2	S1	S2	R1	R3	RHS
R1	1	0	-1	0	1	0	4
S2	1	4	0	1	0	0	32
R3	3	2	0	0	0	1	24
Z	-2	-5	0	0	+ M	+ M	0



R1, S2, R3 are basic variables.

- Make z consistent; (R1, R3) in z-row coefficient (+M,+M) it must be zero; By apply:

New z-row = old z-row + (-M * R1 row – M * R3 row)

} MAX objective function

New z-row = old z-row + (M * R1 row +M * R3 row)

} MIN objective function

Solving For the optimal solution of [Maximization] when there are artificial variables (cont.)

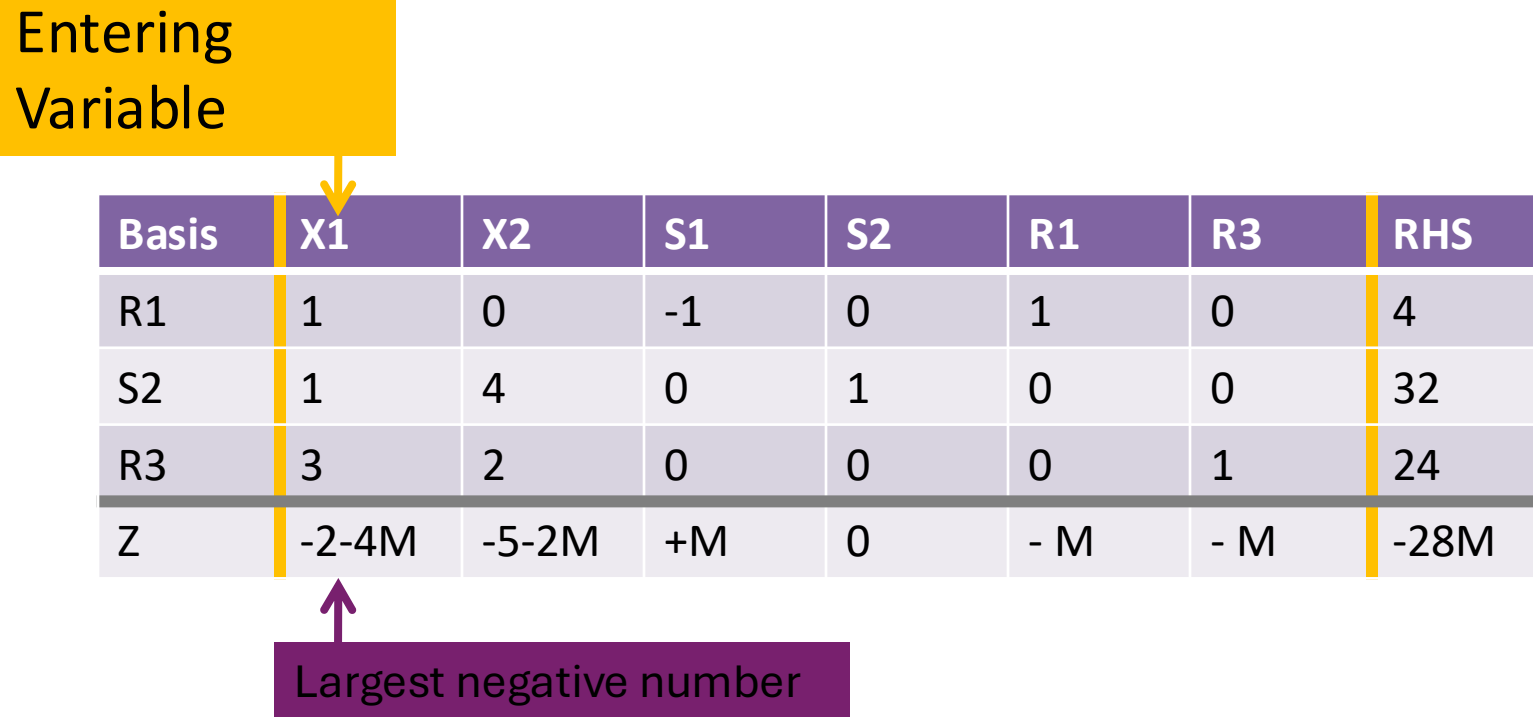
- Starting table:

Basis	X1	X2	S1	S2	R1	R3	RHS
R1	1	0	-1	0	1	0	4
S2	1	4	0	1	0	0	32
R3	3	2	0	0	0	1	24
Z	-2-4M	-5-2M	+M	0	- M	- M	-28M

Solving For the optimal solution of [Maximization] when there are artificial variables (cont.)

- To determine **Entering Variable**; We should look to the largest negative number in z-row.

Entering
Variable



Basis	X1	X2	S1	S2	R1	R3	RHS
R1	1	0	-1	0	1	0	4
S2	1	4	0	1	0	0	32
R3	3	2	0	0	0	1	24
Z	-2-4M	-5-2M	+M	0	- M	- M	-28M

Largest negative number

Solving For the optimal solution of [Maximization] when there are artificial variables (cont.)

- Calculate the ratio; then, determine the smallest positive number as **Leaving Variable**

Basis	X1	X2	S1	S2	R1	R3	RHS	Leaving Variable
R1	1	0	-1	0	1	0	4	4
S2	1	4	0	1	0	0	32	32
R3	3	2	0	0	0	1	24	8
Z	-2-4M	-5-2M	+M	0	-M	-M	-28M	

- Pivot element = $(1, 0, -1, 0, 1, 0, 4) / (1)$
 $(1, 0, -1, 0, 1, 0, 4)$

Solving For the optimal solution of [Maximization] when there are artificial variables (cont.)

- First iteration

Entering Variable								
Basis	X1	X2	S1	S2	R1	R3	RHS	Ratio
X1	1	0	-1	0	1	0	4	
S2	0	4	1	1	-1	0	28	28
R3	0	2	3	0	-3	1	12	4
Z	0	-5-2M	-2-3M	0	2+3M	-M	8-12M	

Leaving Variable

Solving For the optimal solution of [Maximization] when there are artificial variables (cont.)

- Second iteration

Entering Variable			S1	S2	R1	R3	Leaving Variable	
B							RHS	Ratio
X1	1	2/3	0	0	0	1/3	8	12
S2	0	10/3	0	1	0	-1/3	24	7.2
S1	0	2/3	1	0	-1	1/3	4	6
Z	0	-11/3	0	0	0	2/3	+16	

Solving For the optimal solution of [Maximization] when there are artificial variables (cont.)

- Third iteration

Basis	X1	X2	S1	S2	R1	R3	RHS	Ratio
X1	1	0	-1	0	1	0	4	
S2	0	0	-5	1	5	-2	4	
X2	0	1	3/2	0	-3/2	1/2	6	
Z	0	0	11/3	0	-11/2	5/2	38	

Solving For the optimal solution of **[Maximization]** when there are artificial variables (cont.)

points	Classification	Reason
X1=0, X2=0	Not Feasible	R1, R3 both Positive (4, 24)
X1=4, X2=0	Not Feasible	R3 positive= 12
X1=8, X2=0	Feasible but not optimal	X2 is negative
X1=4, X2=6	Feasible and optimal	All $x_1, x_2 \geq 0$

Solving For the optimal solution of **[Minimization]** when there are artificial variables

Example # 2:

$$\text{Min } 4x_1 + x_2$$

ST

$$3x_1 + x_2 = 3$$

$$4x_1 + 3x_2 \geq 6$$

$$x_1 + 2x_2 \leq 4$$

$$x_1, x_2 \geq 0$$

Solving For the optimal solution of [**Minimization**] when there are artificial variables (cont.)

The Solution

- By adding the appropriate slack, surplus, and artificial variables, we obtain the following:

$$\text{Min } 4x_1 + x_2 + MR_1 + MR_2$$

ST

$$3x_1 + x_2 + R_1 = 3$$

$$4x_1 + 3x_2 - s_1 + R_2 = 6$$

$$x_1 + 2x_2 + s_2 = 4$$

$$x_1, x_2, s_1, s_2, R_1, R_2 \geq 0$$

Solving For the optimal solution of [Minimization] when there are artificial variables (cont.)

- The initial table:

Basis	X1	X2	S1	R1	R2	S2	RHS
R1	3	1	0	1	0	0	3
R2	4	3	-1	0	1	0	6
S2	1	2	0	0	0	1	4
Z	-4	-1	0	-M	-M	0	0

- New z-row = old z-row + (M * R1 row + M * R3 row)

Solving For the optimal solution of [Minimization] when there are artificial variables (cont.)

- Starting table:

Entering
Variable

Basis	X1	X2	S1	R1	R2	S2	RHS
R1	3	1	0	1	0	0	3
R2	4	3	-1	0	1	0	6
S2	1	2	0	0	0	1	4
Z	-4+7M	-1+4M	-M	0	0	0	9M

Leaving
Variable



Solving For the optimal solution of [Minimization] when there are artificial variables (cont.)

- First iteration

Entering Variable							Leaving Variable	
Basis	X1	X2	S1	R1	R2	S2	RHS	
X1	1	1/3	0	1/3	0	0	1	
R2	0	5/3	-1	-4/3	1	0	2	←
S2	0	5/3	0	-1/3	0	1	3	
Z	0	$(1+5M)/3$	-M	$(4-7M)/3$	0	0	$4+2M$	

Solving For the optimal solution of [Minimization] when there are artificial variables (cont.)

- Second iteration

Basis	X1	X2	S1	R1	R2	S2	
X1	1	0	1/5	3/5	-1/5	0	3/5
X2	0	1	-3/5	-4/5	3/5	0	6/5
S2	0	0	1	1	-1	1	1
Z	0	0	1/5	8/5 - M	-1/5 - M	0	18/5

Entering Variable

Leaving Variable



Solving For the optimal solution of [Minimization] when there are artificial variables (cont.)

- Third iteration

Basis	X1	X2	S1	R1	R2	S2	RHS
X1	1	0	0	2/5	0	-1/5	2/5
X2	0	1	0	-1/5	0	3/5	9/5
s1	0	0	1	1	-1	1	1
Z	0	0	0	7/5 - M	-M	-1/5	17/5

- Optimal solution : $x_1 = 2/5$, $x_2 = 9/5$, $z = 17/5$

Simplex Algorithm – Special cases

- There are four special cases arise in the use of the simplex method.
 1. Degeneracy
 2. Alternative optima
 3. Unbounded solution
 4. Nonexisting (infeasible) solution

Simplex Algorithm – Special cases (cont.)

1. Degeneracy (no improve in objective)

- It typically occurs in a simplex iteration when in the minimum ratio test more than one basic variable determine 0, *hence two or more variables go to 0*, whereas only one of them will be leaving the basis.
- This is in itself not a problem, but making simplex iterations from a degenerate solution may give rise to cycling, meaning that after a certain number of iterations without improvement in objective value the method may turn back to the point where it started.

Simplex Algorithm – Special cases (cont.)

Example:

$$\text{Max } 3x_1 + 9x_2$$

ST

$$x_1 + 4x_2 \leq 8$$

$$x_1 + 2x_2 \leq 4$$

$$x_1, x_2 \geq 0$$

Simplex Algorithm – Special cases (cont.)

The solution:

- The constraints:

$$X_1 + 4x_2 + s_1 = 8$$

$$X_1 + 2x_2 + s_2 = 4$$

$$X_1, x_2, s_1, s_2 \geq 0$$

Simplex Algorithm – Special cases (cont.)

Entering Variable						Leaving Variable	
Basis	X1	X2	S1	S2	RHS		
s1	1	4	1	0	8	←	
s2	1	2	0	1	4		
Z	-3	-9	0	0	0		

Simplex Algorithm – Special cases (cont.)

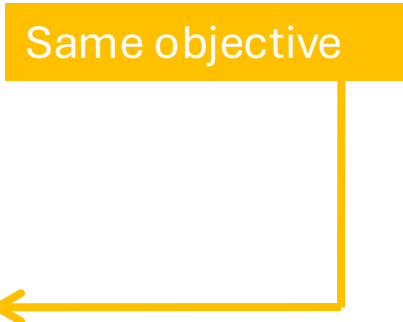
Entering
Variable

Basis	X1	X2	S1	S2	
X2	1/4	1	1/4	0	2
s2	1/2	0	-1/2	1	0
Z	-3/4	0	2/4	0	18

Leaving
Variable



Simplex Algorithm – Special cases (cont.)

Basis	X1	X2	S1	S2	RHS	Same objective 
X2	0	1	$\frac{1}{2}$	$-\frac{1}{2}$	2	
X1	1	0	-1	2	0	
Z	0	0	$\frac{3}{2}$	$\frac{3}{2}$	18	

- Same objective no change and improve (cycle)
- It is possible to have no improve and no termination for computation.

Simplex Algorithm – Special cases (cont.)

2. Alternative optima

- If the z-row value for one or more nonbasic variables is 0 in the optimal tableau, alternate optimal solution exists.

Simplex Algorithm – Special cases (cont.)

Example:

$$\text{Max } 2x_1 + 4x_2$$

ST

$$x_1 + 2x_2 \leq 5$$

$$x_1 + x_2 \leq 4$$

$$x_1, x_2 \geq 0$$

Simplex Algorithm – Special cases (cont.)

The solution

$$\text{Max } 2x_1 + 4x_2$$

ST

$$x_1 + 2x_2 + s_1 = 5$$

$$x_1 + x_2 + s_2 = 4$$

$$x_1, x_2, s_1, s_2 \geq 0$$

Simplex Algorithm – Special cases (cont.)

Entering Variable				Leaving Variable	
Basis	X1	X2	S1	S2	
s1	1	2	1	0	4
s2	1	1	0	1	5
Z	-2	-4	0	0	0



Simplex Algorithm – Special cases (cont.)

- Optimal solution is 10 when $x_2=5/2$, $x_1=0$.

Basis	X1	X2	S1	S2	RHS
x2	1/2	1	1/2	0	5/2
s2	1/2	0	-1/2	1	3/2
Z	0	0	2	0	10

- How do we know from this tableau that alternative optima exist ?

Simplex Algorithm – Special cases (cont.)

- By looking at z-row coefficient of the nonbasic variable.

Entering Variable		X1	X2	S1	S2	Leaving Variable	
Basis							
x2		1/2	1	1/2	0	5/2	
s2		1/2	0	-1/2	1	3/2	
Z		0	0	2	0	10	



- The coefficient for x1 is 0, which indicates that x1 can enter the basic solution without changing the value of z.

Simplex Algorithm – Special cases (cont.)

- The second alternative optima is:

Basis	X1	X2	S1	S2	RHS
x2	0	1	1	-1	1
x1	1	0	-1	2	3
Z	0	0	2	0	10

- The new optimal solution is 10 when $x_1=3$, $x_2=1$

Simplex Algorithm – Special cases (cont.)

3. Unbounded solution

- It occurs when nonbasic variables are zero or negative in all constraints coefficient (max) and variable coefficient in objective is negative

Simplex Algorithm – Special cases (cont.)

Example

$$\text{Max } 2x_1 + x_2$$

ST

$$x_1 - x_2 \leq 10$$

$$2x_1 \leq 40$$

$$x_1, x_2 \geq 0$$

Simplex Algorithm – Special cases (cont.)

The solution

Max $2x_1 + x_2$

ST

$$x_1 - x_2 + s_1 = 10$$

$$2x_1 + s_2 = 40$$

$$x_1, x_2, s_1, s_2 \geq 0$$

Simplex Algorithm – Special cases (cont.)

Basis	X1	X2	S1	S2	RHS
x2	1	-1	1	0	10
x1	2	0	0	1	40
Z	-2	-1	0	0	0

- All value if x2(nonbasic variable) either zero or negative.
- So, solution space is unbounded

Simplex Algorithm – Special cases (cont.)

4. Infeasible solution

- R coefficient at end $\neq 0$
- This situation can never occur if all the constraints are of the type “ $\leq$ ” with nonnegative RHS

MME8215: Operations Research

Continuous Linear Programming

Continuous Linear Programming (CLP) spans a wide set of OR problems that can be modeled based on the following assumptions:

- Unknown variables are continuous: The result variable and the decision variables can take any real value. Also, all the different coefficients can take any real value.
- Objectives and constraints are linear expressions: The relationships between the variables are expressed as linear expressions.

CLP establishes a rather simplified model of real world problems. Many assumptions are needed to represent real problems as a set of linear expressions. Yet, CLP is a very powerful tool in *Operations Research* (OR) and it can provide valuable insights to decision-making in different fields. The main reason is that, once these assumptions are taken, linear algebra provides valuable tools to find an optimal solution and analyse it.

The main types of problems that will be covered in the CLP exercises belong to two problem types:

- **Production mix:** Production mix problems help us determine the optimal allocation of resources in the production of good or services to either minimise costs or maximise profits.
- **Blending problems:** In blending problems, the objective is to find the optimal combination of components in a set of products that either minimise costs or maximise profits.

1 Setup

In CLP, the objective variable, noted as z , can take any real value:

$$z \in \mathbb{R}.$$

The value of this unknown variable z is a function of another set of unknowns, which are called decision variables and that represent our decisions. These decision variables are noted as x_j , where j is an integer (sub)index that goes from 1 to n :

$$z = f(x_1, x_2, \dots, x_n)$$

and $x_1, x_2, \dots, x_n \in \mathbb{R}$.

Therefore, we will have n different decision variables, that are going to be continuous. And, in extension, z is also going to be continuous. So now we know from where does the C in *CLP* comes from.

Next, in *CLP*, the function f is a linear function. What we want is to find the **optimal value** of z , which, depending on the type of problem will be either the maximum value (e.g. maximise profits) or the minimum value (e.g. minimise costs).

Hence, we want to maximise or minimise z , which as we said is a linear function and therefore can be expressed as the sum of the product of the decision variables multiplied by a set of coefficients, which are noted as c_1 to c_n .

$$\max \text{ or } \min z = f(x_1, x_2, \dots, x_n) = c_1 \cdot x_1 + c_2 \cdot x_2 + \dots + c_n \cdot x_n.$$

Now, we know that linear functions do not have any absolute minimum or maximum, but the objective function is subject to a set of constraints, which are also linear functions of the decision variables, that is, the sum product of the decision variables times a set of coefficients noted as a_{ij} must be less than, greater than, or equal to another coefficient noted as b_j .

s.t.

$$\begin{aligned} a_{11} \cdot x_1 + a_{12} \cdot x_2 \dots + a_{1n} \cdot x_n &\leq b_1 \\ a_{21} \cdot x_1 + a_{22} \cdot x_2 \dots + a_{2n} \cdot x_n &\leq b_2 \\ &\dots \\ a_{m1} \cdot x_1 + a_{m2} \cdot x_2 \dots + a_{mn} \cdot x_n &\leq b_m \end{aligned}$$

Note that coefficients have two sub-indexes, the second is equal to the index of the corresponding decision variable, and the first is equal to the sub-index of b . These new index i ranges from 1 to m and therefore, our optimisation function is subject to a set of m constraints. We will refer to the expressions to the left of the inequality as the Left Hand Side (LHS), and to the other side of the relationship as the Right Hand Side (RHS).

Now, note that we can use the sum operator to represent the objective function in a more compressed form, that is, the LHS is equal to the sum of the product of c_j times x_j for j equal to 1 up to n . Note that this can also be represented as the dot product of two vectors, the vector x with the decision variables and the vector c of the coefficients.

$$\max \text{ or } \min z = c_1 \cdot x_1 + c_2 \cdot x_2 + \dots + c_n \cdot x_n = \sum_{j=1}^n c_j \cdot x_j = c \cdot x$$

where $x = [x_1, x_2, \dots, x_n]^T$ and $c = [c_1, c_2, \dots, c_n]^T$.

We can do the same transformations to the left hand sides of the constraints, expressing them as the sum product of the decision variables times the left hand side coefficients. And, we may as well express all the left hand sides of the constraints as the product of a matrix A that contains all the different coefficients times the decision variable vector.

$$\begin{aligned}
& \max \text{ or } \min \quad z = \sum_{j=1}^n c_j \cdot x_j \\
& \text{subject to} \\
& \quad \sum_{j=1}^n a_{1j} \cdot x_j \leq b_1, \\
& \quad \sum_{j=1}^n a_{2j} \cdot x_j \leq b_2, \\
& \quad \dots \\
& \quad \sum_{j=1}^n a_{mj} \cdot x_j \leq b_m.
\end{aligned}$$

Or, we can write

$$\begin{aligned}
& \max \text{ or } \min \quad z = c \cdot x \\
& \text{subject to} \\
& \quad A \cdot x \leq b.
\end{aligned}$$

where $A = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \dots & & & \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}$ and $b = [b_1, b_2, \dots, b_m]$.

These alternative compact forms allow us to deal with problems with an arbitrary number of decision variables and an arbitrary number of constraints.

2 Types of problems that can be solved with CLP

2.1 Production Mix Problems

In production mix problems, the decision maker needs to create products to satisfy demand using limited resources. The objective is to maximise the profits, taking into account the demand and the limited availability of resources. Production mix problems represent the simplest form of optimization problem in operations research, where the objective is to maximise a utility function (in this case profits) bounded by a set of constraints. Normally, production mix problems are not found in real world applications, but rather, they are part of larger problems.

Objective function Let us note the objective variable profits as z , expressed in a monetary unit (e.g. €).

- z : profits in €.

The decision maker needs to decide how many units of each product type to manufacture. Let us assume there are n different products. Now, let us note the decision variables $x_j \in \mathbb{R}$ as the amount of product $j \in [1, n]$ to be produced, in a production unit:

- x_j : Quantity of product j to be produced in product units.

The objective is to maximize profits. Let us note the profit per unit of product j as p_j :

- p_j : Profit per unit of product j to be produced in €/unit.

Now, the objective function is to maximise the sum of the product of the number of units produced times the profit per unit, for each product type:

$$\max z = x_1 \cdot p_1 + x_2 \cdot p_2 + \dots + x_n \cdot p_n = \sum_{j=1}^n x_j \cdot p_j.$$

Demand constraints There might be requirements on the demand. Let us note the demand of product j as d_j . The demand constraints will be of the type:

$$\begin{aligned} x_1 &\geq d_1, \\ x_n &\geq d_2, \\ &\dots \\ x_n &\geq d_n. \end{aligned}$$

Or $x_j \geq d_j \forall j \in [1, n]$.

Availability constraints Different limited resources are used to produce each product. Let us assume that we have m different resources. Let us note as b_i as the quantity available of resource i , $i \in [1, m]$, expressed in a given unit:

- b_i : Availability of resource i in resource unit.

Further, to produce a unit of a product j we will require a given amount of resource i . Let us note as a_{ij} the requirement of resource i to produce a unit of product j :

- a_{ij} : Requirement of resource i to produce a unit of product j in resource unit / product unit.

Then, for each resource i , we need to ensure that the total amount used is less or equal the total amount available. The amount used to produce a specific number of products will be the number of units products (e.g. x_j) multiplied by the amount required to produce one unit x_{ij} . If we sum across all products, we obtain the total amount used, which needs to be lower than the availability. Let's take the resource indexed 1 as an example:

$$x_1 \cdot a_{11} + x_2 \cdot a_{12} + \dots + x_n \cdot a_{1n} \leq b_1.$$

Note that we can write this down this constraint in a compact form like:

$$\sum_{j=1}^n x_j \cdot a_{ij} \leq b_1.$$

Now, we need to take into account the other resources (we can write all this m equations in a compact form):

$$\sum_{j=1}^n x_j \cdot a_{ij} \leq b_m.$$

That is, for every resource, we need to ensure that the sum of the quantities used to produce each type of product are less or equal than the total amount available.

Summary Putting it all together, the generic formulation of the production mix problem as a CLP is:

$$\begin{aligned} \max \quad & z = \sum_{j=1}^n x_j \cdot p_j \\ \text{subject to} \quad & \sum_{j=1}^n x_j \cdot a_{ij} \leq b_i, \forall i \in [1, m] \\ & x_j \geq d_j, \forall j \in [1, n] \end{aligned}$$

Example 1. *Production Mix:* A company produces two kinds of products, $P1$ and $P2$. The selling price of $P1$ is 300€. The selling price of $P2$ is 250€.

- $P1$ requires 2 operator man-hours and 1 hour of machine operating time.
- $P2$ requires 1 operator man-hours and 3 hours of machine time.
- Operator man-hours cannot be higher than 40 hours per day.
- Machine operating time cannot be higher than 45 hours per day.
- Marketing states that $P1$ units per day cannot be higher than 12.

How many units of $P1$ and $P2$ should the company produce every day to **maximise income**?

Model We want to find the maximum solution to:

$$\max x = 300 \cdot x_1 + 250 \cdot x_2.$$

This is our objective function. The decision variables are:

- x_1 : units of Product 1,
- x_2 : units of Product 2.

The objective function is subject to the following constraints:

$$\begin{aligned} 2 \cdot x_1 + x_2 &\leq 40, \\ x_1 + 3 \cdot x_2 &\leq 45, \\ x_1 &\leq 12. \end{aligned}$$

We define the problem in its standard form as:

$$\max z = 300 \cdot x_1 + 250 \cdot x_2 + 0 \cdot s_1 + 0 \cdot s_2 + 0 \cdot s_3.$$

And the constraints as:

$$\begin{aligned} 2 \cdot x_1 + x_2 + s_1 &= 40, \\ x_1 + 3 \cdot x_2 + s_2 &= 45, \\ x_1 &= 12. \end{aligned}$$

2.2 Blending Problems

In blending problems, the decision maker needs to find the optimal blending (mixing or combination) of materials or resources to create **products** that satisfy a given demand. Each resource or material will have a different costs and therefore, in its primary form, the objective is to minimise the overall costs of the materials needed to satisfy the demand. Additionally, the required materials or resources needed may be subject to availability constraints (i.e. we cannot consider that the amount of materials or resources available is unlimited).

Objective function Let us assume that we have m different materials or resources to be considered in the problem and n different products.

Let us note the cost of material i as $c_i, i \in [1, m]$. Let us also note x_{ij} the amount of resource or material i used to produce products $j, j \in [1, n]$.

Note that these variables x_{ij} represent our decision variables: we are looking for the optimal combination of resources or materials to manufacture our products. Note also that we have $m \cdot n$ different decision variables (the combination of products and materials).

With these definitions, our objective function can be defined as:

$$\min z = \sum_{i=1}^m \sum_{j=1}^n c_i \cdot x_{ij}.$$

That is, the objective is to minimize the overall costs, which is the sum of the cost of the different materials needed to manufacture all our products.

Availability constraints Since the amount of resources or materials available is limited, let us plug in the availability constraints, let us note as a_m as the availability of the material or resource m . The total amount used to manufacture all products cannot exceed the availability of that specific material or resource. For instance, for material or resource 1, the sum of the amount used to manufacture the different products cannot exceed the availability a_1 , that is:

$$\sum_{j=1}^n x_{1j} \leq a_1.$$

The same applies to other material or resources:

$$\sum_{j=1}^n x_{2j} \leq a_2,$$

...

$$\sum_{j=1}^n x_{mj} \leq a_m.$$

We can write this set of constraints in a compact form as:

$$\sum_{j=1}^n x_{ij} \leq a_i, \forall i \in [1, m].$$

Demand constraint As mentioned above, we need to satisfy the demand for every product, which means that the total amount of a product that we need to manufacture needs to be greater or equal than its demand. Let us note d_j as the demand for product j .

Note that we do not have explicitly defined a decision variable for the quantity of each product to manufacture, but in terms of the blending problem, if all the materials are considered, it is equivalent to think of the total amount of a product as the total amount of the materials needed to manufacture a product. Hence, the demand constraints can be expressed as:

$$\begin{aligned} \sum_{j=1}^m x_{i1} &\geq d_1, \\ \sum_{j=1}^m x_{i2} &\geq d_2, \\ &\dots \\ \sum_{j=1}^m x_{im} &\geq d_n, \\ &\dots \end{aligned}$$

Note that we can express this set of constraints in a compact form as:

$$\sum_{i=1}^m x_{ij} \geq d_j, \forall j \in [1, n].$$

Additional requirements: Minimum amount We may find additional requirements in the problem definition, for instance, we may find a requirement for a minimum amount of an ingredient. Sometimes the requirement is expressed as a percentage or ratio of the total composition of the product.

As an example, let us define that the minimum amount of material or resource 2 for product 3 needs to be 20%, or 0.2. If we are dealing with all the materials and resources used, we can again assume that the amount of material or resource x_{23} needs to be at least 20% of the sum of all ingredients used to manufacture product 3, that is:

$$x_{23} \leq 0.2 \sum_{i=1}^m x_{i3}.$$

We could bring all the dependant coefficients to the left-hand side and express this constraint as:

$$\begin{aligned} 0 &\leq -1 \cdot x_{23} + 0.2 \sum_{i=1}^m x_{i3} \\ 0 &\leq -0.8 \cdot x_{23} + 0.2 \sum_{i=1, i \neq 2}^m x_{i3} \\ 0.8 \cdot x_{23} - 0.2 \sum_{i=1, i \neq 2}^m x_{i3} &\geq 0 \end{aligned}$$

Combination of production mix and blending problems Instead of just minimizing the overall cost, our objective can be to maximise profit. This can be regarded as a combination of the production mix and blending problems. Let us define the unitary selling prices of our n products as $[p_1, p_2, \dots, p_n]$.

If we focus on one product, e.g. product 1, the profits can be calculated as the difference between the selling price and the cost obtained for the total amount of the products sold. Again, if we can

assume that the total amount of ingredients is equal to the total amount of products, expressed in the same unit as the unitary selling price, the overall profits for product 1 can be expressed as:

$$p_1 \cdot \sum_{i=1}^n x_{i1} - \sum_{i=1}^n c_i \cdot x_{i1}.$$

Our objective is to maximize the overall profits, so we need to factor in the profits of the rest of the products with:

$$\max z = \sum_{j=1}^n p_j \cdot \sum_{i=1}^n x_{ij} - \sum_{i=1}^n c_i \cdot x_{ij}.$$

Now, we can re-arrange the terms of the summations as:

$$\max z = \sum_{j=1}^n \sum_{i=1}^n (p_j - c_i) \cdot x_{ij}.$$

Example 2. *Blending Problem:* The Kahuna company manufactures sausages using three kinds of meat. The relevant information about the ingredients is provided in the table below:

Ingredient	Cost (€/kg)	Availability (kg)
Pork	4.31	30
Wheat	2.46	20
Starch	1.86	17

The company makes two types of sausages:

- Economy ($\geq 40\%$ Pork)
- Premium ($\geq 60\%$ Pork)

One sausage has 50 grams (0.05 kg). According to government regulations, the most starch we can use in our sausages is 25%. The company has a contract with a butcher, and has already purchased 23 kg pork, that will go bad if it's not used. The company has a demand for 350 economy sausages and 500 premium sausages.

Write a linear program to figure out the most cost-effective recipe to manufacture our sausages.

Example 3. *Petroleum Blending:* Harry Stamper is an engineer working on a petroleum company. The company produces three grades of motor oil – *super*, *premium*, and *extra*. The three grades are made from the same 3 ingredients, named component 1, component 2 and component 3. The company wants to determine the optimal mix of the 3 components in each grade of motor oil that will maximize profit.

The maximum availability of each component and their cost per barrel are as follows: To ensure

Component	Max barrels available / day	Cost / barrel
1	4500	12€
2	2700	10€
3	3500	14€

the appropriate blend, each grade must have a minimum amount of component 1 plus a combination of other components as follows:

Grade	Component 1	Component 2	Component 3	Selling price/barrel
Super	$\geq 50\%$	$\leq 30\%$	-	23€
Premium	$\geq 40\%$	-	$\leq 25\%$	20€
Extra	$\geq 60\%$	$\geq 10\%$	-	18€

The company wants to make at least 3000 barrels of each blend. Formulate an LP to help Harry Stamper find the optimal blend that maximises profit.

Analysis questions (Do this part once you have reviewed CLP analysis including sensitivity analysis questions)

- What is the total profit obtained by the company with the Extra grade with the solution provided by the solver?
- What is the maximum value that the profit per barrel of component 2, in the Extra grade, can take without changing the base of the optimal solution?
- If the cost of the barrel of component 3 is increased by 1€, will the optimal composition change? Motivate your response
- Do we use every barrel available of every component in the optimal solution? Or do we have any surplus availability for any component? Motivate your response
- The provider of Component 2 may have a problem with the delivery and the availability may be reduced to 400 barrels. Will this shortage affect the optimal composition? Motivate your response
- Grace wants to use the results to renegotiate the terms and conditions with the suppliers of the different components. Which component is a better candidate to increase the availability? How would you use the model to negotiate new terms with the provider, assuming you do not want any change in the base of the optimal solution? Motivate your response

Example 4. *Blending craft beer:* Duff Beer is an american brewery that sells alcoholic beverages worldwide. Duff beer has a very large market share in the US. One of the new business lines of Duff Beer is in the craft beer sector. Duff beer buys 3 types of wort from local suppliers. The availability of wort (in liters) and the price for each of them is specified in the table below. The

Wort	Availability (liters)	Cost (€cents/liter)
Type 1	2500	120
Type 2	1200	95
Type 3	2000	150

blend of the three types of wort must comply with the following quality requirements:

- For Beer Angels, no less than 60% of Type 3 and no more of 20% of Type 2. The price of one bottle of Beer Angel is 4.10€/liter.
- For Beer Beast, no less than 15% of Type 3 and no more than 60% of Type 2. The price of one bottle of Beer Beast is 2.80€/liter.
- For Beer Cactus, no more than 50% of type 2. The selling price is 2.45€/liter.

Write a linear program to find the most profitable blend of wort to elaborate the three kinds of beers.

3 Standard Form

Some algorithms like the Simplex need to operate with problems formulated in a standard form. Every CLP problem described in the setup section can be converted to a standard form where:

- The objective function is of type **maximise**.
- All the constraints are of type **equal**.

That is, any CLP problem can be expressed in the standard form as:

$$\begin{aligned} \max \quad & z = \sum_{i=1}^n c_i \cdot x_i = c^T \\ \text{subject to} \quad & a_{11} \cdot x_1 + a_{12} \cdot x_2 + \dots + a_{1n} \cdot x_n + s_1 = b_1 \\ & a_{21} \cdot x_1 + a_{22} \cdot x_2 + \dots + a_{2n} \cdot x_n + s_2 = b_2 \\ & \dots \\ & a_{m1} \cdot x_1 + a_{m2} \cdot x_2 + \dots + a_{mn} \cdot x_n + s_m = b_m \end{aligned}$$

which can be expressed using a matrix product as:

$$A \cdot x = b$$

and $x_i \geq 0, \forall i$.

3.1 Conversion of minimization problems

If we need to convert a problem where the objective function is of type minimize to the standard form, we need just need to make a change of variable:

$$z = -z.$$

With this change of variable the objective function:

$$\min z = c \cdot x$$

is equivalent to:

$$\max z = -c \cdot x.$$

And now the objective function is of type maximise.

3.2 Conversion of constraints and slack variables

Note that we have introduced a new set of decision variables $s = [s_1, s_2, \dots, s_n]$ used to make all the constraints of type equal. These are new variables that are introduced to change the type of constraint to a constraint of type equal. Imagine for instance that our problem has a constraint of type less or equal:

$$a_{11} \cdot x_1 + a_{12} \cdot x_2 + \dots + a_{1n} \cdot x_n \leq b_1.$$

Since the left hand side is less or equal than the right hand side, we can make the left hand side equal to the right hand side by adding a non-negative variable s_1 to the left hand side:

$$a_{11} \cdot x_1 + a_{12} \cdot x_2 + \dots + a_{1n} \cdot x_n + s_1 = b_1.$$

Obviously, we need to add a non-negativity constraint to the new variable to make sure that both constraints are equivalent: $s_1 \geq 0$.

Equivalently, if we have a constraint of type greater or equal, we can make the same modification, but now, the new variable needs to be lower or equal than zero to make the problem equivalent. For instance, let us consider without loss of generality that we have the following constraint in our problem:

$$a_{21} \cdot x_1 + a_{22} \cdot x_2 + \dots + a_{2n} \cdot x_n \geq b_2.$$

This is equivalent to:

$$a_{21} \cdot x_1 + a_{22} \cdot x_2 + \dots + a_{2n} \cdot x_n + s_2 = b_2.$$

where $s_2 \leq 0$.

Also, if our constraint is of type equal, we do not need to introduce any change, but just to make sure we treat all constraints in the same way, let us introduce again a new variable:

$$a_{31} \cdot x_1 + a_{32} \cdot x_2 + \dots + a_{3n} \cdot x_n + s_3 = b_3.$$

This is equivalent to:

$$a_{31} \cdot x_1 + a_{32} \cdot x_2 + \dots + a_{3n} \cdot x_n + s_3 + s_3 = b_3.$$

where $s_3 = 0$.

3.3 Slack variables

These new variables are called **slack variables**, and they represent the difference between the left hand side and the right hand side. When slack variables are non-negative (meaning that the constraint is of type less or equal), they are called **surplus** variables. Note that normally these constraints represent limited resources, so the slack value will represent the surplus of resource that we have available in the optimal solution. When slack variables are non-positive (meaning that the constraint is of type greater or equal), they are called **excess** variables. Note that these types of constraints will normally represent additional requirements, so the value of the slack variable will represent the excess of the solution with respect to the requirement. When slack variables are zero, they are referred to **artificial variables**. Since they must be zero, they need to be treated differently by the solvers.

3.4 Conversion of negative decision variables

Finally, note that in the standard form, all the decision variables need to be non-negative, we can achieve this with a change of variable:

$$\begin{aligned} x &= x^i - x^{ii} \\ x^i, x^{ii} &\in [0, \infty] \end{aligned}$$

That is, we replace the decision variable x by two non-negative decision variables, when x^{ii} is greater than x^i , x takes negative values.

3.5 Summary

In order for the standard form to hold, we need to plug in the new variables into the model. We first introduce the slack variables into our decision variable vector, so that it becomes:

$$x = [x_1, x_2, \dots, x_n, s_1, s_2, \dots, s_p].$$

To make sure that there is no impact in the objective function, we multiply the non-zero slack variables by a factor of 0. Since artificial variables are equal to zero, we multiply them by an arbitrarily large negative number $-M$. This number needs to be introduced by algorithms such as the Simplex. Let us assume without loss of generality that we have p non-zero slack variables and k artificial variables. Let us note artificial variables as $a_1, a_2, \dots, a_k$. Our objective function becomes:

$$\max z = \sum_{i=1}^n c_i \cdot x_i + 0 \cdot s_1 + 0 \cdot s_2 + \dots + 0 \cdot s_p - M \cdot a_1 - M \cdot a_2 - \dots - M \cdot a_k.$$

And the constraints:

$$\begin{aligned} a_{11} \cdot x_1 + a_{12} \cdot x_2 \dots + a_{1n} \cdot x_n + s_1 + 0 \cdot s_2 + \dots + 0 \cdot s_p + 0 \cdot a_1 + 0 \cdot a_2 + \dots + 0 \cdot a_k &\leq b_1 \\ a_{21} \cdot x_1 + a_{22} \cdot x_2 \dots + a_{2n} \cdot x_n + 0 \cdot s_1 + s_2 + \dots + 0 \cdot s_p + 0 \cdot a_1 + 0 \cdot a_2 + \dots + 0 \cdot a_k &\leq b_2 \\ &\dots \\ a_{p1} \cdot x_1 + a_{p2} \cdot x_2 \dots + a_{pn} \cdot x_n + 0 \cdot s_1 + 0 \cdot s_2 + \dots + s_p + 0 \cdot a_1 + 0 \cdot a_2 + \dots + 0 \cdot a_k &\leq b_p \\ a_{(p+1)1} \cdot x_1 + a_{(p+1)2} \cdot x_2 \dots + a_{(p+1)n} \cdot x_n + 0 \cdot s_1 + 0 \cdot s_2 + \dots + 0 \cdot s_p + a_1 + 0 \cdot a_2 + \dots + 0 \cdot a_k &\leq b_{(p+1)} \\ a_{(p+2)1} \cdot x_1 + a_{(p+2)2} \cdot x_2 \dots + a_{(p+2)n} \cdot x_n + 0 \cdot s_1 + 0 \cdot s_2 + \dots + 0 \cdot s_p + 0 \cdot a_1 + a_2 + \dots + 0 \cdot a_k &\leq b_{(p+2)} \\ &\dots \\ a_{m1} \cdot x_1 + a_{m2} \cdot x_2 \dots + a_{mn} \cdot x_n + 0 \cdot s_1 + 0 \cdot s_2 + \dots + 0 \cdot s_p + 0 \cdot a_1 + 0 \cdot a_2 + \dots + a_k &\leq b_m \end{aligned}$$

3.6 Standard tabular representation

We can express the standard form as stated at the beginning of this section as:

$$\begin{aligned} \max \quad & z = c \cdot x \\ \text{subject to} \quad & A \cdot x = b \end{aligned}$$

where

$$\begin{aligned} z &= [x_1, x_2, \dots, x_n, s_1, s_2, \dots, s_p, a_1, a_2, \dots, a_k]^T \\ c &= [c_1, c_2, \dots, c_n, 0, \dots, 0, -M, \dots, -M]^T \\ A &= \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} & 1 & 0 & \dots & 0 \\ a_{21} & a_{22} & \dots & a_{2n} & 0 & 1 & \dots & 0 \\ \dots & & & & & & & \\ a_{m1} & a_{m2} & \dots & a_{mn} & 0 & 0 & \dots & 1 \end{bmatrix} \end{aligned}$$

x and c are two $n + m$ vectors and A is a $m \times (n + m)$ matrix where the right most n columns represent the original LHS coefficients and the right most n columns represent the $n \times n$ identity matrix.

Now, if we move all the dependent terms to the LHS, the objective function becomes:

$$z - c_1 \cdot x_1 - c_2 \cdot x_2 - \dots - c_n \cdot x_n - 0 \cdot s_1 - 0 \cdot s_2 - \dots - 0 \cdot s_p + M \cdot a_1 + M \cdot a_2 + \dots + M \cdot a_k = 0$$

z	x_1	x_2	...	x_n	s_1	s_2	...	s_p	a_1	a_2	...	a_k	RHS
1	$-c_1$	$-c_2$	...	$-c_n$	0	0	...	0	-M	-M	...	-M	0
0	a_{11}	a_{12}	...	a_{1n}	1	0	0	...	0	0	...	0	b_1
0	a_{21}	a_{22}	...	a_{2n}	0	1	0	...	0	0	...	0	b_2
...	...	...	...	...	...	...	...	...	...	...	...	...	...
0	a_{m1}	a_{m2}	...	a_{mn}	0	0	0	...	0	0	...	1	b_m

We can write all the coefficients in a data frame where each column represents the coefficients of a variable and each row represents an equation:

This representation of the problem is called the standard tabular form.

4 Simplex example

We will use a problem as an example to show the simplex method.

Problem Definition Production Mix A company produces two kinds of products, P1 and P2. The selling price of P1 is 300€ The selling price of P2 is 250€

- P1 requires 2 operator man hours and 1 hour of machine operating time.
- P2 requires 1 operator man hours and 3 hours of machine time.
- Operator man hours cannot be higher than 40 hours per day.
- Machine operating time cannot be higher than 45 hours per day.
- Marketing states that P1 units per day cannot be higher than 12.

How many units of P1 and P2 should the company produce every day to maximise income?

Model We want to find the maximum solution to:

$$\max z = 300 \cdot x_1 + 250 \cdot x_2.$$

This is our objective function. The decision variables are:

- x_1 : units of Product 1,
- x_2 : units of Product 2.

The objective function is subject to the following constraints:

$$\begin{aligned} 2 \cdot x_1 + x_2 &\leq 40, \\ x_1 + 3 \cdot x_2 &\leq 45, \\ x_1 &\leq 12. \end{aligned}$$

We define the problem in its standard form as:

$$\max z = z = 300 \cdot x_1 + 250 \cdot x_2 + 0 \cdot s_1 + 0 \cdot s_2 + 0 \cdot s_3.$$

Where:

- s_1 : Slack variable of the operator man hour constraint ($s_1 \geq 0$),
- s_2 : Slack variable of the machining time constraint ($s_2 \geq 0$),
- s_3 : Slack variable of the operator man hour constraint ($s_3 \geq 0$).

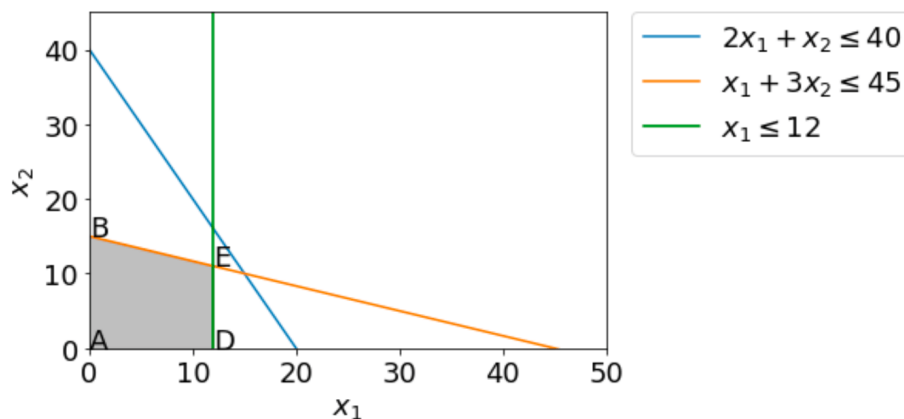
And thus, by entering the slack variables, we can now express the constraints as:

$$2 \cdot x_1 + x_2 + s_1 = 40,$$

$$x_1 + 3 \cdot x_2 + s_2 = 45,$$

$$x_1 + s_3 = 12.$$

Solution We will use the Simplex Method and iterate between the graphical and the algebraic form of the problem to describe the results. First let us represent the problem: The feasible region



(in grey) contains all possible solutions to the objective function. Let us now represent the same problem in a basic Tableau: From the Tableau, we can easily find a first Feasible Basic Solution

Z	x_1	x_2	s_1	s_2	s_3	RHS
1	-300	-250	0	0	0	0
0	2	1	1	0	0	40
0	1	3	0	1	0	45
0	1	0	0	0	1	12

(FSB), by making x_1 and x_2 non basic ($= 0$). The three equations of the constraints provide us the value of the slack variables in this solution:

$$x_1 = 0, x_2 = 0, s_1 = 40, s_2 = 45, s_3 = 12.$$

Let us note this solution as $(0, 0, 40, 45, 12)$ and $z = 0$. Note that this FSB corresponds to Vertex A in the above figure. *How can we know if it is optimal?* We have 2 non basic variables x_1 and x_2 which have negative coefficients in the objective function and positive values of these variables would increase z . If we increase x_1 in one unit, z would increase 300 units and if we increase x_2 in one unit z would increase 250. Therefore, the solution is not optimal, since we could find a higher z for a positive value of these variables in the objective function.

Hence, this is not the optimal solution and we need to look for it in another vertex. From point A , we could either move to vertex B or to vertex D . Moving towards D would mean making x_1 higher than zero, thus making it a **basic variable**. Likewise, moving towards B would mean making x_2 higher than zero and basic (enter the base). We select x_1 as the **entering basic variable**, since its coefficient (300) is higher and therefore z will increase at a higher rate when we move towards D . We have just selected the pivot variable and the pivot column in the Tableau.

Now, we are going to move towards D , which is equivalent to make $x_1 = \delta$. As this value increases, the objective value will also increase, but we have to bear in mind that all variables must be positive and therefore δ cannot take any given value: we have to make sure the solution is **feasible**. If we write down all the constraints in algebraic form, we can find the possible values of δ

$$\begin{aligned} 2 \cdot \delta + 1 \cdot 0 + s_1 &= 40, \\ 1 \cdot \delta + 3 \cdot 0 + s_2 &= 45, \\ 1 \cdot \delta + s_3 &= 12. \end{aligned}$$

To ensure that all the surplus variables are positive δ cannot exceed the maximum positive value that guarantees that all the surplus variables are non negative. The strategy to find a feasible solution is to make one slack variable **non-basic** (equal to zero), but making sure that all the constraints are met. We only have δ and a slack variable in each expression, this means that the maximum value it can take is $\delta = \min\{40/2, 45/1, 12/1\}$. Any value above this makes the solution unfeasible. This is called the **Ratio test**. If we make $x_1 = \delta = 12$ then that implies that s_3 is now a non-basic variable. With the Ratio test, we have selected s_3 as the leaving variable which will no longer be basic in the FSB. Let us add two new columns to the basic Tableau, one to the right to indicate which variable is the basic variable in the equation and another one to the left to express this Ratio test.

Basic	z	x_1	x_2	s_1	s_2	s_3	RHS	Ratio
.	1	-300	-250	0	0	0	0	.
s_1	0	2	1	1	0	0	40	40/2=20
s_2	0	1	3	0	1	0	45	45/1=45
s_3	0	1	0	0	0	1	12	12/1=12

Now, notice that from the Ratio test, we have easily determined that the variable that leaves the basic solution is s_3 and using the last equation we can easily determine that $x_1 = 12$ in the new solution. We will move to another solution $(12, 0, ?, ?, 0)$ which corresponds exactly to point D . Now notice that in the first iteration, the problem was expressed in a very convenient way since we had only a non-zero (1) value for each basic variable in every row. To accomplish this, we are going to transform the different rows. Since the equations are linearly independent, we can make linear operations in the different equations and still obtain a linear independent system of equations. We want to obtain a system like this:

Basic	z	x_1	x_2	s_1	s_2	s_3	RHS	Ratio
.	1	0	?	0	0	?	?	.
s_1	0	0	?	1	0	?	?	?
s_2	0	0	?	0	1	?	?	?
x_1	0	1	?	0	0	?	?	?

Note that when the problem is in this form, we will have each basic variable in a row with coefficient 1 and the rest with coefficient 0 and we will be able to obtain the value for each basic variable rather easily. First we transform the **leaving** equation, by dividing the entire row by the coefficient of the entering variable in that equation (in this case is 1) and the row remains unaltered:

Basic	z	x_1	x_2	s_1	s_2	s_3	RHS	Ratio
.	1	0	?	0	0	?	?	.
s_1	0	0	?	1	0	?	?	?
s_2	0	0	?	0	1	?	?	?
x_1	0	1/1=1	0/1=0	0/1=0	0/1=0	1/1=1	12/1=12	?

Now, for the rest of the row, we need to make a linear operation that gives a zero in the cell corresponding to the entering variable. To accomplish this, we subtract the **entering equation** multiplied by the value of the coefficient that multiplies the entering variable x_1 .

Basic	z	x_1	x_2	s_1	s_2	s_3	RHS	Ratio
.	1	$-300 - (-300 \cdot 1) = 0$	$-250 - (-300 \cdot ?) = 0$	$0 - (-300 \cdot 0) = 0$	$0 - (-300 \cdot 0) = 0$	$0 - (-300 \cdot 1) = 300$	$0 - (-300 \cdot 12) = 3600$	.
s_1	0	$2 - 2 \cdot 1 = 0$	$1 - 2 \cdot 0 = 1$	$1 - 2 \cdot 0 = 1$	$0 - 2 \cdot 0 = 0$	$0 - 2 \cdot 1 = -2$	$40 - 12 \cdot 2 = 16$	?
s_2	0	$1 - 1 \cdot 1 = 0$	$3 - 1 \cdot 0 = 3$	$0 - 1 \cdot 0 = 0$	$1 - 1 \cdot 0 = 1$	$0 - 1 \cdot 1 = -1$	$45 - 12 \cdot 1 = 33$	?
x_1	0	1	0	0	0	1	12	?

We end up with the following Tableau:

Basic	z	x_1	x_2	s_1	s_2	s_3	RHS	Ratio
.	1	0	-250	0	0	300	3600	.
s_1	0	0	1	1	0	-2	16	?
s_2	0	0	3	0	1	-1	33	?
x_1	0	1	0	0	0	1	12	?

And the new FSB solution is $(12, 0, 16, 32, 0)$ which as noted corresponds to point D .

At this point, z has increased from 0 to 3600, but we still have a coefficient in x_2 that is lower than zero and therefore we could find a higher value if again we move from D to E by increasing x_2 , just as we did with x_1 . We have selected the next entering variable and the pivot column and we can perform the Ratio test again to select the variable leaving the basic solution:

Basic	z	x_1	x_2	s_1	s_2	s_3	RHS	Ratio
.	1	0	-250	0	0	300	3600	.
s_1	0	0	1	1	0	-2	16	16
s_2	0	0	3	0	1	-1	33	11
x_1	0	1	0	0	0	1	12	12

The minimum value that makes all variables positive is 11. Therefore, the variable leaving the solution is s_2 . We first transform the new entering row:

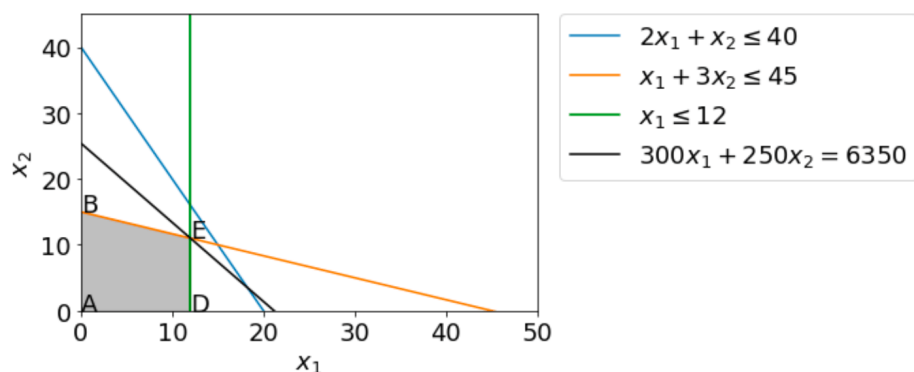
Basic	z	x_1	x_2	s_1	s_2	s_3	RHS	Ratio
.	1	?	0	0	?	?	?	.
s_1	0	0	0	1	?	?	?	?
x_2	0	0	1	0	1/3	-1/3	11	?
x_1	0	1	0	0	?	?	?	?

And then the rest of the rows using linear transformations as we did before we obtain. The next two tables show this. (The second table shows only the variables and RHS)

Basic	z	x_1	x_2	s_1	s_2	s_3	RHS	Ratio
.	1	$0 - (-250 \cdot 0) =$	$-250 - (-250 \cdot 0) =$	$0 - (-250 \cdot 0) =$	$0 - (-250 \cdot 1/3) =$	$300 - (-250 \cdot -1/3) =$	$3600 - (-250 \cdot 11) =$	.
s_1	0	0	$1 - 1 = 0$	$1 - 0 = 0$	$0 - 1/3 = -1/3$	$-2 - (-1/3) = -5/3$	$16 - 11 = 5$	.
x_2	0	0	1	0	1/3	-1/3	11	.
x_1	0	1	0	0	0	1	12	.

z	x_1	x_2	s_1	s_2	s_3	RHS
1	$0 - (-250 \cdot 0) = 0$	$-250 - (-250 \cdot 1) = 0$	$0 - (-250 \cdot 0) = 0$	$0 - (-250 \cdot 1/3) = 250/3$	$300 - (-250 \cdot -1/3) = 650/3$	$3600 - (-250 \cdot 11) = 6350$
0	0	$1 - 1 = 0$	$1 - 0 = 0$	$0 - 1/3 = -1/3$	$-2 - (-1/3) = -5/3$	$16 - 11 = 5$
0	0	1	0	1/3	-1/3	11
0	1	0	0	0	1	12

Now we obtain a new solution $(12, 11, 5, 0, 0)$ where $z = 6350$ and we know for sure that this is a maximum because there are no negative coefficients for any of the non-basic variables that could yield to a higher value of z .



5 Duality

5.1 Introduction

The duality principle states that every optimization problem can be represented and analysed in two different formulations, the **primal** and the **dual**. This means that every problem that we have analysed so far has a **dual** problem formulation with the following characteristics:

- The dual has an optimal solution **if and only if** the primal has an optimal solution (and vice versa).
- The **optimal value** of the objective function both cases (respective optimal solutions) **is the same** (strong duality principle).
- The dual of dual is the primal.
- The solution of the dual problem can be obtained from the solution of the primal and vice versa (using algorithms like the like Simplex)

Basic rules for obtaining the dual The dual of a problem can be obtained applying the following rules:

- If the **primal problem objective function** is of type **maximise**, the **dual problem objective function** is of type **minimise** (and vice versa).
- If the primal has m constraints, the dual has m decision variables, noted as $u_1, u_2, \dots, u_m$.
- The coefficients of the objective function of the dual are the independent terms (RHS) of the constraints of the primal (and vice versa).
- The coefficients of the dependent terms (LHS) of the constraint of the dual can be represented by the transposed matrix A^T that contains the LHS coefficients of the primal (and vice versa).
- All the constraints are of type greater or equal ($\geq$).
- For maximization problems:
 - If the i^{th} constraint of the primal is of type less or equal ($\leq$), then u_i is non-negative (≥ 0).
 - If the i^{th} constraint of the primal is of type greater or equal ($\geq$), then u_i is non-positive (≤ 0).
- For minimization problems:
 - If the i^{th} constraint of the primal is of type less or equal ($\leq$), then u_i is non-positive (≤ 0).
 - If the i^{th} constraint of the primal is of type greater or equal ($\geq$), then u_i is non-negative (≥ 0).
- If any case, if the i^{th} constraint of the primal is of type equal ($=$), then u_i is unrestricted (can be positive or negative).

Summary These rules are summarised in the following table:

Primal problem	Dual problem
Objective of type maximize	Objective of type minimize
Objective of type minimize	Objective of type maximize
Constraint limits, RHS	Decision Variable coefficients
Decision Variable coefficients	Constraint limits, RHS
Constraint coefficient of decision variable i (a_{ij})	Constraint coefficient of constraint i (a_{*ji})

5.2 Simplex example

Let us consider the following generic problem with two decision variables:

$$\begin{aligned}
 \max \quad & z = c_1 \cdot x_1 + c_2 \cdot x_2 \\
 \text{subject to} \quad & \\
 & a_{11} \cdot x_1 + a_{12} \cdot x_2 \leq b_1 \\
 & a_{21} \cdot x_1 + a_{22} \cdot x_2 \leq b_2 \\
 & a_{31} \cdot x_1 + a_{32} \cdot x_2 \leq b_2
 \end{aligned}$$

Applying the rules above, the objective function of the dual is of type minimise, there are 3 decision variables, as the primal has 3 constraints, and the coefficients of the objective function of the dual are the RHS of the primal, hence, the objective function of the dual becomes:

$$\min z = b_1 \cdot u_1 + b_2 \cdot u_2 + b_3 \cdot u_3.$$

Now, we have two constraints, one per every decision variable. The RHS are the coefficients of the objective function, and the dependent coefficient are obtained by transposing the coefficients of the primal, hence the dual problem is subject to the following constraints:

$$\begin{aligned}
 a_{11} \cdot u_1 + a_{21} \cdot u_2 + a_{31} \cdot u_3 &\geq c_1 \\
 a_{12} \cdot u_1 + a_{22} \cdot u_2 + a_{32} \cdot u_3 &\geq c_2
 \end{aligned}$$

Now, since the constraints of the primal are of type $\leq$, the three decision variables are non-negative: $u_1, u_2, u_3 \geq 0$.

5.3 Bounding explanation

One way to deduct the dual is by acknowledging that, if the objective function of the primal is of type maximize and all the constraints are of type less or equal, linear combinations of the constraints provide **upper bounds** for the objective function. For instance, let us consider the following problem:

$$\begin{aligned}
 \max \quad & z = 4 \cdot x_1 + x_2 + 5 \cdot x_3 + 3 \cdot x_4 \\
 \text{subject to} \quad & \\
 & x_1 - x_2 - x_3 + 4 \cdot x_4 \leq 1 \\
 & 5 \cdot x_1 + x_2 + 3 \cdot x_3 + 8 \cdot x_4 \leq 55 \\
 & -x_1 + 2 \cdot x_2 + 3 \cdot x_3 + 5 \cdot x_4 \leq 3
 \end{aligned}$$

Note that, if we multiply the second constraint by a factor of 2, we obtain an equivalent constraint:

$$10 \cdot x_1 + 2 \cdot x_2 + 6 \cdot x_3 + 16 \cdot x_4 \leq 110.$$

Note also that all the LHS coefficients of this constraint are higher or equal than the coefficients of the objective function, and therefore:

$$\max z = 4 \cdot x_1 + x_2 + 5 \cdot x_3 + 3 \cdot x_4 \leq 10 \cdot x_1 + 2 \cdot x_2 + 6 \cdot x_3 + 16 \cdot x_4 \leq 110.$$

This constraint is an upper bound for the objective function, we do not know yet which is the maximum value of z , but we know that it cannot be higher than 110.

Note that we can find another upper bound using a linear combination of the second of third constraint, just by adding both constraints, we obtain:

$$4 \cdot x_1 + 3 \cdot x_2 + 6 \cdot x_3 + 14 \cdot x_4 \leq 58.$$

Again, this linear combination is an upper bound of the objective function, since all the coefficients are higher or equal the coefficients of the objective value, for any value of the decision variables, we have:

$$\max z = 4 \cdot x_1 + x_2 + 5 \cdot x_3 + 3 \cdot x_4 \leq 4 \cdot x_1 + 3 \cdot x_2 + 6 \cdot x_3 + 14 \cdot x_4 \leq 58.$$

Therefore, even if we do not know which is the optimal value of the objective function, 58 is an upper bound, the optimal value cannot exceed this bound.

Note that finding the maximum value of z , is equivalent to finding the **minimum upper bound**, hence, the minimum linear combination of the constraints, or equivalently, the minimum linear combination of the RHSs:

$$\min z = 1 \cdot u_1 + 55 \cdot u_2 + 4 \cdot u_3$$

which yields coefficients that are higher or equal the coefficients of the objective function for every decision variable:

$$\begin{aligned} u_1 + 5 \cdot u_2 - u_3 &\geq 4 \\ -u_1 + u_2 + 2 \cdot u_3 &\geq 1 \\ -u_1 + 3 \cdot u_2 + 3 \cdot u_3 &\geq 5 \\ 4 \cdot u_1 + 8 \cdot u_2 + 5 \cdot u_3 &\geq 3 \end{aligned}$$

Note that this is the dual of the primal problem.

5.4 Complementary slackness

Following the reasoning of the previous section, note that the decision variables of the dual are equivalent to the **Lagrangian multipliers** of the constraints in the Lagrangian of the primal:

$$\max z = \sum_{i=1}^n c_i \cdot x_i + u_1(a_{11} \cdot x_1 + a_{12} \cdot x_2 + \dots + a_{1n} \cdot x_n) + u_2(a_{21} \cdot x_1 \dots)$$

Maximizing the objective function is equivalent to minimizing the part of the Lagrangian multipliers.

But most importantly, Dual variables represent changes in the primal objective function per unit of constraint, that is, if an RHS of a constraint i is modified in 1 unit, the objective function is going to change in u_i units.

For this reason, the decision variables of the dual are referred to as the **shadow prices** of the constraints. The term shadow highlights that they are not factored in the objective function, and the term price means that they represent the impact in the objective function.

It follows that if a constraint is non-binding, meaning that the corresponding slack variable is non-zero, then the corresponding decision variable, or shadow price must necessarily be zero (that is, the shadow price must be zero, because we have a surplus or excess, and a change in the RHS of the constraint is not going to change the objective variable, just the value of the surplus or excess). Conversely, if a constraint is binding (meaning that the corresponding slack variable is zero), then the corresponding dual variable (shadow price) is basic (non-zero), because a change in the RHS of the constraint can cause a change in the objective variable.

Same applies for the dual: If a constraint of the dual is binding (meaning that the slack variable is zero, or non-basic), then the corresponding primal variable is basic. Also, if a constraint of the dual is non-binding (meaning that the slack variable is non-zero, or basic), then the corresponding primal variable is non-basic.

This principle is called **complementary slackness** meaning that the slack values or slackness of dual and primal are complementary.

6 Sensitivity: Algebraic explanation

6.1 Changes in the coefficients of the objective function

Let us start with the Production Mix Problem example, which in the standard form is:

$$\begin{aligned} \max \quad & z = 300 \cdot x_1 + 250 \cdot x_2 + 0 \cdot s_1 + 0 \cdot s_2 + 0 \cdot s_3 \\ \text{subject to} \quad & 2 \cdot x_1 + x_2 + s_1 = 40, \\ & x_1 + 3 \cdot x_2 + s_2 = 45, \\ & x_1 = 12. \end{aligned}$$

The final Tableau is:

Basic	z	x_1	x_2	s_1	s_2	s_3	RHS	Ratio
•	1	0	0	0	250/3	650/3	6350	•
s_1	0	0	0	1	-1/3	-5/3	5	•
x_2	0	0	1	0	1/3	-1/3	11	•
x_1	0	1	0	0	0	1	12	•

Let us assume we want to make a change in coefficient c_1 of magnitude Δc_1 . The objective function becomes:

$$\max z = (300 + \Delta c_1) \cdot x_1 + 250 \cdot x_2.$$

The final Tableau now becomes as the one presented by table 1.

The Simplex method requires that the Tableau is in canonical form. Thus, we need to add the last row times Δc_1 to the first row for this, table 2 shows this.

And taking into account that the coefficient of s_3 needs to be positive for x_1 to remain in the basis, Δc_1 cannot be lower than $-650/3$ and can increase up to infinity.

Let us now assume we want to make a change in coefficient c_2 of magnitude Δc_2 . The objective function becomes:

$$\max z = 300 \cdot x_1 + (250 + \Delta c_2) \cdot x_2.$$

The final Tableau now becomes as the one presented by table 3.

Again, the Simplex method requires that the Tableau is in canonical form. Thus, we need to add the second last row times Δc_2 to the first row for this, table 4 shows this.

And taking into account that the coefficients of s_2 and s_3 need to be positive for x_2 to remain in the basis, Δc_2 cannot be lower than -250 nor higher than 650 .

Table 1

Basic	z	x_1	x_2	s_1	s_2	s_3	RHS	Ratio
•	1	$-\Delta c_1$	0	0	250/3	650/3	6350	•
s_1	0	0	0	1	-1/3	-5/3	5	•
x_2	0	0	1	0	1/3	-1/3	11	•
x_1	0	1	0	0	0	1	12	•

Table 2

Basic	z	x_1	x_2	s_1	s_2	s_3	RHS	Ratio
•	1	0	0	0	250/3	$650/3 + \Delta c_1$	$6350 + 12 \cdot \Delta c_1$	•
s_1	0	0	0	1	-1/3	-5/3	5	•
x_2	0	0	1	0	1/3	-1/3	11	•
x_1	0	1	0	0	0	1	12	•

Table 3

Basic	z	x_1	x_2	s_1	s_2	s_3	RHS	Ratio
•	1	0	$-\Delta c_2$	0	250/3	650/3	6350	•
s_1	0	0	0	1	-1/3	-5/3	5	•
x_2	0	0	1	0	1/3	-1/3	11	•
x_1	0	1	0	0	0	1	12	•

6.2 Changes in the constraint independent terms

Now let us see what is the effect of changing b , for instance decreasing b_1 an amount equal to Δb_1 . In the original problem formulation, this means:

$$2 \cdot x_1 + x_2 + s_1 = 40 - \Delta b_1,$$

$$x_1 + 3 \cdot x_2 + s_2 = 45,$$

$$x_1 + s_3 = 12.$$

Changing the Right Hand Side by Δb_1 is equivalent to changing s_1 an amount of Δb_1 . Note that

Table 4

Basic	z	x_1	x_2	s_1	s_2	s_3	RHS	Ratio
•	1	0	0	0	$250/3 + \Delta c_2/3$	$650/3 - \Delta c_2/3$	$6350 + 11 \cdot \Delta c_2$	•
s_1	0	0	0	1	$-1/3$	$-5/3$	5	•
x_2	0	0	1	0	$1/3$	$-1/3$	11	•
x_1	0	1	0	0	0	1	12	•

s_1 is in the basis, thus for it to remain in the basis, it must satisfy:

$$s_1 = 5 + \Delta b_1 \geq 0,$$

$$\Delta b_1 \geq -5.$$

Now, take the second constraint and let us apply the same change:

$$2 \cdot x_1 + x_2 + s_1 = 40,$$

$$x_1 + 3 \cdot x_2 + s_2 = 45 - \Delta b_2,$$

$$x_1 + s_3 = 12.$$

Again, this is equivalent to:

$$2 \cdot x_1 + x_2 + s_1 = 40,$$

$$x_1 + 3 \cdot x_2 + s_2 + \Delta b_2 = 45,$$

$$x_1 + s_3 = 12.$$

Which can be regarded as a change of Δb_2 in s_2 . Now, if we change s_2 from $s_2 = 0$ to $s_2 = \Delta b_2$ in the final Tableau:

Basic	z	x_1	x_2	s_1	s_2	s_3	RHS	Ratio
•	1	0	0	0	$250/3 \cdot \Delta b_2$	$650/3$	6350	•
s_1	0	0	0	1	$-1/3 \cdot \Delta b_2$	$-5/3$	5	•
x_2	0	0	1	0	$1/3 \cdot \Delta b_2$	$-1/3$	11	•
x_1	0	1	0	0	$0 \cdot \Delta b_2$	1	12	•

The basis remains unchanged as long as all the constraints are still met:

$$s_1 - 1/3 \cdot \Delta b_2 = 5,$$

$$x_2 + 1/3 \cdot \Delta b_2 = 11,$$

$$x_1 + 0 \cdot \Delta b_2 = 12,$$

$$s_1 = 5 + 1/3 \cdot \Delta b_2 \geq 0 (\Delta b_2 \geq -15),$$

$$x_2 = 11 - 1/3 \cdot \Delta b_2 \geq 0 (\Delta b_2 \leq 33),$$

$$x_1 = 12 - 0 \cdot \Delta b_2 \geq 0 (\Delta b_2 \leq \infty).$$



Operations Research

05 – Transportation model

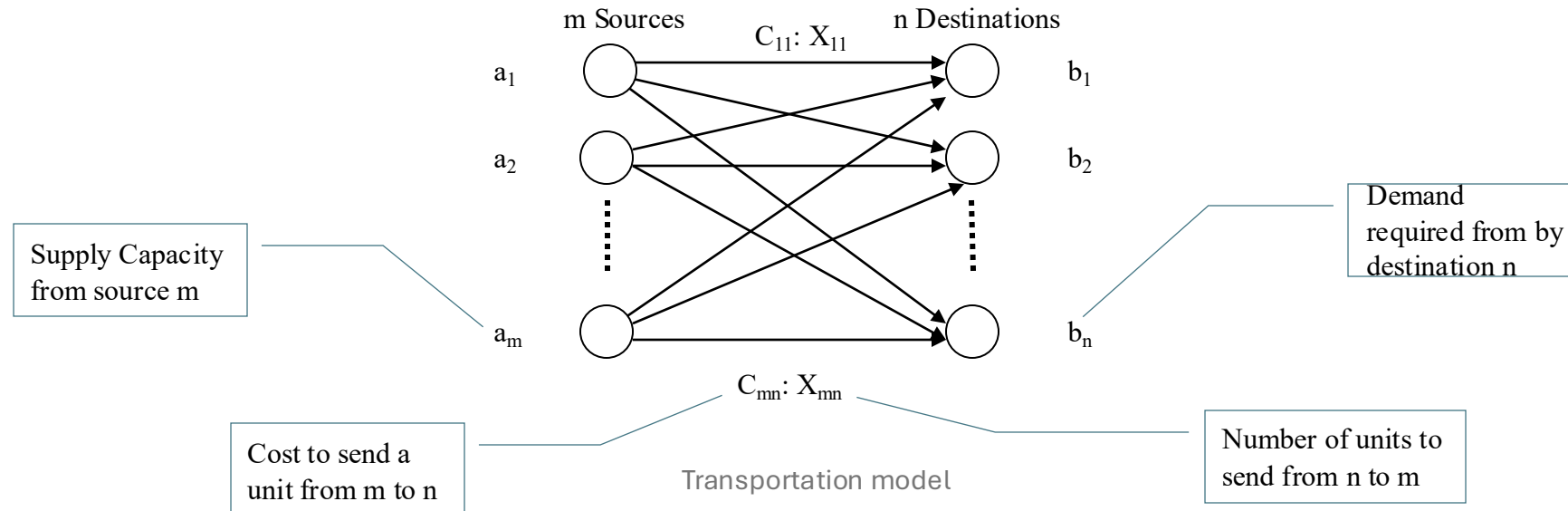
Faculty of Mathematics and Computer Science
Babeş-Bolyai University

Objectives

- Given a situation, identify when the transportation algorithm can be applied
- Understand the basics of the transportation algorithm
- Generate a basic feasible solution for the transportation problem

The Transportation Problem

- The transportation problem is a special case of an LP problem
- Because of its characteristics, the transportation problem can be solved very efficiently with a special algorithm called the transportation algorithm
- The problem is concerned with specifying how to disposition a single product from several sources to several destinations at minimum cost



Car-Distribution Problem (from Taha)

The MG Auto Company has plants in LA, Detroit, and New Orleans. Its major distribution centers are in Denver and Miami. The capacities of the three plants during the next quarter are: 1000, 1500, and 1200 cars. The quarterly demands at the two distribution centers are 2300 and 1400 cars. The cost of shipping in \$'s per car is given by:

	Denver	Miami
Los Angeles	80	215
Detroit	100	108
New Orleans	102	68

Find the best strategy to send cars from the plants to the distribution centers.

Model of the Car Distribution Problem

Example: The car distribution problem

Let X_{ij} = units sent from location i (L,N,D) to destination j (V,M).

Then, the problem can be stated as:

$$\text{Minimize } z = 80X_{LV} + 215X_{LM} + 100X_{DV} + 108X_{DM} + 102X_{NV} + 68X_{NM}$$

Subject to:

$$X_{LV} + X_{LM} = 1000$$

$$X_{DV} + X_{DM} = 1500$$

$$X_{NV} + X_{NM} = 1200$$

$$X_{LV} + X_{DV} + X_{NV} = 2300$$

$$X_{LM} + X_{DM} + X_{NM} = 1400$$

$$X_{LV}, X_{LM}, X_{DV}, X_{DM}, X_{NV}, X_{NM} \geq 0$$

The General Transportation Problem

- In general, a transportation problem can be expressed as:

$$\min \quad z = \sum_{i=1}^m \sum_{j=1}^n C_{ij} X_{ij}$$

Subject to:

$$\sum_{j=1}^n X_{ij} = a_i, i = 1, 2, \dots, m$$

$$\sum_{i=1}^m X_{ij} = b_j, j = 1, 2, \dots, n$$

$$X_{ij} \geq 0$$

Transportation Tableau

- Usually, it is not necessary to explicitly build the LP model of the transportation problem. Instead, we usually represent the transportation problem by using a transportation tableau

Decision Variable

		Markets (Destinations)				
		M_1	M_2	M_3	M_4	
Sources	S_1	X_{11} C_{11}	X_{12} C_{12}	X_{13} C_{13}	X_{14} C_{14}	a_1
	S_2	X_{21} C_{21}	X_{22} C_{22}	X_{23} C_{23}	X_{24} C_{24}	a_2
	S_3	X_{31} C_{31}	X_{32} C_{32}	X_{33} C_{33}	X_{34} C_{34}	a_3
		b_1	b_2	b_3	b_4	

Number of units required in destination 1 (j)

Cost for sending a unit from 3 (i) to 1 (j)

Number of available units to ship from source 3 (i)

Transportation Tableau Example

- Example: Car Distribution Matrix

		Markets (Destinations)		Supply
		Denver	Miami	
Plant	L.A.	X_{11} 80	X_{12} 215	1000
	Detroit	X_{21} 100	X_{22} 108	1500
	New Orl.	X_{31} 102	X_{32} 68	1200
	Demand	2300	1400	

$$\text{Min } z = 80 X_{11} + 215 X_{12} + 100 X_{21} + 108 X_{22} + 102 X_{31} + 68 X_{32}$$

$$X_{11} + X_{12} = 1000 \text{ (LA)}$$

$$X_{21} + X_{22} = 1500 \text{ (Detroit)}$$

$$X_{31} + X_{32} = 1200 \text{ (New O.)}$$

$$X_{11} + X_{21} + X_{31} = 2300 \text{ (Denver)}$$

$$+ X_{12} + X_{22} + X_{32} = 1400 \text{ (Miami)}$$

Characteristics of the Transportation Problem

- The transportation problem could be solved using the regular simplex method. However, because of its special characteristics, a more efficient procedure is used. The procedure is called the transportation (simplex) method.
- Because of the uni-modularity property (special structure of the constraints), transportation problems with supplies and demands given by integers will have integer basic solutions.
- The transportation problem is solved in two phases:
 1. Determination of an initial basic feasible solution
 2. Finding an optimal solution through the sequential improvement of the initial feasible solution

Finding an Initial Basic Solution

- There are many ways to find a feasible solution. We will examine several below. The first is simple but ineffective, and we will then look at more complex but effective (producing near optimal solutions) methods.
- **Northwest Corner Rule (NWC)**
 - We begin in the Northwest (upper-left) corner of the matrix and assign as much as we can (considering supply and demand) and update remaining supply and demand. We move either down or to the right (depending on whether supply or demand has been depleted). We again assign as much as possible and continue to the Southeast (lower-right).

Northwest Corner Rule (NWC)

- **Step1.** Allocate as much as possible to the selected cell and adjust the associated amounts of supply and demand by subtracting the allocated amount.
- **Step2.** Cross out the row or column with zero supply or demand. If both a row and a column net to zero simultaneously, cross out one only, and leave a zero supply (demand) in the uncrossed-out row (column).
- **Step3.** If exactly one row or column is left uncrossed out, stop. Otherwise, move to the cell to the right if a column has just been crossed out or below if a row has been crossed out. Go step 1

Example of NW Corner

		Markets (Destinations)		Supply
		Denver	Miami	
Plant	L.A.	X_{11} 80	X_{12} 215	1000
	Detroit	X_{21} 100	X_{22} 108	1500
	New Orl.	X_{31} 102	X_{32} 68	1200
	Demand	2300	1400	

		Markets (Destinations)		Supply
		Denver	Miami	
Plant	L.A.	1000 ↓	215	1000
	Detroit	1300 ↓	200 →	1500
	New Orl.	102	1200 ↓	1200
	Demand	2300	1400	

TC=313,200

Example 2 of NWC

- Using the NW corner rule, make the initial assignment for the following transportation problem

	Demand				
Source	A	B	C	D	
1	X_{11} 10	X_{12} 2	X_{13} 20	X_{14} 11	15
2	X_{21} 12	X_{22} 7	X_{23} 9	X_{24} 20	25
3	X_{31} 4	X_{32} 14	X_{33} 16	X_{34} 18	10
	5	15	15	15	

- the starting basic solution are:

$$\begin{aligned}
 X_{11} &= 5, & X_{12} &= 10; \\
 X_{22} &= 5, & X_{23} &= 15, & X_{24} &= 5 \\
 X_{34} &= 10
 \end{aligned}$$

- the association cost is

$$z = 5 \cdot 10 + 10 \cdot 2 + 5 \cdot 7 + 15 \cdot 9 + 5 \cdot 20 + 10 \cdot 18 = \$520$$

The Least Cost Rule (LCR)

- Note that the NWC method did not look at the costs! Thus, it may produce a terrible solution.
- The Least Cost Rule examines the costs to build an initial solution.
- The cell with the lowest cost is chosen, and we assign as many units as possible to the cell (considering supply and demand).
- We then reduce supplies and demands by the assignment and mark out ineligible cells (those in rows or columns where the supply or demand has been depleted).
- We repeat this process until all supplies and demands are depleted.

Example LCR

		Markets (Destinations)		Supply
		Denver	Miami	
Plant	L.A.	1000		1000
		80	215	0
	Detroit	1300	200	1500
		100	108	200 0
	New Orl.		1200	1200
		102	68	0
	Demand	2300	1400	
		1300	200 0	
		0		

Least Cost

Least Cost

TC=313,200

Example 2 of LCR

- Using the LC Rule, make the initial assignment for the following Transportation Problem

	Demand				
Source	A	B	C	D	
1	X_{11} 10	X_{12} 2	X_{13} 20	X_{14} 11	15
2	X_{21} 12	X_{22} 7	X_{23} 9	X_{24} 20	25
3	X_{31} 4	X_{32} 14	X_{33} 16	X_{34} 18	10
	5	15	15	15	0

Least
Cost

Least Cost

Least Cost

Least Cost

Least Cost

Least Cost

Example 2 of LCR (cont.)

	Demand				
Source	A	B	C	D	
1	X_{11} 10	X_{12} 2	X_{13} 20	X_{14} 11	15
2	X_{21} 12	X_{22} 7	X_{23} 9	X_{24} 20	25
3	X_{31} 4	X_{32} 14	X_{33} 16	X_{34} 18	10
	5	15	15	15	

- the starting basic solution are:

$X_{12}=15$; $X_{14}=0$;
 $X_{23}=15$; $X_{24}=10$;
 $X_{31}=5$ $X_{34}=5$

- the association cost is

$$z = 15 \cdot 2 + 0 \cdot 11 + 15 \cdot 9 + 10 \cdot 20 + 5 \cdot 4 + 5 \cdot 18 = \$475$$

Vogel's Approximation Method (VAM)

- This method recognizes that it may be wise to make a small sacrifice for a bigger gain. It computes a penalty for each row and column if the lowest cost cell is not selected. That is, it figures out what it would cost to take the second-best cost.
- The penalty is the cost difference between the lowest cost cell and next lowest cost value in each row and column.
- We select the cell associated with the largest penalty to assign units to and proceed essentially like the LCR. We will have to recalculate some of the penalties on each iteration.

Vogel's Approximation Method (VAM)

- **Step 1:** Determine the difference between the lowest two cells in all rows and columns, including dummies.
- **Step 2:** Identify the row or column with the largest difference. Ties may be broken arbitrarily.
- **Step 3:** Allocate as much as possible to the lowest-cost cell in the row or column with the highest difference. If two or more differences are equal, allocate as much as possible to the lowest-cost cell in these rows or columns.
- **Step 4:** Stop the process if all row and column requirements are met. If not, go to the next step.
- **Step 5:** Recalculate the differences between the two lowest cells remaining in all rows and columns. Any row and column with zero supply or demand should not be used in calculating further differences. Then go to Step 2.

VAM

When this assignment is made, we deplete both the column and the row; however, we eliminate just one. In this case, we arbitrarily eliminate the row

	Demand					Penalty
Source	A	B	C	D	Avail	
1	10	15	20	11	15	10
2	0	0	15	10	25	2
3	5				5	14
Demand	5	15	15	10		

High
est
Penal
ty

When this assignment is made, we deplete both the column and the row; however, we eliminate just one. In this case, we arbitrarily eliminate the row

Highest
Penalty.

VAM Final Solution

	Demand				
Source	A	B	C	D	Avail
1	10	15 0	20	11	15
2	0 12	0 7	15 9	10 20	25
3	5 0	14	16	18	5
Demand	5	15	15	10	

TC = 335

Example 2 of VAM

	Demand				
Source	A	B	C	D	Avail
1	10	2	20	11	15
2	12	7	9	20	25
3	4	14	16	18	10
Demand	5	15	15	10	

Penalty	High est Penal
8	9
2	11
10	2

- the starting basic solution are:

$$\begin{aligned} X_{12} &= 15; & X_{14} &= 0; \\ X_{23} &= 15; & X_{24} &= 10; \\ X_{31} &= 5 & X_{33} &= 5 \end{aligned}$$

- the association cost is

$$z = 15 \cdot 2 + 0 \cdot 11 + 15 \cdot 9 + 10 \cdot 20 + 5 \cdot 4 + 5 \cdot 18 = \$475$$

The solution happens to have the same objective value as in LCR

Dealing with Unbalanced Problems

- If the problem is unbalanced (demand and supply are not equal), the problem can be transformed into a balanced one by creating dummy sources or destinations with a cost of zero (usually). These dummy nodes will absorb the difference between the supply and demand.

Dealing with Unbalanced Problems

- Balanced: total demand = total supply
- Example: We have three reservoirs with daily supplies of 15, 20, and 25 million liters of fresh water, respectively. On each day we must supply four cities- A, B, C and D, whose demands are 8, 10, 12, and 15, respectively. The cost of pumping per million liters is given below:

	Cities			
Reservoirs	A	B	C	D
1	2	3	4	5
2	3	2	5	2
3	4	1	2	3

- Set up the transportation Tableau to determine the cheapest pumping schedule if excess water can be disposed of at no cost.

	City					
Reservoir	A	B	C	D	E (Dummy)	
1	X_{11} 2	X_{12} 3	X_{13} 4	X_{14} 5	X_{15} 0	15
2	X_{21} 3	X_{22} 2	X_{23} 5	X_{24} 2	X_{25} 0	20
3	X_{31} 4	X_{32} 1	X_{33} 2	X_{34} 3	X_{35} 0	25
Demand	8	10	12	15	15	

Dummy
Destination
Added

Tutorial

1. Compare the starting solutions obtained by the northwest-corner, least-cost, and Vogel methods for each of the following models:

***(a)**

0	2	1	6
2	1	5	7
2	4	3	7
5	5	10	

(b)

1	2	6	7
0	4	2	12
3	1	5	11
10	10	10	

(c)

5	1	8	12
2	4	0	14
3	6	7	4
9	10	11	

Iterative computation of the transportation algorithms

- After determining the starting solution; use the following algorithms to determine the optimum solution

Step1 : use the simplex optimality condition to determine the Entering Variables as the current nonbasic variable that can improve the solution. If the optimality condition is satisfied, stop. Otherwise, go to step 2

Step 2: determine the Leaving variables using the simplex feasibility condition. Change the basis and return to step 1.

- The optimality and feasibility do not involve the row operational that used in simplex method. Instead, the special structure of transportation allow simpler computation

Iterative computation of the transportation algorithms

- The determination of the Entering variable is done by computing the nonbasic coefficient in z-row , using **Method of multipliers**
- In the method of multiplier, associate the multipliers **u_i** and **v_j** with **row i** and **column j** of the transportation tubule.
- These multipliers satisfy the following equations:
$$u_i + v_j = C_{ij}, \text{ for each basic } X_{ij}$$
- To solve these equation, the method of multipliers call for arbitrarily setting any $u_i=0$, and then solving for the remaining variables

Example

- By using the NW corner rule, the starting basic solution are:

	Demand				
Source	A	B	C	D	
1	X_{11} 10 5	X_{12} 2 10	X_{13} 20	X_{14} 11	15
2	X_{21} 12	X_{22} 7 5	X_{23} 9 15	X_{24} 20 5	25
3	X_{31} 4	X_{32} 14	X_{33} 16	X_{34} 18 10	10
	5	15	15	15	

Example

	Demand				
Source	A	B	C	D	
1	X_{11} 5 10	X_{12} 10 2	X_{13} 20	X_{14} 11	15
2	X_{21} 12	X_{22} 5 7	X_{23} 15 9	X_{24} 5 20	25
3	X_{31} 4	X_{32} 14	X_{33} 16	X_{34} 10 18	10
	5	15	15	15	

Basic variable	(u,v) Equation	Solution
X11	$U_1 + V_1 = 10$	Set $U_1=0 \rightarrow V_1=10$
X12	$U_1 + V_2 = 2$	$U_1=0 \rightarrow V_2=2$
X22	$U_2 + V_2 = 7$	$V_2=2 \rightarrow U_2=5$
X23	$U_2 + V_3 = 9$	$U_2=5 \rightarrow V_3=4$
X24	$U_2 + V_4 = 20$	$U_2=5 \rightarrow V_4=15$
X34	$U_3 + V_4 = 18$	$V_4=15 \rightarrow U_3=3$

**We got: $U_1=0, U_2=5, U_3=3$
 $V_1=10, V_2=2, V_3=4, V_4=15$**

Example

- Use u_i , v_j to evaluate the nonbasic variable by computing:

$U_i + v_j - c_{ij}$, for each nonbasic x_{ij}

	Demand				
Source	A	B	C	D	
1	X_{11} 5 10	X_{12} 10 2	X_{13} 20	X_{14} 11	15
2	X_{21} 12	X_{22} 5 7	X_{23} 15 9	X_{24} 5 20	25
3	X_{31} 4	X_{32} 14	X_{33} 16	X_{34} 10 18	10
	5	15	15	15	

Nonbasic variable	$U_i + V_j - c_{ij}$
X13	$U_1 + V_3 - C_{13} = 0 + 4 - 20 = -16$
X14	$U_1 + V_4 - C_{14} = 0 + 15 - 11 = 4$
X21	$U_2 + V_1 - C_{21} = 5 + 10 - 12 = 3$
X31	$U_3 + V_1 - C_{31} = 3 + 10 - 4 = 9$
X32	$U_3 + V_2 - C_{32} = 3 + 2 - 14 = -9$
X33	$U_3 + V_3 - C_{33} = 3 + 4 - 16 = -9$

Example

- $u_i + v_j - c_{ij} = 0$ for each basic x_{ij} is important to computing the z-row of the simplex tableau, as the following:

Basic	X11	X12	X13	X14	X21	X22	X23	X24	X31	X32	X33	X34
Z	0	0	-16	4	3	0	0	0	9	-9	-9	0

- The transportation seek to minimize so, EV is the largest positive value coefficient in z-row thus X31 is EV

Basic	X11	X12	X13	X14	X21	X22	X23	X24	X31	X32	X33	X34
Z	0	0	-16	4	3	0	0	0	9	-9	-9	0

Example

- Construct closed loop that start and end at EV;
 - The loop consist of connected horizontal and vertical segments only.
 - Except EV, each corner must coincide with basic variable.
 - It must alternate between subtracting and adding (an addition to one cell in the loop is followed by a subtraction from the next cell in the loop)

	Demand				
Source	A	B	C	D	
1	X_{11} - 5 10	X_{12} + 10 2	X_{13} 20	X_{14} 11	15
2	X_{21} 12	X_{22} - 5 7	X_{23} + 15 9	X_{24} - 5 20	25
3	X_{31} + 4 4	X_{32} 14	X_{33} 16	X_{34} + 10 18	10
	5	15	15	15	

each unit shipped here will save \$9

Example

- To determine the LV it should Determine Which Current Basic Variable Reaches 0 First.
 - If we add one unit to X_{31} , it must subtract a unit from X_{34} (leaving 9); add it back to X_{24} (giving 6), and subtract it from X_{22} (leaving 4); add it back to X_{12} (giving 11), and subtract it from X_{11} (leaving 4); this saves $(1)(\$9)$.
 - If we add five unit to X_{31} , it must subtract a 5 unit from X_{34} (leaving 5); add it back to X_{24} (giving 10), and subtract it from X_{22} (leaving 0); add it back to X_{12} (giving 15), and subtract it from X_{11} (leaving 0); this saves $(5)(\$9) = \45 .

	Demand				
Source	A	B	C	D	
1	$X_{11} - 5$ 10	$X_{12} + 10$ 2	X_{13} 20	X_{14} 11	15
2	X_{21} 12	$X_{22} 5$ 7	$X_{23} 15$ 9	$X_{24} 5$ 20	25
3	$X_{31} + 4$ 4	X_{32} 14	X_{33} 16	$X_{34} 10$ 18	10
	5	15	15	15	

Both X_{11} , X_{22} reach zero, arbitrary choose X_{11} to leave the solution. The new cost is $520 - 45 = \$475$

Example

- The next transportation tableau is

	Demand				
Source	A	B	C	D	
1	X_{11} 10	X_{12} 15 2	X_{13} 20	X_{14} 11	15
2	X_{21} 12	X_{22} 0 7	X_{23} 15 9	X_{24} 10 20	25
3	X_{31} 5 4	X_{32} 14	X_{33} 16	X_{34} 5 18	10
	5	15	15	15	

Basic variable	(u,v) Equation	Solution
X12	$U_1 + V_2 = 2$	Set $U_1=0 \rightarrow V_2=2$
X22	$U_2 + V_2 = 7$	$V_2=2 \rightarrow U_2=5$
X23	$U_2 + V_3 = 9$	$U_2=5 \rightarrow V_3=4$
X24	$U_2 + V_4 = 20$	$U_2=5 \rightarrow V_4=15$
X34	$U_3 + V_4 = 18$	$V_4=15 \rightarrow U_3=3$
X31	$U_3 + V_1 = 4$	$U_3=3 \rightarrow V_1=1$

Example

- Use u_i , v_j to evaluate the nonbasic variable by computing:

$U_i + v_j - c_{ij}$, for each nonbasic x_{ij}

	Demand				
Source	A	B	C	D	
1	X_{11} 10	X_{12} 15 2	X_{13} 20	X_{14} 11	15
2	X_{21} 12	X_{22} 0 7	X_{23} 15 9	X_{24} 10 20	25
3	X_{31} 5 4	X_{32} 14	X_{33} 16	X_{34} 5 18	10
	5	15	15	15	

Nonbasic variable	$U_i + V_j - c_{ij}$
X_{11}	$U_1 + V_1 - C_{11} = 0 + 1 - 10 = -9$
X_{13}	$U_1 + V_3 - C_{13} = 0 + 4 - 20 = -16$
X_{14}	$U_1 + V_4 - C_{14} = 0 + 15 - 11 = 4$
X_{21}	$U_2 + V_1 - C_{21} = 5 + 1 - 12 = -6$
X_{32}	$U_3 + V_2 - C_{32} = 3 + 2 - 14 = -9$
X_{33}	$U_3 + V_3 - C_{33} = 3 + 4 - 16 = -9$

Example

- EV is

Basic	X11	X12	X13	X14	X21	X22	X23	X24	X31	X32	X33	X34
Z	-9	0	-16	4	-6	0	0	0	0	-9	-9	0

- closed loop

	Demand				
Source	A	B	C	D	
1	X ₁₁ 10	X ₁₂ 2	X ₁₃ 20	X ₁₄ 11	15
2	X ₂₁ 12	X ₂₂ 7	X ₂₃ 9	X ₂₄ 20	25
3	X ₃₁ 4	X ₃₂ 14	X ₃₃ 16	X ₃₄ 18	10
	5	15	15	15	

- If we add 10 unit to X₁₄, it must subtract a 10 unit from X₂₄ (leaving 0); add it back to X₂₂ (giving 10), and subtract it from X₁₂ (leaving 5; this saves (10)(\$4)= \$45.

X₂₄ reach zero, so it leaves the solution

The new cost is 475 - 40 = \$435

Example

- The next transportation tableau is

	Demand				
Source	A	B	C	D	
1	X_{11} 10	X_{12} 5 2	X_{13} 20	X_{14} 10 11	15
2	X_{21} 12	X_{22} 10 7	X_{23} 15 9	X_{24} 20	25
3	X_{31} 5 4	X_{32} 14	X_{33} 16	X_{34} 5 18	10
	5	15	15	15	

Basic variable	(u,v) Equation	Solution
X12	$U_1 + V_2 = 2$	Set $U_1=0 \rightarrow V_2=2$
X14	$U_1 + V_4 = 11$	$U_1=0 \rightarrow V_4=11$
X22	$U_2 + V_2 = 7$	$V_2=2 \rightarrow U_2=5$
X23	$U_2 + V_3 = 9$	$U_2=5 \rightarrow V_3=4$
X34	$U_3 + V_4 = 18$	$V_4=11 \rightarrow U_3=7$
X31	$U_3 + V_1 = 4$	$U_3=7 \rightarrow V_1=-3$

Example

- Use u_i , v_j to evaluate the nonbasic variable by computing:

$U_i + v_j - c_{ij}$, for each nonbasic x_{ij}

	Demand				
Source	A	B	C	D	
1	X_{11} 10	X_{12} 5 2	X_{13} 20	X_{14} 10 11	15
2	X_{21} 12	X_{22} 10 7	X_{23} 15 9	X_{24} 20	25
3	X_{31} 5 4	X_{32} 14	X_{33} 16	X_{34} 5 18	10
	5	15	15	15	

Nonbasic variable	$U_i + V_j - c_{ij}$
X_{11}	$U_1 + V_1 - C_{11} = 0 - 3 - 10 = -13$
X_{13}	$U_1 + V_3 - C_{13} = 0 + 4 - 20 = -16$
X_{21}	$U_2 + V_1 - C_{21} = 5 - 3 - 12 = -10$
X_{32}	$U_3 + V_2 - C_{32} = 7 + 2 - 14 = -5$
X_{33}	$U_3 + V_3 - C_{33} = 7 + 4 - 16 = -5$
X_{24}	$U_2 + V_4 - C_{24} = 5 + 11 - 20 = -4$

Example

Basic	X11	X12	X13	X14	X21	X22	X23	X24	X31	X32	X33	X34
Z	-13	0	-16	0	-10	0	0	-4	0	-5	-5	0

- Since no positive , the optimal solution is

From Silo	To mill	Number truckload
1	2	5
1	4	10
2	2	10
2	3	15
3	1	5
3	4	5

- Optimal cost is \$435



Operations Research

06 – Network Models

Faculty of Mathematics and Computer Science
Babeş-Bolyai University

Network Models

- There are many situations in operation research where the solution is modeled and solved as network (nodes are connected by branches)
- Network models algorithms
 - 1 - minimum spanning tree
 - 2 - shortest-route algorithms
 - 3 - maximum-flow algorithms
 - 4 - minimum cost capacitated network algorithms
 - 5 - critical path algorithms

Network Models (CONT.)

- 1- Design of an offshore gas pipeline network connecting wellheads in gulf of Mexico to inshore delivery points.; the objective of the model is: minimize the cost constructing the pipeline.
 - The situation represented as a *minimum spanning tree*.
- 2- Determine the shortest route between two cities in a network of roads.
 - This situation is *shortest-route algorithms*

Network Models (CONT.)

3- determination the maximum capacity (in ton per year) of a coal slurry pipeline network

- This situation is *maximum flow algorithms*

4- determination of the minimum-cost flow schedule from oil field to refineries through a pipeline network.

- This situation is *minimum-cost capacitated network algorithms*

Network Models (CONT.)

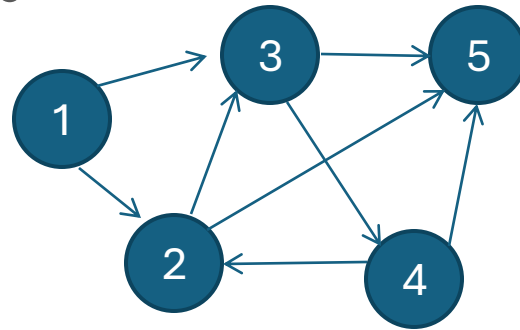
- 5- determination the time schedule (start and completion date) for activities
 - This situation is *(CPM) algorithms*

Network definitions

- A network consist of set of **nodes** linked by **arcs** (or **branches**)
- The notion for describing a network is (N, A) where:
 - N is set of nodes
 - A set of arc
- Or as a graph $G(V, E)$, $w:E \rightarrow \mathbb{R}$

Network definitions (cont.)

- Example



$N = \{ 1, 2, 3, 4, 5 \}$

$A = \{ (1, 2), (1, 3), (2, 3), (2, 5), (3, 4), (3, 5), (4, 2), (4, 5) \}$

- *Flow*: the amount sent from node i to node j , over an arc that connects i to j .

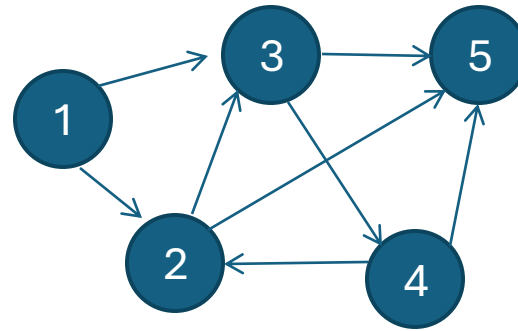
Network definitions (cont.)

- *Directed/undirected arcs* :
 - when flow is allowed in one direction the arc is **directed**; (that means allow positive flow in one direction and zero flow in the opposite direction)
 - When flow is allowed in two directions, the arc is **undirected**.
- *Path* : sequence of distinct arcs that join two nodes through other nodes regardless of the direction of flow in each arcs
- The nodes are said to be **connected** if there is a path between them.

Network definitions (cont.)

- *Cycle* : a path starting at a certain node and returning to the same node without using any arc twice. (or connects a node to itself through other nodes)

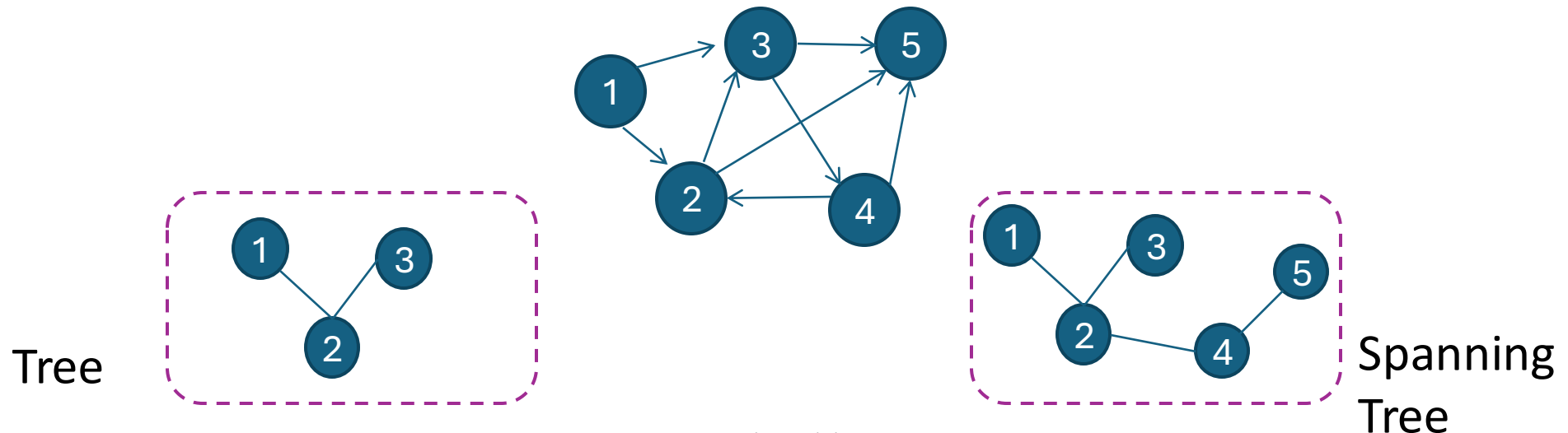
Example:



- (2,3),(3,5),(5,2) form of loop
- Cycle is directed if it consists of directed path (2,3),(3,4) and(4,2)

Network definitions (cont.)

- *Tree* : is connected network that may involve only a subset of all nodes of network without cycle.
- *Spanning tree* : a tree that connects all the nodes in a network with no cycle (it consists of $n - 1$ arcs).



Minimal Spanning tree

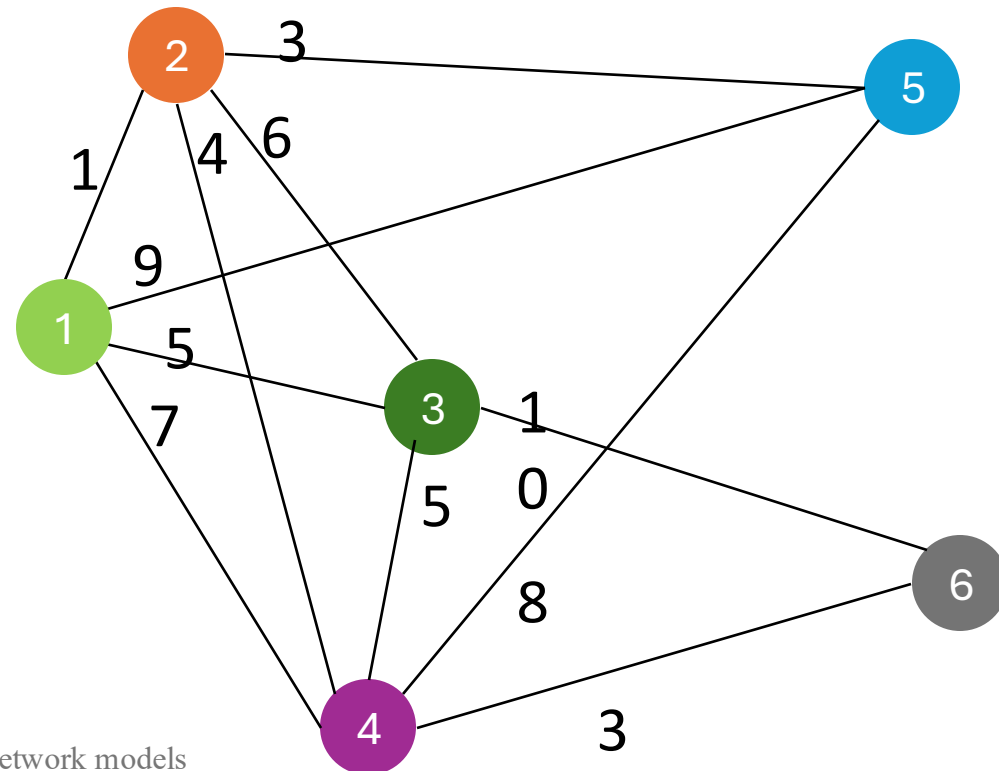
- It deals with linking the nodes of network, directly or indirectly, using shortest length of connecting branches.
- The typical application occurs in construction of paved roads that link several towns.

Minimal Spanning tree

- The step of procedure are given as follows:
 - Let $N=\{1,2,\dots,n\}$ set of nodes
 - C_k = set of nodes that have been permanently connected at iteration K
 - C_k' = set of nodes as yet to be connected permanently.
- Step 0: set $C_0= \emptyset$, $C_0'=N$
- Step 1: start with any node i ; set $C_1=\{i\}$, $C_1'=N-\{i\}$
- General step: selected node j in unconnected set C_{k-1}' that yield in shortest arcs to a node in the connected set C_{k-1} . Link j permanently to C_{k-1} and remove it from C_{k-1}'
- If the set of unconnected nodes is empty stop. Otherwise set $k=K+1$ and repeat the step

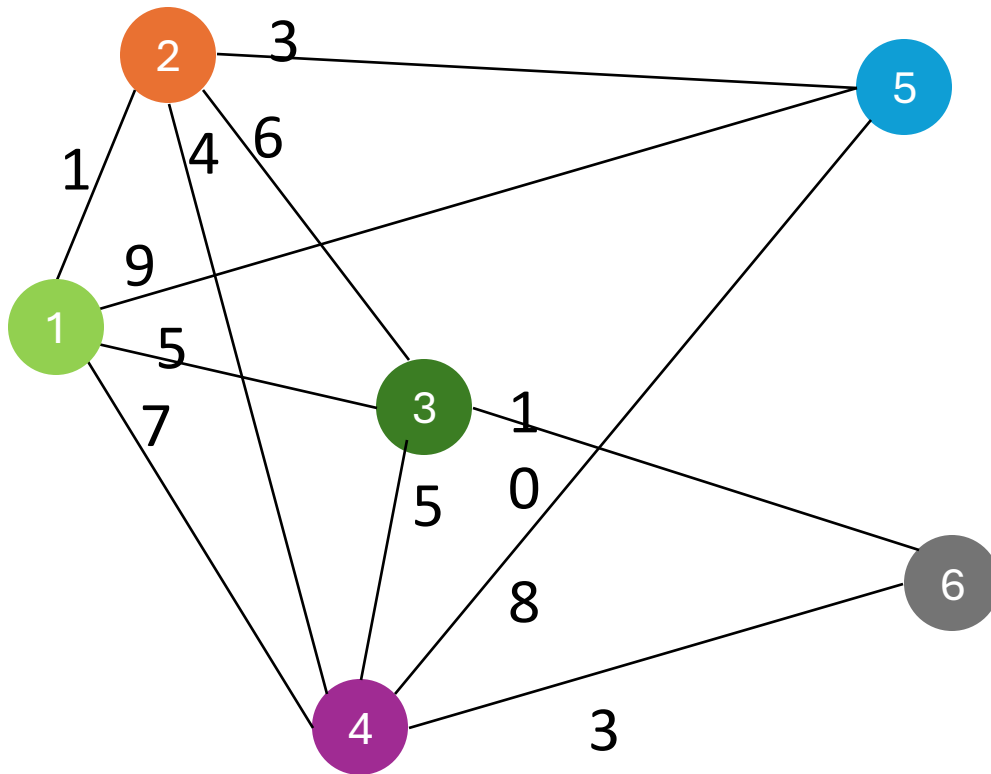
Example (cont.)

- Midwest TV cable company is in the process of providing cable service to five new housing development service areas.

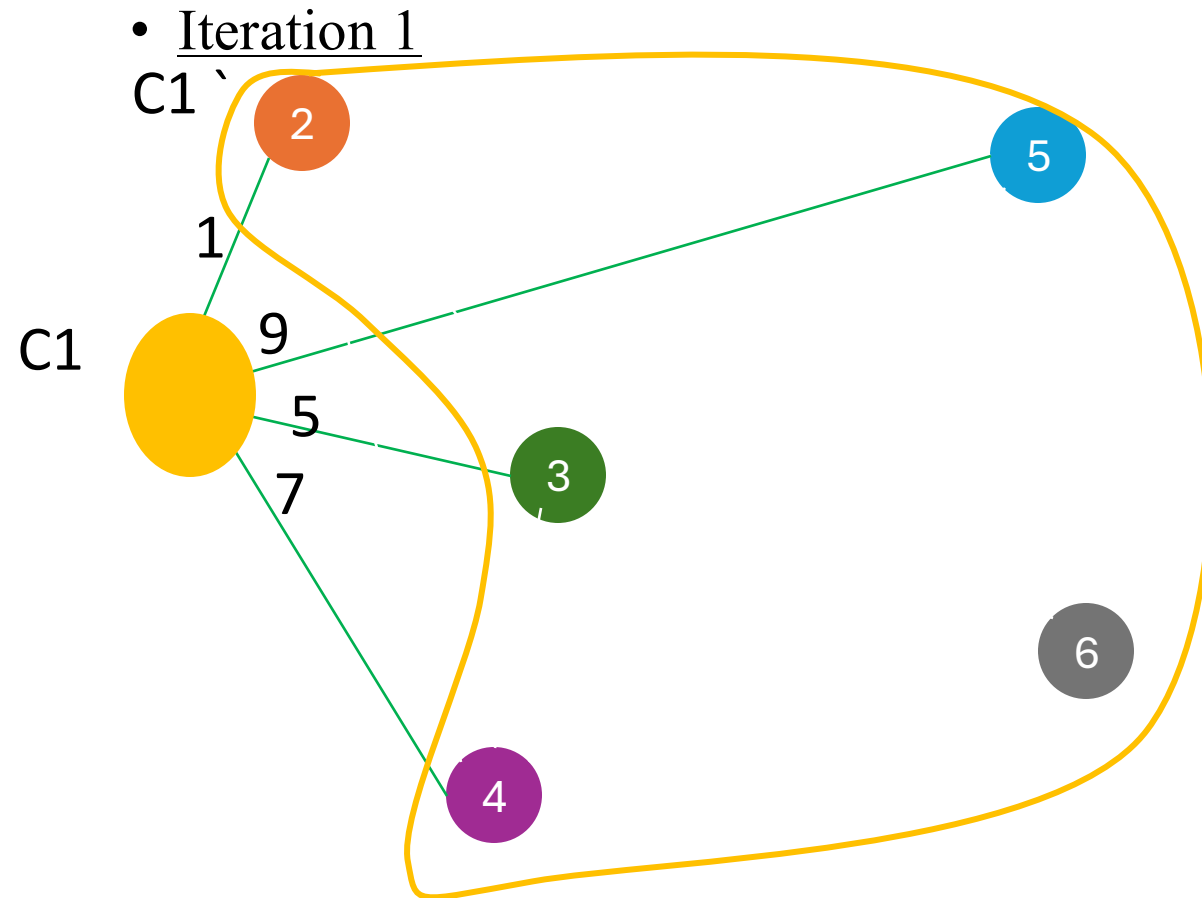


Example (cont.)

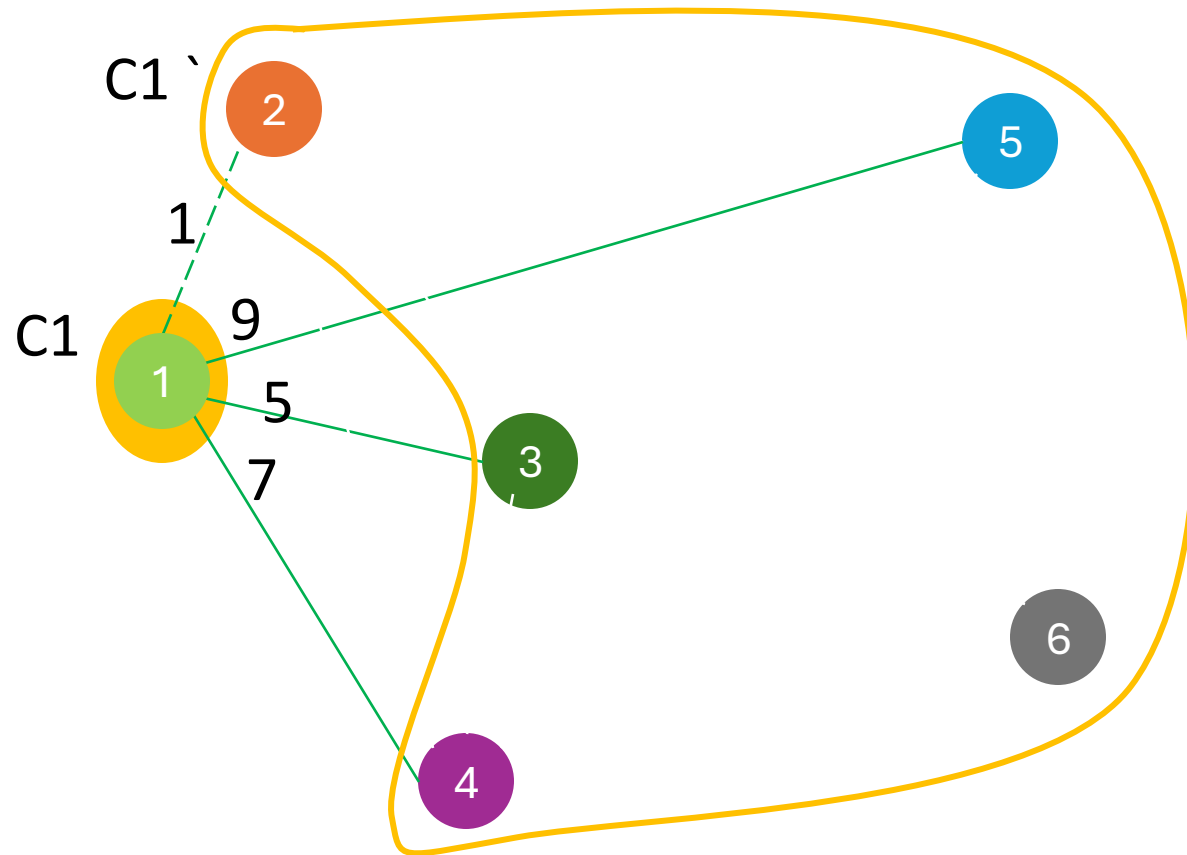
- The algorithms start at node 1



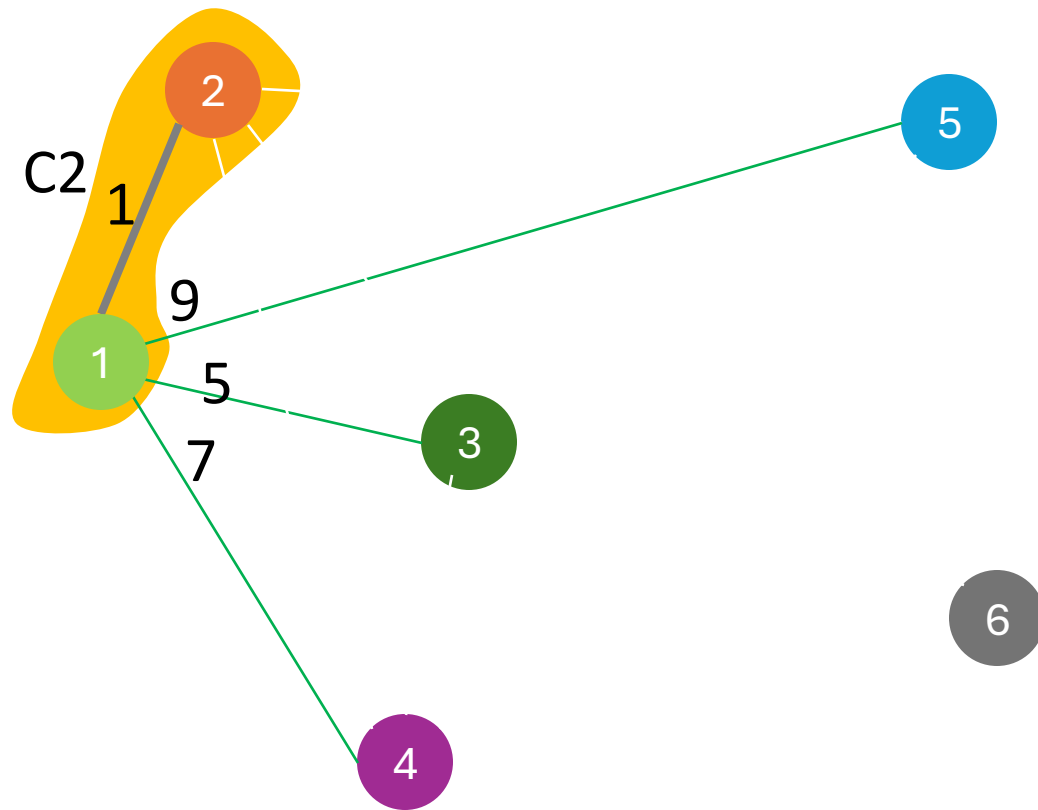
Example (cont.)



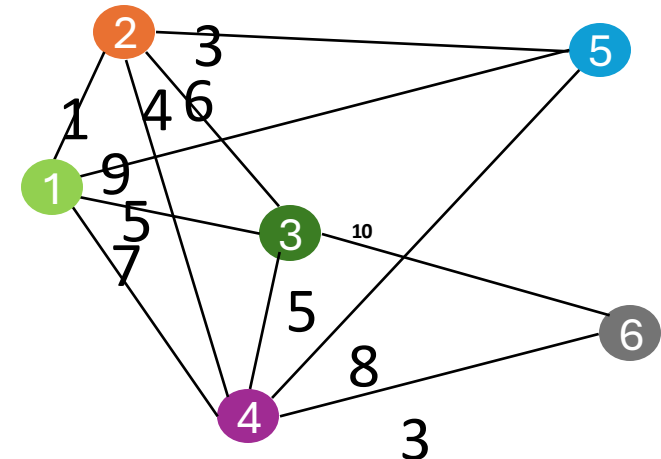
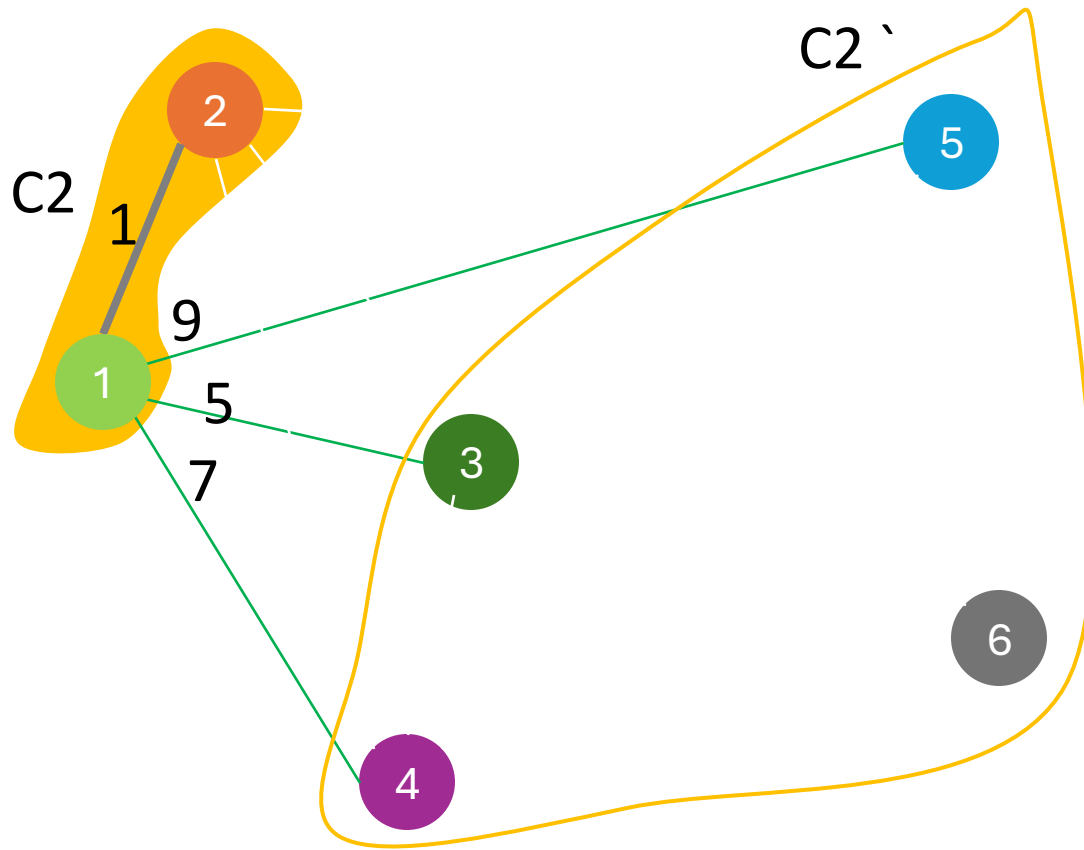
Example (cont.)



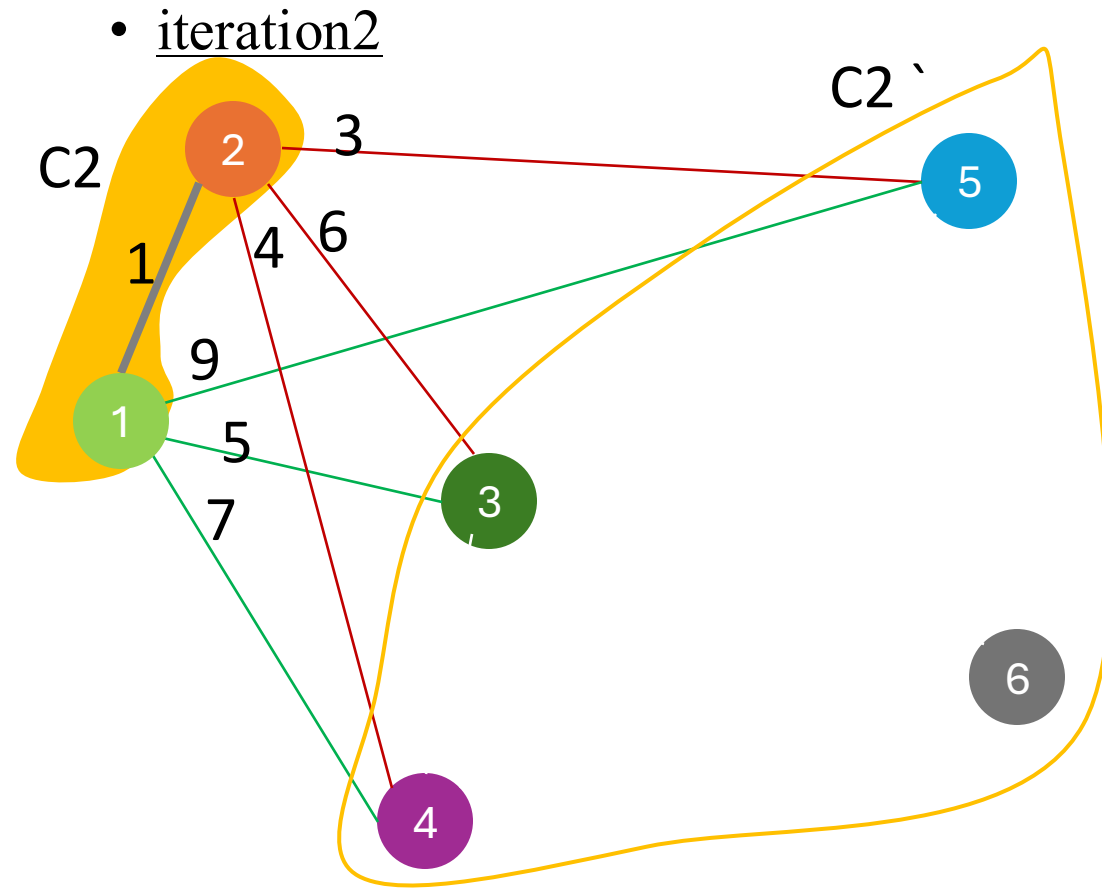
Example (cont.)



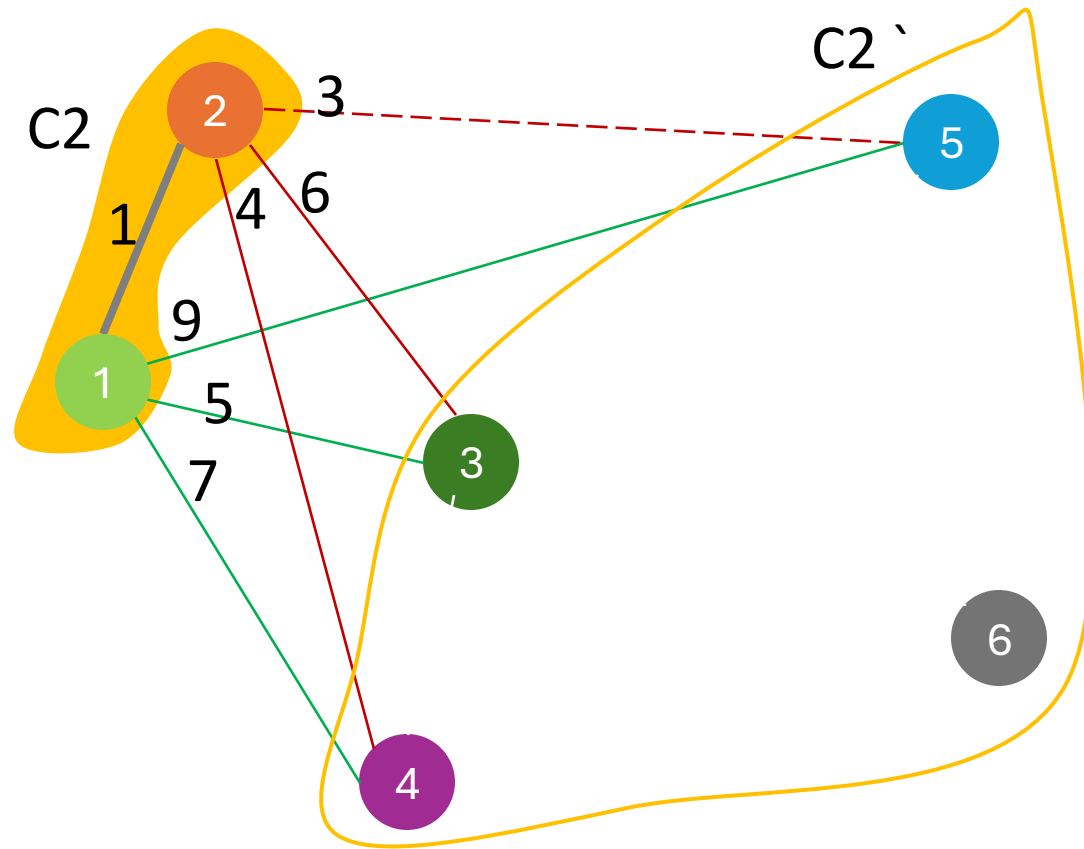
Example (cont.)



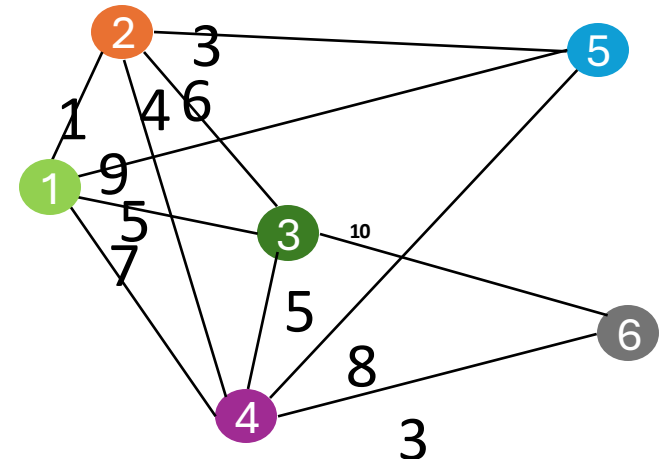
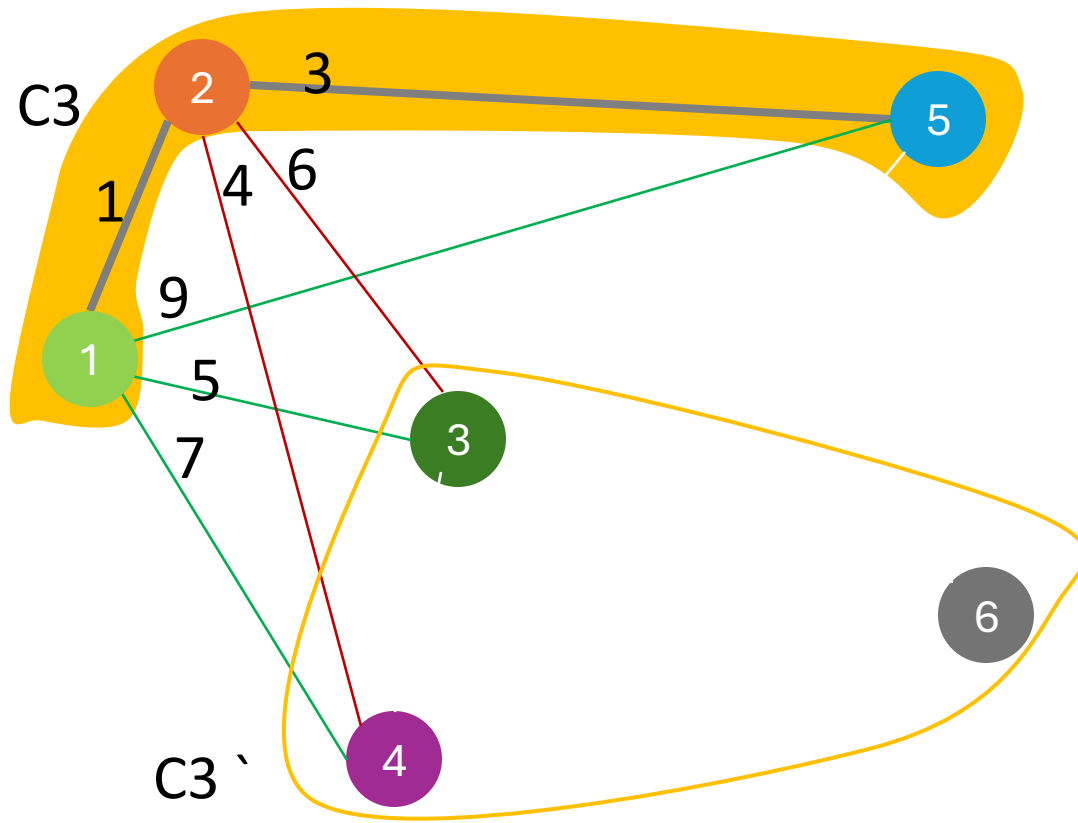
Example (cont.)



Example (cont.)

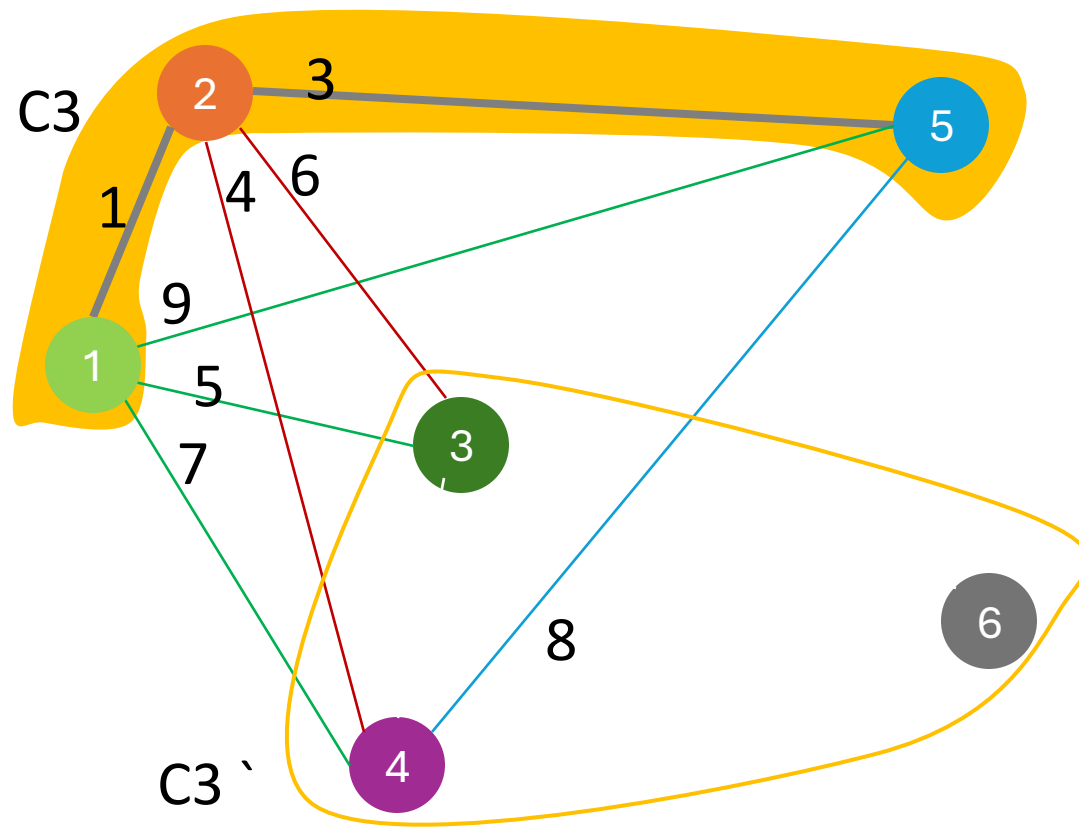


Example (cont.)

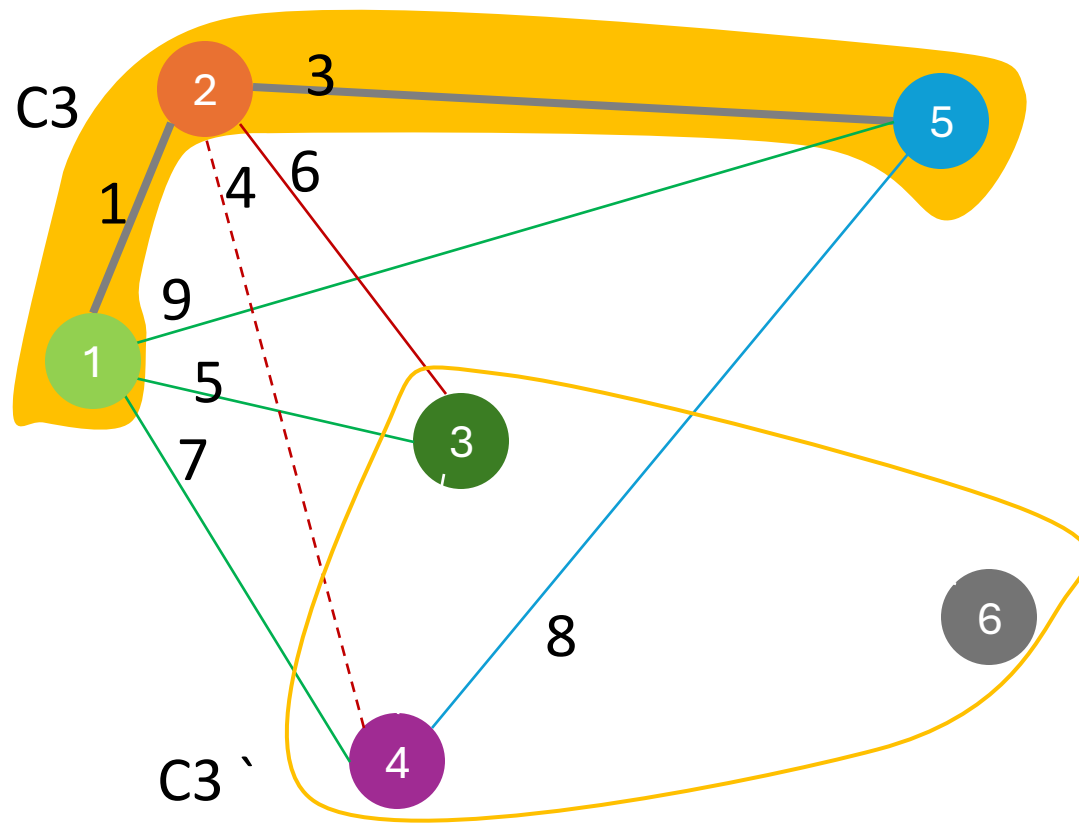


Example (cont.)

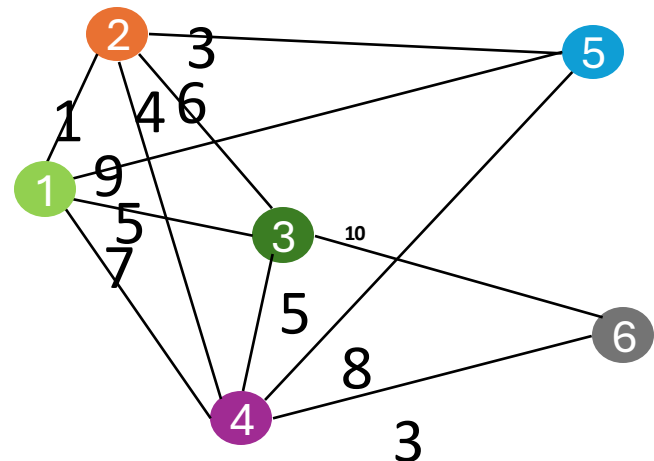
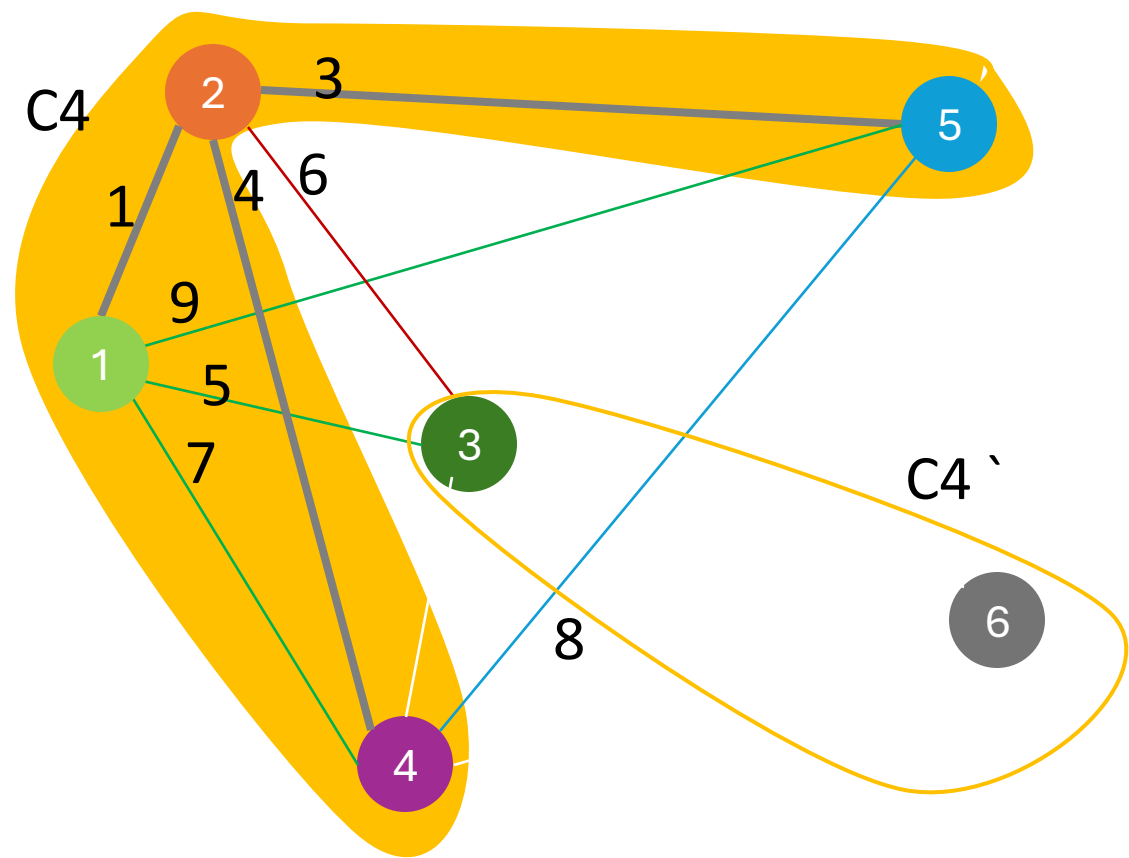
- iteration3



Example (cont.)

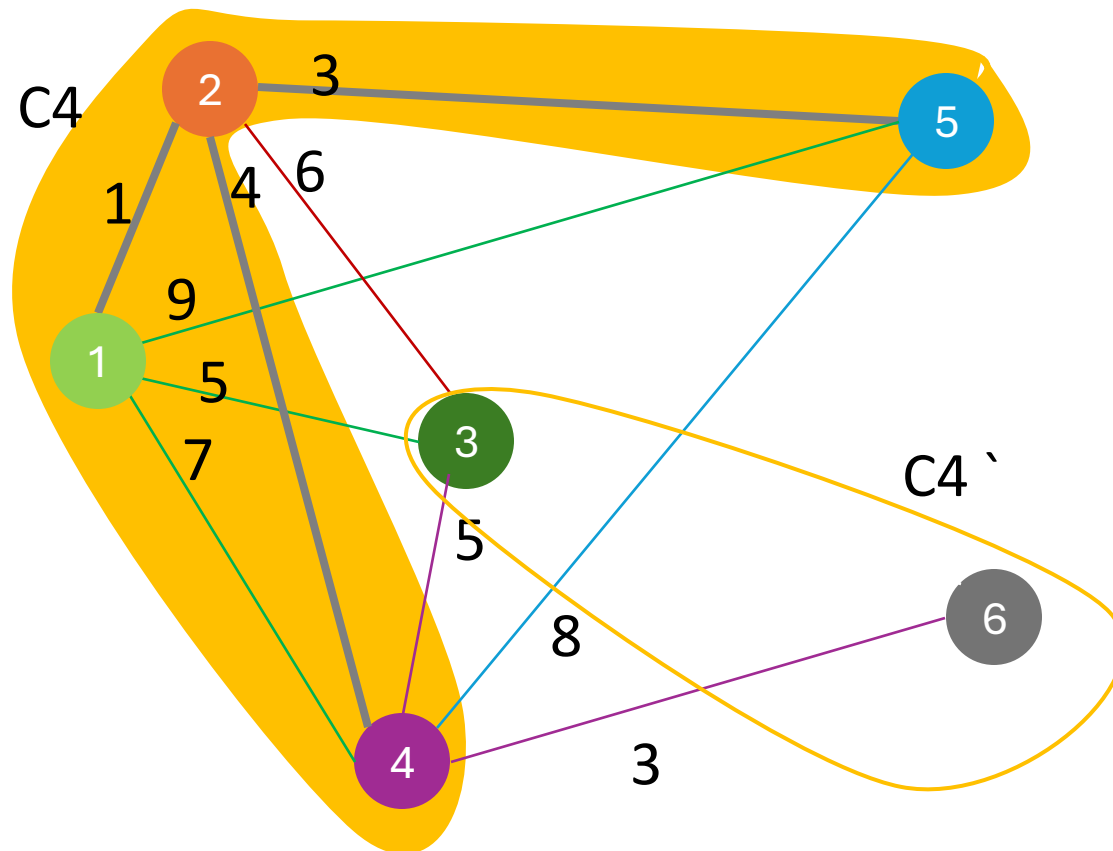


Example (cont.)

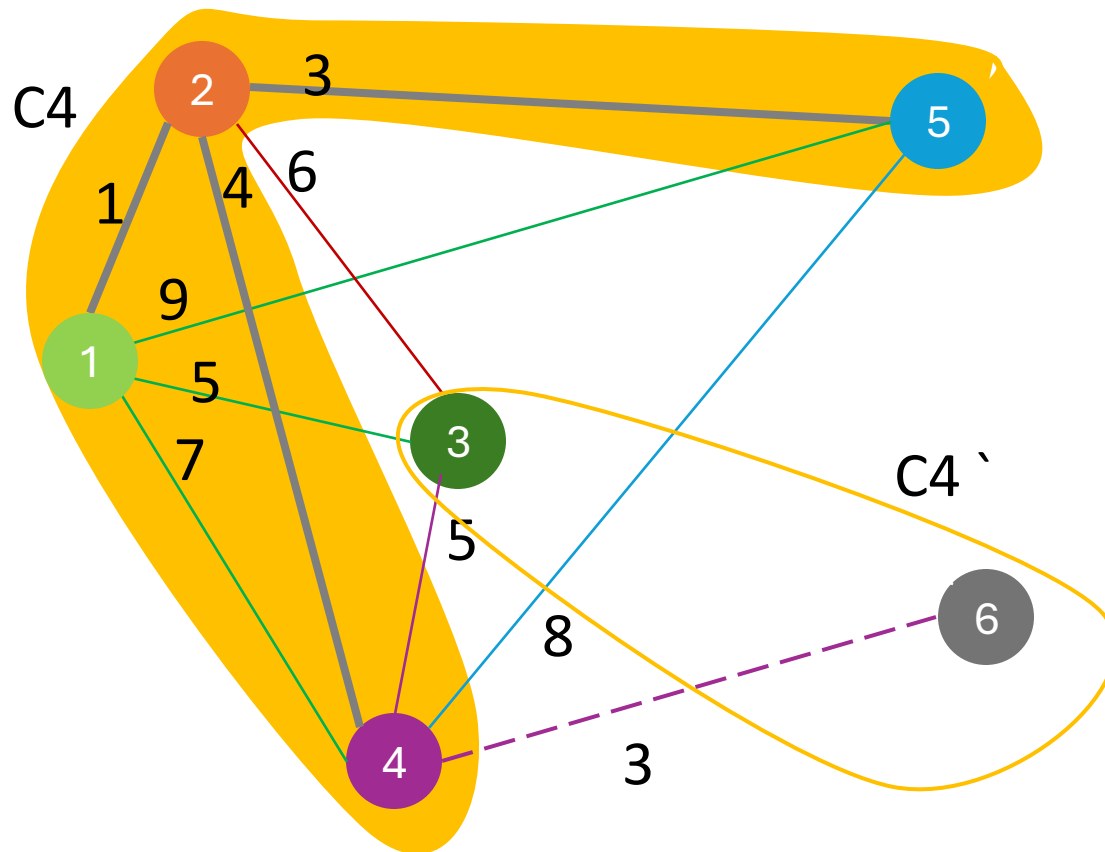


Example (cont.)

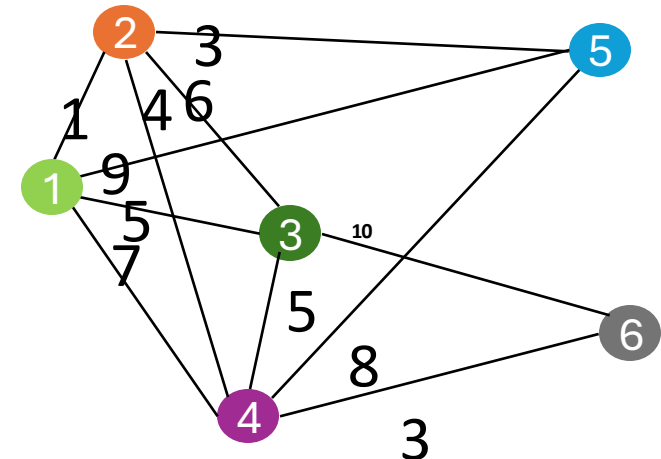
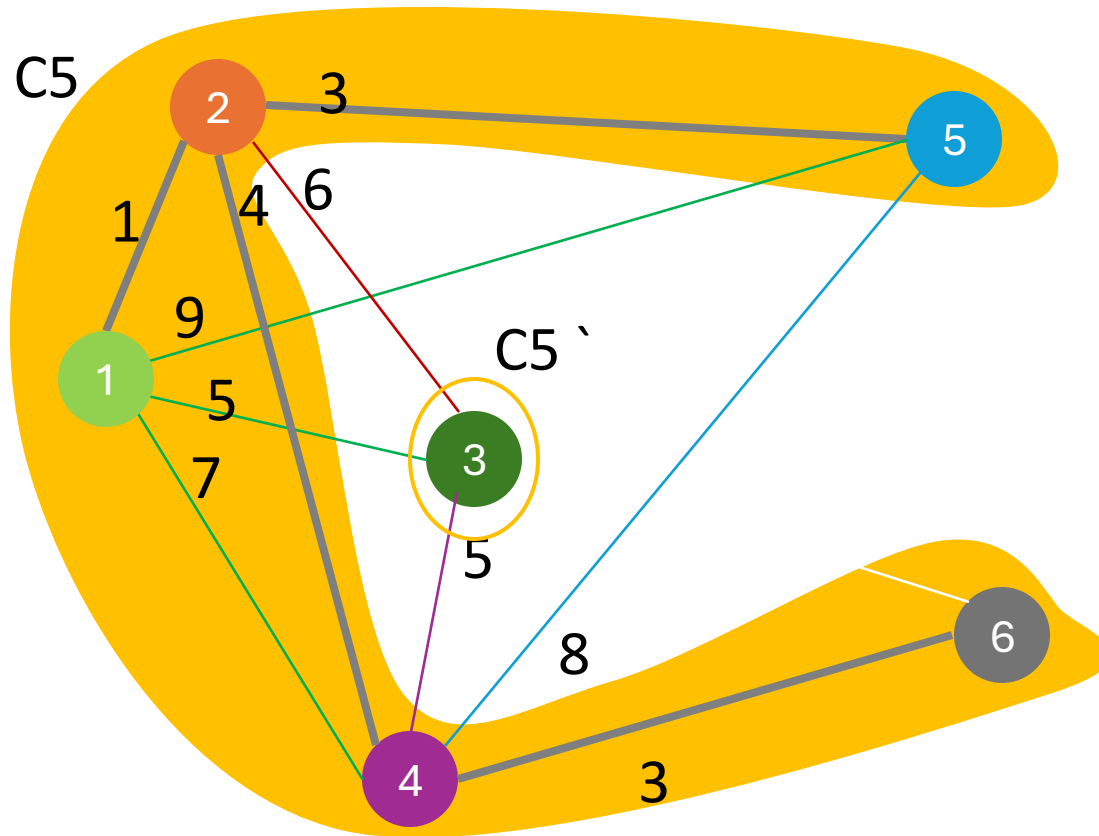
- iteration4



Example (cont.)

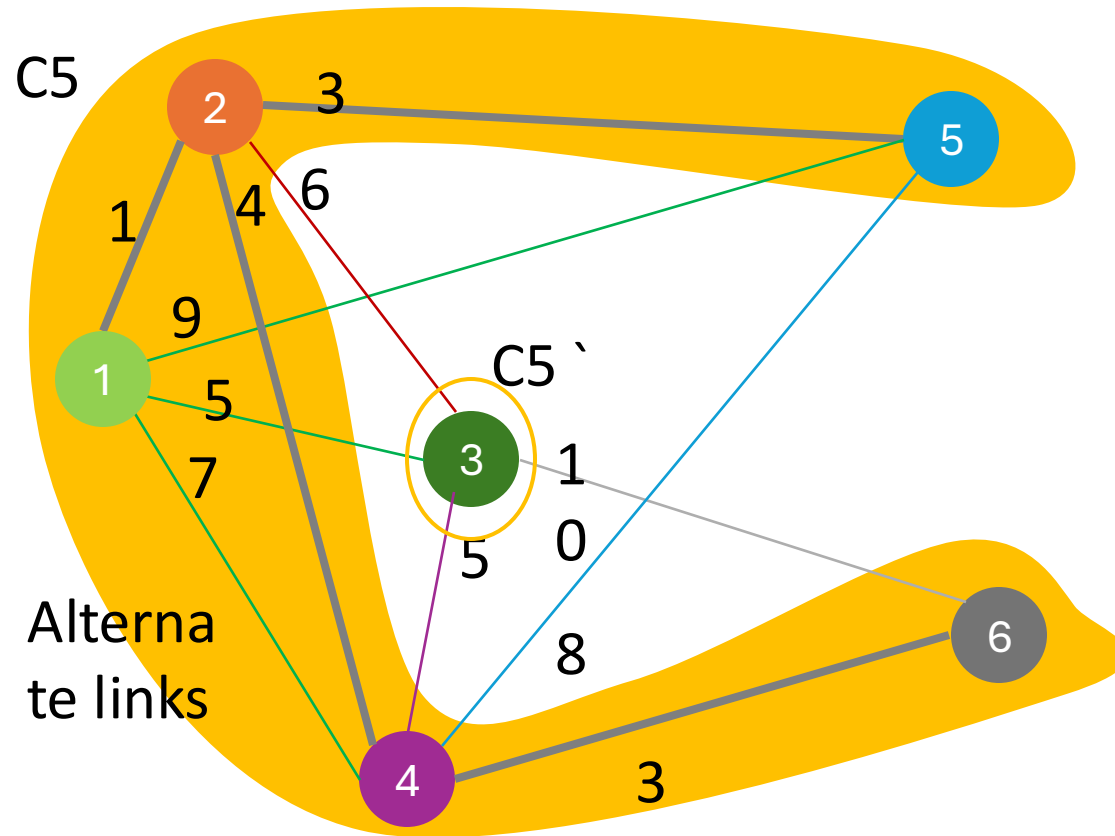


Example (cont.)

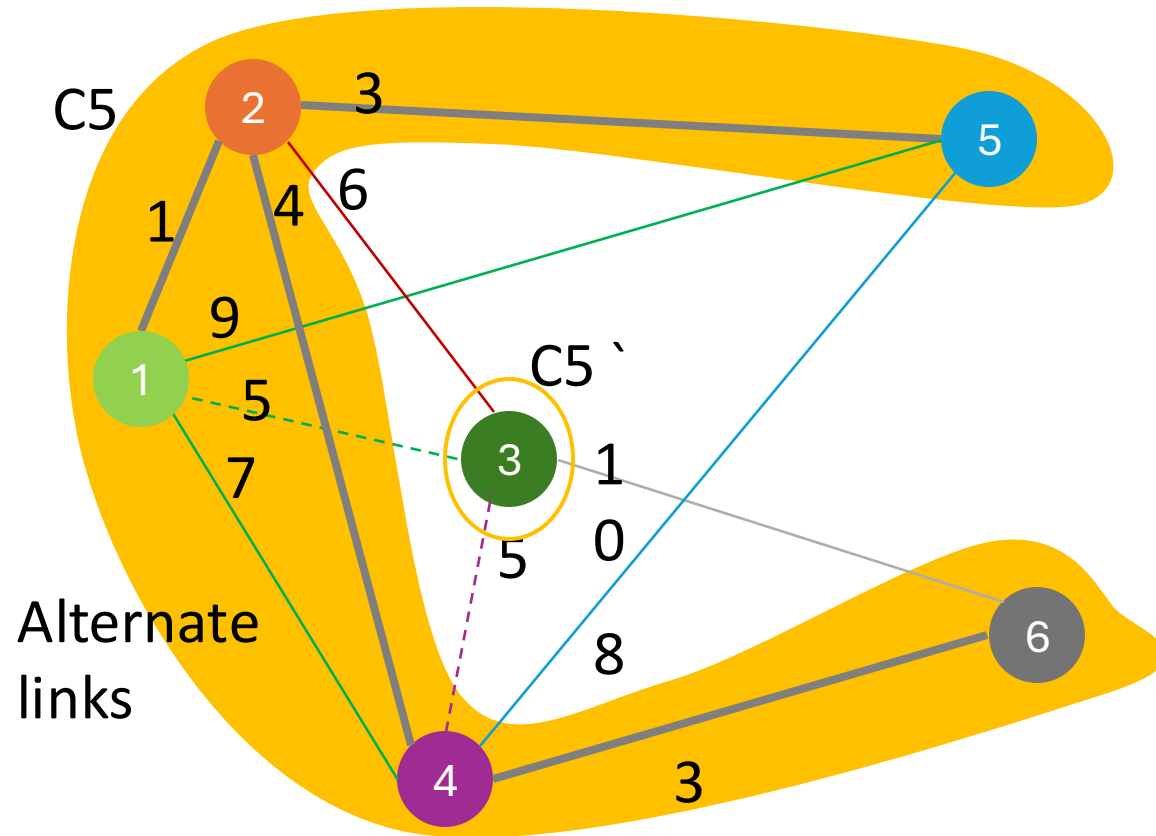


Example (cont.)

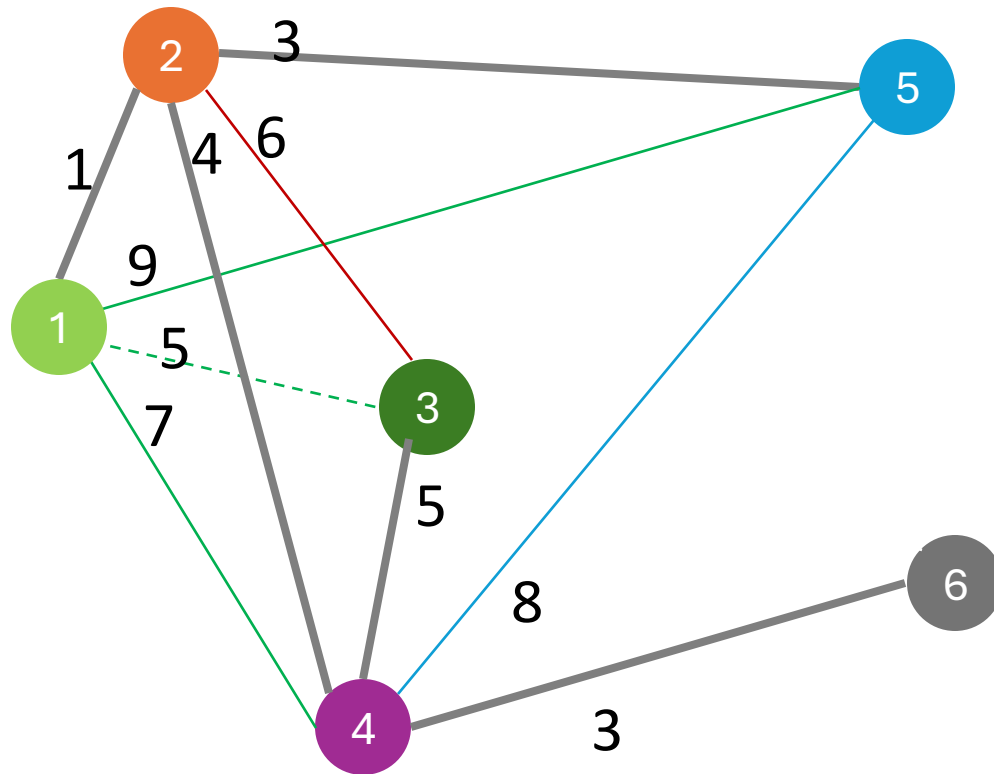
- iteration5



Example (cont.)



Example (cont.)



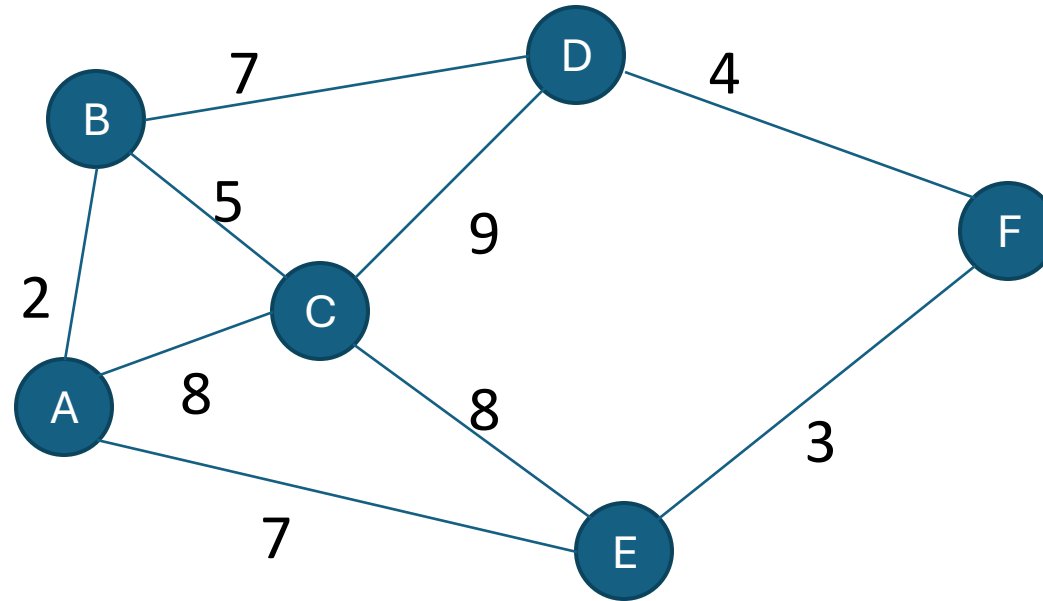
Example (cont.)

- Summary of solution

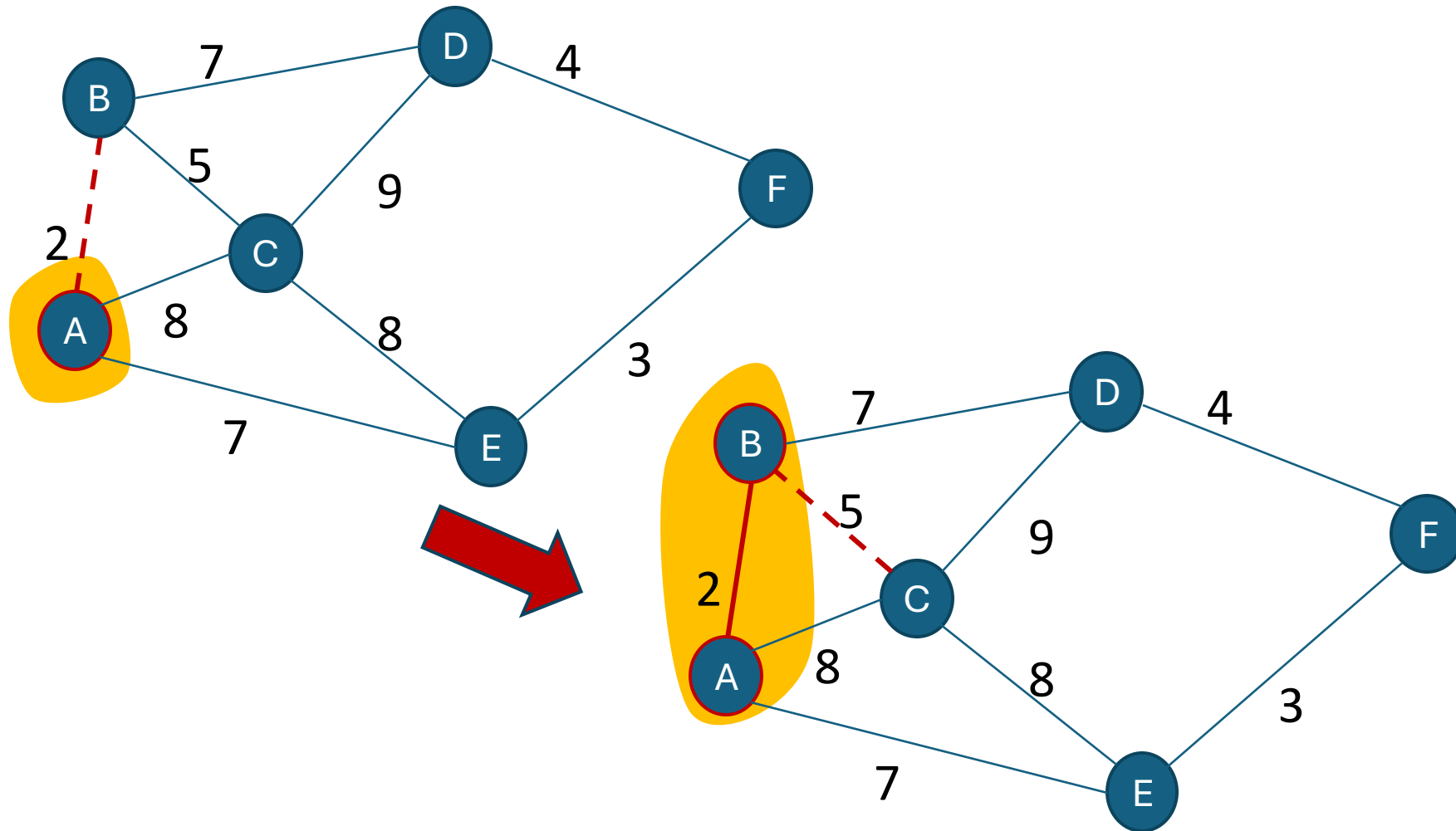
iteration	Minimum distance connecting arc	distance	Add arc to tree?	Cumulative tree distance
1	(1,2)	1	yes	1
2	(2,5)	3	yes	4
3	(2,4)	4	yes	8
4	(4,6)	3	yes	11
5	(4,3)	5	yes	16

Example 2

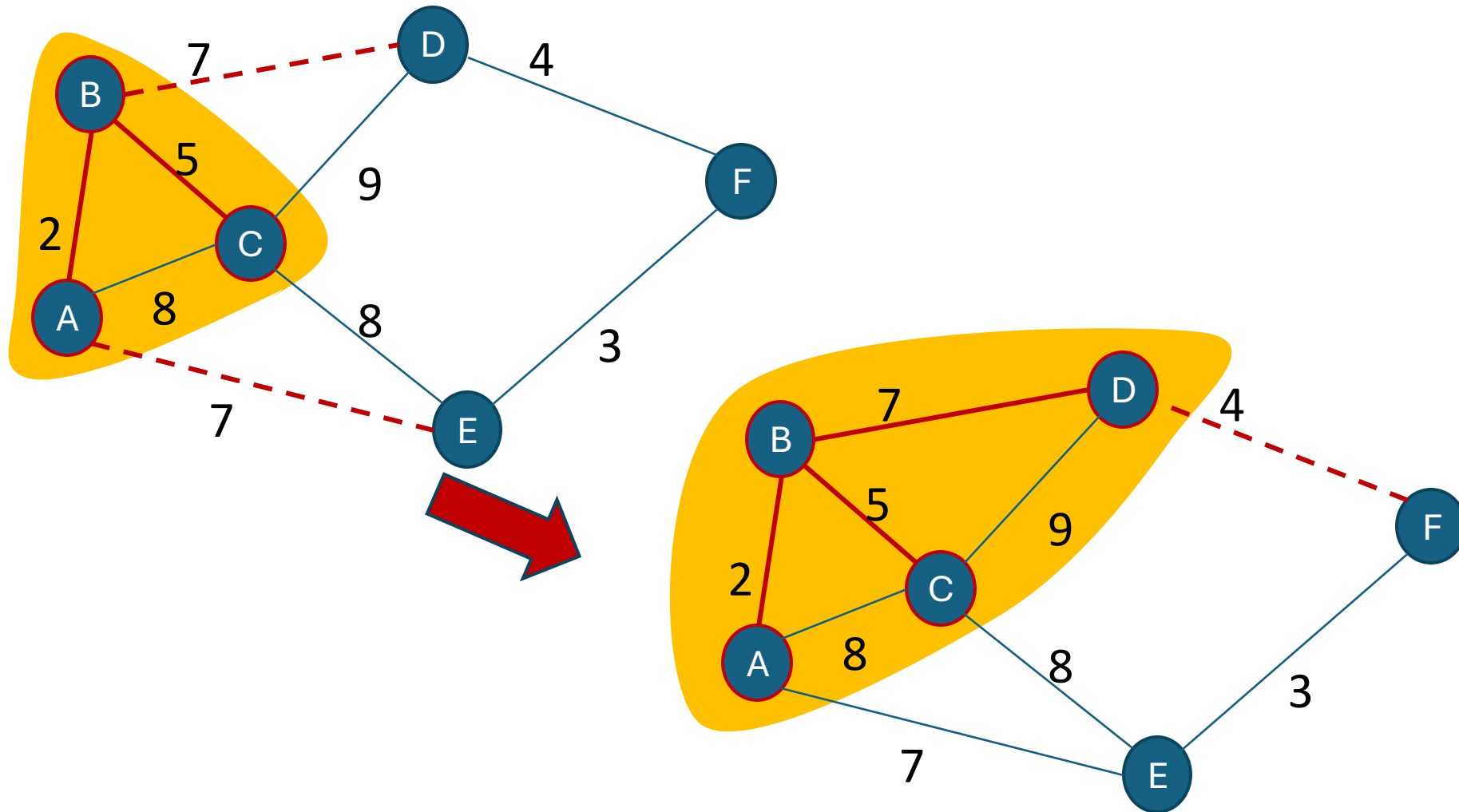
- Minimum spanning tree



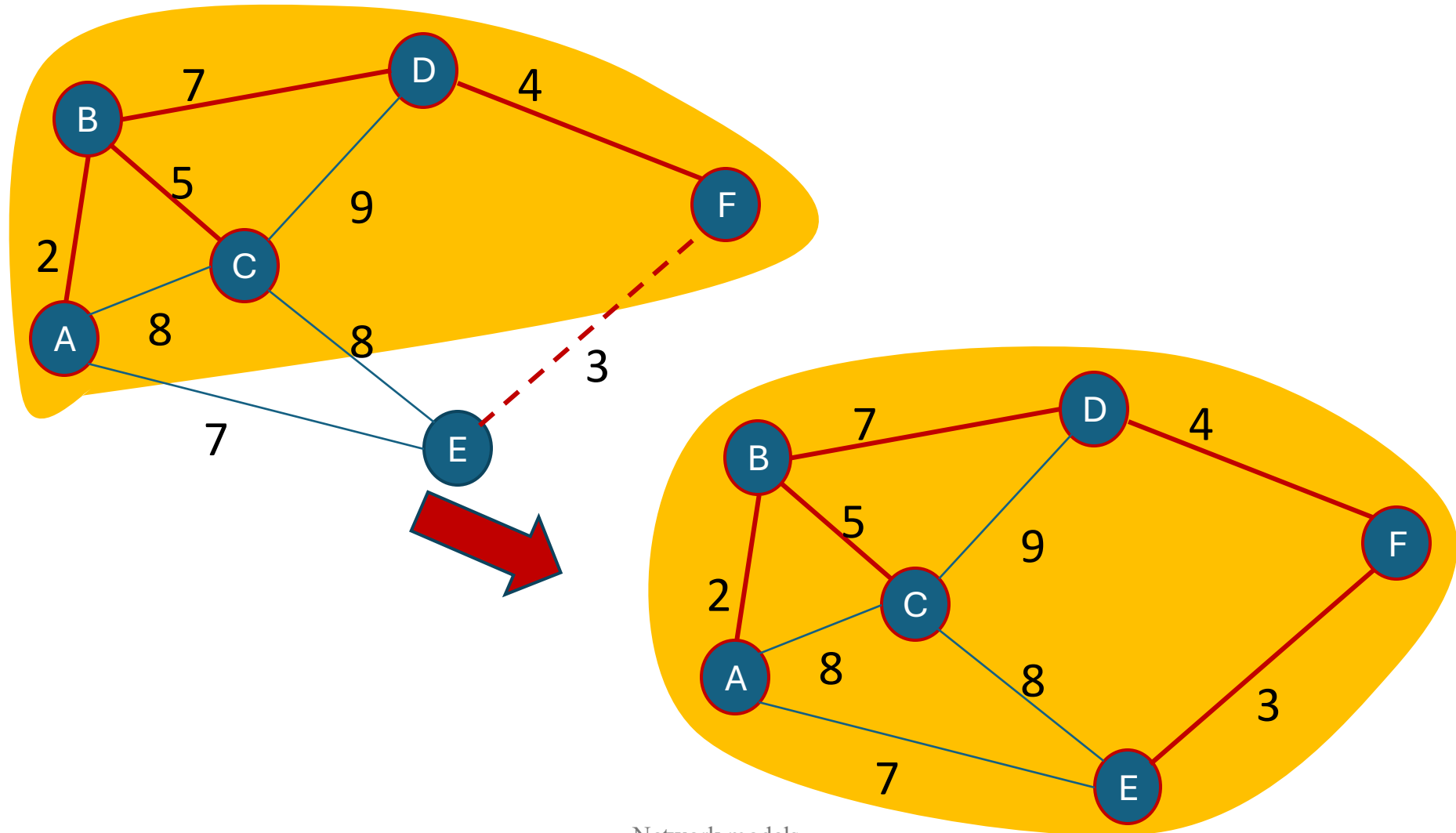
Solution



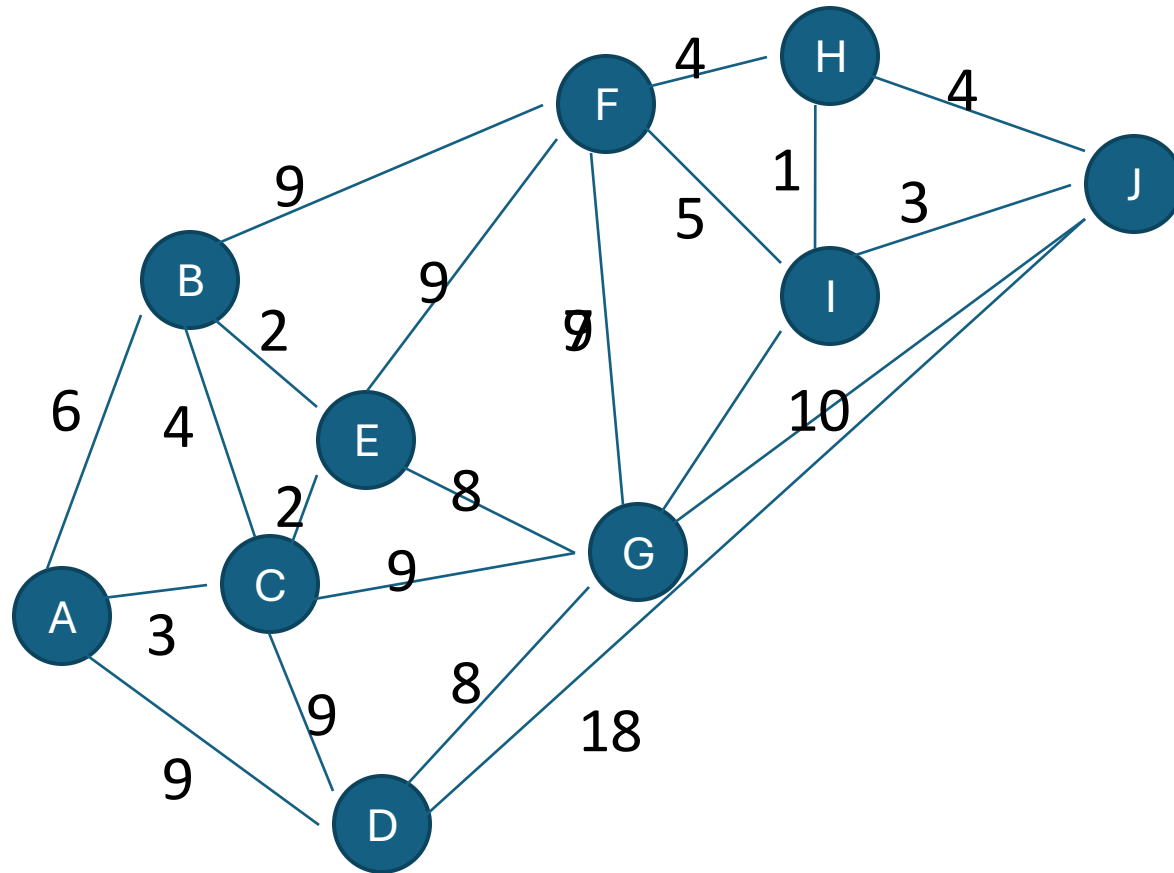
Solution (cont.)



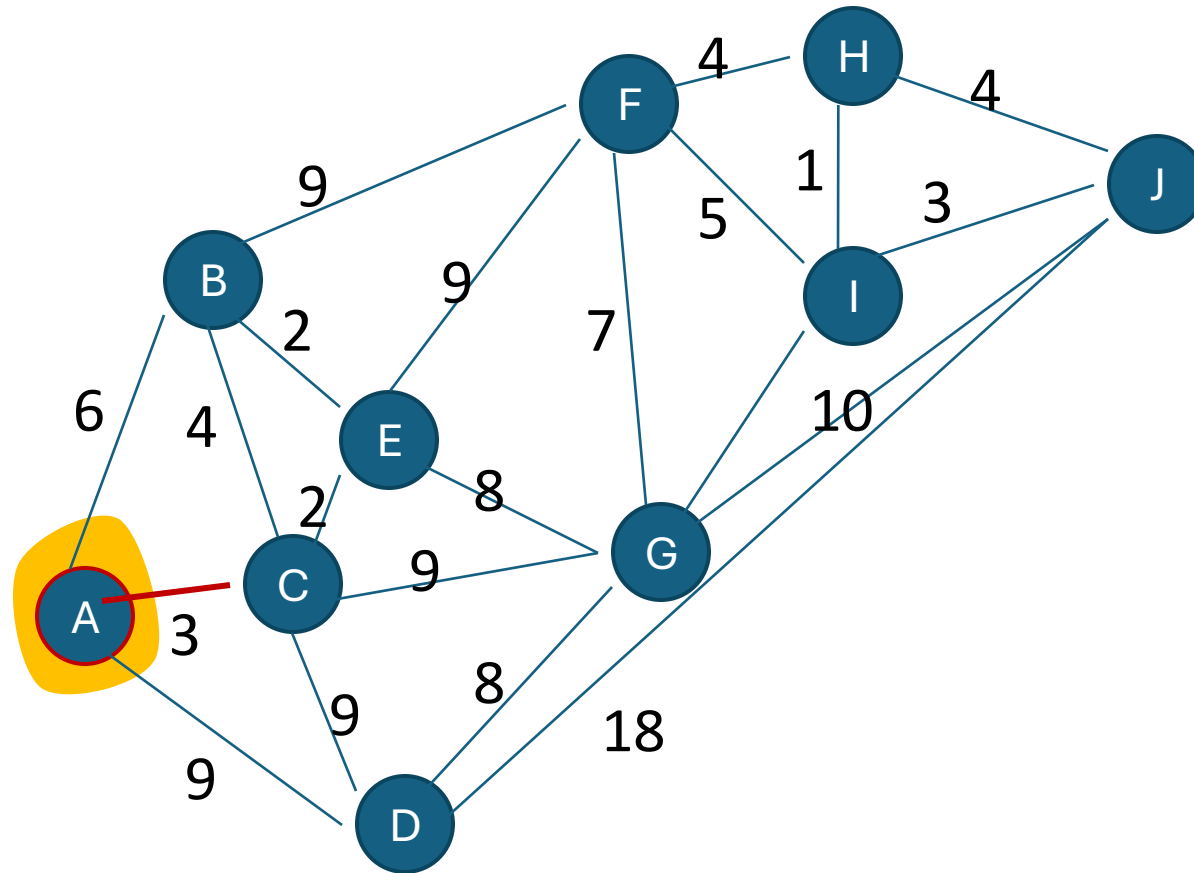
Solution (cont.)



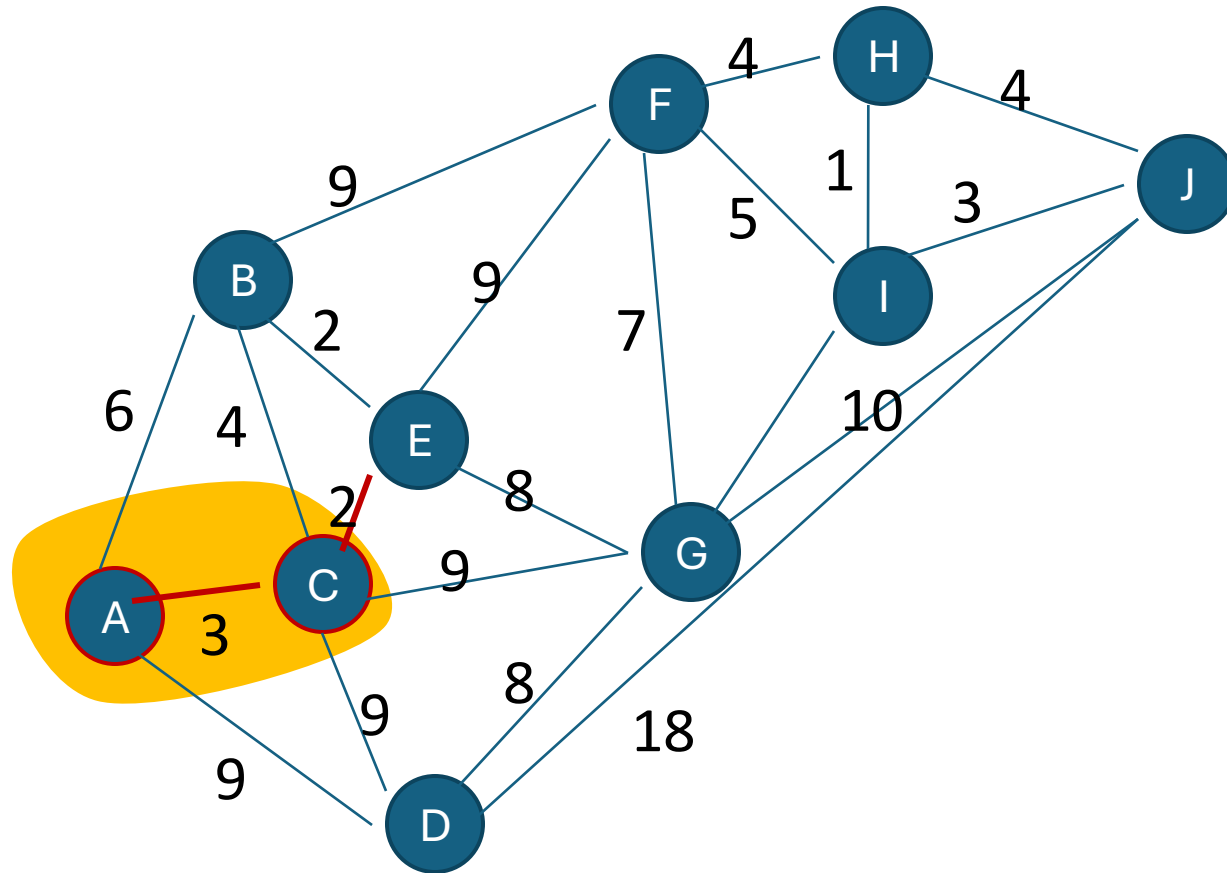
Example 3



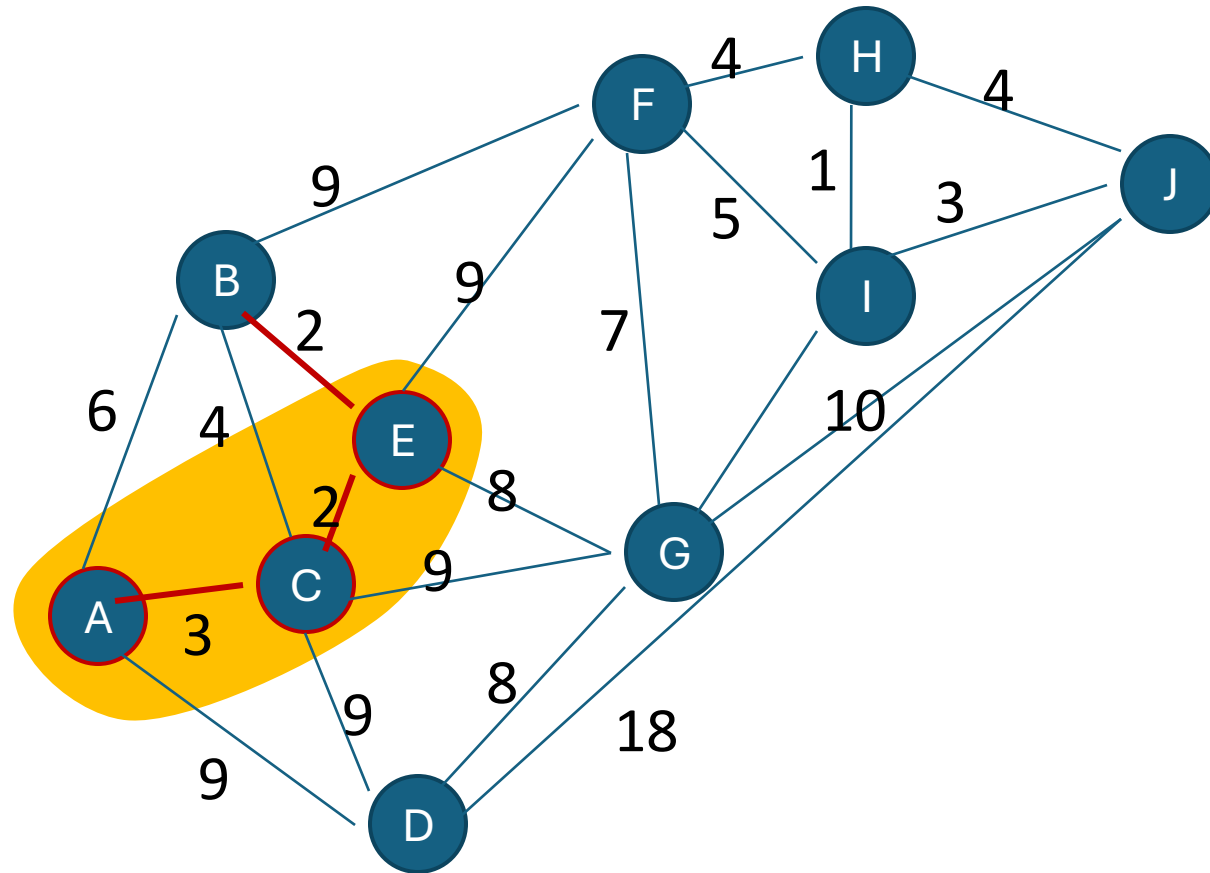
Solution



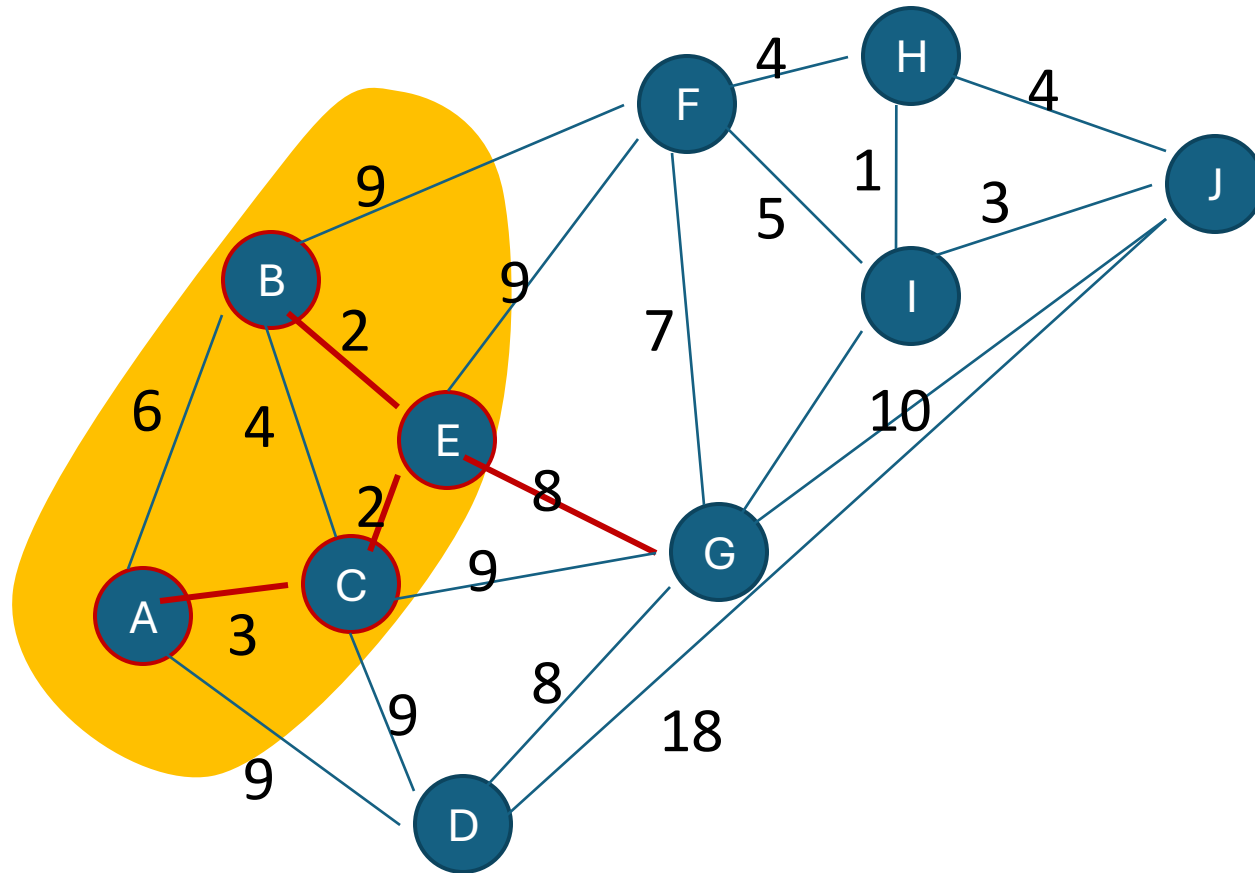
Solution (cont.)



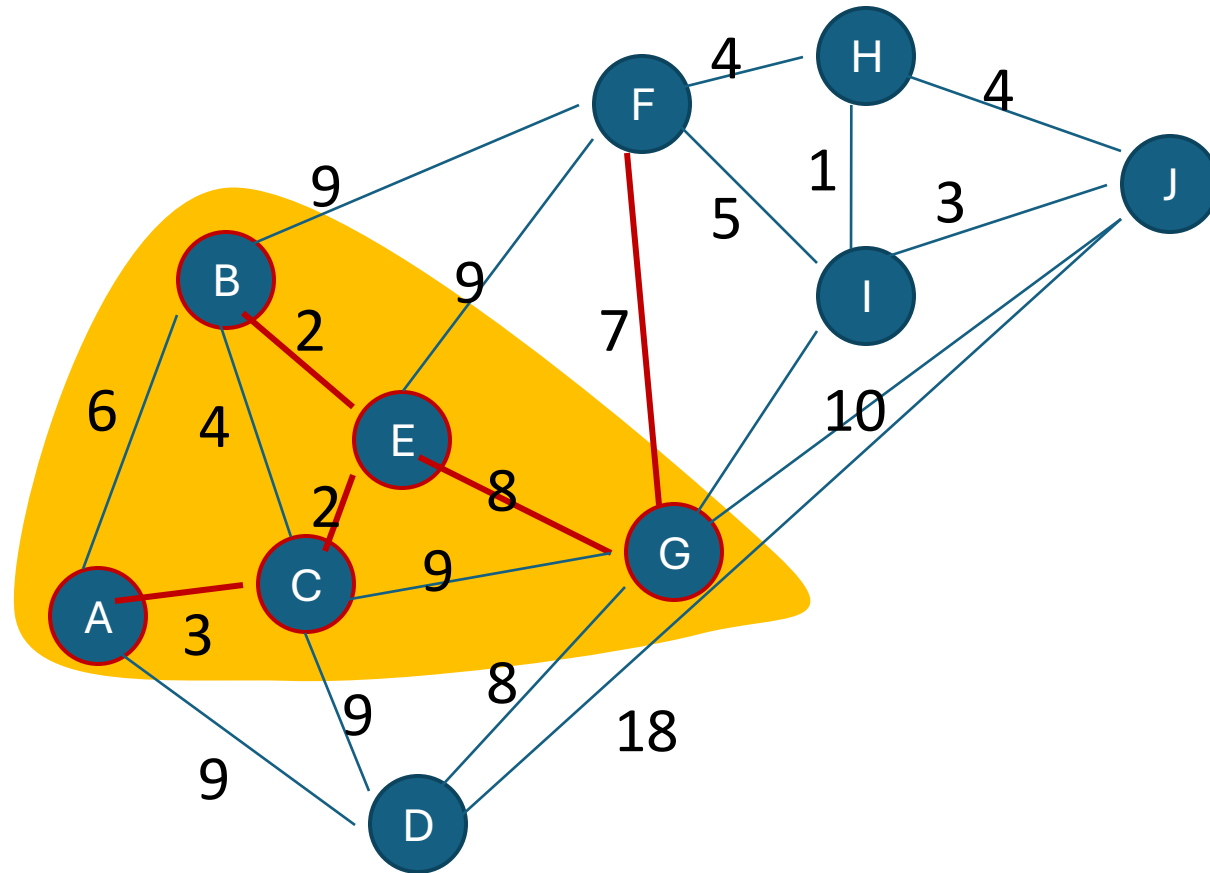
Solution (cont.)



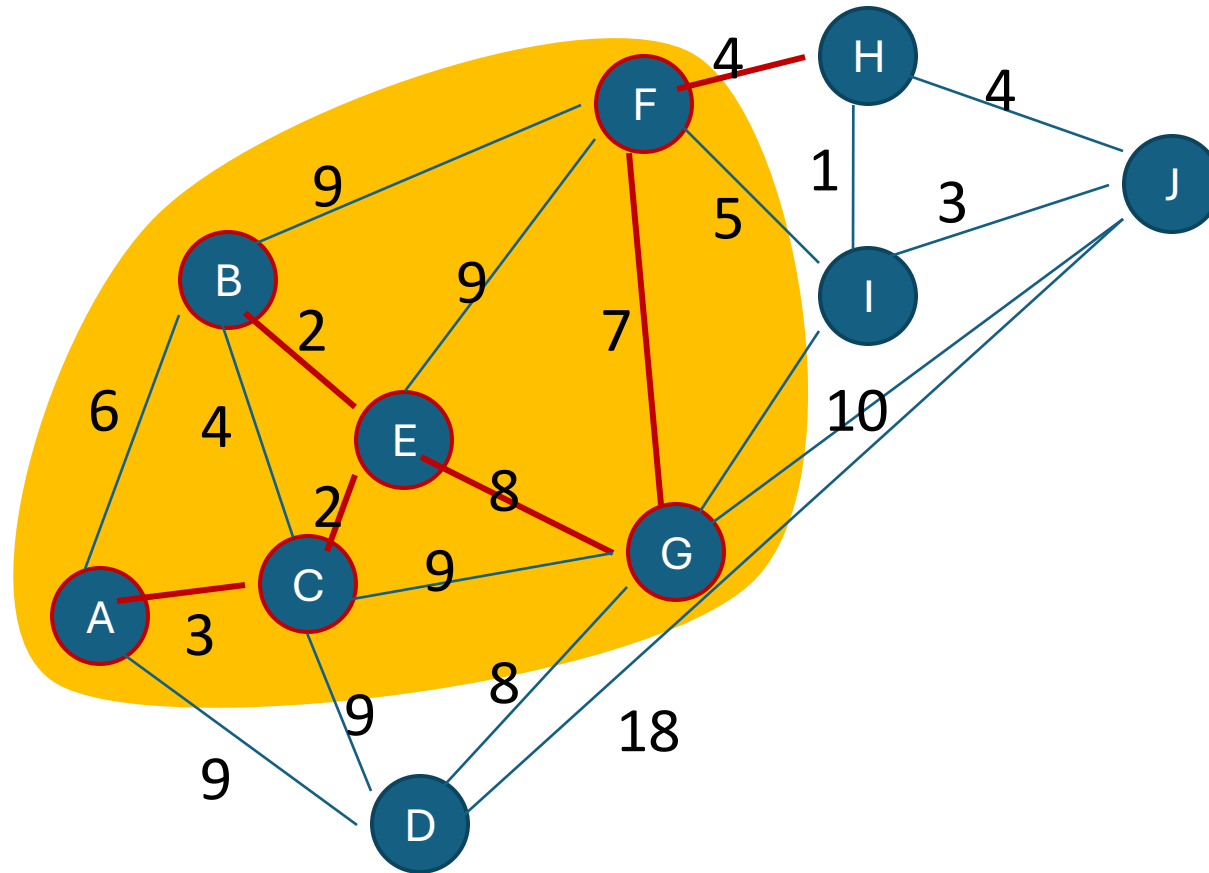
Solution (cont.)



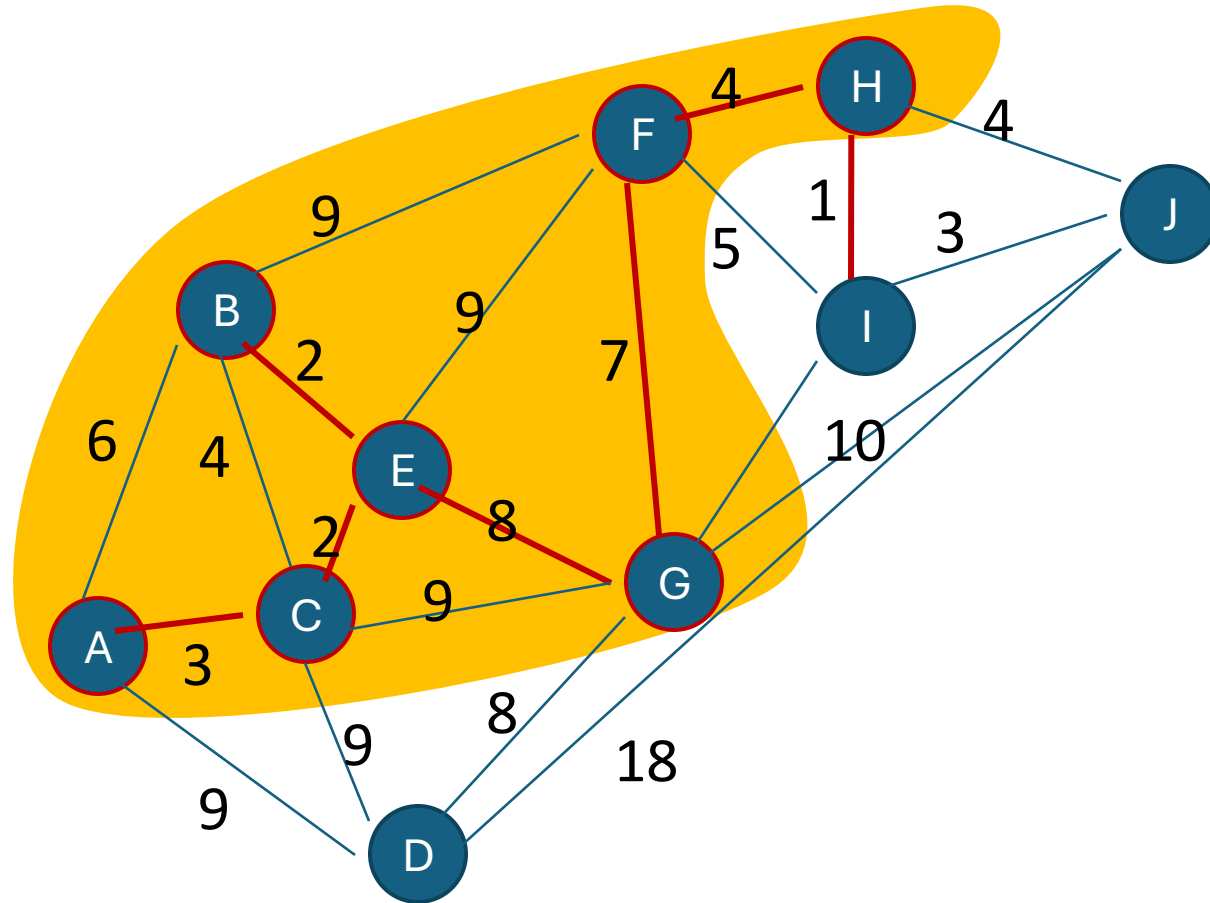
Solution (cont.)



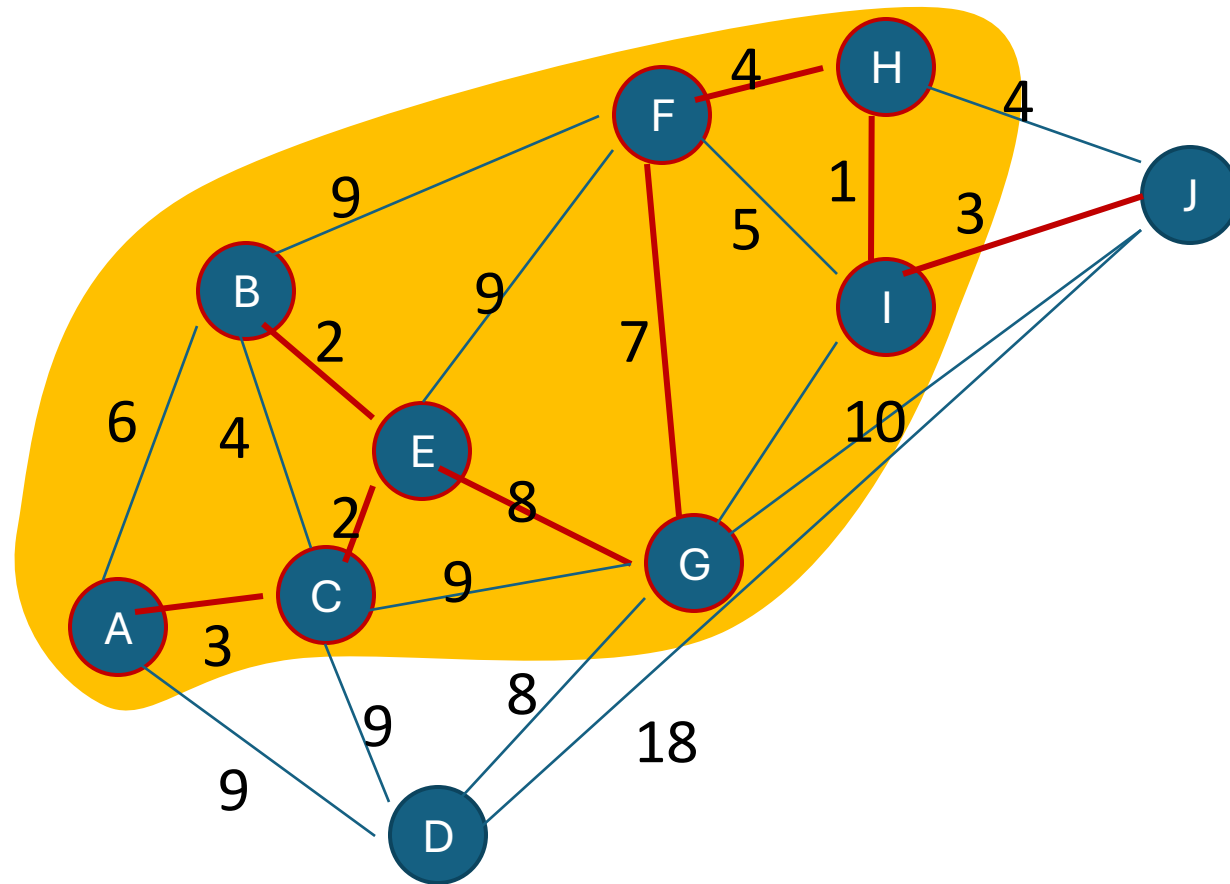
Solution (cont.)



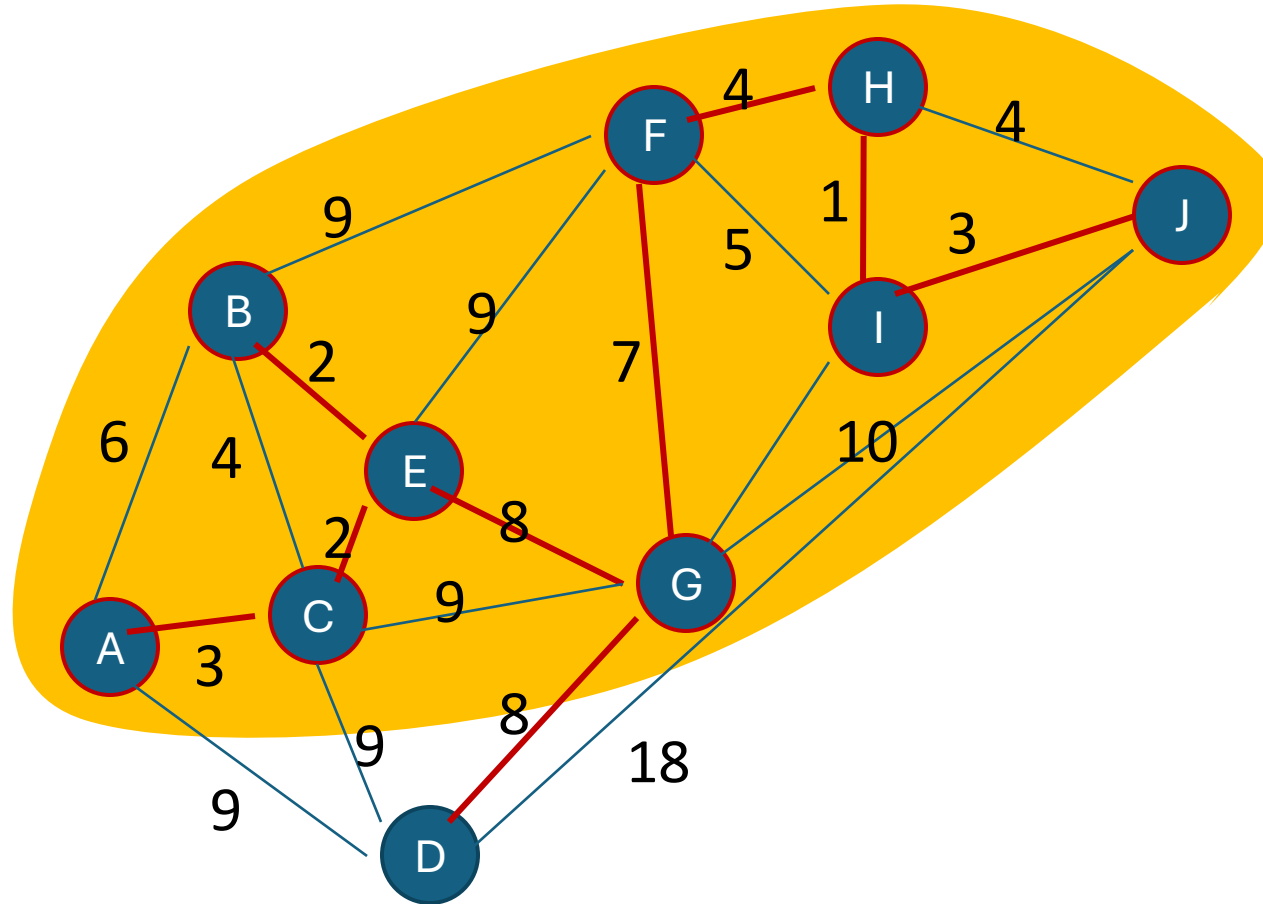
Solution (cont.)



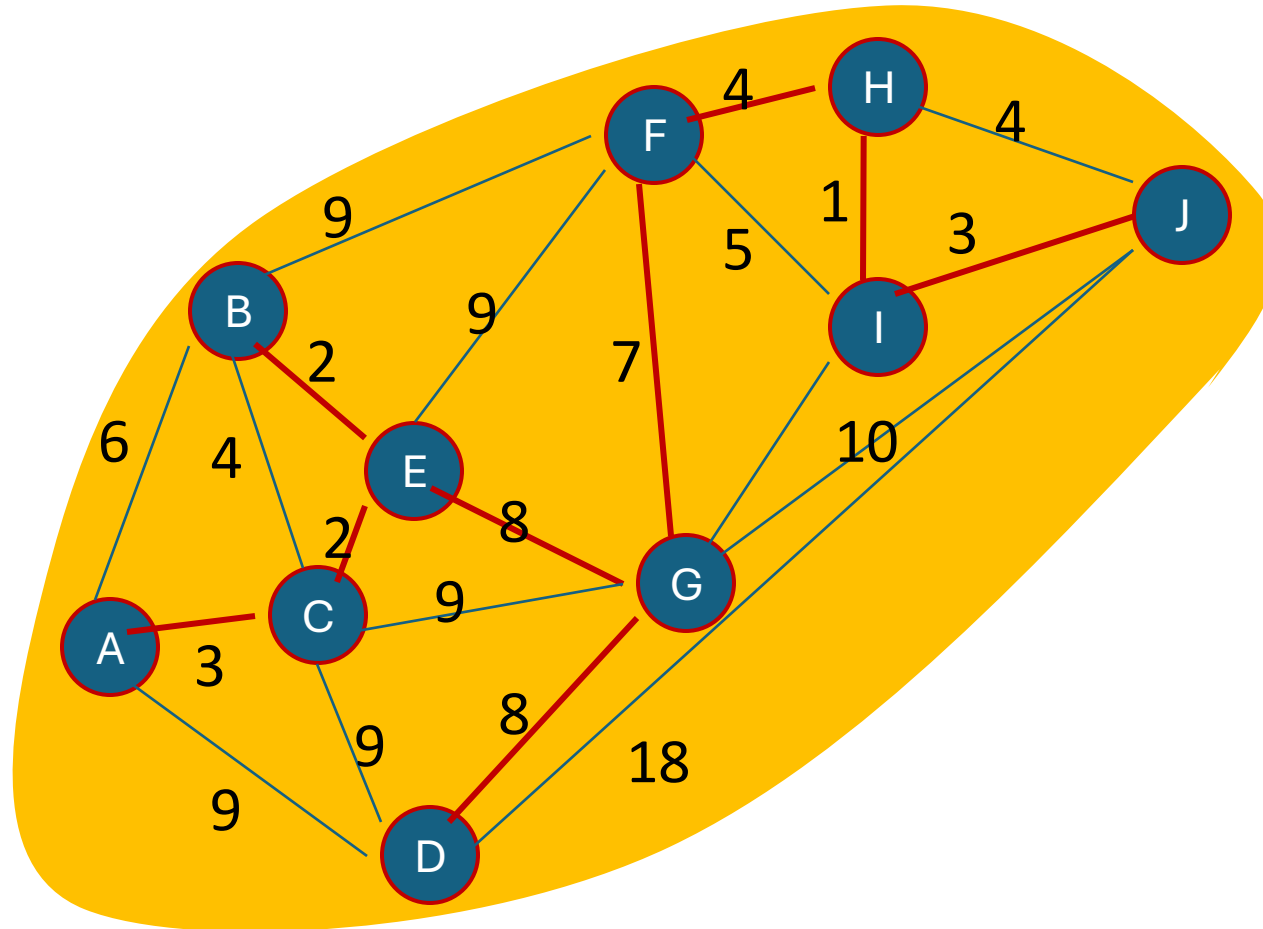
Solution (cont.)



Solution (cont.)



Solution (cont.)



Shortest - Route problem

- The shortest route problem determines the shortest route between a source and destination.
- There are two algorithms to solve shortest-route problems:
- 1- **Dijkstra's algorithm** that design to determine the shortest routes between the source node every other node in the network
- 2- **Floyd's algorithms** is general because it allow the determination of the shortest route between any two node in network

Dijkstra's algorithm

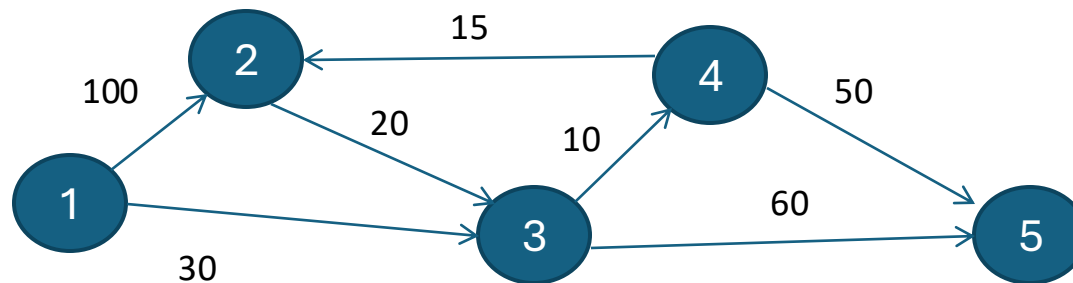
Step0: label the source node(node1) with the permanent label $[0, -]$. Set $i=1$

Step i = (a) compute the temporary labels $[u_i + d_{ij}, i]$ for each node j that can be reached through node i . provided j is not permanently label. If node j is already label with $[u_j, k]$ through another node k and if $u_i + d_{ij} < u_j$, replace $[u_j, k]$ with $[u_i + d_{ij}, i]$

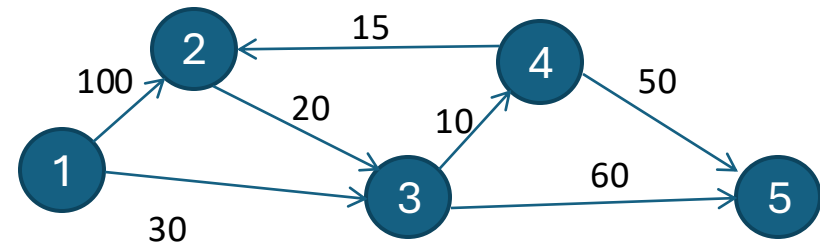
(b) if all node have permanent label stop. Otherwise select the label $[U_r, s]$ having the shortest distance ($=u_i$) among all temporary label. Set $i=r$ and repeat step i

Example

- The figure gives the route and their length in miles between city 1 and four other cities. Determine the shortest route between city 1 and each of the remaining four cities.



Example(cont.)

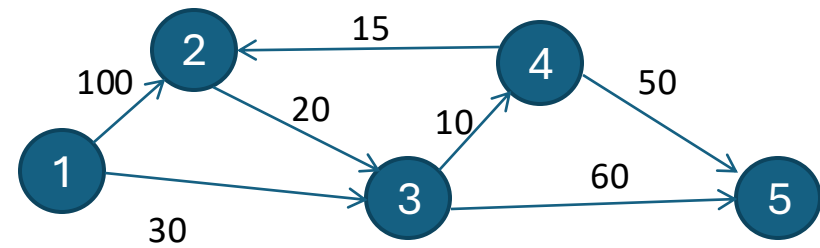


- Iteration 0: assign permanent label $[0, \infty]$ to node 1
- Iteration 1: node 2 and 3 can be reached from (the last permanent labeled) node 1 thus the list labeled node (temporary and permanent) becomes

Node	label	status
1	$[0, \infty]$	permanent
2	$[0+100, 1]$	temporary
3	$[0+30, 1]$	temporary

- For both two temporary label $[100, 1]$ and $[30, 1]$ **node 3** is smallest distance so, status of node 3 is changed to permanent

Example(cont.)

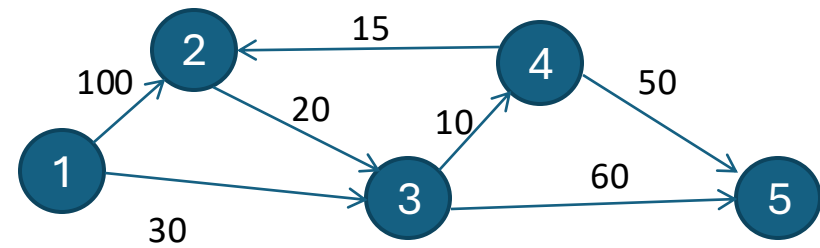


- Iteration2: node 4, and 5 can be reached from node 3 and the list labeled node becomes:

Node	label	status
1	[0,--]	permanent
2	[100, 1]	temporary
3	[30,1]	Permanent
4	[30+10,3]=[40,3]	temporary
5	[30+60,3]=[90,3]	temporary

- **node 4** is smallest distance so from the temporaries list. so, status of node 4 is changed to permanent

Example(cont.)



- Iteration 3: node 2 and 5 can be reached from node 4. the list of labeled is updated as

Node	label	status
1	[0,--]	permanent
2	[40+12,4]=[55,4]	temporary
3	[30,1]	Permanent
4	[40,3]	Permanent
5	[90,3] or [40+50,4]	temporary

- Node 2** is permanent

Example(cont.)

- Iteration 4: only node 3 can be reached from node 2, the node 3 is permanent, so the new list remain the same

Node	label	status
1	[0,--]	permanent
2	[55,4]	permanent
3	[30,1]	Permanent
4	[40,3]	Permanent
5	[90,3] or [40+50,4]	temporary

- Because node 5 is not lead to other node, it is status will convert to permanent

Example(cont.)

- The process ends

Node	label	status
1	[0,--]	permanent
2	[55,4]	permanent
3	[30,1]	Permanent
4	[40,3]	Permanent
5	[90,3] or [40+50,4]	Permanent

- The shortest route between node1 and node2 is:
- (2) [55,4]→(4) [40,3]→(3) [30,1]→(1)
- So, the desired route is 1→3→4→2 with total length 55 miles

Floyd's algorithm

- Step0: define starting distance matrix D_0 and node sequence matrix S_0 . the diagonal elements are marked with(-). Set $k=1$
- General step k : define row k and column as pivot row and pivot column. Apply the *triple operation* to each element d_{ij} in D_{k-1} . if the condition:

$$D_{ik} + d_{kj} < d_{ij}$$

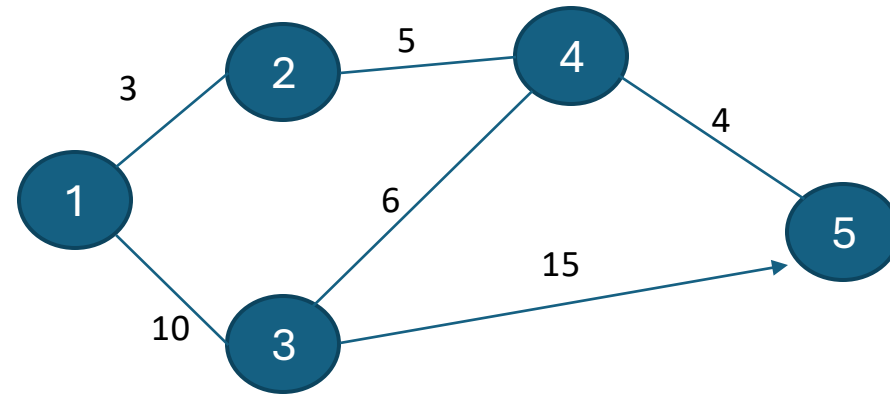
Is satisfied, make the following changes:

- (a) create D_k by replacing d_{ij} in D_{k-1} with $d_{ik} + d_{kj}$
- (b) create S_k by replacing s_{ij} in s_{k-1} with k . set $k=k+1$ and repeat step k .

Floyd's algorithm (cont.)

- Step k : if the sum elements on the pivot row and povot coumn is smaller than associated intersection elements, then it is optimal to replace the intersection distance by the sum of pivot distance.
- After n step, it can determine the shortest route by using the following rules:
 - 1- from D dij gives the shortest distance
 - 2- from S determine the intermediate node.

Example



D0

	1	2	3	4	5
1	-	3	10	∞	∞
2	3	-	∞	5	∞
3	10	∞	-	6	15
4	∞	5	6	-	4
5	∞	∞	∞	4	-

s0

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	3	4	5
3	1	2	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

Example (cont.)

- **K=1**
- We highlight the **first** column and **first** row of the Distance matrix and compare all other items with the sum of the items highlighted in the same row and column.
- If the sum is less than the item then it should be replaced with the sum.

	1	2	3	4	5
1	-	3	10	∞	∞
2	3	-	∞	5	∞
3	10	∞	-	6	15
4	∞	5	6	-	4
5	∞	∞	∞	4	-

Example (cont.)

D0

	1	2	3	4	5
1	-	3	10	∞	∞
2	3	-	∞	5	∞
3	10	∞	-	6	15
4	∞	5	6	-	4
5	∞	∞	∞	4	-

S0

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	3	4	5
3	1	2	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

- When (-) is involved, we leave the item.

Example (cont.)

D0

	1	2	3	4	5
1	-	3	10	∞	∞
2	3	-	∞	5	∞
3	10	∞	-	6	15
4	∞	5	6	-	4
5	∞	∞	∞	4	-

S0

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	3	4	5
3	1	2	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

- $10+3=13$ is less than ∞

So change

Example (cont.)

D0

	1	2	3	4	5
1	-	3	10	∞	∞
2	3	-	13	5	∞
3	10	∞	-	6	15
4	∞	5	6	-	4
5	∞	∞	∞	4	-

S0

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	3	4	5
3	1	2	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

- $10+3=13$ is less than ∞

So change

Example (cont.)

D0

	1	2	3	4	5
1	-	3	10	∞	∞
2	3	-	13	5	∞
3	10	∞	-	6	15
4	∞	5	6	-	4
5	∞	∞	∞	4	-

S0

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	3	4	5
3	1	2	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

- When ∞ is involved we leave the item.

Example (cont.)

D0

	1	2	3	4	5
1	-	3	10	∞	∞
2	3	-	13	5	∞
3	10	∞	-	6	15
4	∞	5	6	-	4
5	∞	∞	∞	4	-

S0

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	3	4	5
3	1	2	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

- When ∞ is involved we leave the item.

Example (cont.)

D0

	1	2	3	4	5
1	-	3	10	∞	∞
2	3	-	13	5	∞
3	10	∞	-	6	15
4	∞	5	6	-	4
5	∞	∞	∞	4	-

S0

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	3	4	5
3	1	2	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

- $3+10=13$ less than ∞ ,
So change

Example (cont.)

D0

	1	2	3	4	5
1	-	3	10	∞	∞
2	3	-	13	5	∞
3	10	13	-	6	15
4	∞	5	6	-	4
5	∞	∞	∞	4	-

S0

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	3	4	5
3	1	2	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

- When (-) is involved we leave the item.

Example (cont.)

D0

	1	2	3	4	5
1	-	3	10	∞	∞
2	3	-	13	5	∞
3	10	13	-	6	15
4	∞	5	6	-	4
5	∞	∞	∞	4	-

S0

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	3	4	5
3	1	2	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

- When ∞ is involved we leave the item.

Example (cont.)

D0

	1	2	3	4	5
1	-	3	10	∞	∞
2	3	-	13	5	∞
3	10	13	-	6	15
4	∞	5	6	-	4
5	∞	∞	∞	4	-

S0

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	3	4	5
3	1	2	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

When ∞ is involved we leave the item.

Example (cont.)

D0

	1	2	3	4	5
1	-	3	10	∞	∞
2	3	-	13	5	∞
3	10	13	-	6	15
4	∞	5	6	-	4
5	∞	∞	∞	4	-

S0

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	3	4	5
3	1	2	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

- When ∞ is involved we leave the item.

Example (cont.)

D0

	1	2	3	4	5
1	-	3	10	∞	∞
2	3	-	13	5	∞
3	10	13	-	6	15
4	∞	5	6	-	4
5	∞	∞	∞	4	-

S0

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	3	4	5
3	1	2	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

- When ∞ is involved we leave the item.

Example (cont.)

D0

	1	2	3	4	5
1	-	3	10	∞	∞
2	3	-	13	5	∞
3	10	13	-	6	15
4	∞	5	6	-	4
5	∞	∞	∞	4	-

S0

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	3	4	5
3	1	2	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

- When (-) is involved we leave the item.

Example (cont.)

D0

	1	2	3	4	5
1	-	3	10	∞	∞
2	3	-	13	5	∞
3	10	13	-	6	15
4	∞	5	6	-	4
5	∞	∞	∞	4	-

S0

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	3	4	5
3	1	2	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

- When ∞ is involved we leave the item.

Example (cont.)

D0

	1	2	3	4	5
1	-	3	10	∞	∞
2	3	-	13	5	∞
3	10	13	-	6	15
4	∞	5	6	-	4
5	∞	∞	∞	4	-

S0

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	3	4	5
3	1	2	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

- When ∞ is involved we leave the item.

Example (cont.)

D0

	1	2	3	4	5
1	-	3	10	∞	∞
2	3	-	13	5	∞
3	10	13	-	6	15
4	∞	5	6	-	4
5	∞	∞	∞	4	-

S0

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	3	4	5
3	1	2	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

- When ∞ is involved we leave the item.

Example (cont.)

D0

	1	2	3	4	5
1	-	3	10	∞	∞
2	3	-	13	5	∞
3	10	13	-	6	15
4	∞	5	6	-	4
5	∞	∞	∞	4	-

S0

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	3	4	5
3	1	2	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

- When ∞ is involved we leave the item.

Example (cont.)

D0

	1	2	3	4	5
1	-	3	10	∞	∞
2	3	-	13	5	∞
3	10	13	-	6	15
4	∞	5	6	-	4
5	∞	∞	∞	4	-

S0

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	3	4	5
3	1	2	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

- When ∞ is involved we leave the item.

Example (cont.)

	1	2	3	4	5
1	-	3	10	∞	∞
2	3	-	13	5	∞
3	10	13	-	6	15
4	∞	5	6	-	4
5	∞	∞	∞	4	-

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	1	4	5
3	1	1	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

Example (cont.)

- We have now completed one iteration. We rename the new matrices:

D1

	1	2	3	4	5
1	-	3	10	∞	∞
2	3	-	13	5	∞
3	10	13	-	6	15
4	∞	5	6	-	4
5	∞	∞	∞	4	-

S1

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	1	4	5
3	1	1	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

Example (cont.)

- Set $k=2$
- We highlight the **second** column and **second** row of the Distance matrix and compare all other items with the sum of the items highlighted in the same row and column.
- If the sum is less than the item then it should be replaced with the sum.

Example (cont.)

D1

	1	2	3	4	5
1	-	3	10	∞	∞
2	3	-	13	5	∞
3	10	13	-	6	15
4	∞	5	6	-	4
5	∞	∞	∞	4	-

S1

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	1	4	5
3	1	1	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

- When (-) is involved we leave the item.

Example (cont.)

D1

	1	2	3	4	5
1	-	3	10	∞	∞
2	3	-	13	5	∞
3	10	13	-	6	15
4	∞	5	6	-	4
5	∞	∞	∞	4	-

S1

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	1	4	5
3	1	1	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

- $3+13=16$ Not Less than 10

Example (cont.)

D1

	1	2	3	4	5
1	-	3	10	∞	∞
2	3	-	13	5	∞
3	10	13	-	6	15
4	∞	5	6	-	4
5	∞	∞	∞	4	-

S1

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	1	4	5
3	1	1	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

- $3+5=8$ less than ∞

So change

Example (cont.)

D1

	1	2	3	4	5
1	-	3	10	8	∞
2	3	-	13	5	∞
3	10	13	-	6	15
4	∞	5	6	-	4
5	∞	∞	∞	4	-

S1

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	1	4	5
3	1	1	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

- $3+5=8$ less than ∞

So change

Example (cont.)

D1

	1	2	3	4	5
1	-	3	10	8	∞
2	3	-	13	5	∞
3	10	13	-	6	15
4	∞	5	6	-	4
5	∞	∞	∞	4	-

S1

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	1	4	5
3	1	1	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

- When ∞ is involved we leave the item.

Example (cont.)

D1

	1	2	3	4	5
1	-	3	10	8	∞
2	3	-	13	5	∞
3	10	13	-	6	15
4	∞	5	6	-	4
5	∞	∞	∞	4	-

S1

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	1	4	5
3	1	1	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

- $3+13=16$ Not less than 10

Example (cont.)

D1

	1	2	3	4	5
1	-	3	10	8	∞
2	3	-	13	5	∞
3	10	13	-	6	15
4	∞	5	6	-	4
5	∞	∞	∞	4	-

S1

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	1	4	5
3	1	1	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

- When (-) is involved we leave the item.

Example (cont.)

D1

	1	2	3	4	5
1	-	3	10	8	∞
2	3	-	13	5	∞
3	10	13	-	6	15
4	∞	5	6	-	4
5	∞	∞	∞	4	-

S1

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	1	4	5
3	1	1	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

- $5+13=18$ Not less than 6

Example (cont.)

D1

	1	2	3	4	5
1	-	3	10	8	∞
2	3	-	13	5	∞
3	10	13	-	6	15
4	∞	5	6	-	4
5	∞	∞	∞	4	-

S1

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	1	4	5
3	1	1	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

- When ∞ is involved we leave the item.

Example (cont.)

D1

	1	2	3	4	5
1	-	3	10	8	∞
2	3	-	13	5	∞
3	10	13	-	6	15
4	∞	5	6	-	4
5	∞	∞	∞	4	-

S1

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	1	4	5
3	1	1	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

- $3+5=8$ less than ∞

So change

Example (cont.)

D1

	1	2	3	4	5
1	-	3	10	8	∞
2	3	-	13	5	∞
3	10	13	-	6	15
4	8	5	6	-	4
5	∞	∞	∞	4	-

S1

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	1	4	5
3	1	1	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

- $3+5=8$ less than ∞

So change

Example (cont.)

D1

	1	2	3	4	5
1	-	3	10	8	∞
2	3	-	13	5	∞
3	10	13	-	6	15
4	8	5	6	-	4
5	∞	∞	∞	4	-

S1

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	1	4	5
3	1	1	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

- $13+5=18$ Not less than 6

Example (cont.)

D1

	1	2	3	4	5
1	-	3	10	8	∞
2	3	-	13	5	∞
3	10	13	-	6	15
4	8	5	6	-	4
5	∞	∞	∞	4	-

S1

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	1	4	5
3	1	1	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

- When (-) is involved we leave the item.

Example (cont.)

D1

	1	2	3	4	5
1	-	3	10	8	∞
2	3	-	13	5	∞
3	10	13	-	6	15
4	8	5	6	-	4
5	∞	∞	∞	4	-

S1

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	1	4	5
3	1	1	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

- When ∞ is involved we leave the item.

Example (cont.)

D1

	1	2	3	4	5
1	-	3	10	8	∞
2	3	-	13	5	∞
3	10	13	-	6	15
4	8	5	6	-	4
5	∞	∞	∞	4	-

S1

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	1	4	5
3	1	1	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

- When ∞ is involved we leave the item.

Example (cont.)

D1

	1	2	3	4	5
1	-	3	10	8	∞
2	3	-	13	5	∞
3	10	13	-	6	15
4	8	5	6	-	4
5	∞	∞	∞	4	-

S1

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	1	4	5
3	1	1	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

- When ∞ is involved we leave the item.

Example (cont.)

D1

	1	2	3	4	5
1	-	3	10	8	∞
2	3	-	13	5	∞
3	10	13	-	6	15
4	8	5	6	-	4
5	∞	∞	∞	4	-

S1

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	1	4	5
3	1	1	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

- When ∞ is involved we leave the item.

Example (cont.)

D1

	1	2	3	4	5
1	-	3	10	8	∞
2	3	-	13	5	∞
3	10	13	-	6	15
4	8	5	6	-	4
5	∞	∞	∞	4	-

S1

	1	2	3	4	5
1	-	2	3	4	5
2	1	-	1	4	5
3	1	1	-	4	5
4	1	2	3	-	5
5	1	2	3	4	-

- When ∞ is involved we leave the item.

Example (cont.)

D1

	1	2	3	4	5
1	-	3	10	8	∞
2	3	-	13	5	∞
3	10	13	-	6	15
4	8	5	6	-	4
5	∞	∞	∞	4	-

S1

	1	2	3	4	5
1	-	2	3	2	5
2	1	-	1	4	5
3	1	1	-	4	5
4	2	2	3	-	5
5	1	2	3	4	-

Example (cont.)

- We have now completed two iteration. We rename the new matrices:

D2

	1	2	3	4	5
1	-	3	10	8	∞
2	3	-	13	5	∞
3	10	13	-	6	15
4	8	5	6	-	4
5	∞	∞	∞	4	-

S2

	1	2	3	4	5
1	-	2	3	2	5
2	1	-	1	4	5
3	1	1	-	4	5
4	2	2	3	-	5
5	1	2	3	4	-

Example (cont.)

- Set $k=3$
- We highlight the **third** column and **third** row of the Distance matrix and compare all other items with the sum of the items highlighted in the same row and column.
- If the sum is less than the item then it should be replaced with the sum.

Example (cont.)

D2

	1	2	3	4	5
1	-	3	10	8	∞
2	3	-	13	5	∞
3	10	13	-	6	15
4	8	5	6	-	4
5	∞	∞	∞	4	-

S2

	1	2	3	4	5
1	-	2	3	2	5
2	1	-	1	4	5
3	1	1	-	4	5
4	2	2	3	-	5
5	1	2	3	4	-

Example (cont.)

D2

	1	2	3	4	5
1	-	3	10	8	∞
2	3	-	13	5	∞
3	10	13	-	6	15
4	8	5	6	-	4
5	∞	∞	∞	4	-

S2

	1	2	3	4	5
1	-	2	3	2	5
2	1	-	1	4	5
3	1	1	-	4	5
4	2	2	3	-	5
5	1	2	3	4	-

Example (cont.)

D2

	1	2	3	4	5
1	-	3	10	8	25
2	3	-	13	5	28
3	10	13	-	6	15
4	8	5	6	-	4
5	∞	∞	∞	4	-

S2

	1	2	3	4	5
1	-	2	3	2	3
2	1	-	1	4	3
3	1	1	-	4	5
4	2	2	3	-	5
5	1	2	3	4	-

Example (cont.)

- We have now completed third iteration. We rename the new matrices:

D3

	1	2	3	4	5
1	-	3	10	8	25
2	3	-	13	5	28
3	10	13	-	6	15
4	8	5	6	-	4
5	∞	∞	∞	4	-

S3

	1	2	3	4	5
1	-	2	3	2	3
2	1	-	1	4	3
3	1	1	-	4	5
4	2	2	3	-	5
5	1	2	3	4	-

Example (cont.)

- Set $k=4$
- We highlight the **fourth** column and **fourth** row of the Distance matrix and compare all other items with the sum of the items highlighted in the same row and column.
- If the sum is less than the item then it should be replaced with the sum.

Example (cont.)

D3

	1	2	3	4	5
1	-	3	10	8	25
2	3	-	13	5	28
3	10	13	-	6	15
4	8	5	6	-	4
5	∞	∞	∞	4	-

S3

	1	2	3	4	5
1	-	2	3	2	3
2	1	-	1	4	3
3	1	1	-	4	5
4	2	2	3	-	5
5	1	2	3	4	-

Example (cont.)

D3

	1	2	3	4	5
1	-	3	10	8	12
2	3	-	11	5	9
3	10	11	-	6	10
4	8	5	6	-	4
5	12	9	10	4	-

S3

	1	2	3	4	5
1	-	2	3	2	4
2	1	-	4	4	4
3	1	4	-	4	4
4	2	2	3	-	5
5	4	4	4	4	-

Example (cont.)

- We have now completed fourth iteration. We rename the new matrices:

D4

	1	2	3	4	5
1	-	3	10	8	12
2	3	-	11	5	9
3	10	11	-	6	10
4	8	5	6	-	4
5	12	9	10	4	-

S4

	1	2	3	4	5
1	-	2	3	2	4
2	1	-	4	4	4
3	1	4	-	4	4
4	2	2	3	-	5
5	4	4	4	4	-

Example (cont.)

- Set $k=5$
- We highlight the **fifth** column and **fifth** row of the Distance matrix and compare all other items with the sum of the items highlighted in the same row and column.
- If the sum is less than the item then it should be replaced with the sum.

Example (cont.)

D4

	1	2	3	4	5
1	-	3	10	8	12
2	3	-	11	5	9
3	10	11	-	6	10
4	8	5	6	-	4
5	12	9	10	4	-

S4

	1	2	3	4	5
1	-	2	3	2	4
2	1	-	4	4	4
3	1	4	-	4	4
4	2	2	3	-	5
5	4	4	4	4	-

- No further improvement are possible in this iteration, D5,S5 are the same D4 and S4

Example (cont.)

D4

	1	2	3	4	5
1	-	3	10	8	12
2	3	-	11	5	9
3	10	11	-	6	10
4	8	5	6	-	4
5	12	9	10	4	-

S4

	1	2	3	4	5
1	-	2	3	2	4
2	1	-	4	4	4
3	1	4	-	4	4
4	2	2	3	-	5
5	4	4	4	4	-

- Shortest distance is $d_{15} = 12$
- Associated route: recall $\text{segment}(I, j)$ if $S_{ij} = J$ is direct link otherwise they link through intermediate node.

Example (cont.)

	1	2	3	4	5
1	-	3	10	8	12
2	3	-	11	5	9
3	10	11	-	6	10
4	8	5	6	-	4
5	12	9	10	4	-

	1	2	3	4	5
1	-	2	3	2	4
2	1	-	4	4	4
3	1	4	-	4	4
4	2	2	3	-	5
5	4	4	4	4	-

- $S_{15} = 4 \neq 5$ so. The initial link is $1 \rightarrow 4 \rightarrow 5$
- Now, $s_{14} = 2$. is not direct link and $1 \rightarrow 4$ must replaced with $1 \rightarrow 2 \rightarrow 4$, so the road from 1 to 5 will be change to $1 \rightarrow 2 \rightarrow 4 \rightarrow 5$.
- Now $s_{12} = 2$, $s_{24} = 4$, $s_{45} = 5$. the route $1 \rightarrow 2 \rightarrow 4 \rightarrow 5$ need no further dissecting and the process end

Maximal flow algorithm

- In a maximal flow problem, we seek to find the maximum volume of *flow* from a source node to terminal sink node in a capacitated network.
- Maximum flow algorithm is straightforward.

How it works

- In maximum flow algorithm, we determine if there is any path from source to sink that can carry flow.
- If there is , the flow is augmented as much as possible along this path; and residual capacities of the arc used on the path are reduced accordingly.

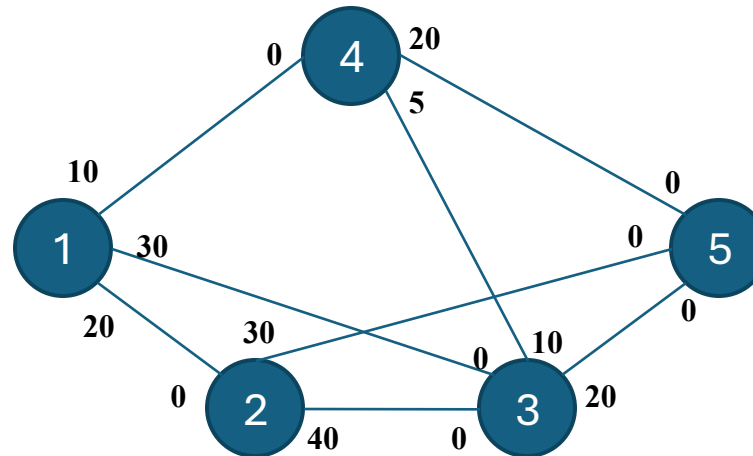
Steps of maximum flow algorithm

- *Step1*: find path from the source to the sink that has positive *residual capacities*. If no path have positive, **STOP**; the maximum flow have been found
- *Step2*: Find the minimum *residual capacity* of the arc on the path (call it K) and augment the *flow* on each involved arc by K
- *Step3*: Adjust the *residual capacities* of arcs on the path by **decreasing** the *residual capacities* in direction of *flow* by **K**; and **increasing** the *residual capacities* in the direction opposite the *flow* by **K**;

GO TO STEP 1

Example

- Determine the maximum flow in the network.



Example (cont.)

- Iteration1:

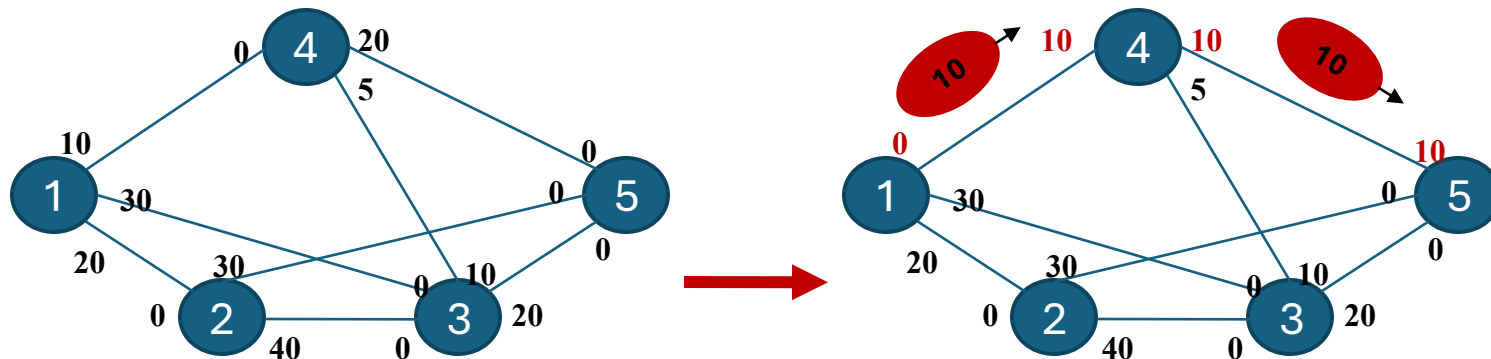
Select Path: $1 \rightarrow 4 \rightarrow 5$

Residual capacities	
1-4	10
2-5	20

Augment **flow** by 10

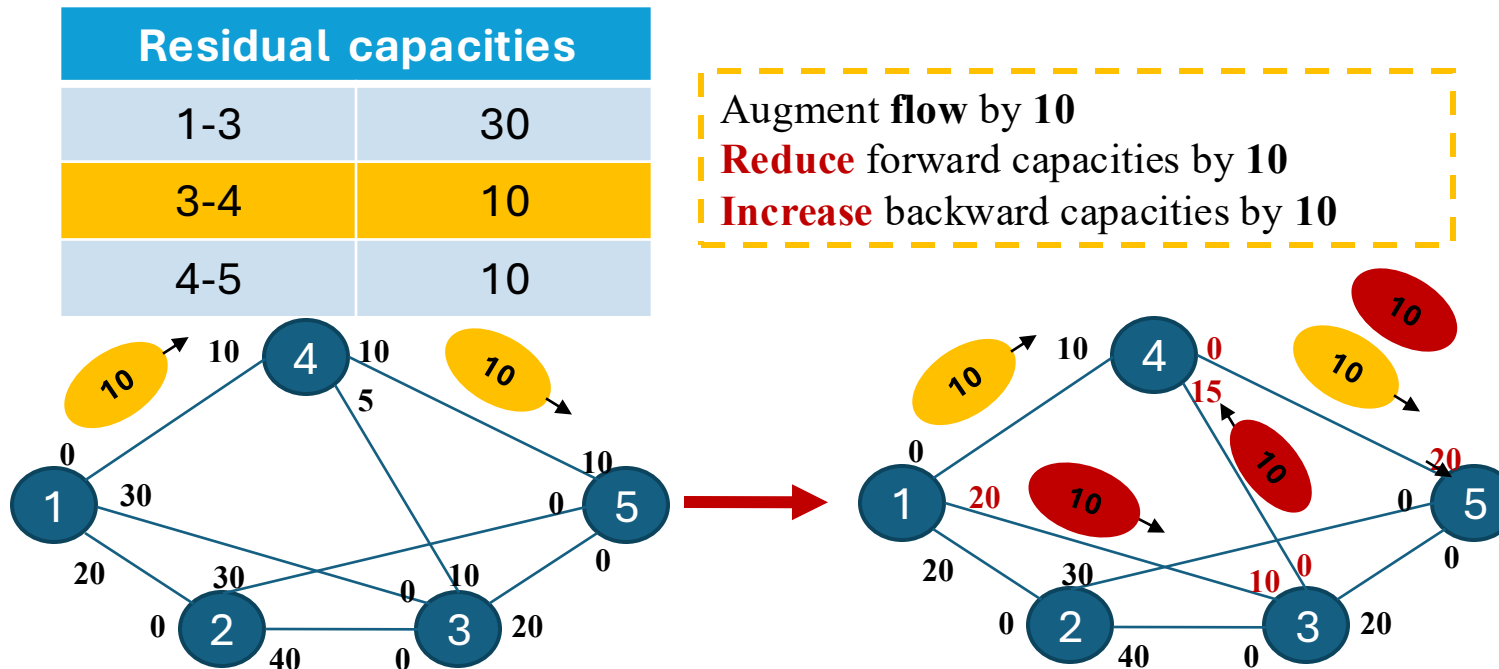
Reduce forward capacities by 10

Increase backward capacities by 10



Example (cont.)

- Iteration 2:
- No additional possible flow along arc(1,4); thus find new path; Select path $1 \rightarrow 3 \rightarrow 4 \rightarrow 5$

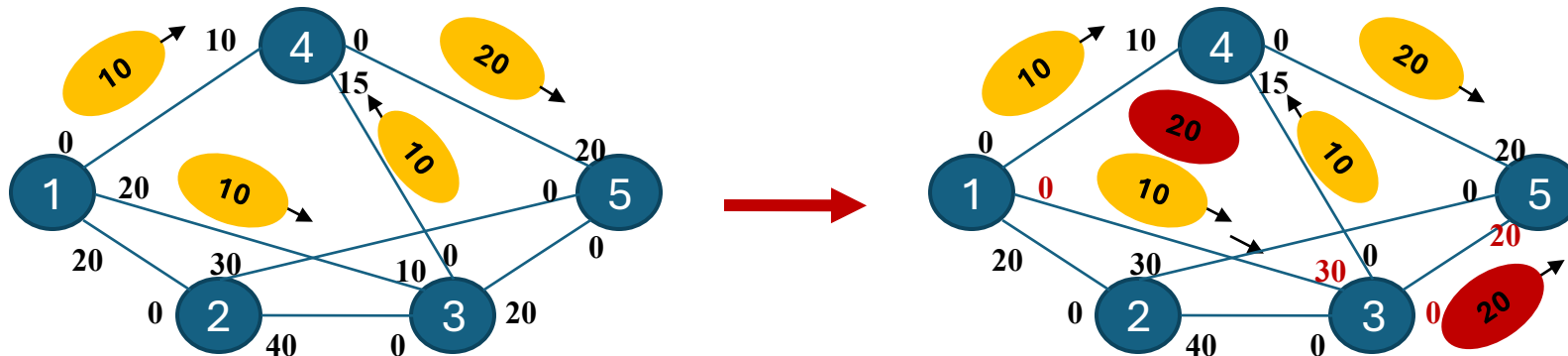


Example (cont.)

- Iteration 3:
- No additional possible flow along arc(3,4) and (4,5); thus find new path; Select path $1 \rightarrow 3 \rightarrow 5$

Residual capacities	
1-3	20
3-5	20

Augment **flow** by 20
Reduce forward capacities by 20
Increase backward capacities by 20

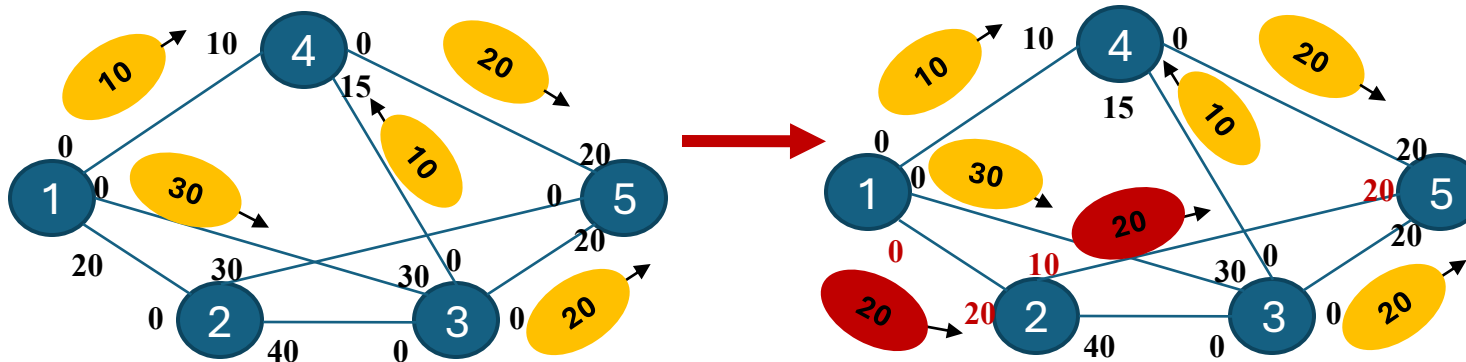


Example (cont.)

- Iteration 4:
- No additional possible flow along arc(1,3) and (3,5); thus find new path; Select path $1 \rightarrow 2 \rightarrow 5$

Residual capacities	
1-2	20
2-5	30

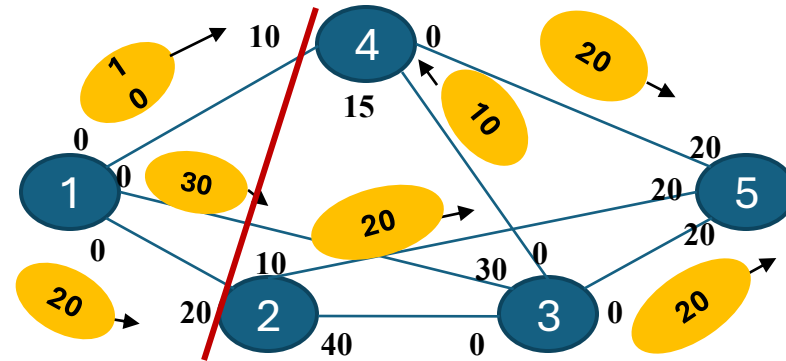
Augment **flow** by 20
Reduce forward capacities by 20
Increase backward capacities by 20



Example (cont.)

- Iteration 4:
- No more flow is possible because there is no residual capacity left on the cut consisting (1,2),(1,3), and (1,4); so maximum flow is $20+30+10=60$.

From	To	Flow
1	2	20
1	3	30
1	4	10
2	5	20
3	4	10
3	5	20
4	5	20



MME8215: Operations Research

Mixed Integer Programming

Contents

1 Combinatorial Optimization	1
1.1 Introduction	1
1.2 Types of techniques	2
1.3 Efficiency of Techniques	3
1.3.1 Classification of problems in complexity theory	4
2 Mixed integer Programming	6
2.1 Introduction	6
2.2 Set up	6
2.3 Examples	7
2.3.1 Knapsack Problem	7
2.3.2 Assignment problems	8
2.3.3 Sequencing Problems	8

Next we cover basic theoretical and practical aspects of Mixed integer Programming (MIP) both theoretical and practical topics related to Continuous Linear Programming (CLP) and Network Theory. Like, Continuous Linear Programming, MIP uses linear equations to model optimization problems. However, some of the decision variables can only take Integer values. Feasible solutions are no longer found at vertices of the feasibility region, and this fundamental fact drastically increases the complexity of MIP problems. Network Theory provides a framework rather useful to solve MIP problems, and therefore it is also introduced in this course. CLP allows us to model, solve and analyse the solution of optimization problems using linear equations. “The world is not linear” but still, CLP is a really powerful tool to solve a plethora of real world optimisation problems.

1 Combinatorial Optimization

1.1 Introduction

Combinatorial optimization gathers different types of optimization problems where the decision maker needs to select an alternative from a finite set of possible alternatives with different outcomes, given a set of requirements (i.e. constraints). Some examples of problems in combinatorial optimization are:

- **Grouping:** Find a subset of members within a set that optimizes an objective function (e.g. minimises costs or maximises profits) given a set of constraints. For instance, select the best starting 11 in a football team given their performance statistics.
- **Ordering:** Find the optimal ordering of the members of a set that maximise or minimise an objective function with some constraints. For instance the optimal sequence of production orders that minimises production time.
- **Assignment:** Assign members of a set to members of another set to optimise an objective function given a set of constraints. For instance, assign tasks to employees at a minimum cost

Combinatorial optimization is strongly connected with set theory and the concept of search, or (iteratively) explore the set to find the optimal solution. Heuristics will explore the set of alternatives looking for the optimal solution.

The following types of problems are outstanding instances of combinatorial optimization problems:

- **Integer Programming:** Any Linear Programming Problems with Integer or binary variables. When this happens, the feasibility region and objective function are no longer convex, and we cannot apply the same principles and methods as with CLP. When we combine Integer, Binary and Continuous variables, we are dealing with Mixed Integer Programming (MIP). MIP covers a wide range of OR problems, which we will be covering extensively in this unit.
- **Travelling salesman problems:** In the classic formulation of the Travelling Salesman Problem (TSP), a salesman needs to visit a set of cities and wants to find the optimal route that would take him through all cities and back to his/her original departure city. Thus, given a list of cities and the distances between each pair of cities, the TSP aims to determine what is the shortest possible route that visits each city and returns to the origin city. The TSP is thus an ordering problem. There are many OR problems, specially in logistics and manufacturing that can be modeled as TSP.
- **Knapsack problems:** In knapsack problems, given a set of items, each with a weight and a value, we need to determine the number of each item to include in a collection so that the total value is maximised while the total weight is less than or equal than the maximum weight the knapsack can hold.
- **Minimum Spanning tree:** Minimum Spanning Trees (MST) problems have a wide number of applications in fields like logistics or telecommunications when we deal with networks that interconnect different objects or edges through vertices. In this context, MST are grouping problems where we want to find a subset of edges in a graph that interconnects certain vertices at a minimum cost. We will address MST problems, among others, in Network Theory.

As you can already imagine by now, there is a really wide range of applications for combinatorial optimization. Some examples are Planning and Scheduling of activities, product and process design, portfolio selection, networking and routing (Internet traffic, airlines routes, logistics).

1.2 Types of techniques

Given the importance of the field, scientist have developed many techniques to solve combinatorial optimization problems over the years. They can be classified into the following categories:

- **Numeric Techniques:** Numeric techniques gather algorithms that provide an optimal solution by exploring all possible options. The simplest example is brute force, although there are many other algorithms that fall into this category. Whenever you use numeric functions like Python's Numpy to compute a minimum, maximum, or ordering of a set, internally the library is applying a numeric technique to compute the result. As we will soon see, for problem instances where the set is large, it might not be computationally efficient to rely solely on numeric techniques, and that is why we have other methods. Numeric techniques are out of the scope of this unit.
- **Problem-solving heuristics:** These techniques find a (sub-)optimal solution in a given number of steps. We will focus on some of these techniques in this unit. Some examples are:
 - **State space search:** These algorithms test successive combinations (or states) of the decision variables iteratively, in such a way that at every step the solution is closer to the optimal solution, until the optimum solution is found. One example of search heuristic is the Branch and Bound algorithm that will be covered in this unit.
 - **Greedy algorithms:** Greedy algorithms first sort the members of the set according to their impact in the objective function, and add them to the solution iteratively.
 - **Genetic algorithms:** Explore the possible solutions combining the features of an initial set of possible solutions, selecting the fittest candidates to combine their features in the next iteration.
 - **Graph Theory:** Graph theory is not really a type of technique, but rather, a separate field of mathematics dedicated to the study of relations between objects through mathematical structures known as graphs. Many combinatorial optimization problems can be easily modeled through these structures, and there are many efficient algorithms that effectively compute graph-like structures to find MSTs or shortest paths which provide the optimal solution to our problem.

1.3 Efficiency of Techniques

But why do we need this plethora of techniques in the first place? The main reason is that it is not feasible to use numeric techniques in relatively large instances of problems, because the number of possible combinations grows exponentially with the number of members in the sets. The image below, taken from Dynamic logic by Harel et al., 2000¹ highlights this fact:

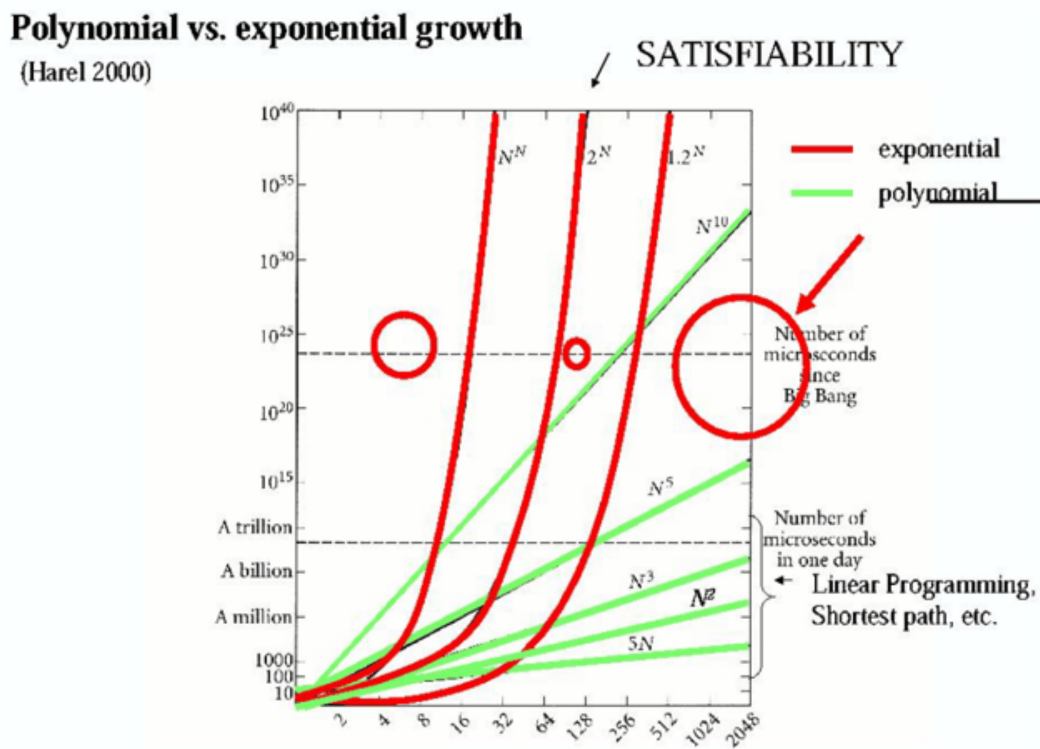
As shown in the graph, for relatively medium instances of combinatorial optimization problems like Integer Programming, the time required to explore all possible combinations might be larger than time itself! We need techniques that need fewer operations to find an optimal solution, or that at least can provide a suboptimal solution in a given interval of time.

Not all techniques provide results in the same number of operations, that is, not all techniques are equally efficient. An informal definition of an efficient algorithm is an algorithm in which the time required to solve a problem instance does not increase too fast as the problem size increases. Below, there is a formal definition of efficiency:

Definition 1 *An algorithm is efficient (good, polynomial) if the time necessary to solve any instance of input size L is bounded above by a polynomial function of L . (Thus it does not increase as fast as an exponential function such as s^L .)*

The Big $\mathcal{O}$ Notation is very handy to compare the efficiency of algorithms. For instance, an algorithm that solves a problem of size L in a time proportional to the square of L is a polynomial

¹David Harel, Dexter Kozen, Jerzy Tiuryn. 2000 Dynamic Logic. MIT Press.



algorithm in $\mathcal{O}(n^2)$, where as an algorithm that solves the same problem in an exponential time proportional to 2^L is noted as $\mathcal{O}(2^L)$. The latter is less efficient because the time required to solve the problem increases faster as L becomes larger. An important consideration that needs to be made at this point is that efficiency does not explicitly mention the number of operations, but rather the computing time. This has different implications. The first is that algorithms are formulated in mathematical language, and we need a computing model to translate operations into computational time. In complexity theory, which is the field of mathematics dedicated to the analysis of the complexity of problems, the Turing Machine, which is a mathematical model of a “perfect” machine is used to define time as a function of the number of operations in the definitions above of efficiency above.

Based on this definition of a computational machine model, problems can be classified according to their complexity, in a classification or taxonomy of problems.

1.3.1 Classification of problems in complexity theory

Based on the definitions above, a problem is as complex as the most efficient algorithm devised to solve it. An algorithm is considered efficient if it can solve an instance of a problem of input size L in a polynomial function of L . But what about other problems? Are there different categories of complex problems? Complexity theory is a field of mathematics that focuses precisely on the analysis of the complexity of problems, and its findings help scientist guide the search for new algorithms and mathematical techniques. The following sections describe the different problem classes, which are categories defined for problems according to their complexity.

Class Polynomial - Class P (Easy problems) A problem is in the class P (or polynomially solvable) if it admits a polynomial algorithm. This category gathers all problems for which we have found an algorithm which is able to find an efficient algorithm, able to solve it in a number of operations which is a polynomial function of the size of the problem.

Class Non-deterministic Polynomial time - Class NP (Possibly easy problems) A problem is in the class non-deterministic polynomial (NP) if the problem can be verified in polynomial time, meaning that we can find a bound for the optimal solution in polynomial time. Think for instance of a maximization problem, if we can find an upper bound for the objective function in polynomial time, we might not know which is the optimal solution, but we can already use this information in our problem, as we know that the optimal value will not exceed the upper bound.

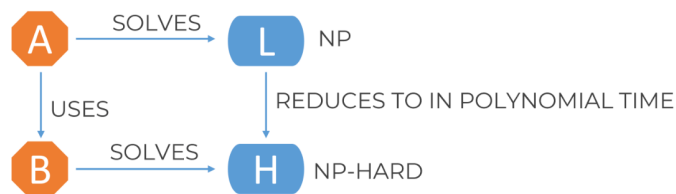
From this, we can define the associated decision problem as a problem that allows us to verify if a given number is a valid bound for our problem. If the result of the associated decision problem is True (meaning, yes, the provided number is a valid bound), then we can use this information to provide an approximation to the optimal solution.

Problems for which there is an algorithm that solves the associated decision problem in polynomial time are classified as NP problems.

Class Non-deterministic Polynomial time Hard - Class NP-Hard (Possibly Hard Problems) Reduction

To understand the NP-Hard category, we first need to introduce the concept of reduction. A reduction is basically an algorithm that allows us to convert an instance of a problem A into instances of another problem B. Formally, a problem H is NP-Hard if there is a Polynomial algorithm to reduce every problem L in class NP to it.

Let us use the following figure to illustrate this:



If a problem L can be reduced to H in polynomial time, and we can find a solution for H using an algorithm B in one unit of time, then we can solve L in polynomial time with an algorithm A which basically reduces L to H and then uses B to solve the problem.

The Traveling Salesman Problem (TSP) is an example of an NP-Hard problem.

Informal definition Based on the definition above, we can argue that NP-Hard problems are at least as hard as the hardest NP problem, since we can reduce every problem in NP to it in polynomial time, it follows that algorithms for NP-Hard problems are at least as efficient as algorithms for NP problems.

Class Non-Deterministic Polynomial time class Complete - NP-Complete (Hard Problems)

Formally, a problem is NP-Complete if:

- A given solution can be verified in polynomial time (that is, it is an NP Problem).
- It can be used to simulate any other problem for which we can verify that a solution in polynomial time (that is, it is an NP-Hard problem).

It follows that, if a problem is both NP and NP-Hard, it can be used to validate the solution of the complete NP class

Note however that although solutions for NP-Problems can be verified in polynomial time, this does not imply that an optimal solution can be found in polynomial time. That is, we can use heuristics to verify a bound and use this bound as an approximation of the optimal solution, and

we can do this efficiently in polynomial time, but we cannot find an optimal solution in polynomial time, which would in turn imply that the problem is in class P.

2 Mixed integer Programming

2.1 Introduction

Mixed Integer Programming (MIP) can be regarded as an extension of Continuous Linear Programming where any of or all the defined decision variables cannot take fractional values. MIP introduces two new types of variables:

- **Integer variables:** Integer variables can only take Integer values. Normally, Integer variables are introduced by logical restrictions that make rational valuables unfeasible. For instance, in some instances of production mix problems, it does not make sense to produce fractional parts of a product and treating them as real number is just an approximation.
- **Binary variables:** Binary variables or Boolean variables can only take values 1 or 0. Normally, binary variables are used to factor into our model binary decisions, e.g. to decide whether to use a machine configuration or another, or whether to select one element from a set or not.

In CLP, the continuity of the decision variables ensures that the feasibility region and the objective function intersect in a vertex of the feasible region at the optimal value. But this is no longer the case when some decision variables are Integer values. Intuitively, one can think that the optimal value may be found in one of the feasible Integer values around the theoretical intersection, as long as the solution is feasible. This makes the number of candidate solutions larger and consequently, MIP problems are more difficult to solve. From a mathematical point of view, when the decision variables are not continuous, the feasible region and the objective function are no longer convex. Convexity of feasibility region and objective function ensures that the optimal solution is at one vertex. We will explain this concept further in Non-Linear Programming. For now, it is sufficient to understand that in MIP, the feasibility region is no longer convex since not all values are possible and this makes it harder to find a solution.

This limitation makes convergence to an optimal solution more complex. MIP solution is several orders of magnitude more complex than linear programming. For instance, the assumptions made to design the Simplex algorithm do no longer hold and although adaptations may find an optimal solution, convergence is so slow that they cannot be applied to practical problems.

2.2 Set up

In general, the set-up of MIP problems is the same as the set-up of CLP problems, except that some decision variables can only take Integer values. For instance:

Decision variables Let us note our decision variable as $X = [x_1, x_2, \dots, x_n]$ where n is the number of decision variables. In continuous linear programming, the decision variables could take any real value, in mathematical notation this is expressed as: $x_j \in \mathbb{R}$.

Now, in *integer programming*, decision variables can only take integer numbers. Without loss of generalization, let us consider that our decision variables can only take positive integers, or natural numbers. We refer to this type of decision variables as *integer variables*, and are noted as: $x_j \in \mathbb{N}$.

A special type of integer variables are binary variables which are variables that can only take values 0, or 1. These variables are very convenient to model 'yes or no' decisions or 'true or false'

(boolean) logic decisions into our problem. We note can use this notation for binary variables: $x_j \in [0, 1]$.

In general, not all decision variables have to be necessarily of the same type. Integer Programming (IP) problems only have integer or binary decision variables, whereas MIP problems can have any combination of decision variables.

Objective function As with CLP, our objective function is a linear function of the decision variables, noted as z and which we would like to maximize or minimize. For instance, if our objective function is of type maximize, we can use:

$$\max z = \sum_{j=1}^n c_j \cdot x_j.$$

Constraints The objective function will be subject to some limitations or constraints:

subject to

$$\sum_{j=1}^n x_j \cdot a_{ij} \leq b_i, \forall i \in [1, m]$$

2.3 Examples

Let us now see some prominent examples of Integer Programming types. The next sections cover the formulation of some well known combinatorial optimization problems as IP problems.

2.3.1 Knapsack Problem

The Knapsack problem is an optimization problem that is often found in practical applications. The classical formulation reads

Definition 2 *Given a set of items, each with a weight and a value, we need to determine the number of each item to include in a collection so that the total value is maximised while the total weight is less than or equal than the maximum weight the knapsack can hold.*

Decision variables The decision variables x_j are binary decision variables that determine whether we include item j in the Knapsack: $x_j \in [0, 1]$, 0: item not included, 1: item included.

That is, the decision variable can only take values 0, or 1. The value 0 means that the object is not selected and therefore has no impact in the objective function. Likewise, the value 1 means that the object is selected.

Objective coefficient Each item j will have a different value v_j . We can express the objective function as the value of the items selected: $\max z = \sum_{j=1}^n v_j \cdot x_j$.

Constraints Now, our only constraint is related to the maximum weight W the knapsack can support. Each item will have a different weight w_j , and the sum of the weights w_j of the selected variables has to be less or equal than the maximum weight.

$$\sum_{j=1}^n w_j \cdot x_j \leq W$$

Note that, if a given x_j is zero, the product $w_j \cdot x_j$ will also be zero and will have no impact on the total value of the left hand side summation.

Now, it is important to note that in this formulation, weights and values as abstract concepts, and the same formulation can be applied to many practical problems.

2.3.2 Assignment problems

Assignment problems are a fundamental class of combinatorial optimization problems. In essence, assignment problems consists of assigning elements of a set of resources R (like workers, machines, etc.) to a set of tasks T to optimize a certain objective (like minimizing cost or maximizing efficiency), subject to various constraints.

Indices Now, since we have two different sets, we will use two different indices to uniquely identify the elements in each set:

- **i:** We will use index i to identify each element in set R . Let us note the size of set R as m , therefore, $i \in [1, 2, \dots, m]$
- **j:** We will use index j to identify each element in set T . The size of set T is noted as n , therefore, $j \in [1, 2, \dots, n]$

Decision Variables Let us define a binary variable x_{ij} for each resource i and task j . $x_{ij} = 1$ if resource i is assigned to task j , and 0 otherwise: $x_{ij} \in [0, 1]$ 0: resource i is not assigned to task j , 1: otherwise $\forall i, \forall j$.

Note that there are a total of $m * n$ decision variables!

Objective function The objective is to minimize costs or maximize benefits associated with assigning a particular resource to a particular task. As with Knapsack problems, we need to think of costs or benefits as abstract concepts, and the model can be applied to many practical applications.

For instance, let us note as c_{ij} the cost of assigning resource i to task j . The objective function then becomes:

$$\min z = \sum_{i=1}^m \sum_{j=1}^n c_{ij} * x_{ij}.$$

Constraints Now, the constraints need to ensure that one element of R is assigned to one element T , that is, we need to ensure that there are not two or more resources assigned to a task:

$$\sum_{j=1}^n x_{ij} = 1, \forall i.$$

Note first that the for a given i , the left hand side $\sum_{j=1}^n x_{ij}$ is the sum of all the decision variables associated to a resource i . Since the different x_{ij} can only be 0 or 1, by establishing an upper bound limit of 1 to the sum of all the elements, we ensure that the resource can only be assigned at most to one task.

Similarly, we need to ensure that a task is only associated to one resource:

$$\sum_{i=1}^m x_{ij} = 1, \forall j.$$

Note that the expression is similar, but now the left hand side $\sum_{i=1}^m x_{ij}$ represents the sum of all the decision variables associated to task j .

2.3.3 Sequencing Problems

Traveling Salesman Problem Recall the classic formulation of the Travelling Salesman Problem (TSP):

Definition 3 *Given a list of cities and the distances between each pair of cities, what is the shortest possible route that visits each city and returns to the origin city.*

In practical applications, cities represent different steps in a process, and distances represent the cost of a sequence of steps in the process, from one step to another.

Thus, for the setup of the TSP as an IP, we have the following variables and coefficients:

- x_{ij} : Binary decision variables that represent a sequence from source i to destination j . 1 means that the step is selected in the final sequence, 0 otherwise.
- c_{ij} : Represents the cost the step in the sequence, in the classical formulation, the cost of travelling from city i to city j .
- u_i : Integer variable that represents the step of i in the sequence. If u_i is equal to y then city i is departed at step t .

The IP formulation is then as follows:

$$\min z = \sum_{i=1}^N \sum_{j=1}^N c_{ij} * x_{ij}.$$

Subject to: we can only depart from 1 city:

$$\sum_{j=1}^N x_{ij} = 1, \forall i.$$

We can only arrive to a city from a city:

$$\sum_{i=1}^N x_{ij} = 1, \forall j.$$

We cannot travel in loops in a city:

$$x_{ij} = 0, \forall i = j.$$

We travel from city i to city j exactly at step u_i :

$$u_i - u_j + n * x_{ij} = n - 1, \forall i \in 1 \leq i \neq j \leq n.$$

There are no loops, we do not allow u_i to be arbitrarily large:

$$0 \leq u_i \leq n, \forall i \in 2 \leq i \leq n.$$

MME8215: Operations Research

IX. Non-Linear Programming

Contents

1 Introduction	1
2 Unconstrained Non-Linear Programming	4
2.1 One dimensional problems set-up	4
2.2 Multidimensional problems set-up	4
3 Gradient Descent Methods	6

This course is an introduction to Non-Linear Programming (NLP), the branch of optimisation that deals with problem models where the functions that define the relationship between the unknowns (either objective function or constraints) are not linear. The fact that the functions are not linear makes NLP a lot harder to deal with from a mathematical point of view. We will focus on specific cases that have many applications in real life problems, particularly quadratic functions and their relationship to machine learning.

1 Introduction

A non-linear programming (NLP) problem is similar to a linear programming problem in the sense that it is also composed of an objective function and constraints, but a NLP includes at least a non-linear function. We will deal with continuous decision variables and problems where either the objective function, or any of the constraints are not linear. NLP problems are in many cases more difficult to solve than linear programming problems. The mathematical foundations of NLP optimisation date back to the work of Joseph Louis Lagrange. He developed a method to take into account non-linear constraints into optimisation problems, which is called Lagrange multipliers, and that will be used in constrained NLP.

Hard to distinguish local and global optimal values

In linear programming, methods like the simplex need only to know the value of the unknowns and the value of the objective function to determine if the solution is optimal, because the maximum is always at a vertex of the feasibility region. Non-linear functions have critical points, which can be either local or global maxima or minima:

- *Local maximum*: The value of the function at the critical point is higher than the values of the function around the critical point.
- *Global maximum*: The value of the function at the critical point is the highest value of the function.
- *Local minimum*: The value of the function at the critical point is lower than the values of the function around the critical point.
- *Global minimum*: The value of the function at the critical point is the lowest value of the function.

For instance, consider the function presented in figure 1. The function has many critical points. Any critical point can be a local maximum or a global maximum depending on the range of values that x can take. For instance, if x could only take values in the range $x \in [-3, 0]$, then the local maximum would become a global maximum. This makes it somewhat more difficult to find the optimal solution in an optimisation problem.

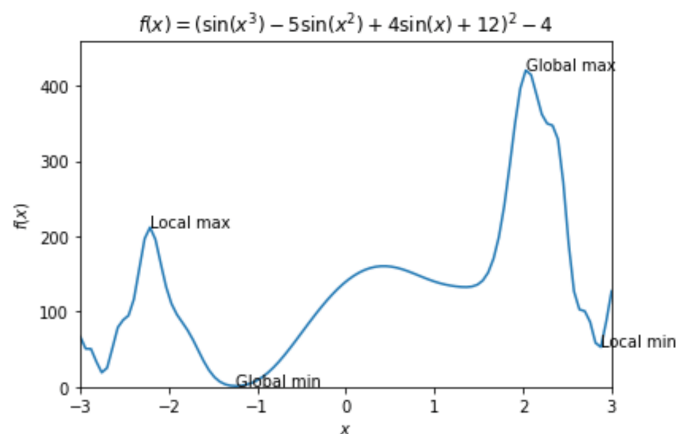


Figure 1: An example of a function with many critical points.

Optima not restricted to vertex

With non-linear functions, the optimal (maximum or minimum) value might no longer be at one of the vertices. For instance, let us take as an example the function presented in figure 2.

The example shows a non-linear function on two decision variables, x and y . The black lines represent some constraints limiting the feasibility region. The contour graph shows that the maximum value of the objective function is within the interior of the feasibility region and not in one of the vertices.

The following situations are examples where the optimal value might not be at one vertex of the feasibility region:

- *Multimodal function*: As in the example from figure 2, the function might be multimodal and depend on several decision variables, increasing its complexity.
- *Interior optima*: As in the above example, the optimal value might be found in an interior point of the feasibility region.
- *Discontinuous*: The objective function might be discontinuous and again, the optimal value might not be surely located in one vertex.

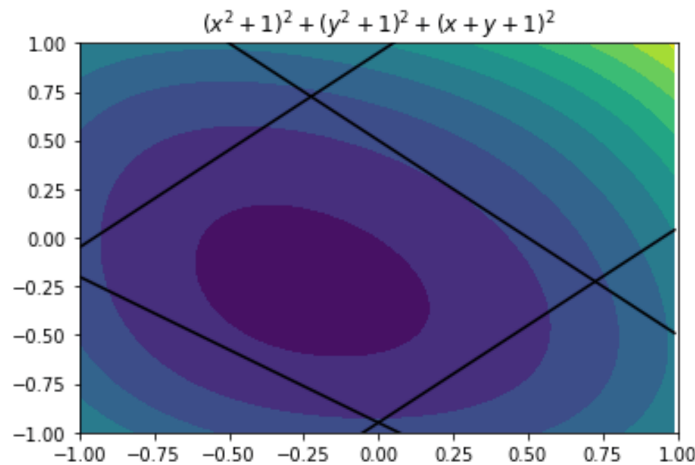


Figure 2: A non-linear function on two decision variables.

Given that the objective functions and the constraints can be of any family of functions, potentially, there is a huge body of mathematical theory that can be applied to NLP, since non-linear functions have a wide range of characteristics. The theoretical framework is more sophisticated, and modeling is also more difficult, since the researcher needs to define the types of function to use in the model.

Solver selection

Due to the complex mathematical theory behind, selecting the right solver to solve a specific problem can be complex. In fact, different algorithms and solvers can provide different results. Normally, algorithms deal with local optimisation, rather than global optimisation, and for that reason, the selection of the initial point used to start the algorithm might have an impact in the solution.

Hence, different algorithms might provide different solutions, but when a specific algorithm is selected, its configuration (particularly the initial point) might yield different results.

Types of problems

We can divide NLP into two different classes of problems with increasing complexity:

- *Unconstrained NLP Problems:* This class of NLP problems have no constraints, we look for a maximum or minimum of the objective function within some bounded decision variables, but not subject to any constraint.
- *Constrained NLP Problems:* This class of problems have constraints, i.e. there are restrictions to the objective function solution determined by a set of functions of the decision variables.

Additionally, to better present the mathematical framework behind NLP, we will consider One-Dimensional unconstrained problems, where there is only one decision variable, and Multi - Dimensional or multi-modal, where there are several decision variables

2 Unconstrained Non-Linear Programming

2.1 One dimensional problems set-up

Let us first consider just one decision variable. The objective is to search a maximum or minimum of a non-linear function. Without loss of generalisation, let us consider that the objective is to minimize the objective function:

$$\min f(x), x \in \mathbb{R}.$$

Any (local or global) minimum of $f(x)$, denoted as x^* , will have the following properties:

1. **First derivative test:** The first derivative of a function at a point x^* represents the slope of a line tangent to $f(x)$ at x^* . In a minimum, the first derivative is equal to zero, $\frac{df(x)}{dx} = 0$, meaning that the tangent is horizontal.
2. **Second derivative test:** The second derivative is positive $\frac{d^2f(x)}{dx^2}$, meaning that the slope of the tangent is increasing and therefore, it must be less than zero for $x < x^*$, meaning that the function decreases before x^* , and greater than zero for $x > x^*$, meaning that the function increases after x^* . Therefore $f(x)$ takes its minimum value at x^* , at least in its proximity.

By definition, x^* is a critical point. For a maximum, the second derivative is negative and using the same logic, $f(x)$ decreases in the proximity of x^* .

Bi-section algorithm Since the sign of the first derivative changes at a critical point, it is possible to search a critical point if we find two initial points such that the sign of the first derivative is different. The simplest algorithm that uses this property is known as the *bi-section method*, and it can be formulated as this:

Initialization:

1. Choose an acceptable error ϵ
2. Find an initial left point x_L such that $\frac{df(x_L)}{dx} < 0$
3. Find an initial right point x_R such that $\frac{df(x_R)}{dx} > 0$

Iteration:

1. Bisect the range $[x_L, x_R]$, $x' = \frac{x_R + x_L}{2}$,
2. If $\frac{df(x')}{dx} < 0$ then make $x_L = x'$, else if $\frac{df(x')}{dx} > 0$ then make $x_R = x'$,
3. If $x_R - x_L \leq 2\epsilon$ then the solution is $\frac{x_R + x_L}{2}$, else repeat.

That is, we start with two initial values for which the first derivative has different sign. If the function is continuously differentiable, then there must be a value somewhere in between these values where the first derivative is zero. Then, we make a bi-section and continue looking in the part of the range for which there is a change in the sign of the first derivative. The process stops either when we find exactly a critical point or when the distance between both points is less than the acceptable error.

For instance, suppose that your equation $f(x) = 0$ had a solution precisely at $x = \frac{1}{\pi} - 0.3183$. (You don't know that of course; you're trying to find the solution!) You'd first narrow down the interval from $[0, 1]$ to $[0, \frac{1}{2}]$; then to $[\frac{1}{4}, \frac{1}{2}] = [0.25, 0.5]$; then to $[\frac{1}{4}, \frac{3}{8}] = [0.25, 0.375]$; then to $[\frac{5}{16}, \frac{3}{8}] = [0.3125, 0.375]$; and so on. See figure 3 for a visual example.

2.2 Multidimensional problems set-up

For multidimensional problems the logic applied is the same, except that in this case it is necessary to consider partial derivatives in the different decision variables or dimensions of the problem.

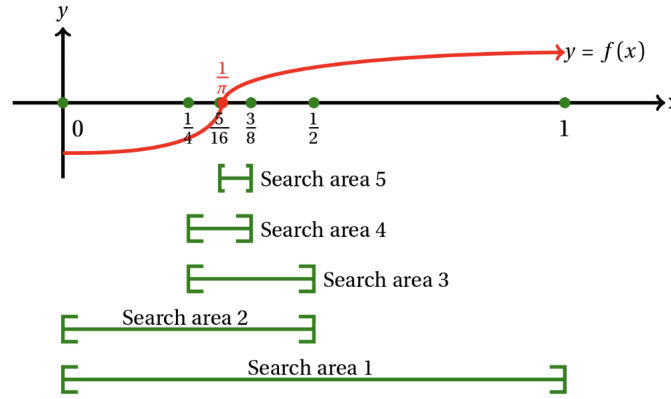


Figure 3: Bi-section example.

Thus, given an n -dimensional function:

$$f(x), x = [x_1, x_2, \dots, x_n],$$

the **gradient vector** is defined as a vector containing the first partial derivatives:

$$\nabla f(x) = \left[\frac{\delta f(x)}{\delta x_1}, \frac{\delta f(x)}{\delta x_2}, \dots, \frac{\delta f(x)}{\delta x_n} \right]^T$$

The gradient represents the slopes of the tangent planes to $f(x)$ in every dimension. For a minimum, the gradient must be zero, as an extension of the first derivative test of one dimensional problems.

1. Gradient test The gradient must be zero at a critical point x^* , $\nabla f(x^*) = [0, \dots, 0]^T$.

To know if the function decreases in every direction around x^* , we need to define the Hessian:

$$Hess f(x) = \begin{bmatrix} \frac{\delta^2 f}{\delta x_1^2} & \frac{\delta^2 f}{\delta x_1 \delta x_2} & \cdots & \frac{\delta^2 f}{\delta x_1 \delta x_n} \\ \frac{\delta^2 f}{\delta x_2 \delta x_1} & \frac{\delta^2 f}{\delta x_2^2} & \cdots & \frac{\delta^2 f}{\delta x_2 \delta x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\delta^2 f}{\delta x_n \delta x_1} & \frac{\delta^2 f}{\delta x_n \delta x_2} & \cdots & \frac{\delta^2 f}{\delta x_n^2} \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & \cdots & h_{1n} \\ h_{21} & h_{22} & \cdots & h_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ h_{n1} & h_{n2} & \cdots & h_{nn} \end{bmatrix}$$

That is, the Hessian is a $n \times n$ matrix that contains the second order derivatives.

To determine how the function increases or decreases in any direction around a given point, we can check *Sylvester's criterion*. This criterion evaluates the *leading principal minors* of the Hessian matrix. A leading principal minor k of a Hessian matrix is the determinant of the square sub-matrix that includes columns and rows from 1 to k , that is, if we note, H_k as the leading minor of the Hessian matrix A , we can compute the leading principal minors as:

$$H_1 = |h_{11}|, H_2 = \begin{vmatrix} h_{11} & h_{12} \\ h_{21} & h_{22} \end{vmatrix}, \dots$$

Note that an n -dimensional function has n Hessian leading minors. Now, the Hessian determinants can help us decide whether a function is a local or global minimum, according to the following definitions:

- *Definitive positive Hessian*: all leading principal minors are greater than zero $H_i > 0, \forall i \in [1, \dots, n]$.
- *Semi-definitive positive Hessian*: all leading principal minors are greater or equal than zero $H_i \geq 0, \forall i \in [1, \dots, n]$.
- *Definitive negative Hessian*: all leading principal minors are lower than zero $H_i < 0, \forall i \in [1, \dots, n]$.
- *Semi-definitive negative Hessian*: all leading principal minors are less or equal than zero $H_i \leq 0, \forall i \in [1, \dots, n]$.

2. Second partial derivative test Now, the second derivative test becomes:

- In a global maximum, the Hessian is definitive negative;
- In a global minimum, the Hessian is definitive positive;
- In a local maximum, the Hessian is semi-definitive negative;
- In a local minimum, the Hessian is semi-definitive positive.

3 Gradient Descent Methods

Gradient Descent Methods constitute a large family of heuristic methods based on the gradient definitions presented in the set-up of unconstrained NLP. Gradient descent methods are useful to find critical points and test whether they are global minimum (for minimisation problems) or global maximum (for maximisation problems).

Basically, they consist of two types functions:

- *search functions*: to find points where the gradient is zero or close to zero, i.e. functions to find critical points.
- *test functions*: to verify whether the critical point is a global or local maximum or minimum.

The different implementations have different performance trade-offs depending on the characteristics of the optimisation problem. Next we will cover two different methods, the simple gradient descent method and the stochastic gradient descent (you probably know these methods from your AI, optimisation courses).

Simple Gradient descent

In the Simple gradient descent method, the algorithm starts at an initial point, calculates the gradient, and if it's different from zero, it looks for the next point to test in the direction opposite to the gradient. Recall that, at any given point x^* the gradient of the objective function $\nabla f(x^*)$ represents a vector tangent to $f(x^*)$. This tangent vector represents the direction of the maximum change of x towards infinity in the n -dimensional space. Therefore, the algorithm continues the search in the opposite direction to this vector, assuming that this will provide the maximum change of the function towards its minimum value. With this, the Simple Gradient descent method can be defined as:

Start:

Choose an acceptable error ϵ

Choose a starting point x'

Iterate:

1. Calculate $\nabla f(x')$,
2. If $|\nabla f(x')| \leq \epsilon$ then stop with x' as the solution, else
3. set $x'' = x' - t \cdot \nabla f(x')$, $t \geq 0$, $t \in \mathbb{R}$ and iterate again.

Note that since t is a positive value and $\nabla f(x')$ is a vector, we are changing the value of x in the direction of the vector. The change is proportional to t , which can be a fixed parameter or a calculated parameter depending on the implementation. One implementation alternative is to use a one-dimensional search (e.g. using a bisection algorithm) to find x^* such that $f(x'')$ is a local minimum. Note that since both x' and $\nabla f(x')$ are vectors, the only unknown is t , and this bisection is a reduction of the multidimensional minimisation problem to a one-dimensional minimisation problem.

Stochastic Gradient Descent (SGD)

The stochastic gradient descent method can be applied in problems where the gradient of the objective function can be estimated from a subset of the parameters needed to compute the closed form of the gradient. The stochastic gradient descent is one of the fundamental optimization methods used in machine learning. Let us consider the following objective function:

$$\min R_n(\theta) = \frac{1}{n} \sum_t^n L_t(\theta).$$

The stochastic gradient descent method is defined as:

Start:

Choose an acceptable error ϵ

Choose a starting point θ , e.g. $\theta = 0$

Choose a learning factor, ζ_k , for each iteration k

Iterate:

1. Select a random value t^*
2. Calculate the gradient of $\nabla L_{t^*}(\theta_k)$
3. Update θ_k in the direction contrary to the gradient at t^* : $\theta_{k+1} = \theta_k - \zeta_k \cdot \nabla L_{t^*}(\theta_k)$
4. Repeat until the difference between two consecutive estimations is sufficiently small, or until the maximum number of iterations is reached.

MME8215: Operations Research

X. Decision Theory and Game Theory (I)

Contents

1 Decision Theory	1
1.1 Payoff Matrix	2
1.2 Dominance	3
1.3 Decision Rules	3
1.3.1 MinMax	4
1.3.2 MaxMax	4
1.3.3 Bayesian	5
2 Decision Trees	5
2.1 Solving a Decision Tree	7
2.2 Imperfect Information	7
2.3 Value of Perfect Information	8

This course provides an introduction to decision theory, where we will focus on solving decision problems. In this case a decision maker must select among various alternatives, the alternatives provide different performance. This course also gives an introduction to game theory, a field dedicated to modelling and solving decision problems involving several rational decision makers. For the game theory part we focus on solving decision problems where the decision maker must select among various alternatives, focusing on competition and collaboration.

1 Decision Theory

Decision theory spans a combination of problem-solving techniques to find the best decision in complex decision problems. We will use the following terminology:

- **Alternatives:** Decision variables which are controllable and depend on the decision maker's decision.
- **Uncertainty and states of nature:** External variables, these variables are uncontrollable and need to be estimated or assumed.
- **Performances:** Profit or Cost (utility) of the result of a decision

The **objective** is to find the alternative with the highest performance for the decision maker, possibly in situations where the performance may depend on external variables which cannot be controlled.

We will be focusing on the following aspects:

- **Impacts over time** All the important consequences of a problem do not occur at the same instant in time.
- **Uncertainty** At the time the decision-maker must select an alternative, the consequences are not known with certainty.
- **Possibility of acquiring information** Often we can acquire additional information to support decision-making at a cost. For instance, to collect seismic information to decide whether to drill for oil. Decision theory provides methods to evaluate if it is worth acquiring additional information or not.
- **Dynamic aspects** The problem might not end immediately after an alternative is chosen but might require further analysis (e.g. further decisions)

1.1 Payoff Matrix

The payoff matrix is a tool to represent the performance of the different alternatives against the possible future outcomes of an uncontrolled event, or states of nature. The **different rows** of the matrix represent the alternatives of the decision maker, and the **different columns** of the matrix, the different possible states of nature. Every cell in the matrix contains the performance of each alternative given the occurrence of each state of nature.

Let us note the decision maker's alternatives as $a_1, a_2, \dots, a_m$, and the different states of nature as $s_1, s_2, \dots, s_n$. Let us note the performance of alternative a_i when s_j has occurred as u_{ij} , then the pay-off matrix is:

$$\begin{bmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ u_{21} & u_{22} & \cdots & u_{2n} \\ \vdots & \vdots & \vdots & \vdots \\ u_{m1} & u_{m2} & \cdots & u_{mn} \end{bmatrix}.$$

Note that the payoff matrix represents all possible outcomes of our decision under uncertainty, given that the states of nature are exhaustive (together they describe all the possible outcomes of the uncontrolled variable) and mutually exclusive (if one occurs, then the rest cannot occur). The payoff matrix can also be represented in tabular form as depicted by table 1.

Table 1: Payoff matrix as tabular.

Decision alternatives	state 1	state2	...	state n
a_1	u_{11}	u_{12}	$\dots$	u_{1n}
a_2	u_{21}	u_{22}	$\dots$	u_{2n}
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
a_m	u_{m1}	u_{m2}	$\dots$	u_{mn}

Example 1. Let us, at this point, set one example to be used in the next sections of this course. In the example, the decision maker needs to select a financial product among a set of market alternatives: Gold, bonds, stock options, deposit, or hedge fund. Each product provides different benefits or losses depending on the behaviour of the market. The behaviour of the market is not

controlled by the decision maker, therefore, it is characterised in this example as a set of states of natures:

- *Accumulation Phase*: The market is stable and characterised by a slow raise.
- *Mark-up Phase*: The market has been stable for a while, investors feel secure and the market is characterised by a fast raise.
- *Distribution Phase*: Most investors start selling to collect benefits, the market stabilises and there is little or no change.
- *Mark-down Phase*: Some investors try to hold positions as there is a large fall of the market.

Table 2: Payoff matrix for example 1.

Decision alternatives	Accumulation	Mark-up	Distribution	Mark-down
Gold	−100	100	200	0
Bonds	250	200	150	−150
Stock options	500	250	100	−600
Deposit	60	60	60	60
Hedge fund	200	150	150	−150

The payoff matrix presented by table 2 shows the expected performance in euros of the investment of each product under each market cycle phase.

1.2 Dominance

Dominance is a property of alternatives that provide a better performance than others for every state of nature. In example 1, it can be seen that the bond provides a higher benefit than the option hedge in any market scenario, therefore the bond option is dominant over the hedge fund option. Dominance allows us to simplify the decision-making process, eliminating the dominated alternatives in favour of the dominant alternatives. Taking into account dominance, the payoff matrix from table 2 can be written as the one in table 3.

Table 3: Payoff matrix for example 1, taking into account dominance.

Decision alternatives	Accumulation	Mark-up	Distribution	Mark-down
Gold	−100	100	200	0
Bonds	250	200	150	−150
Stock options	500	250	100	−600
Deposit	60	60	60	60

Any rational decision maker would select bonds before hedge fund option under any market phase and therefore, the latter can be ignored.

1.3 Decision Rules

Decision rules are criteria used by a rational decision maker to make systematic decisions, i.e., to select the best alternative. This section describes some of the most important criteria in related literature.

1.3.1 MinMax

The MinMax criteria aims to minimise loss in the worst-case scenario. It is therefore a *conservative*, or pessimistic criteria. Also known as Maximin (minimise the maximum loss), this criteria first finds the minimum value of the performance of each decision variables in every scenario:

$$m_i = \min\{u_{i1}, u_{i2}, \dots, u_{in}\}, \forall i = [1, 2, \dots, m].$$

These values represent the worst case scenario in every decision alternative, then the MinMax criteria selects the maximum of these values:

$$d = \operatorname{argmax}(m_1, m_2, \dots, m_m).$$

The function *argmax* above returns the index of the maximum value rather than the maximum value. Hence, it will return the index of the alternative for which the maximum is value.

For example 1, table 4 with the associated explanations, presents the result of the the MinMax criteria.

Table 4: Payoff matrix for example 1, taking into account dominance and Minmax.

Decision alternatives	Accumulation	Mark-up	Distribution	Mark-down	m_i
Gold	-100	100	200	0	-100
Bonds	250	200	150	-150	-150
Stock options	500	250	100	-600	-600
Deposit	60	60	60	60	60

$$d = \operatorname{argmax}(-100, -150, -600, 60) = 4, \text{ and } v = \max(-100, -150, -600, 60) = 60.$$

The MinMax criteria models the decisions of a conservative rational decision maker and in this example, a conservative investor will invest in deposits.

1.3.2 MaxMax

The MaxMax criteria on the other hand aims to maximise profit in the best-case scenario. It is therefore an *aggressive*, and optimistic criteria. This criteria first finds the maximum value of the performance of each decision variables in every scenario:

$$o_i = \max\{u_{i1}, u_{i2}, \dots, u_{in}\}, \forall i = [1, 2, \dots, m].$$

These values represent the best case scenario in every decision alternative, then the MaxMax criteria selects the maximum of these values:

$$d = \operatorname{argmax}(o_1, o_2, \dots, o_m).$$

For example 1, table 5 with the associated explanations, presents the result of the the MaxMax criteria.

$$d = \operatorname{argmax}(200, 250, 500, 60) = 3, \text{ and } v = \max(200, 250, 500, 60) = 500.$$

The MaxMax criteria models the decisions of a an *aggressive* and optimistic decision maker and in this example, an optimistic investor will invest in stock options.

Table 5: Payoff matrix for example 1 taking into account dominance and MaxMax.

Decision alternatives	Accumulation	Mark-up	Distribution	Mark-down	o_i
Gold	-100	100	200	0	200
Bonds	250	200	150	-150	250
Stock options	500	250	100	-600	500
Deposit	60	60	60	60	60

1.3.3 Bayesian

The Bayesian criteria takes into account additional information about the likelihood of each state of nature. It is therefore the criteria applied by an informed decision maker which wants to factor in the decision the likelihood of each alternative. The probabilities of each state or nature as defined as $p(s_j)$. The Expected Monetary Value (EMV) of each alternative is defined as:

$$EMV_i = \sum u_{ij} * p(s_j).$$

The EMV can be interpreted as the weighted average value of the alternative given the probabilities of occurrence of each state of nature.

The Bayesian criteria selects the alternative with the maximum EMV :

$$d = \operatorname{argmax}(EMV_1, EMV_2, \dots, EMV_m).$$

Let us use again example 1 to apply this decision criteria, table 6

Table 6: Payoff matrix for example 1 taking into account dominance and Bayesian.

Decision alternatives	Accumulation	Mark-up	Distribution	Mark-down	EMV
Probabilities ($p(s_j)$)	0.2	0.3	0.3	0.2	-
Gold	-100	100	200	0	70
Bonds	250	200	150	-150	125
Stock options	500	250	100	-600	85
Deposit	60	60	60	60	60

$$d = \operatorname{argmax}(70, 125, 85, 60) = 2, \text{ and } v = \max(70, 125, 85, 60) = 125.$$

The Bayesian criteria selects the decision alternative with index 2 (Bonds) and its EMV is 125.

2 Decision Trees

Decision Trees are chronological representations of complex decision processes that can be used to find an optimal decision based on a specific criteria, using a divide-and-conquer approach.

Decision trees have three main graphical components:

- **Decision Nodes:** Represent decisions under certainty. Each branch represents a decision alternative and is represented by a utility or cost as a measure of its performance.
- **Random nodes:** Represent possible outcomes under uncertainty. Each branch corresponds to a possible state and is represented by a probability P .
- **Leafs:** Possible outcomes of the decision process. They are represented by the performance, the utility or cost value corresponding to the connected decision nodes and random nodes.

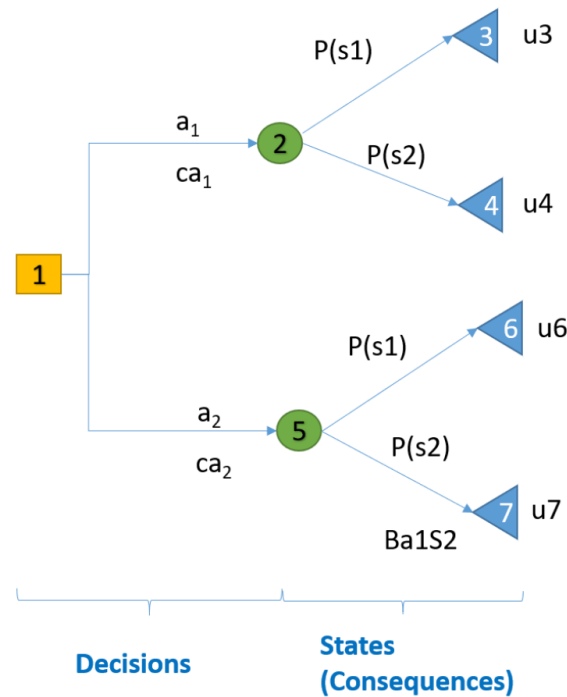


Figure 1: A decision tree.

Figure 1 shows these components and their relationships as a baseline example. The example shows just a decision node, labeled as 1, with two different alternatives, noted as a_1 and a_2 . Each alternative has a different utility value (ca_1 for a_1 and ca_2 for a_2). There is some uncontrolled variable that can impact our decision, and therefore, for each of the output branches of the decision node, there are two random nodes, labelled as 2 and 5, which represent the *uncontrolled* variable. From each node, there are two different branches that represent the two possible states of the decision variable. Each branch is labelled with the corresponding probability of the same state of nature, $P(s_1)$ and $P(s_2)$. Finally, the leaf nodes, are labelled according to the utility values of each decision under each state of nature (u_3, u_4, u_6, u_7). Note that the same information can be represented in a payoff matrix as the one described by table 7.

Table 7: Payoff matrix for decision tree.

Alternatives	state 1	state2
Probabilities	$P(s_1)$	$P(s_2)$
a_1	$u_3 + ca_1$	$u_4 + ca_1$
a_2	$u_6 + ca_2$	$u_7 + ca_2$

The decision tree allows to represent more complex set-ups adding more nodes and uncontrolled variable, even dealing with different combinations of uncontrolled variables and decision nodes at each branch. To define a decision tree, it is necessary to apply the following steps:

1. **Structure the problem:** define decision steps, states, objectives and their performance measures. In general, a decision problem can have multiple objectives. Each objective has a performance measure to represent preferences of the decision maker (e.g. An objective can be to minimize costs and the performance metric can be a monetary unit like euros). If there are multiple objectives, it is necessary to take into account the trade-offs among different

objectives to obtain a single utility function to represent preferences. However, for the rest of the unit, we will deal with single objective decision problems. Utility functions help us quantify the value of each decision. Each alternative can have a positive impact or negative impact on the value.

2. **Describe consequences:** describe the probability of each possible state of nature. In situations of certainty, each decision will have a positive or negative impact in the value of the decision. In situations of uncertainty or risk, each possible state will have a different probability of occurrence. We need an estimation of these probabilities to quantify the value of each decision.
3. **Assign a value to each decision alternative:** Define the utility of each decision. We need to estimate the utility of each possible outcome of the decision problem.
4. **Decide which of the proposed alternatives is preferred:** To decide which of the proposed alternative is preferred, it is necessary to solve the decision tree, as described in the next section.

2.1 Solving a Decision Tree

Decision trees are solved from right to left, that is, from the leaves, back to the initial decision node. At every decision step, it is necessary to quantify the value of each alternative, taking into account the probability of states of nature. Moreover, the values of random nodes and decision nodes are calculated differently.

The value of a random node is calculated using a Bayesian approach, as the weighted expected value across the different states:

$$V_r = \sum_{i=1}^n P(s_i) * V_i.$$

Where $P(s_i)$ is the probability of state i and V_i is the value at branch or leaf i .

The value of a decision node on the other hand is calculated as:

$$V_n = \max\{V_i - ca_k, V_j - ca_p, \dots\}$$

That is, a rational decision would always select the alternative with the highest value across the different alternatives, taking into account their costs.

With this two simple rules, the value of a tree can be calculated from its leaves, no matter how complex the set-up is and regardless of the number of steps.

2.2 Imperfect Information

Sometimes we need to decide whether to incorporate into the decision *imperfect information*. Imperfect information is information about uncontrolled variables that we can incorporate into the decision (possibly at a cost) to improve the performance. For instance, coming back to the previous example, imagine that our decision problem is to decide a financial product. To make a better decision, we may purchase a forecast report of the market trends for the following year. The report forecasts a given market trend, but the state is not known with certainty. It is possible to evaluate the usefulness of incorporating this information into the problem. Applying the *Bayes Theorem*, the probabilities of the different states of random nodes now will be conditioned on the outcome of the imperfect information (in the example, of the forecast report).

In these cases, it is necessary to review the different probabilities. After imperfect information, the probabilities of occurrence of the different states become revised (posterior) probabilities. The revised probabilities are calculated by applying the Bayes Theorem.

Let us assume that there are n different states and that, consequently, the imperfect information may provide additional information about the occurrence of each of the states. The introduction of imperfect information can be modeled first as a decision node that models the decision of whether to incorporate imperfect information or not, and then as a random node in the branch of the imperfect information state i . For instance, let us consider figure 2, a basic example with two alternatives and two possible states.

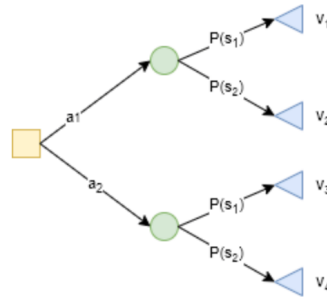


Figure 2: Imperfect information, an example tree.

Let us now introduce the acquisition of imperfect information about the states. First, the decision maker needs to decide whether not to incorporate this information (make the decision without imperfect information) or to incorporate this information (make the decision with additional information, possibly at a cost). The diagram in figure 3 introduces this first decision as an additional decision node. Then, since the result of the forecast is not controlled by the decision maker, its two possible results - i_1 when it forecasts s_1 and i_2 when it forecasts s_2 - are modelled as a random node with two states.

Note that now the probability of state s_j is conditioned to the event i_i in every branch. We can apply the Bayes Theorem to calculate the probability that event s_j occurs given that the imperfect information forecasts i_i :

$$P(s_j|i_i) = \frac{P(s_j) \cdot P(i_i|s_j)}{P(i_i)} = \frac{P(s_j) \cdot P(i_i|s_j)}{\sum_k P(s_k) \cdot P(i_i|s_k)}.$$

In the above example, this yields:

$P(s_1)$ and $P(s_2)$ are the a priori probabilities, or the probabilities without further information. The conditional probabilities: $P(i_1|s_1)$, $P(i_1|s_2)$ refer to the probability that information i_1 occurs under states s_1 and s_2 , respectively. The same applies to $P(i_2|s_1)$ and $P(i_2|s_2)$. This conditional probabilities can be estimated from a historical record of events.

For instance, connecting with our previous example, let us assume that s_1 represents a market raise state and s_2 a market fall state, and a_1 and a_2 two financial products. We may purchase a financial report to guide our investment decision, at a cost. The report has a previous record of predicting 95% of the times a market fall correctly, that is $P(i_1|s_1) = 0.95$ and consequently $P(i_1|s_2) = 0.05$. The record also tells us that the forecast is right 90% of the time when the market falls, therefore $P(s_1|i_2) = 0$ and $P(s_2|i_1) = 0$. With these values, we can calculate the revised probabilities and solve the tree to determine whether to incorporate the report or not.

2.3 Value of Perfect Information

The previous section described how to calculate the value of a decision tree after incorporating additional information, which may come at a cost. An obvious question that arises is, how much should the decision maker be willing to pay for additional information? This quantity is known

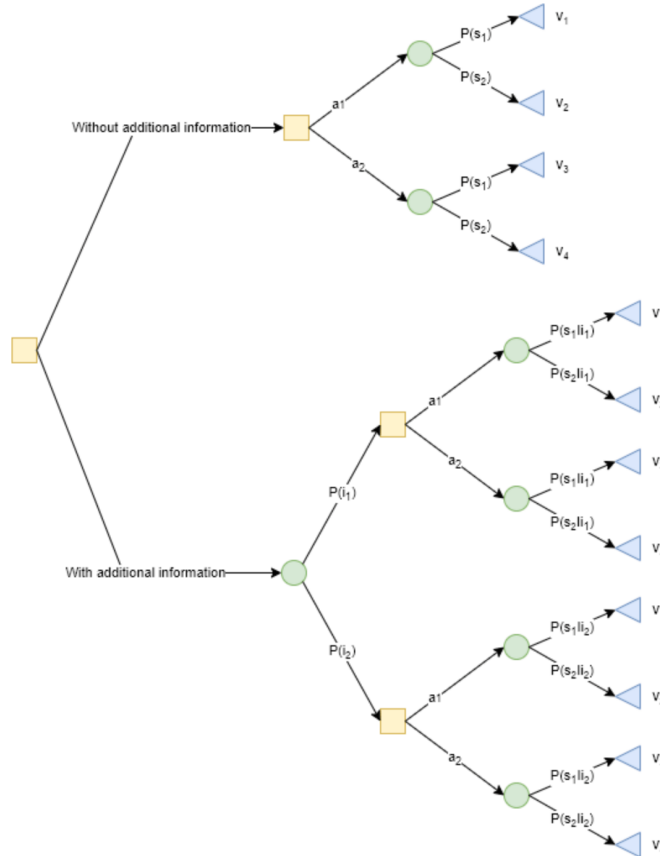


Figure 3: Imperfect information, first decision as an additional decision node.

$$P(i_1) = P(s_1) \cdot P(i_1 | s_1) + P(s_2) \cdot P(i_1 | s_2)$$

$$P(s_1 | i_1) = \frac{P(s_1) \cdot P(i_1 | s_1)}{P(i_1)}$$

$$P(s_2 | i_1) = \frac{P(s_2) \cdot P(i_1 | s_2)}{P(i_1)}$$

$$P(i_2) = P(s_1) \cdot P(i_2 | s_1) + P(s_2) \cdot P(i_2 | s_2)$$

$$P(s_1 | i_2) = \frac{P(s_1) \cdot P(i_2 | s_1)}{P(i_2)}$$

$$P(s_2 | i_2) = \frac{P(s_2) \cdot P(i_2 | s_2)}{P(i_2)}$$

as the Value of Perfect Information. Perfect Information is an abstract concept that represents information that would make the occurrence of a state of nature certain. For instance, a forecast report which can tell us with total certainty which is going to be the trend of the market in the future. Obviously, this type of forecast does not actually exist in reality, but we can model its effect in the decision tree and solve it to calculate its final value. The Expected Value with Perfect Information (EVPI) is the expected final value of the tree given that we had perfect information. It is also an abstract concept to assess the value of the decision-making problem with access to

perfect a priori information.

For instance in the example above, perfect information means that the forecast report is always right, and therefore we can plug in $P(s_1|i_1) = 1$ and $P(s_2|i_2) = 1$ since we know that the information about the states of the report is always right, and since it is always right, then also this means that $P(s_1|i_2) = 0$ and $P(s_2|i_1) = 0$. This simplifies the tree and the value of the node with additional information becomes:

$$EVPI = P(s_1) * \max(v_1, v_3) + P(s_2) * \max(v_2, v_4).$$

Since this is the maximum value of that node provided that we had perfect a priori information, we can compare the *EMV* that we obtain solving the report to the *EVPI* to see how close we are to perfect information. This difference is known as the **Value of Perfect Information** (*PVI*):

$$VPI = EVPI - EMV.$$

MME8215: Operations Research

X. Decision Theory and Game Theory (II)

Contents

1 Game Theory - Introduction	1
1.1 Game examples	3
1.2 Game classification	3
1.3 Describing a game	4
2 Dominant Strategies	5
2.1 How to locate saddle points	7

1 Game Theory - Introduction

The student may have heard that game theory is a mathematical analysis of strategic behavior; and may know that it has found important applications in many fields of study such as warfare, politics, diplomacy, biology, and especially economics.

But what exactly is meant by "analysis of strategic behavior" and what does this have to do with games? The concept of a mathematical game is different from and more general than the usual notion of a parlor game, such as chess, Monopoly, poker, or ticktacktoe. A mathematical game is a special type of optimization problem.

In a game, the outcome depends not only on your decision but also on the decision(s) of other agents over which you have no control. Those other agents could be other people, but they also could be elements of chance or randomness. So you still attempt to make a decision that will lead to the best outcome for you, but you must do so while taking into account what others might do that could affect your outcome. When you make your decisions in a thoughtful way, taking into account the possible actions of others, you are said to be behaving strategically, and those are the situations of mathematical game theory.

Besides parlor games like chess and poker real life examples of games arise naturally. For example,

- in business, if you are deciding between several potential locations for your next franchise store, the eventual success of your new store will at least partly depend on where your competitors open their next stores.
- In warfare, if you are sending supplies and/or reinforcements to your troops located on a remote island, you must choose which supply route to take. Your enemy can potentially thwart your plans – will they attack this route or that? You do not know in advance, but

nevertheless, you must try to anticipate what they might do and what the outcome might be if they do attack. In other words, you must think and act strategically.

- In the next presidential election, if there are three candidates, should you vote for your favorite candidate, who is a long shot, or should you vote for your second-favorite candidate who has a much better chance, just to help make sure the candidate you like the least is not elected? The success of either strategy depends partly on how others vote. Thus, voting can be regarded as a strategic game if there are three or more candidates.

A mathematical game consists of a **set of players** (players can be individuals or any collection of individuals such as teams, corporations, armies, countries, etc.), each with a **collection of actions** open to them, called **strategies**. Once the players select their strategies, the play of the game will result in an **outcome**. We stipulate that each outcome of the game results in a **payoff** for each of the players, which we assume can be measured numerically.

For example, the payoff might be binary in nature – a player is either a winner or a loser, like in ticktacktoe or chess. In this case, we might designate the payoff of 1 to indicate a winner and 0 to indicate a loser. In other games, the payoff might be an integer, such as the number of “points” earned. In some situations the payoff is not easily measured numerically (e.g. what is the payoff for a battle between two armies? number of casualties, difference in casualties?).

Utility theory is that part of game theory that deals with how a player values a particular outcome – the utility of the outcome. It may be quite difficult to assign a specific utility of an outcome. To make matters worse, the same outcome with the same payoff might be valued differently by different people, so the players in a game may not even agree on the utility of an outcome.

To illustrate the difference between payoff and utility, imagine an estate being divided at the reading of a will. One of the heirs might inherit an \$80,000 Rolls Royce, while his brother may inherit \$60,000 cash. Even though the payoff for the car is \$80,000, the person who inherits the car may actually value the \$60,000 cash more – perhaps he doesn’t drive or doesn’t look forward to the bother and expense of maintaining the car. So the utility of the car may actually be less than \$60,000 to the person who inherited it.

We will greatly simplify our treatment of games if we make the following assumption:

Each outcome of a game has a numerical payoff (or value or utility), which is known to all the players before the game starts and upon which all players agree.

We need to make some assumptions:

- all the players have the same information about the game.
- The players will always try to act in a way that will lead to the best possible payoff for themselves.
- Players in a game are skilful, and will act rationally (that is, they will always strive to obtain the best possible payoff for themselves).

We will focus on single-move games (e.g. chess or tick-tack-toe) because they are the easiest to analyse. Thus we make the assumptions:

- *The play of the game consists of each player selecting one of the strategies open to them; then the outcome is reached.*
- *The players will make their selection of strategies simultaneously and independently.*

1.1 Game examples

Example 1. *Two-Finger Morra.* Two players, Ann and Bill, face each other with their right hands hidden behind their backs. On the count of three, they move the hidden hand to the space between them, revealing either one or two fingers extended. If the total number of fingers revealed is odd, Ann pays Bill \$1. If the total number of fingers revealed is even, Bill pays Ann \$1.

Example 2. *Rock Paper Scissors.* Ann and Bill play the well-known game of rock paper scissors, following the usual conventions that rock beats scissors, scissors beat paper, and paper beats rock. They decide to follow the payoff scheme that the loser pays the winner \$1, but there is no exchange of money if the result is a tie.

Example 3. *Saturday Chores at the Anderson Household, (Version I).* Mr. and Mrs. Anderson have three children – Ian, Jon, and Laurie – who will help with certain chores. Mr. Anderson rewards them by paying a total of \$24: \$6 for washing the dishes, \$6 for cleaning the basement, and \$12 for raking the yard. The children can divide up the tasks any way they want, and they can even share tasks (and split the reward two or three ways). However, past experience shows that if the children discuss their preferences ahead of time, they will just get into an argument. To avoid those arguments, Mr. Anderson has the kids write down their job choice or choices secretly on a piece of paper. The only rule is that all the tasks must be done, so if a job is not claimed by one or more of the children, then Mr. Anderson will assign the task as follows: it will be shared equally by all children who do not choose any task, and if all three have chosen something, then it will be shared equally by all three children.

Example 4. *Saturday Chores at the Anderson Household, (Version II).* Several years later, the Anderson children are all older. Ian is off at college, leaving Jon and Laurie to share the chores. However, nobody plays in the basement anymore, so there are only two tasks – raking and dishes. Mr. Anderson has modified the rewards as follows: He'll pay Laurie \$6 if she chooses the dishes, but he wants to encourage Jon to choose that task more often since he tends to avoid it, so if Jon does the dishes, he earns \$8. If they share the job, he pays \$7: \$4 to Jon and \$3 to Laurie. Mr. Anderson does not want to be unfair to Laurie, so he has a similar asymmetry built into the payoffs for raking in Laurie's favor. He'll pay Jon \$12 to do the job himself, and he'll pay Laurie \$15 to do the job herself. If they share the job, he pays \$13: \$7 to Laurie and \$6 to Jon. The rest of the rules are the same: any unclaimed task is assigned to a child who has claimed nothing, or is split between them if both children or neither have claimed a task.

1.2 Game classification

Games are classified into:

- *two-player games* vs. *n-player games* (where n is 3 or more).
- Considering the total payoffs, *zero-sum games* in which the sum of the payoffs for both players is always 0.
- A minor variation of a zero-sum game is a *constant-sum game*, which means that the sum of the payoffs to the players is a constant
- By the game size, i.e. the number of strategies open to each player.

Two person, zero-sum games are the easiest to analyze.

1.3 Describing a game

Let us now narrow our attention to two-person games, and consider two ways to describe and display the important information of the games in a convenient format. The first way is via *tree diagrams* (also known as a *game tree*).

For example, consider the two-finger morra game described in example 1. To represent it select a player (it does not matter which one), and label a node of the tree with his or her name. Draw branches down from this node corresponding to and labelled with the strategies for that player. At the end of each of those branches, put a node labelled with the name of the other player. From each of those nodes, draw further branches down corresponding to and labelled with the second player's strategies. Since the game ends after each of the two players has selected a strategy (that is, a branch of the tree diagram), the ends of these branches are terminal nodes corresponding to the outcomes of the game. We can then label each outcome with its payoff. Figure 1 shows the game representation as a tree.

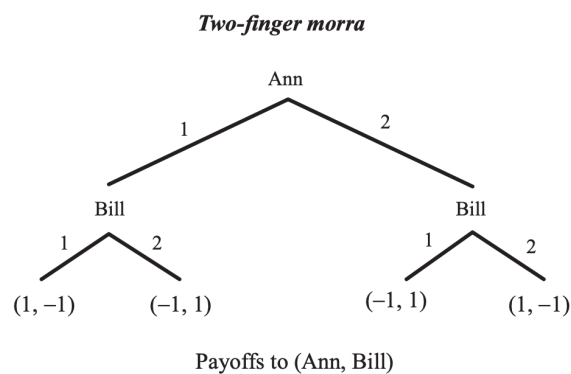


Figure 1: Game tree for two-finger morra.

Since we are assuming that the players make their selections simultaneously it does not matter which player is at the root of the tree. But if we drop the assumption of simultaneous play and look at games where the players move in sequence, then it will be important to draw the game tree with the first player to move at the top, etc.

When a two-person game is depicted by using the game tree, the game is said to be in *extensive form*.

We can make a table of the payoffs so that the rows of the table correspond to Ann's strategies and the columns to Bill's strategies, the following 2×2 matrix, A , called the *payoff matrix* of the game, results.

		Bill	
		1 finger	2 fingers
Ann	1 finger	(1, -1)	(-1, 1)
	2 fingers	(-1, 1)	(1, -1)

Ann, who is called the row player, chooses a row (her strategies), and Bill, the column player, chooses a column (his strategies). The position of the matrix where the row and column intersect is the outcome of the game (more precisely, it is the payoff associated with the outcome).

Since this is a zero-sum game, it suffices to know one player's payoffs (we can easily determine the other, so it is not necessary to use ordered pairs for the payoffs). By convention, we always

display the row-player's payoffs. Thus, for two-person zero-sum (or constant-sum) games, the payoff matrix is an ordinary matrix with numerical entries.

		Bill	
		1 finger	2 fingers
Ann	1 finger	1	-1
	2 fingers	-1	1

In fact, when it is understood which player is the row player and what the strategies are, we omit all the labels on the rows and columns and simply write the payoff matrix as the following:

$$A = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$$

If a game is not constant-sum, the row player's payoffs do not determine the column player's payoffs, so we must retain the ordered pair payoffs as the entries in the matrix. When a **two-person game** is **described** by giving its **payoff matrix**, the game is said to be in **normal form**.

Bellow is the normal form of the game described by example 4. Let Jon be the row player and Laurie the column player.

		Laurie			
		No chore	Dishes	Raking	Both
Jon	No chore	(10, 10)	(12, 6)	(8, 15)	(0, 21)
	Dishes	(8, 15)	(10, 10)	(8, 15)	(4, 18)
	Raking	(12, 6)	(12, 6)	(10, 10)	(6, 13)
	Both	(20, 0)	(16, 3)	(14, 7)	(10, 10)

2 Dominant Strategies

Now, let's see how we can solve a game.

Definition 1 In a two-person game, a strategy S **dominates** a strategy T if each payoff in S is at least as good as the corresponding payoff in T .

In a two-person *constant-sum game*, with payoff matrix A consisting of payoffs to the row player, then Column S dominates Column T if every entry in S is less than or equal to the corresponding entry in T – that is, if $a_{is} \leq a_{it}, \forall i$. On the other hand, row S dominates row T if every entry in S is greater than or equal to the corresponding entry in T – that is, if $a_{sj} \geq a_{tj} \forall j$. We say the **dominance is strict** if at least one of the inequalities is strict.

If you are a player in a two-person game with just two strategies, S and T , and S dominates T , you should clearly play S . If there are more than two strategies and S dominates T , it is not necessarily the case you should play S – you would need to know how S compares to these other strategies besides T . However, one thing is clear: you should not play T . You would always be at least as well off, if not better, by playing S . Therefore, **in any two-person game, rationality dictates that you should never play a dominated strategy**.

Definition 2 In any game (two-person, n -person, constant-sum or variable-sum), an out-come of the game is said to be a **Nash equilibrium** if it has the property that no player stands to gain by a unilateral change in strategy.

Considering the payoff matrix for the game lustrated by example 4, we observe that the largest entries for the first player are all in the bottom row, and the largest entries for the second player are all in the rightmost column. Thus, the strategy of volunteering for both chores dominates all three of the other strategies. So in this game, both players have a dominant strategy. Also, the outcome where these dominant strategies intersect is a Nash equilibrium, (10, 10).

Does this constitute a solution to this game? After all, a Nash equilibrium that results from playing two dominant strategies seems especially stable. For a zero-sum game, yes. As this example is a variable sum game, it can have some subtle dynamics. Variable-sum games are games of partial conflict and that it may be possible by partial cooperation to find outcomes in which both players improve their payoffs (the win-win situations that do not happen with constant-sum games).

In this case, there is no such improvement that is possible (under the assumptions of simultaneous, rational play with no communication between the players and in which the players truly want to maximize the money they make), because by inspecting the payoff matrix, we find no payoffs for both players larger than or equal to 10 simultaneously, except the other payoffs of (10, 10) that all lie on the main diagonal. Thus, each player can see that no further improvements over the (10, 10) payoff are possible for both players at once; any improvement for one player results in the loss for the other player (similar to what happens in a constant-sum game).

Therefore, in this game, it may seem to be safe to say that we have an acceptable solution to the game (although we will not formally discuss variable-sum games until a later chapter). Rational play dictates that both players should volunteer for both jobs, resulting in a payoff of \$10 for each of the children.

Actually, it may be possible to find other outcomes where the payoff to both players is more than \$10 if we use *mixed strategies*,

Definition 3 *The Solution of a two-person game consists of the optimal strategy for each player, as well as the value of the game.*

Definition 4 *Let A be the payoff matrix of a constant-sum game. A saddle point of A is an entry of A that is simultaneously the smallest entry in its row and the largest entry in its column.*

For example, consider the payoff matrix A below where a_{11} is a saddle point. The entry 2 there is the largest entry in its column and is (tied for) the smallest in its row.

$$A = \begin{bmatrix} 2 & 2 \\ 1 & 3 \end{bmatrix}$$

Definition 5 *In the payoff matrix A for a constant-sum game, an entry corresponds to a Nash equilibrium if and only if it is a saddle point.*

Any constant-sum game (in particular, any zero-sum game) in which the payoff matrix has a saddle point is said to be strictly determined.

In a strictly determined game, both players should choose the strategy that leads to the saddle point outcome. Thus, the solution to a strictly determined game is easy to find – choose a strategy containing the saddle point, and the value of the game (for the row player) is the value of the saddle point.

In a strictly determined (constant-sum) game, we can regard the value v of the game as a sort of guaranteed minimum payoff for each player. If the players play optimally, the result will be the saddle point outcome and each player will obtain the value guaranteed there (v for the row player and $c - v$ for the column player, if the payoffs sum to c .) However, if either player deviates from his or her optimal strategy, it may be possible for the other player to have an even better payoff.

2.1 How to locate saddle points

Let us describe an alternate method of locating saddle points. Since a saddle point is the smallest entry in its row, the first step is to record the minimum of each row. Also, since a saddle point is the highest entry in its column, the next step is to record the maximum in each column. Then find the maximum of the row minima (which is the best of the worst-case scenarios for the row player) and the minimum of the column maxima (the best of the worst-case scenarios for the column player. If these values are the same, then the outcome(s) where this occurs will be a saddle point (or saddle points). Suppose the payoff matrix is the following:

$$A = \begin{bmatrix} 1 & -2 & 0 & -1 \\ 3 & 4 & 3 & 6 \\ 5 & 1 & -1 & -4 \end{bmatrix}.$$

We record the row minima and column maxima:

$$A = \begin{array}{ccccc} & & & & \text{Row minima} \\ & \begin{bmatrix} 1 & -2 & 0 & -1 \\ 3 & 4 & 3 & 6 \\ 5 & 1 & -1 & -4 \end{bmatrix} & & & \begin{array}{c} -2 \\ 3 \\ -4 \end{array} \\ \text{Column maxima} & \begin{array}{c} 5 \\ 4 \\ 3 \\ 6 \end{array} & & & \end{array}$$

The largest row minima is 3 (second row), and the smallest column maxima is also 3 (third column), which indicates that the entry $a_{23} = 3$ is a saddle point, which is easily verified.

Consider the following 4×6 payoff matrix, which we may as well assume represents the payoffs in a zero-sum game:

$$A = \begin{bmatrix} 1 & \underline{-2} & 0 & -1 \\ \underline{3} & \underline{\underline{4}} & \underline{\underline{3}} & \underline{\underline{6}} \\ \underline{5} & 1 & -1 & \underline{-4} \end{bmatrix}$$

All four of the entries that are boxed are saddle points. This illustrates that a strictly determined game can have more than one saddle point (i.e., Nash equilibrium). The solution to this game is that the row player should choose either the second or the fourth row, the column player should choose either the second or fifth column, and regardless of these choices, the outcome is at one of the four saddle points. The value v of the game is the common value 1 of all the saddle points.

Theorem 1 *Any two Nash equilibria in a two-person constant-sum game are equivalent and interchangeable.*

MME8215: Operations Research

XI. Decision Theory and Game Theory (III)

Contents

1 Mixed-Strategy Constant-Sum Games	1
1.1 Mixed Strategies, Probabilities, and Expected Payoffs	2
2 Solving Mixed-Strategy games: the Minimax Theorem in the 2x2 case	4
2.1 Derived game	4

1 Mixed-Strategy Constant-Sum Games

Let us solve another strictly determined game that is infinite, the ideas behind it will be useful for solving 2×2 games that are not strictly determined.

Example 1. Antares vs. Bellatrix Antares Industries and Bellatrix, Inc., are the only two companies that sell a certain computer chip, the Pentangle 5.1. There is an estimated market for 400,000 of these chips in the year 2025. Each company must set its prices. Production costs dictate that the price must be at least 80\$ per chip, while industry regulations dictate that the price should be no more than 120\$ per chip. Let x be the price set by Antares and y the price set by Bellatrix. Based on reliable market studies of which both companies are aware, it has been determined that the percentage of the total market which goes to Antares depends on x and y and is reasonably approximated by the function given by

$$E(x, y) = 330 - 3x - 2.7y + 0.003xy.$$

What are the prices x and y that each company should choose, if their goal is to obtain the largest-possible market share?

This can be regarded as a two-person constant-sum game. The strategies for each company are the prices they set (x and y). The values of these variables must be in the real interval $[80, 120]$, so each player has an infinite number of strategies. Once x and y are chosen, then Antares receives a payoff of $E(x, y)$ while Bellatrix receives a payoff of $100 - E(x, y)$.

If the game were finite, we would construct the payoff matrix and search for a saddle point in the matrix. Since we have infinitely many choices for x and y , it does not make sense to construct a matrix, but it should be clear that the function $E(x, y)$ serves the exact same purpose as the payoff matrix - it gives, for each choice (x, y) , the payoff $E(x, y)$ to the row player Antares.

The graph of $E(x,y)$ is a surface, we are concerned with the part of the surface which is defined for $x, y \in [80, 120]$. The Nash equilibrium here is the same as a saddle point for a payoff matrix: a choice of (x, y) for which $E(x, y)$ is simultaneously a minimum in its “row” (being in a row means holding y fixed) and a maximum in its “column” (which means holding x fixed) will represent an outcome of the game where neither player stands to gain by a unilateral change in strategy. For this surface the saddle point is $(x, y) = (90, 100)$ and $E(90, 100) = 60$ for which $80 \leq x, y \leq 120$.

We can also find the revenue for each company, assuming a total market of 400000 chips, the revenue for Antares is $400000(\frac{E}{100})x\$$ and for Bellatrix is $400000(\frac{100-E}{100})y\$$.

1.1 Mixed Strategies, Probabilities, and Expected Payoffs

In a game without a saddle point knowing ahead of time what your opponent will do would affect your strategic decision. For example, in *rock paper scissors*, if you knew before the play that your opponent will choose rock, it is clear that you would then choose paper in order to win, etc. In a game like this, secrecy is important. To be successful in rock paper scissors (and other games without saddle points), one way to keep your opponent guessing about your intentions would be to play different strategies in different plays of the game. Sometimes play rock, other times paper, and other times scissors. This is what is meant by a **mixed strategy**, as opposed to a pure strategy, which means you would always play that strategy no matter how often the game is played.

The first task we face is to devise some notation for these mixed strategies. We do so for the row player by writing a row vector $p = (p_i)$, whose i th component p_i is the relative frequency with which the row player plays the i th row. Similarly, for the column player we write a column vector $q = (q_j)$, whose j th component q_j is the relative frequency with which the column player plays the j th column in repeated play of the game. For example, in rock paper scissors (where the strategies are listed in that order), we might have

$$p = [0.3, 0.2, 0.5], \quad q = \begin{bmatrix} 0.4 \\ 0.4 \\ 0.2 \end{bmatrix},$$

which can be interpreted as the mixed strategies where the row player plays rock 30% of the time, paper 20% of the time, and scissors 50% of the time, and the column player would play rock and paper 40% of the time each and scissors the remaining 20% of the time.

The entries in a mixed strategy are nonnegative fractions whose sum is 1, since the percentages must all sum to 100%. Also note that a pure strategy can also be written in this form – for example, the unit vector $p = (1, 0, 0)$ is the pure strategy of “always play rock.” We view the entries of the mixed strategies as *probabilities*.

Example 2.

Consider the following payoff matrix for a zero-sum game. It does not matter what the actual game is; if you like, you can think of it as a version of two-finger morra, where we have modified the payoffs:

$$A = \begin{array}{cc} & \begin{array}{cc} S & T \end{array} \\ \begin{array}{c} S \\ T \end{array} & \begin{bmatrix} 2 & -2 \\ -4 & 7 \end{bmatrix} \end{array}.$$

It is easy to check that this game is not strictly determined – there is no saddle point. Under repeated play, using mixed strategies, we’d like to compute the expected payoffs for each player.

Suppose that the row player decides to use the mixed strategy $p = [0.2, 0.8]$, and the column player decides to use the mixed strategy $q = \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix}$. If we think of each play of the game as a probability experiment, we can compute the expected payoff for the row player as follows. First, observe that there are four possible outcomes – SS , ST , TS , and TT – depending on what strategies each player chooses in any given play. We already know that the row player chooses row S with probability 0.2 (based on the mixed strategy p), and the column player chooses column S with probability 0.6 (based on the mixed strategy q). So what is the probability of the outcome SS ? In order for SS to occur, both players must choose S : In other words, the outcome SS is the intersection of the two events “row chooses S ” and “column chooses S .” Can we compute the probability of this intersection?

If we have two independent events E and F , the probability that they both happen is:

$$P(E \cap F) = P(E) \cdot P(F).$$

The players choose their strategies simultaneously and independently. Therefore, we are justified in multiplying the individual probabilities to obtain the probability of the intersection: $P(SS) = (0.2) \cdot (0.6) = 0.12$. We can compute the probability of each of the four outcomes in the same manner. Also, the values v_i that go with the outcomes are precisely the entries of the payoff matrix. We are now prepared to compute the expected payoff to the row player. We construct a table in Table 1.

Table 1: Expected payoffs in a mixed-strategy game.

Outcome	p_i	v_i	$p_i v_i$
SS	$(0.2)(0.6) = 0.12$	2	$(0.2)(0.6)(2) = 0.24$
TS	$(0.8)(0.6) = 0.48$	−4	$(0.8)(0.6)(−4) = −1.92$
ST	$(0.2)(0.4) = 0.08$	−2	$(0.2)(0.4)(−2) = −0.16$
TT	$(0.8)(0.4) = 0.32$	7	$(0.8)(0.4)(7) = 2.24$

The expected payoff to the row player is

$$E = 0.24 + (−1.92) + (−0.16) + 2.24 = 0.4.$$

On average, the row player will win 0.4 per game, if the players stick to these strategies.

This payoff E depends on p and q , so we denote it $E(p, q)$. We see that the calculation for E is exactly the same as the matrix product:

$$E = E(p, q) = pAq = [0.2, 0.8] \begin{bmatrix} 2 & -2 \\ -4 & 7 \end{bmatrix} \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix} = 0.4.$$

The itemized calculation (this time, of expected payoff in a game) is made simple and automatic via matrix multiplication. If one repeats this exercise for any payoff matrix, regardless of size, we find, in general, that the same result holds.

Proposition 1 *Let A be the payoff matrix of any two-person, constant-sum game. Suppose the row player decides on the (pure or mixed) strategy p and the column player decides on the (pure or mixed) strategy q . Then, in repeated play, the expected payoff to the row player is the matrix product:*

$$E(p, q) = pAq.$$

2 Solving Mixed-Strategy games: the Minimax Theorem in the 2x2 case

Given a constant-sum game that is not strictly determined, is there an optimal way to play it, and if so, how do we find the optimal strategy?

We consider two cases – when we know what strategy our opponent is using and when we don't.

2.1 Derived game

We consider only 2x2 games in the case when we have no idea of what strategy our opponent might use. Suppose a zero-sum game has the payoff matrix:

$$A = \begin{bmatrix} 4 & -2 \\ -3 & 1 \end{bmatrix}.$$

This matrix has no saddle point. The row player, instead of simply considering the two rows as her strategies, thinks of a strategy as an ordered pair of fractions like $p = (0.2, 0.8)$, where we interpret this to mean she plays the first row with probability 0.2 and the second row with probability 0.8 on any one play of the game, and similarly for the column player. We call these ordered pairs mixed strategies. Furthermore, we have seen that once the row player has settled on a mixed strategy to use – say, $p = (0.2, 0.8)$ – and once the column player has also selected his strategy – say, $q = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}$ – then we can find the expected payoff to the row player as:

$$E(p, q) = pAq = [0.2 \ 0.8] \begin{bmatrix} 4 & -1 \\ -3 & 1 \end{bmatrix} \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix} = -0.6.$$

We have observed that different choices of p and q lead to different expected payoffs to the row player. We are now interested in the question of how each player should choose his or her mixed strategies, p and q , which would lead to the best-possible expected payoff for himself or herself.

Definition 1 *Let A be the payoff matrix for a two-person, zero-sum game. We consider a new game, called the derived game of A , to be the game where the (pure) strategies are precisely the mixed strategies p and q in the original game, and where the payoff $E(p, q)$ to the row player for a particular outcome of the derived game is given by the formula:*

$$E(p, q) = pAq.$$

Remark 1 *The definition of the derived game applies to matrices of any size. Moreover, although we have only stated the definition for zero-sum games, the concept extends to constant-sum or even variable-sum games without change.*

Remark 2 *Since the expected payoff to the column player is just the negative of $E(p, q)$, it follows that the derived game is zero-sum if A is. Similarly, if A represents a constant-sum game, then the derived game is also constant-sum (with the same constant), and finally, if A represents a variable-sum game, the derived game will also turn out to be variable-sum.*

Remark 3 *Since there are an infinite number of mixed strategies (that is, infinitely many different possible choices for p and also for q), the derived game is infinite, even though A is finite.*

Since the derived game is infinite, we cannot write down a payoff matrix. However, sometimes an infinite game can be solved. Let's look more carefully at a mixed strategy $p = (p_1, p_2)$. Since the p_i values are probabilities whose sum is 1, we can rewrite such a mixed strategy in a slightly simplified form, taking $p_1 = x$ and $p_2 = 1 - x$ as $p = (x, 1 - x)$, where $0 \leq x \leq 1$. Similarly, the mixed strategy $q = \begin{bmatrix} q_1 \\ q_2 \end{bmatrix}$ can be rewritten as $q = \begin{bmatrix} y \\ 1 - y \end{bmatrix}$, where $0 \leq y \leq 1$. So the expected payoff $E(p, q)$, in the case of a 2x2 matrix A , really only depends on the two parameters x and y . Let us abuse the notation a bit and call it $E(x, y)$ in this case. And

$$\begin{aligned} E(x, y) &= E(p, q) = pAq \\ &= \begin{bmatrix} x & 1-x \end{bmatrix} \begin{bmatrix} 4 & -2 \\ -3 & 1 \end{bmatrix} \begin{bmatrix} y \\ 1-y \end{bmatrix} \\ &= 10xy - 4y - 3x + 1. \end{aligned}$$

The function $E(x, y)$ is what replaces the payoff matrix, since the players choose x and y so as to make $E(x, y)$ as beneficial to themselves as they can. This function is a saddle surface, so the derived game is strictly determined.

If we write $E(x, y)$ in standard form:

$$\begin{aligned} E(x, y) &= 10xy - 4y - 3x + 1 \\ &= 10(xy - 0.3x - 0.4y) + 1 \\ &= 10(x - 0.4)(y - 0.3) - 0.2. \end{aligned}$$

We see that the saddle point is located at $(x, y) = (0.4, 0.3)$, and E has the value -0.2 there.

$x = 0.4$ means $p = [0.4 \ 0.6]$ and $y = 0.3$ means $q = \begin{bmatrix} 0.3 \\ 0.7 \end{bmatrix}$.

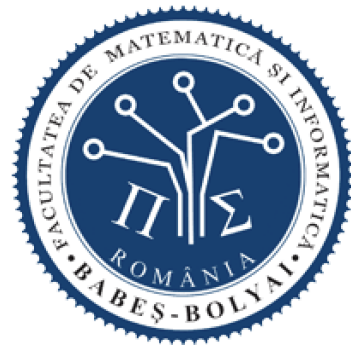
Since this represents a saddle point, it is a **Nash equilibrium** in the derived game. Thus, as long as both players play these strategies, the payoff to the row player is $v = -0.2$. As with any Nash equilibrium, if a player unilaterally deviates from the optimal strategy, it is possible for the payoff for the other player to improve (and, therefore, the first player's payoff would decrease since it is a constant-sum game).

Since the expected payoff $E(x, y)$ gives the long-term average payoff in the original, finite game, played with mixed strategies instead of pure strategies, these statements also hold for the original game. The choice of mixed strategies $\hat{p} = [0.4 \ 0.6]$ and $\hat{q} = \begin{bmatrix} 0.3 \\ 0.7 \end{bmatrix}$ will result in an average payoff of $v = -0.2$ per game to the row player in the long run. If either player deviates from this unilaterally, he or she cannot improve his or her payoff, so we think of $v = -0.2$ as the value of the game, and this solution represents a Nash equilibrium in the original game under repeated play. ($\hat{p}$ indicates an optimal strategy)

Theorem 1 *Let A be the payoff matrix for any 2×2 constant-sum game. If the game is strictly determined, then we already know the solution corresponds to a saddle point. If the game is not strictly determined, then the derived game of A is strictly determined. In particular, in either case, there are optimal strategies $\hat{p}$ and $\hat{q}$ and a real number v (called the value of the game) such that:*

1. $E(\hat{p}, q) \geq v$ for any q ,
2. $E(p, \hat{q}) \leq v$ for any p ,
3. $(\hat{p}, \hat{q}) = v$.

Remark 4 *It can be shown that if A is strictly determined, then $E(x, y)$ is still a saddle surface. However, the saddle point of this surface will be outside the domain $0 \leq x \leq 1$ and $0 \leq y \leq 1$, so the global saddle point of $E(x, y)$ does not correspond to a mixed strategy since x and y are not probabilities outside this domain. Nevertheless, one of the corners of the patch of $E(x, y)$ when $0 \leq x, y \leq 1$ still has the property that (1) – (3) hold. Since the corners correspond to $x = 0$ or 1 and $y = 0$ or 1 , and these correspond to pure strategies in A , we recover the saddle point solution to A in this case. Thus, the statement that the derived game of A is strictly determined always holds, whether or not A is strictly determined. We explore some aspects of this in the exercises.*



Operations Research

13 – Optimization algorithms

Mihai Suciu

Faculty of Mathematics and Computer Science

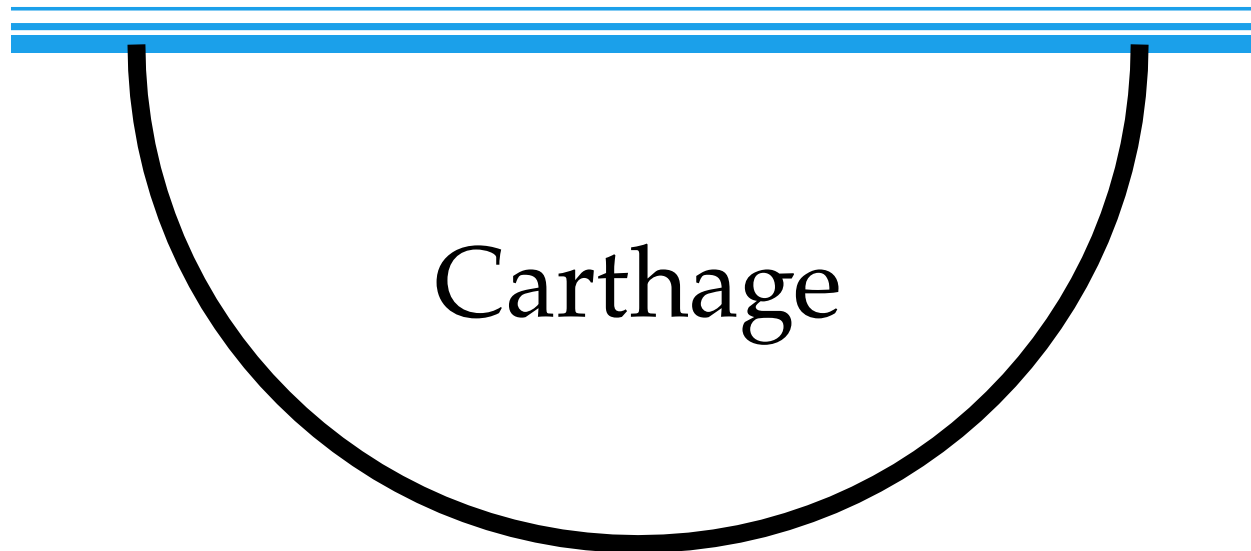
Introduction

- Applications of Optimization
 - Physics
 - Business
 - Biology
 - Engineering
- Objectives to Optimize
 - Efficiency
 - Safety
 - Accuracy

- Constraints
 - Cost
 - Weight
 - Structural Integrity
- Challenges
 - High-Dimensional Search Spaces
 - Multiple Competing Objectives
 - Model Uncertainty

A History

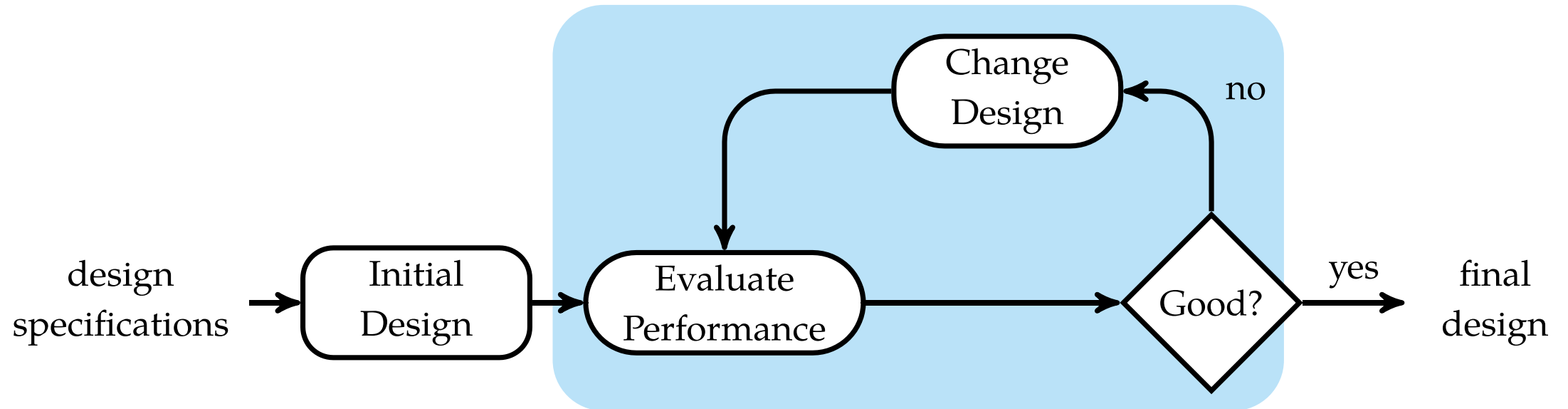
- Queen Dido's Optimization Problem
- sea



A History

- Calculus
- Numerical Algorithms
- Artificial Intelligence

Optimization Process



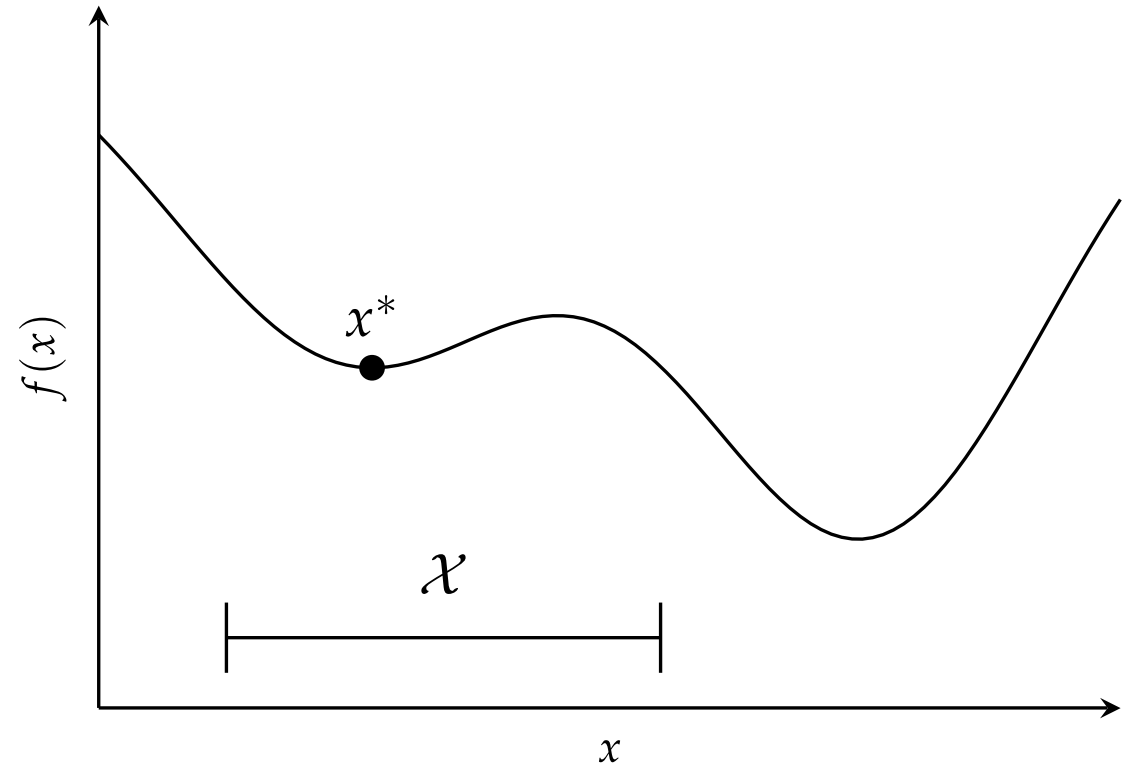
Basic Optimization Problem

$$\begin{array}{ll}\underset{\mathbf{x}}{\text{minimize}} & f(\mathbf{x}) \\ \text{subject to} & \mathbf{x} \in \mathcal{X}\end{array}$$

- Design Point
- Design Variables
- Objective Function
- Feasible Set
- Minimizer

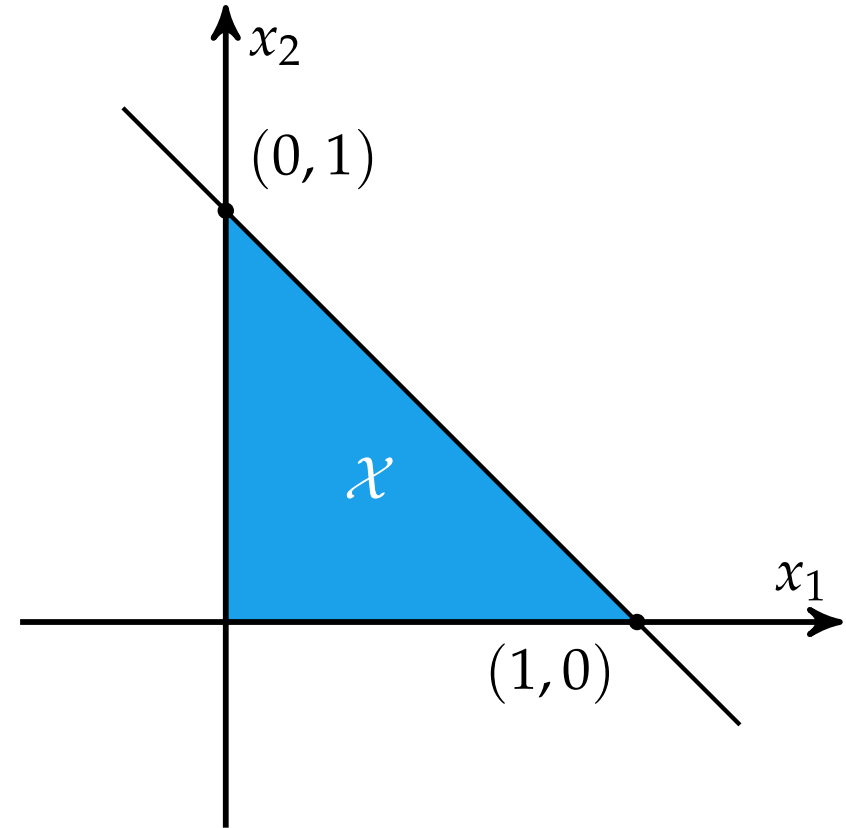
Basic Optimization Problem

$$\begin{array}{ll}\underset{\mathbf{x}}{\text{minimize}} & f(\mathbf{x}) \\ \text{subject to} & \mathbf{x} \in \mathcal{X}\end{array}$$



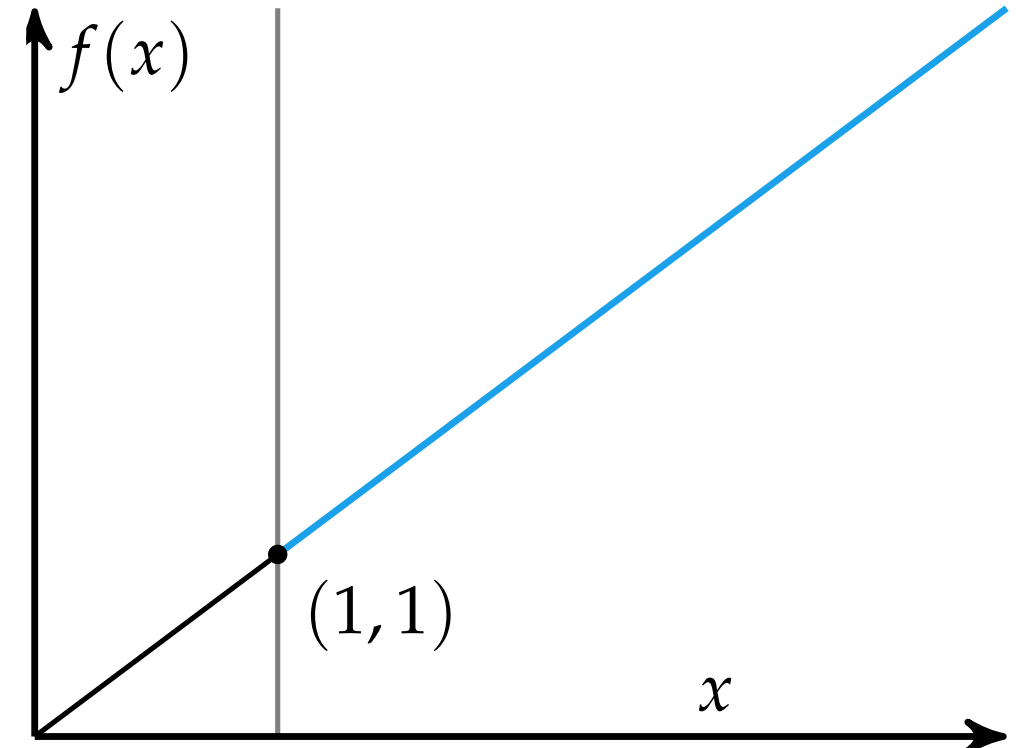
Constraints

$$\begin{array}{ll}\text{minimize} & f(x_1, x_2) \\ \text{subject to} & x_1 \geq 0 \\ & x_2 \geq 0 \\ & x_1 + x_2 \leq 1\end{array}$$



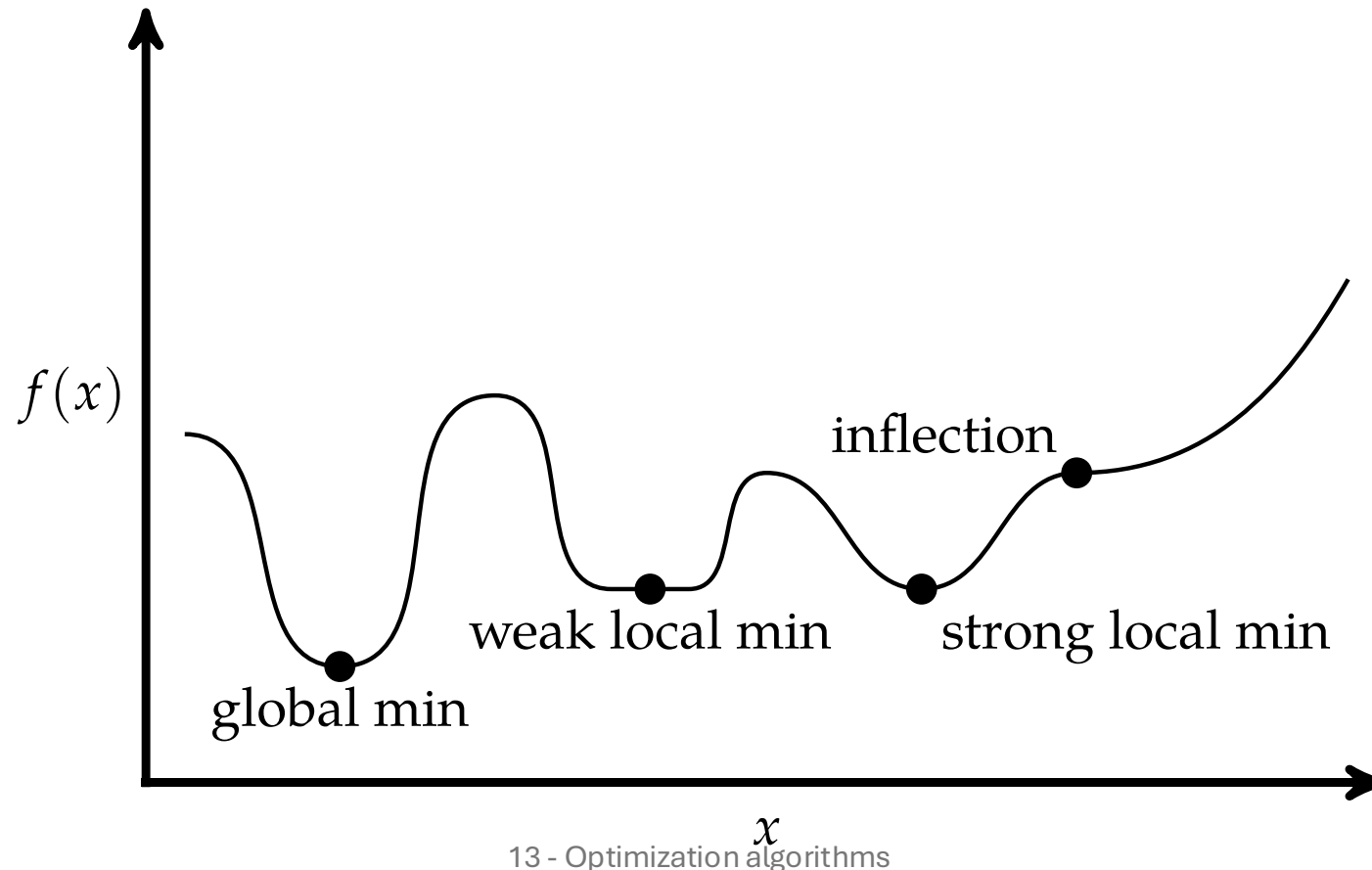
Constraints

$$\begin{array}{ll}\underset{x}{\text{minimize}} & x \\ \text{subject to} & x > 1\end{array}$$



Critical Points

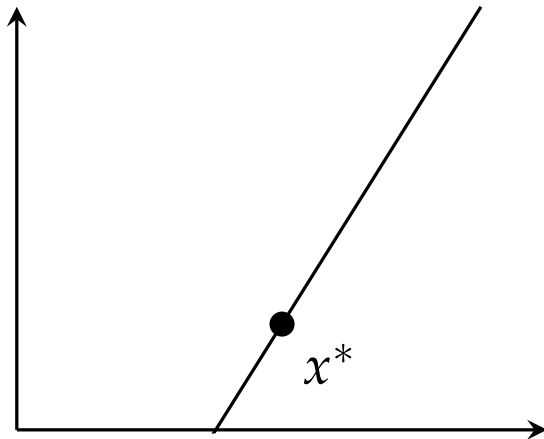
- Univariate Function



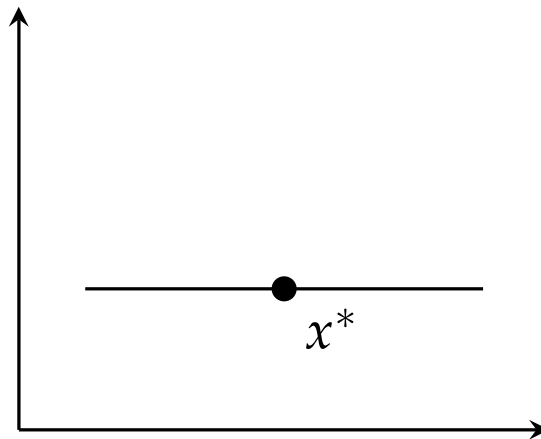
Conditions for Local Minima

- *Univariate Function*

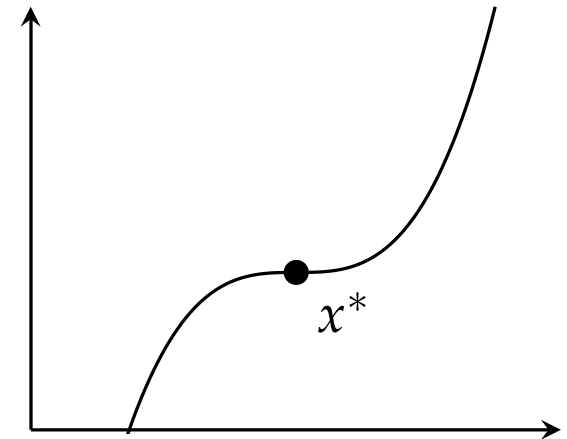
1. $f'(x^*) = 0$, the first-order necessary condition (FONC)
2. $f''(x^*) \geq 0$, the second-order necessary condition (SONC)



SONC but not FONC



FONC and SONC



FONC and SONC

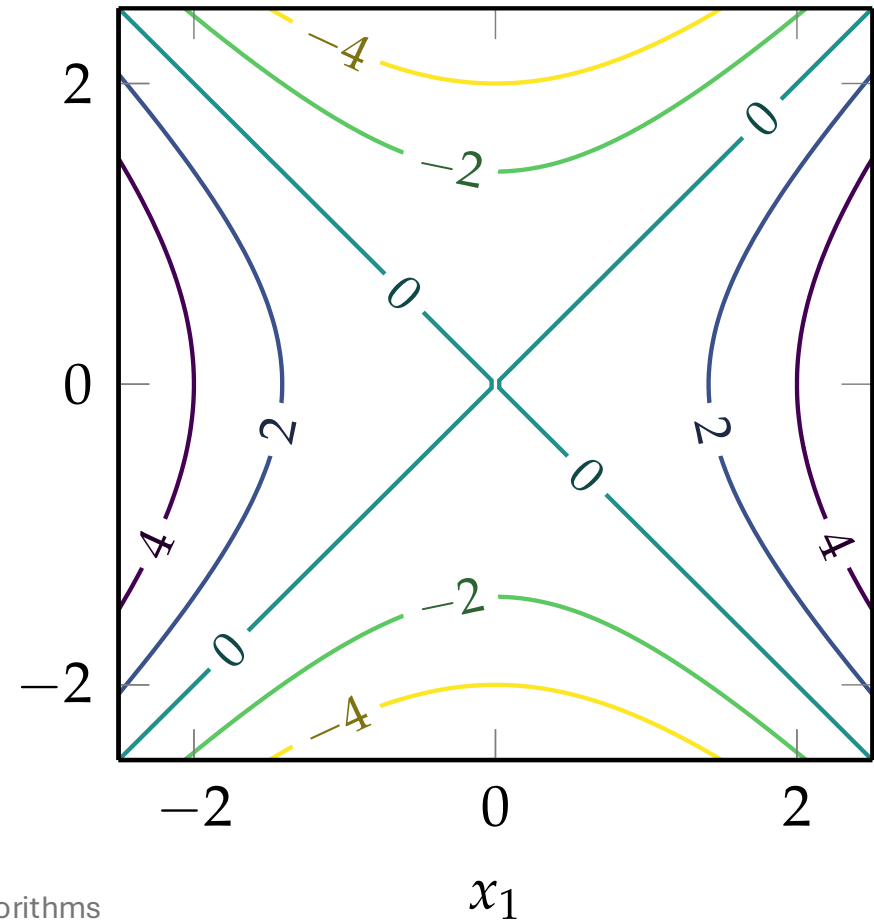
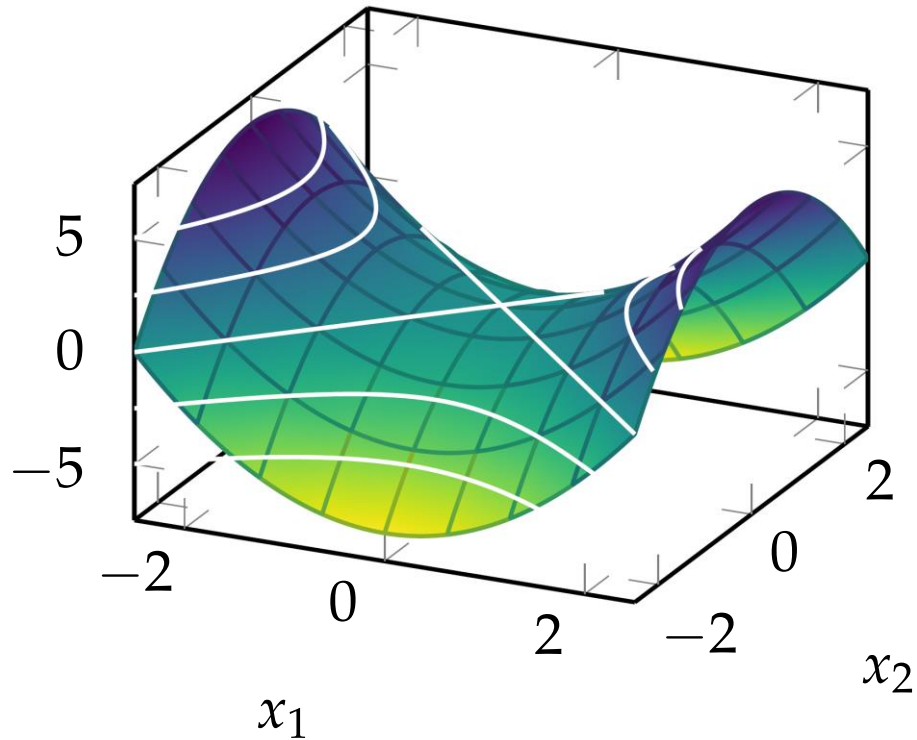
Conditions for Local Minima

- *Multivariate Functions*

1. $\nabla f(\mathbf{x}^*) = 0$, the first-order necessary condition (FONC)
2. $\nabla^2 f(\mathbf{x}^*) \geq 0$, the second-order necessary condition (SONC)

Contour Plots

- $f(x_1, x_2) = x_1^2 - x_2^2$



Population Methods

Population Methods

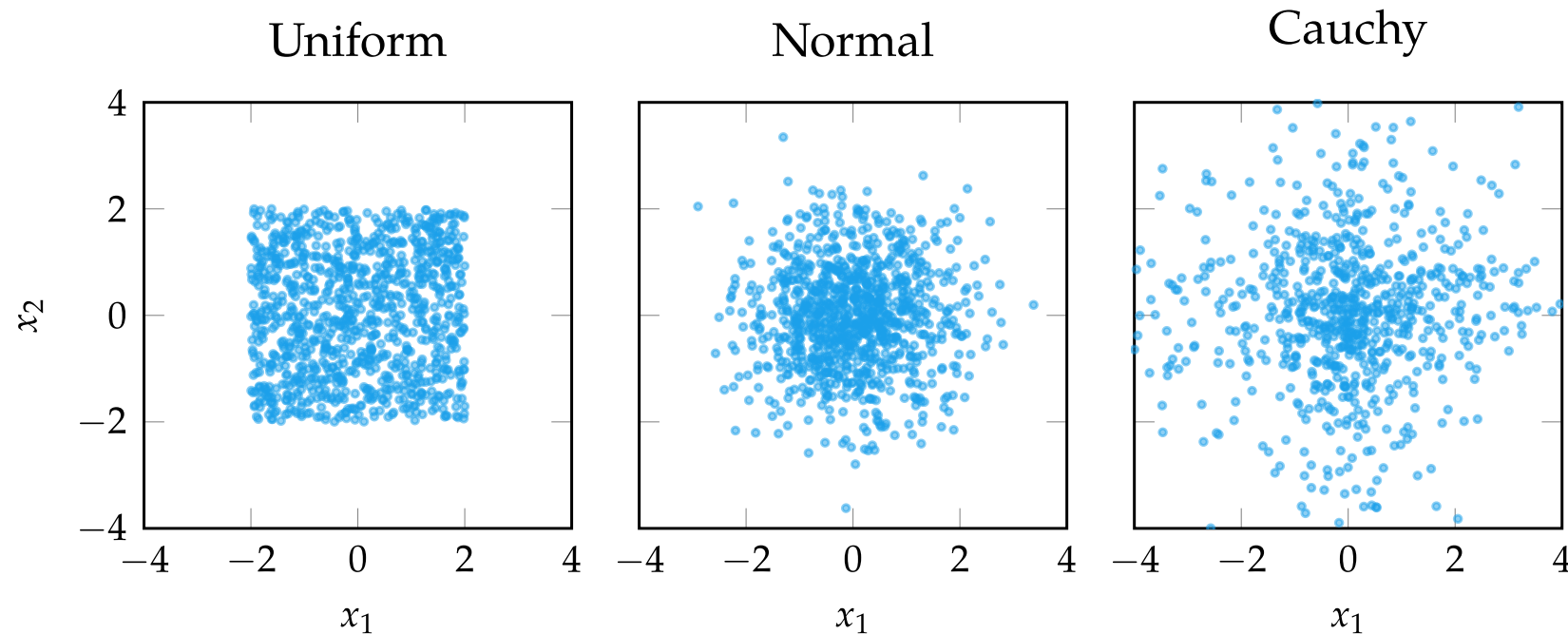
- Instead of optimizing a single design point, population methods optimize a collection of individuals
- Many individuals prevents algorithm from being stuck in a local minimum
- Useful information can be shared between individuals
- Stochastic in nature
- Easy to parallelize

```
abstract type PopulationMethod end

function population_method(M::PopulationMethod, f, designs, k_max)
    population = init!(M, f, designs)
    for k in 1:k_max
        population = step!(M, f, population)
    end
    return population
end
```

Initialization

- Population methods begin with an initial population
- Common initializations are uniform, normal distribution, and Cauchy distribution



Genetic Algorithms

- Inspired by biological evolution where the fittest individuals pass their genetic information to the next generation
- Individuals are interpreted as chromosomes
- The fittest individuals are determined by selection
- The next generation is formed by selecting the fittest individuals and performing crossover and mutation

```

struct GeneticAlgorithm <: PopulationMethod
    S # SelectionMethod
    C # CrossoverMethod
    U # MutationMethod
end

init!(M::GeneticAlgorithm, f, designs) = designs

function step!(M::GeneticAlgorithm, f, population)
    S, C, U = M.S, M.C, M.U
    parents = select(S, f.(population))
    children = [crossover(C,population[p[1]],population[p[2]])
                for p in parents]
    return [mutate(U, c) for c in children]
end

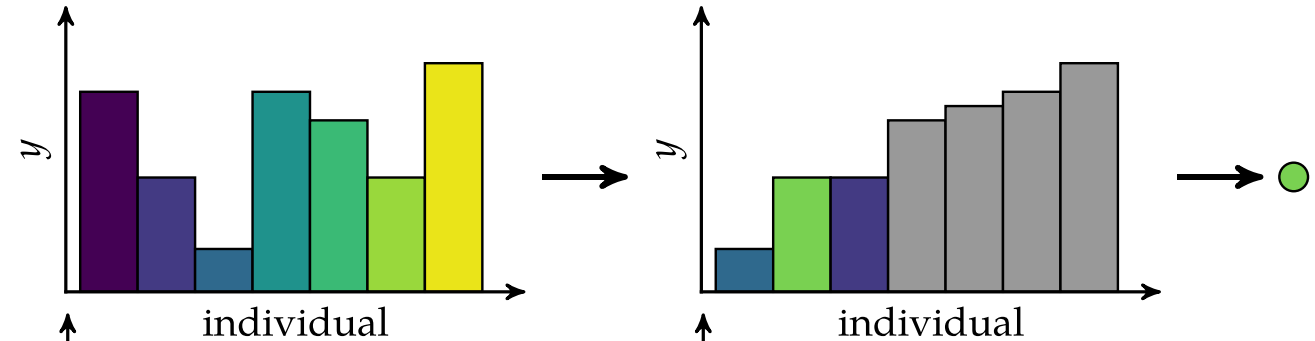
```

Genetic Algorithms: Selection

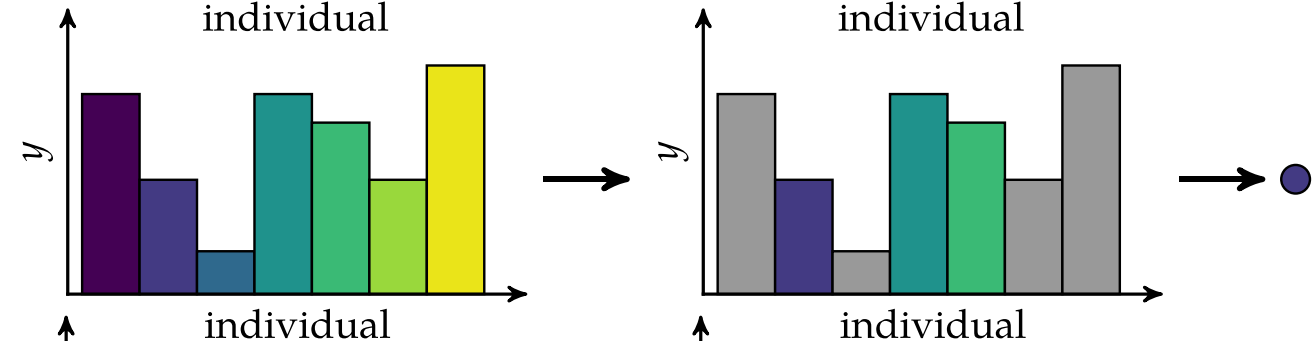
- Determining which individuals pass their genetic information on to the next generation
- Truncation selection: truncate the lowest performers
- Tournament selection: selects fittest out of k randomly chosen individuals
- Roulette Wheel selection: individuals are chosen with probability proportional to their fitness

Genetic Algorithms: Selection

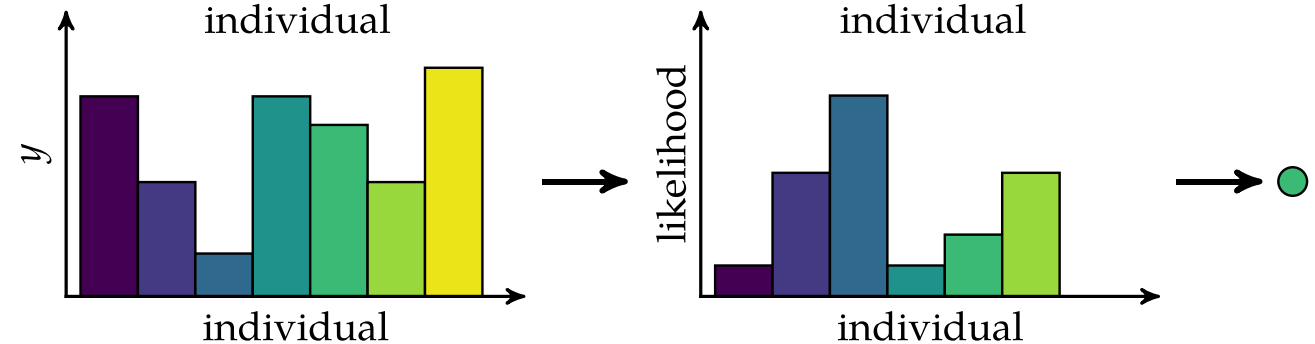
- Truncation Selection



- Tournament Selection



- Roulette Wheel Selection



```
abstract type SelectionMethod end
```

```
# Pick pairs randomly from top k parents  
struct TruncationSelection <: SelectionMethod  
    k # top k to keep  
end
```

```
function select(t::TruncationSelection, y)  
    p = sortperm(y)  
    return [p[rand(1:t.k, 2)] for i in y]  
end
```

```
# Pick parents by choosing best among random subsets  
struct TournamentSelection <: SelectionMethod  
    k # top k to keep  
end
```

```
function select(t::TournamentSelection, y)  
    getparent() = begin  
        p = randperm(length(y))  
        p[argmin(y[p[1:t.k]])]  
    end  
    return [[getparent(), getparent()] for i in y]  
end
```

```
# Sample parents proportionately to fitness  
struct RouletteWheelSelection <: SelectionMethod end
```

```
function select(::RouletteWheelSelection, y)  
    y = maximum(y) .- y  
    cat = Categorical(normalize(y, 1))  
    return [rand(cat, 2) for i in y]  
end
```

Genetic Algorithms: Chromosomes

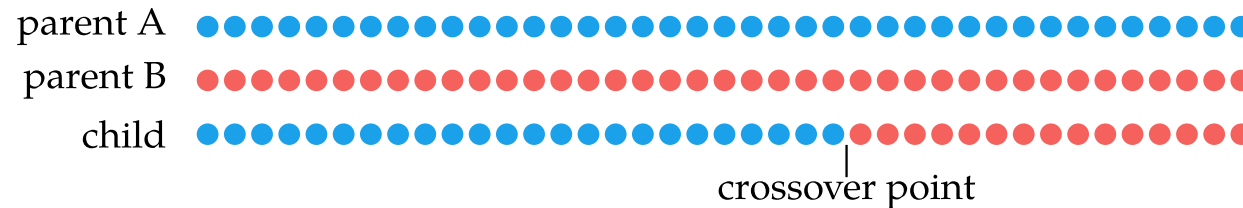
- Simplest representation is the binary string chromosome



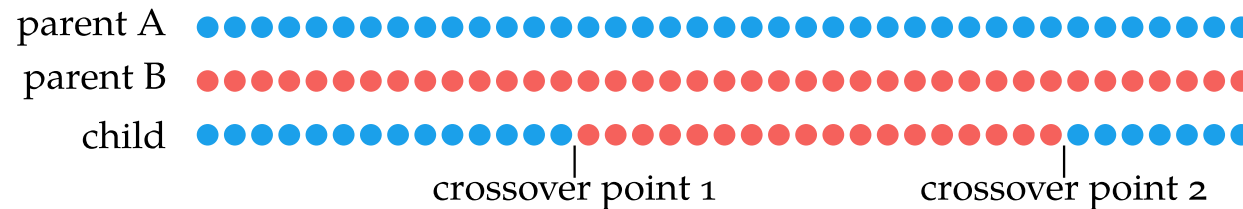
- Chromosomes are more commonly represented as real-valued chromosomes which are simply real-valued vectors
- Typically initialized randomly

Genetic Algorithms: Crossover

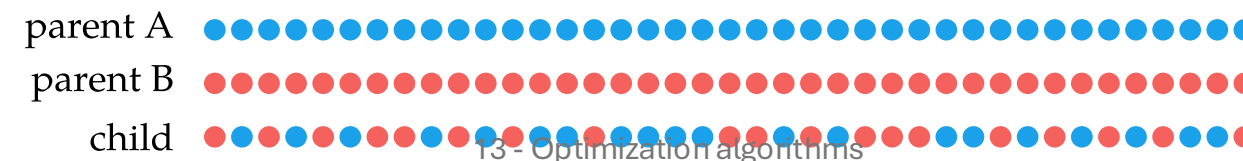
- Combines the chromosomes of the parents to form children
- Single-point crossover: swap occurs after single crossover point



- Two-point crossover: two crossover points



- Uniform crossover: each bit has 50% chance of crossover



```

abstract type CrossoverMethod end

struct SinglePointCrossover <: CrossoverMethod end

function crossover(::SinglePointCrossover, a, b)
    i = rand(eachindex(a))
    return [a[1:i]; b[i+1:end]]
end

struct TwoPointCrossover <: CrossoverMethod end

function crossover(::TwoPointCrossover, a, b)
    n = length(a)
    i, j = rand(1:n, 2)
    if i > j
        (i,j) = (j,i)
    end
    return [a[1:i]; b[i+1:j]; a[j+1:n]]
end

struct UniformCrossover <: CrossoverMethod
    p # crossover probability
end

function crossover(U::UniformCrossover, a, b)
    return [rand() > U.p ? u : v for (u,v) in zip(a,b)]
end

struct InterpolationCrossover <: CrossoverMethod
    λ # interpolant
end

crossover(C::InterpolationCrossover, a, b) = (1-C.λ)*a + C.λ*b

```


Genetic Algorithms: Mutation

- Mutation supports exploration of new areas of design space
- Each bit or real-valued element has a probability of being flipped or modified by noise
- The probability of an element mutating is called mutation rate

before mutation 

after mutation 

```

abstract type MutationMethod end

struct DistributionMutation <: MutationMethod
     $\lambda$  # mutation rate
    D # mutation distribution
end

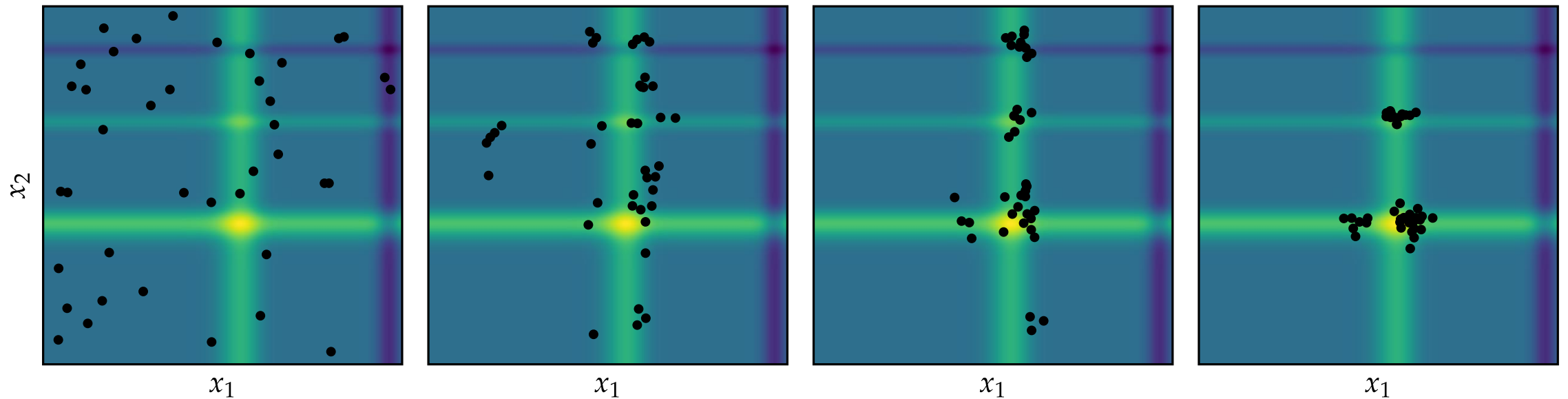
function mutate(M::DistributionMutation, child)
    return [rand() < M. $\lambda$  ? v + rand(M.D) : v for v in child]
end

GaussianMutation( $\sigma$ ) = DistributionMutation(1.0, Normal(0, $\sigma$ ))

```

Genetic Algorithms

- Genetic algorithm with truncation selection, single point crossover, and Gaussian mutation applied to Michalewicz function



```

import Random: seed!
import LinearAlgebra: norm
seed!(0) # set random seed for reproducible results
f = x→norm(x)
m = 100 # population size
k_max = 10 # number of iterations
function rand_population_uniform(m, a, b)
    d = length(a)
    return [a+rand(d).*(b-a) for i in 1:m]
end
population = rand_population_uniform(m, [-3,3], [3,3])
M = GeneticAlgorithm(
    TruncationSelection(10), # select top 10
    SinglePointCrossover(),
    GaussianMutation(0.5)) # perturb with 0.5 standard deviation
population = population_method(M, f, population, k_max)
x = argmin(f, population)
@show x

x = [-0.035070453820023516, 0.03762926957036672]

```

Differential Evolution

- Improves each individual $\mathbf{x}$ by recombining other individuals according to a simple formula
 1. Choose three random, distinct individuals $\mathbf{a}$, $\mathbf{b}$, and $\mathbf{c}$
 2. Construct interim design $\mathbf{z} = \mathbf{a} + w(\mathbf{b} - \mathbf{c})$
 3. Choose a random dimension to optimize in
 4. Construct candidate $\mathbf{x}'$ via binary crossover of $\mathbf{x}$ and $\mathbf{z}$
 5. Insert better design between $\mathbf{x}$ and $\mathbf{x}'$ into next generation

```

mutable struct DifferentialEvolution <: PopulationMethod
    p # crossover probability
    w # differential weight
end

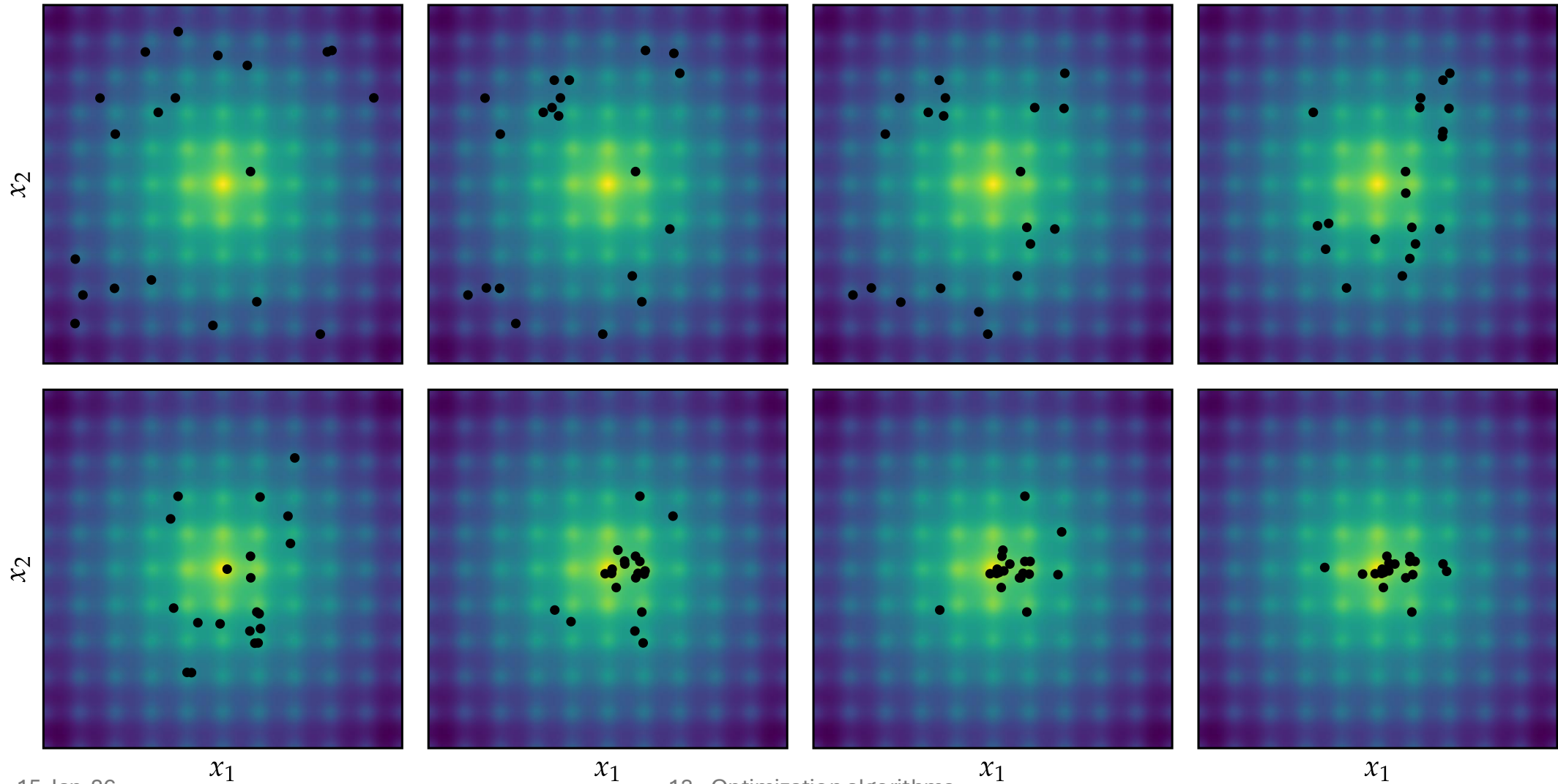
init!(M::DifferentialEvolution, f, designs) = designs

function step!(M::DifferentialEvolution, f, population)
    p, w = M.p, M.w
    n, m = length(population[1]), length(population)
    for x in population
        a, b, c = sample(population, 3, replace=false)
        z = a + w*(b-c)
        x' = crossover(UniformCrossover(p), x, z)
        if f(x') < f(x)
            x .= x'
        end
    end
end

return population
end

```

Differential Evolution



Particle Swarm Optimization

- Each individual, or particle, tracks the following
 - Current position
 - Current velocity
 - Best position seen so far by the particle
 - Best position seen so far by any particle
- At each iteration, these factors produce “force” and “momentum” effects to determine each particle’s movement

$$x^{(i)} \leftarrow x^{(i)} + v^{(i)}$$

$$v^{(i)} \leftarrow wv^{(i)} + c_1r_1\left(x_{\text{best}}^{(i)} - x^{(i)}\right) + c_2r_2\left(x_{\text{best}} - x^{(i)}\right)$$


```

mutable struct Particle
    x      # position
    v      # velocity
    x_best # best design thus far
end

mutable struct ParticleSwarm <: PopulationMethod
    w # inertia
    c1 # first momentum coefficient
    c2 # second momentum coefficient
    V # initial particle velocity distribution
    best # best overall design thus far, and its value
end

function init!(M::ParticleSwarm, f, designs)
    population = [Particle(x,rand(M.V),copy(x)) for x in designs]
    best = (x=copy(population[1].x), y=Inf)
    for P in population
        y = f(P.x)
        if y < best.y; best = (x=P.x, y=y); end
    end
    M.best = best
    return population
end

```

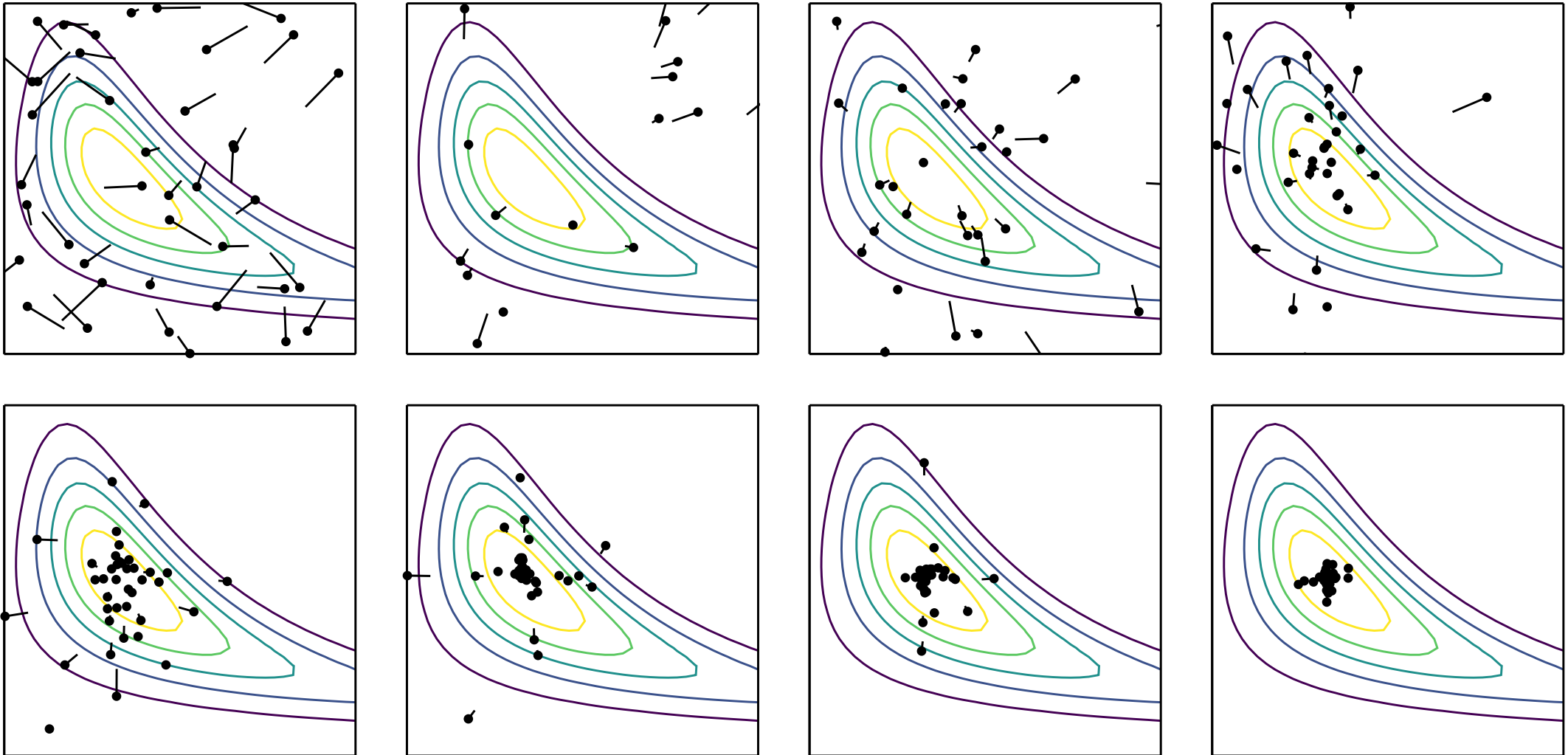
```

function step!(M::ParticleSwarm, f, population)
    w, c1, c2, best = M.w, M.c1, M.c2, M.best
    n = length(best.x)
    for P in population
        r1, r2 = rand(n), rand(n)
        P.x += P.v
        P.v = w*P.v + c1*r1.*(P.x_best - P.x) +
              c2*r2.*(best.x - P.x)

        y = f(P.x)
        if y < best.y; best = (x=copy(P.x), y=y); end
        if y < f(P.x_best); P.x_best .= P.x; end
    end
    M.best = best
    return population
end

```

Particle Swarm Optimization

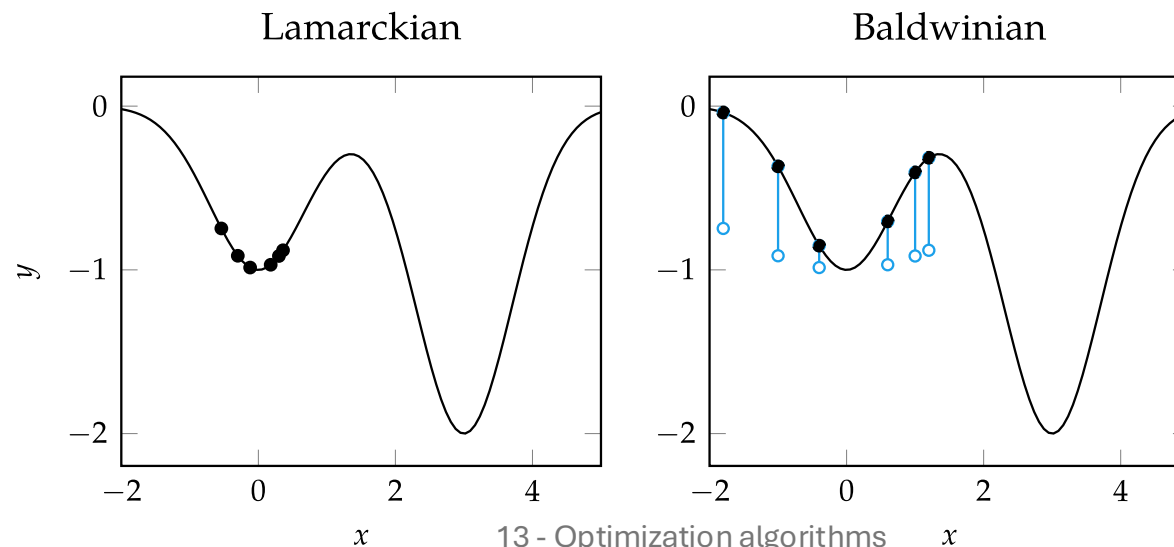


Hybrid Methods

- Generally, population methods are good at finding the best regions in design space, but do not perform as well as descent methods near the minimizer
- Hybrid methods try to leverage the strength of both methods
- Two hybrid approaches
 - Lamarckian learning
 - Baldwinian learning

Hybrid Methods

- Lamarckian learning
 - Performs regular descent method update on each individual
- Baldwinian learning
 - Uses value of descent method update to augment the objective value of each design point



Summary

- Population methods use a collection of individuals in the design space to guide progression toward an optimum
- Genetic algorithms leverage selection, crossover, and mutations to produce better subsequent generations
- Differential evolution, particle swarm optimization, the firefly algorithm, and cuckoo search include rules and mechanisms for attracting design points to the best individuals in the population while maintaining suitable state space exploration
- Population methods can be extended with local search approaches to improve convergence

Summary

- Population methods can be extended to produce individuals that span the Pareto frontier
- Knowing the preferences of experts between pairs of points in the criterion space can help guide the inference of a scalar-valued objective function

Multiobjective Optimization

Multiobjective Optimization

- Some optimization problems have multiple objectives that required tradeoff decisions

Pareto Optimality

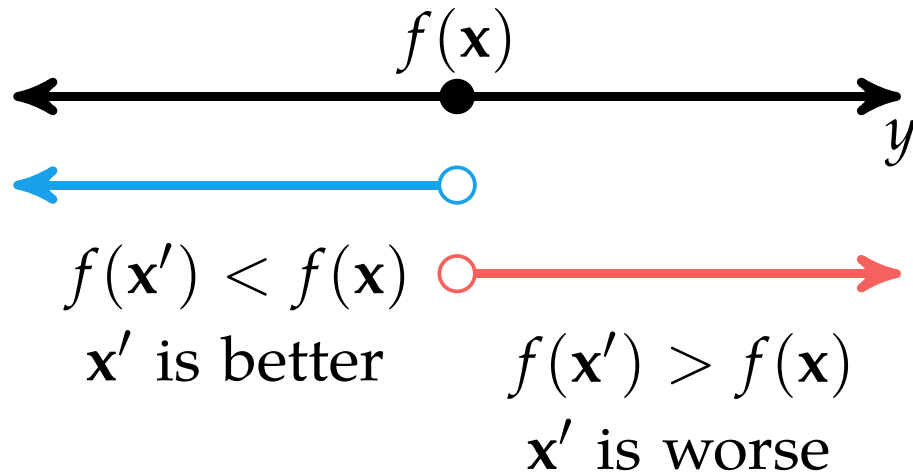
- At a Pareto optimal point, increasing one objective value decreases another
- In multiobjective optimization, one point dominates another if

$$f_i(\mathbf{x}) \leq f_i(\mathbf{x}') \text{ for all } i \text{ in } \{1, \dots, m\}$$

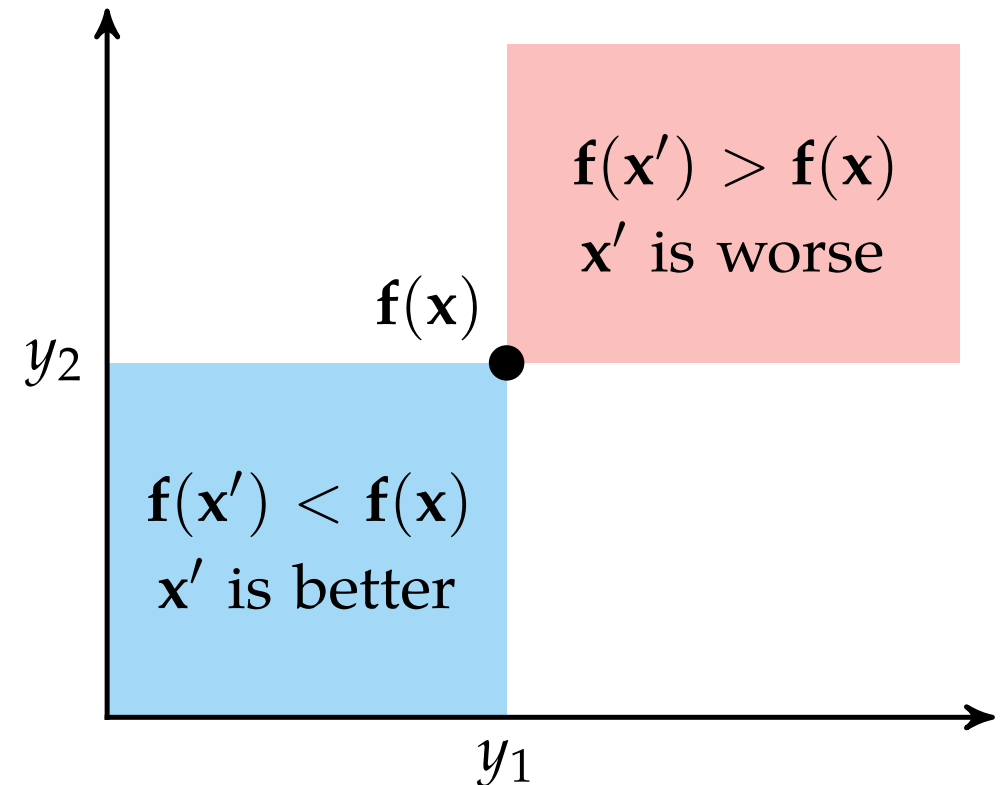
and $f_i(\mathbf{x}) < f_i(\mathbf{x}') \text{ for some } i$

Pareto Optimality

Single Objective

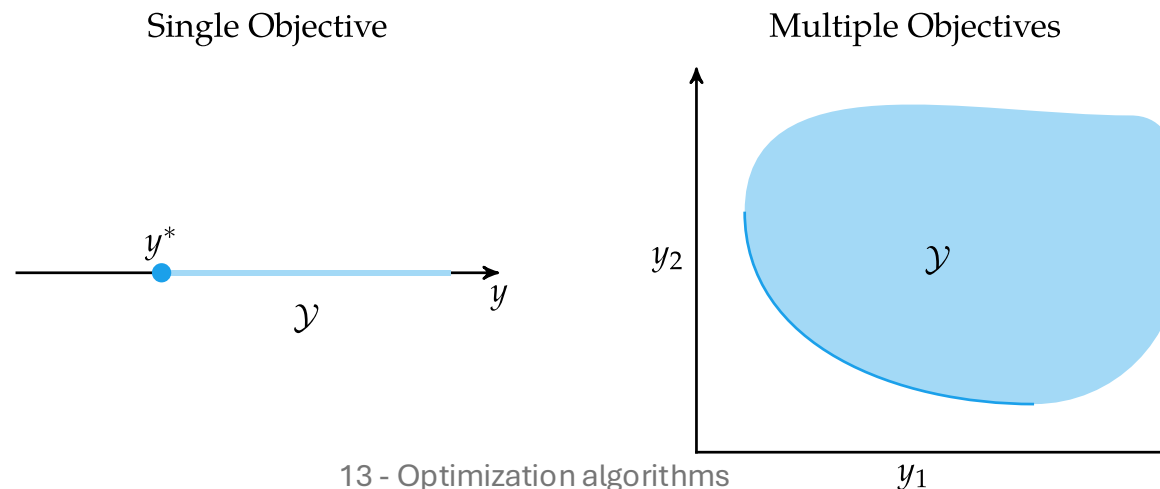


Multiple Objectives



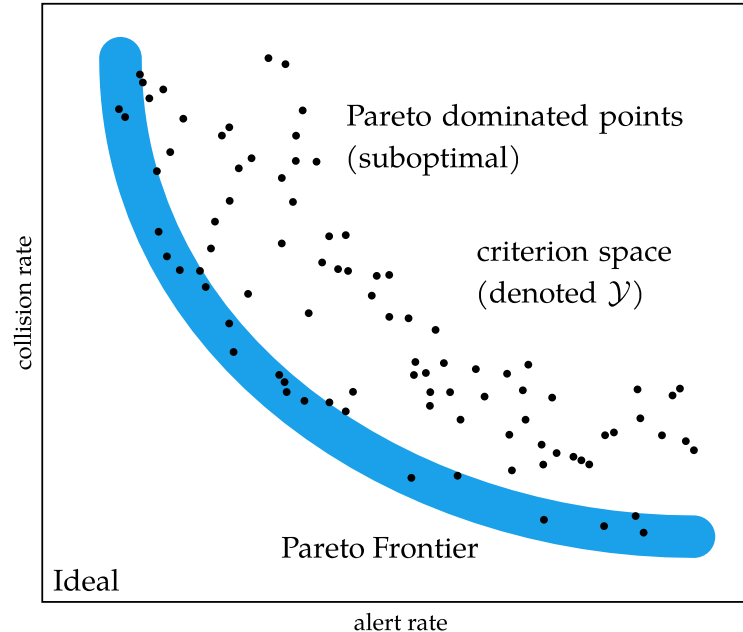
Pareto Optimality: Pareto Frontier

- A criterion space is a set of all outputs possible given a space of inputs
- In a multidimensional optimization problem, the criterion space is m-dimensional.
- In m-dimensional space, it is possible for there to be no clearly optimal point



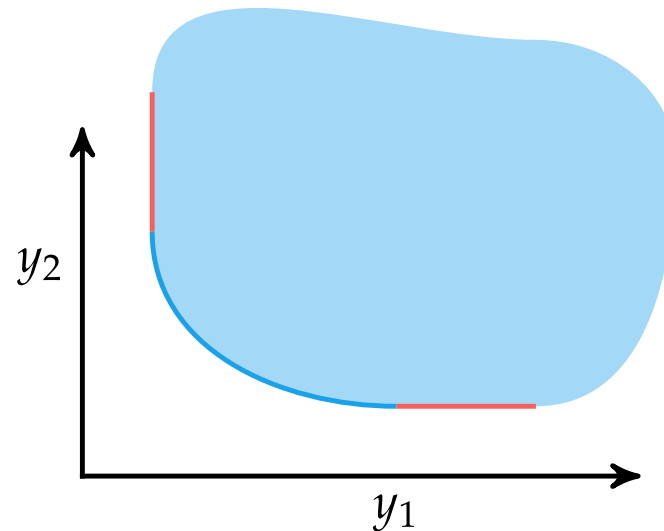
Pareto Optimality: Pareto Frontier

- A design point is considered Pareto-optimal if no other point in the criterion space dominates it
- The set of all Pareto-optimal points forms the Pareto frontier



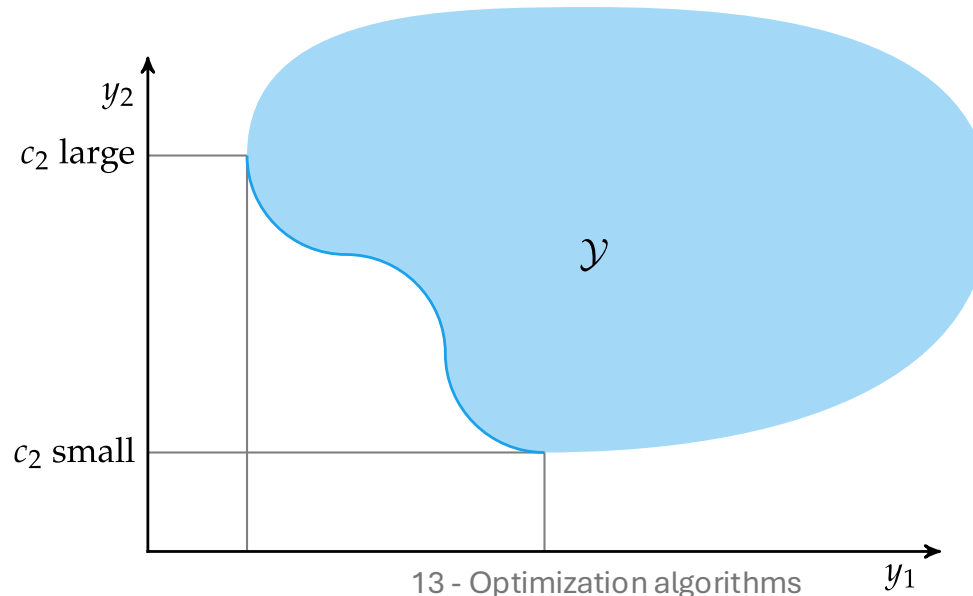
Pareto Optimality: Pareto Frontier

- The utopia point is the component-wise optimal point. It may not be in the actual criterion space
- Weakly Pareto-optimal points are those on the Pareto frontier that cannot be improved simultaneously in all objectives



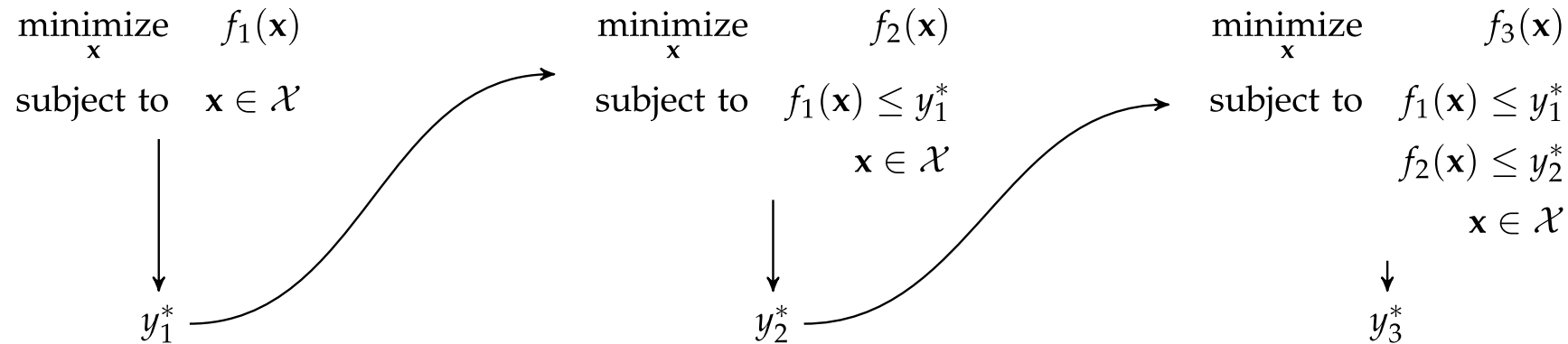
Constraint Methods

- There are various methods for generating the Pareto frontier
- The constraint method constrains all but one of the objectives to produce a unique optimal point in the criterion space
- The constraints are then modified to trace out the Pareto frontier



Constraint Methods

- The lexicographic method ranks the objectives in order of importance and performs a series of single-objective optimizations in order of importance
- Each optimization problem includes constraints to preserve the optimality of the previously optimized objectives



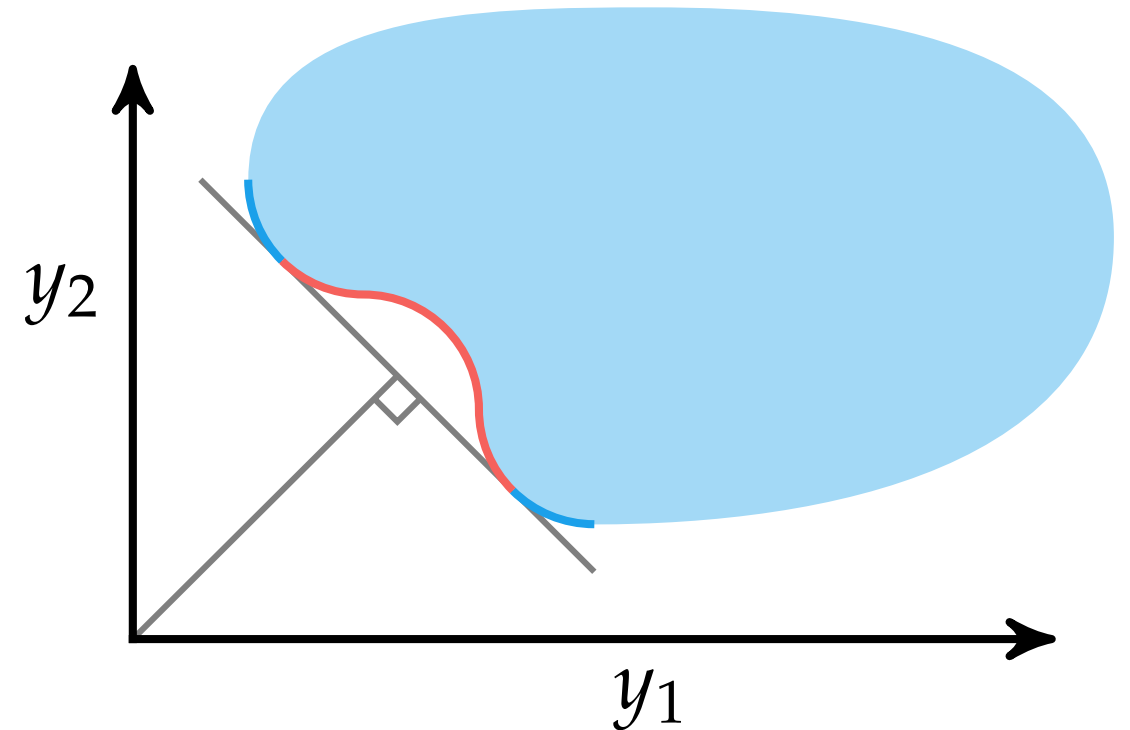
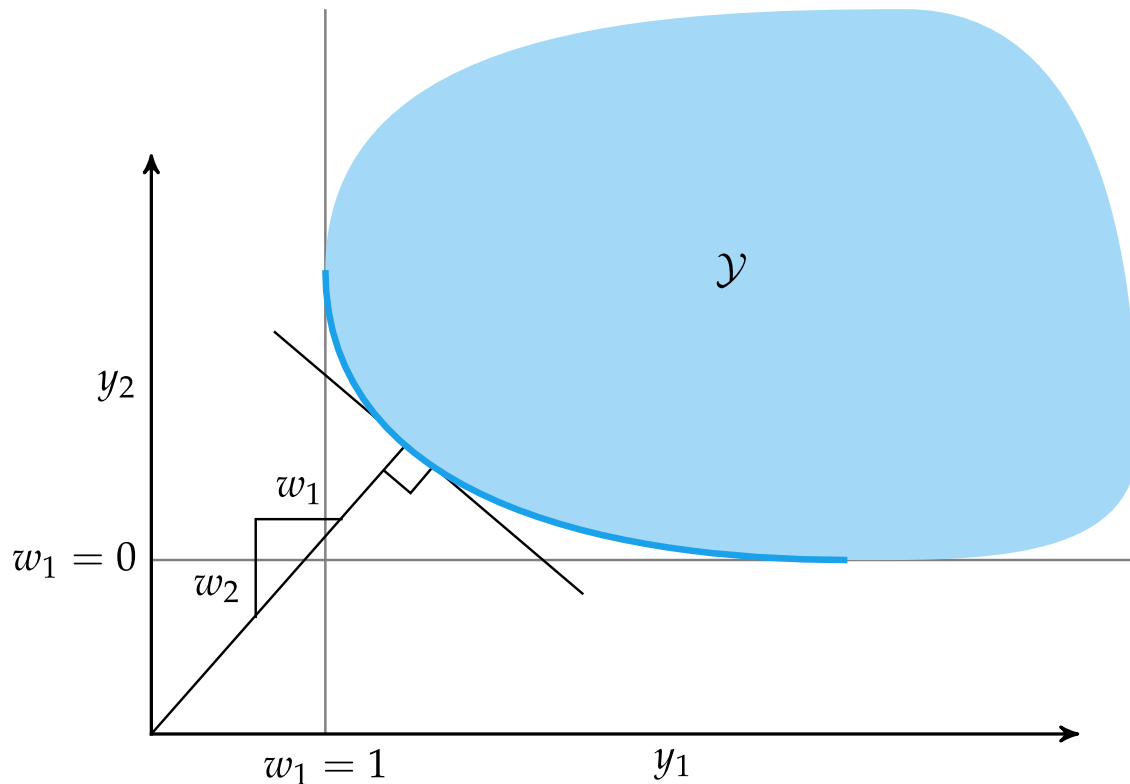
Weight Methods

- A designer can encode preferences over the objectives as a vector of weights
- The Pareto frontier can be generated by sweeping over the space of possible weights
- The weighted sum method combines multiple objective functions into one using a constant vector inner product

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{f}(\mathbf{x})$$

Weight Methods

- Note: weighted sum methods cannot obtain points in red



Weight Methods

- Goal programming converts multiple objectives into a single objective using a norm between $\mathbf{f}(\mathbf{x})$ and a goal point

$$\underset{\mathbf{x} \in \mathcal{X}}{\text{minimize}} \left\| \mathbf{f}(\mathbf{x}) - \mathbf{y}^{\text{goal}} \right\|_p$$

- Typically, the goal point is the utopia point

Weight Methods

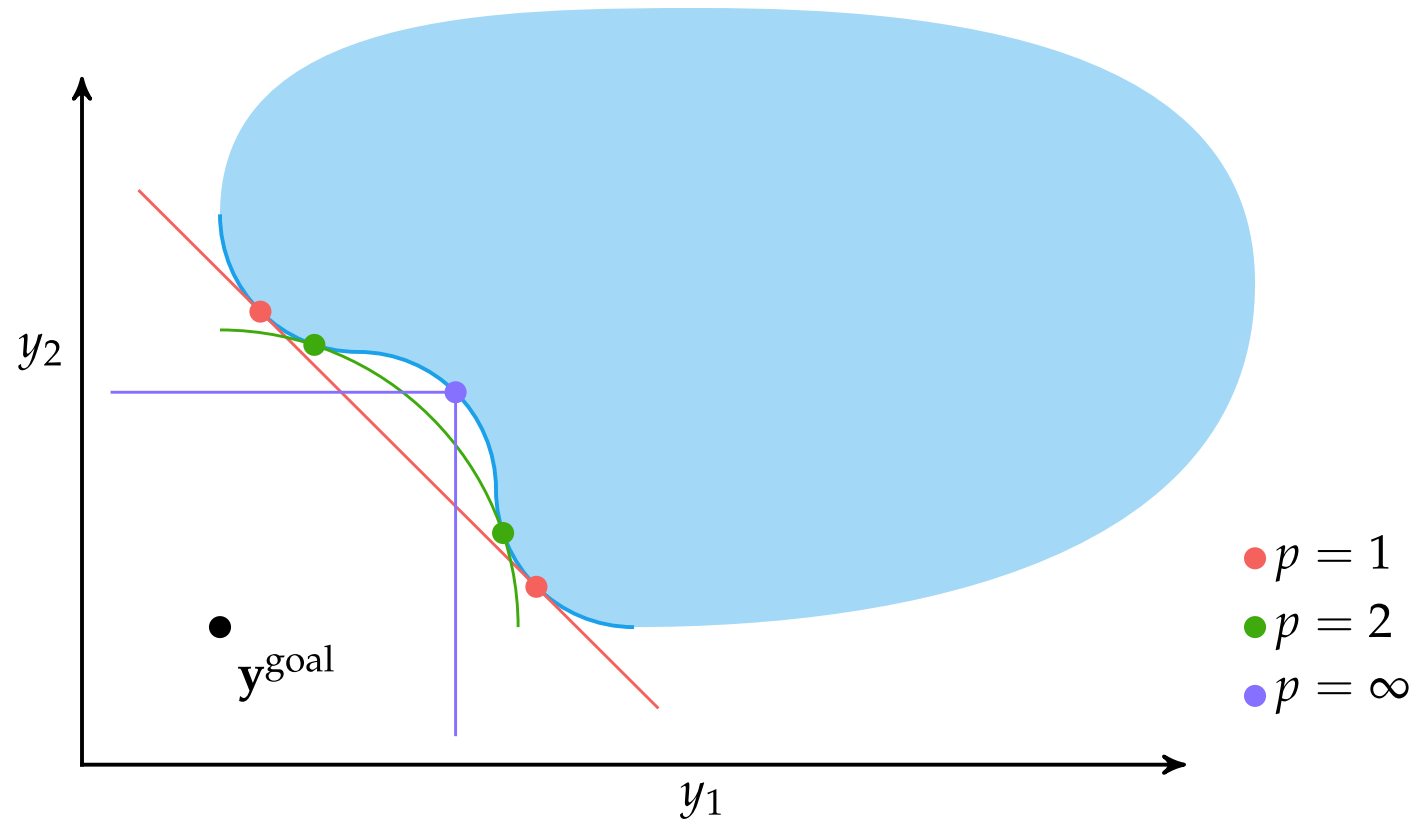
- Weighted exponential sum combines goal programming and the weighted sum method

$$f(\mathbf{x}) = \sum_{i=1}^m w_i \left(f_i(\mathbf{x}) - y_i^{\text{goal}} \right)^p$$

- Here, $\mathbf{w}$ is a vector of weights that sum to 1 and $p \geq 1$

Weight Methods

- Weighted exponential sum for different values of p



Weight Methods

- Previous methods cannot obtain points in nonconvex portions of the Pareto frontier
- The exponential weighted criterion can provide Pareto-optimal points in nonconvex regions
- Note: High values of p can lead to numerical overflow

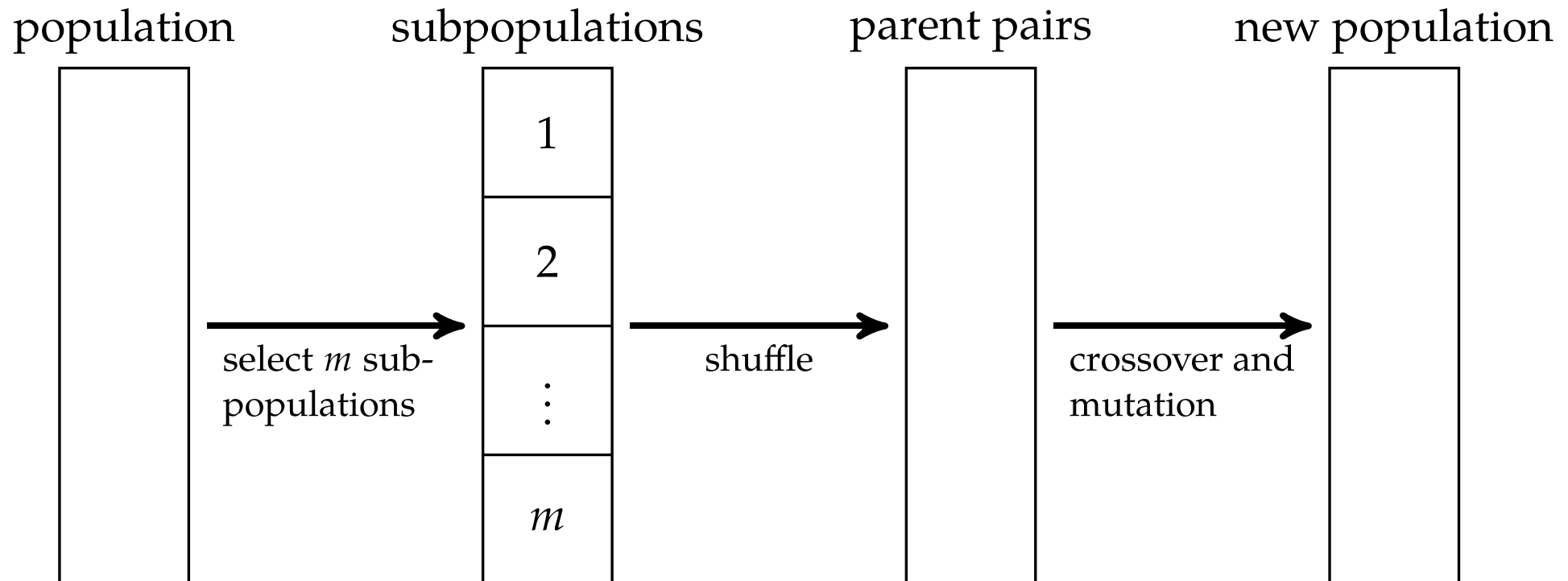
$$f(\mathbf{x}) = \sum_{i=1}^m (e^{pw_i} - 1) e^{pf_i(\mathbf{x})}$$

Multiobjective Population Methods

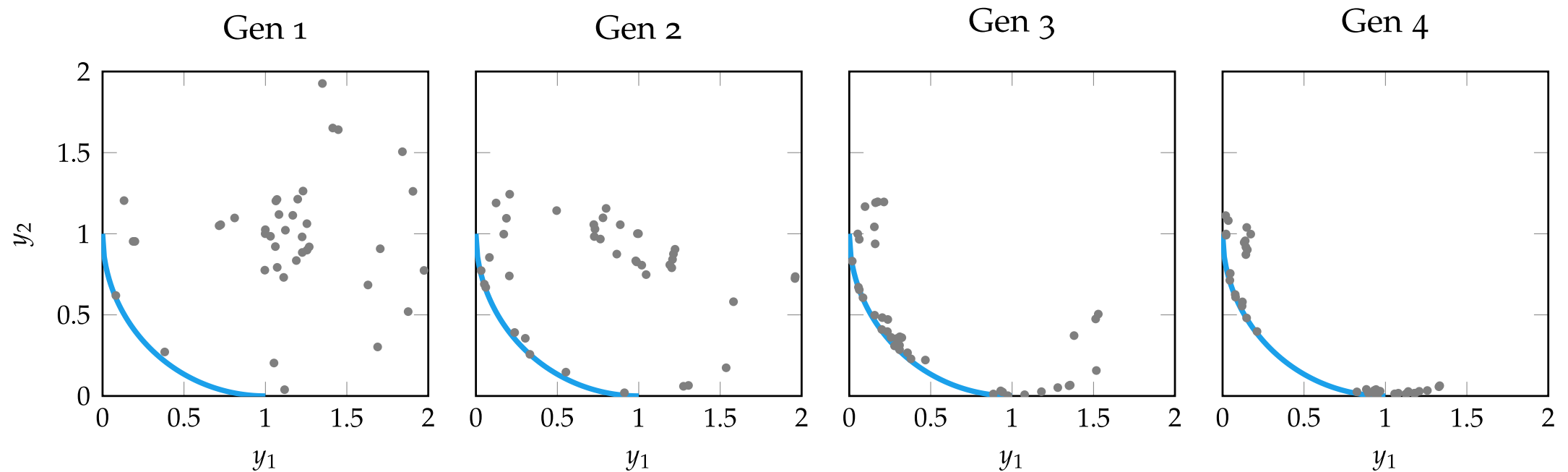
- A population is partitioned into several subpopulations
- Each subpopulation is optimized with respect to a different objective

Multiobjective Population Methods

- The vector evaluated genetic algorithm randomly partitions subpopulations at each generation



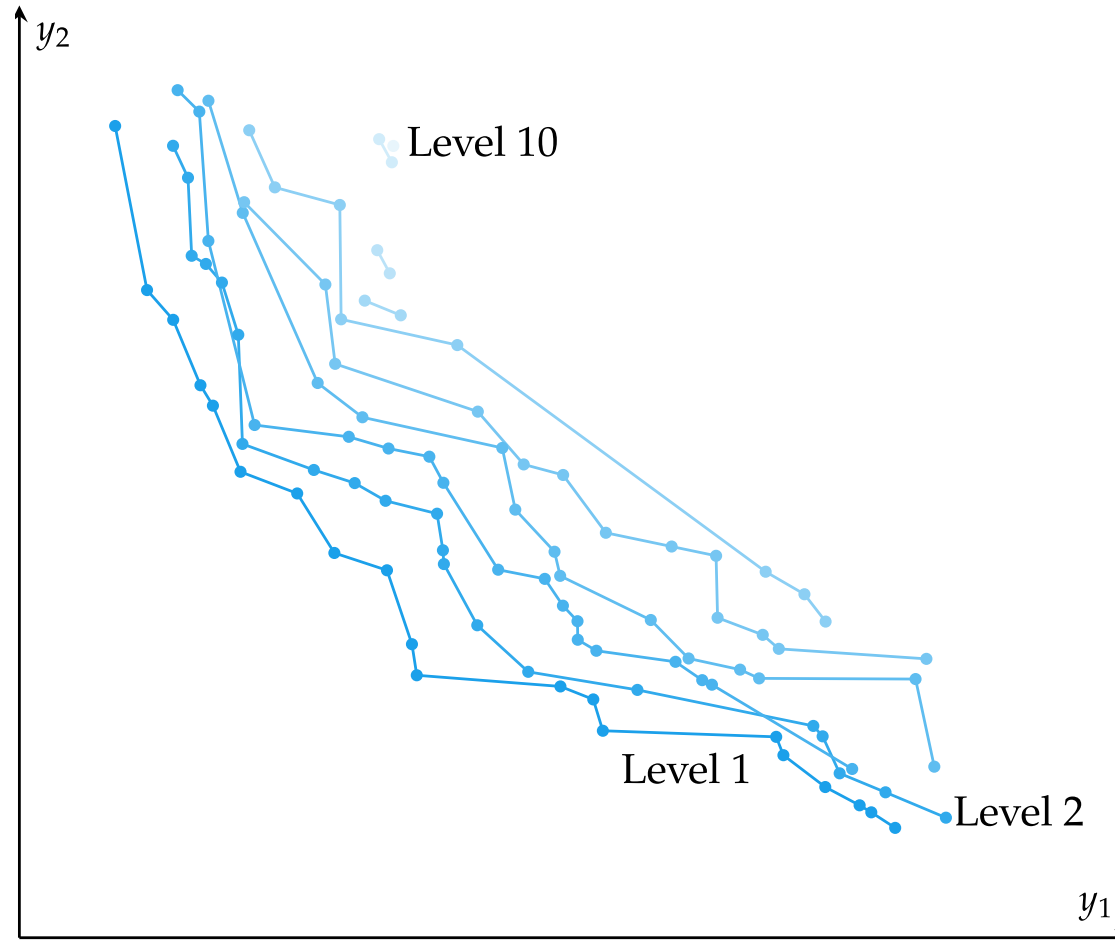
Multiobjective Population Methods



Multiobjective Population Methods

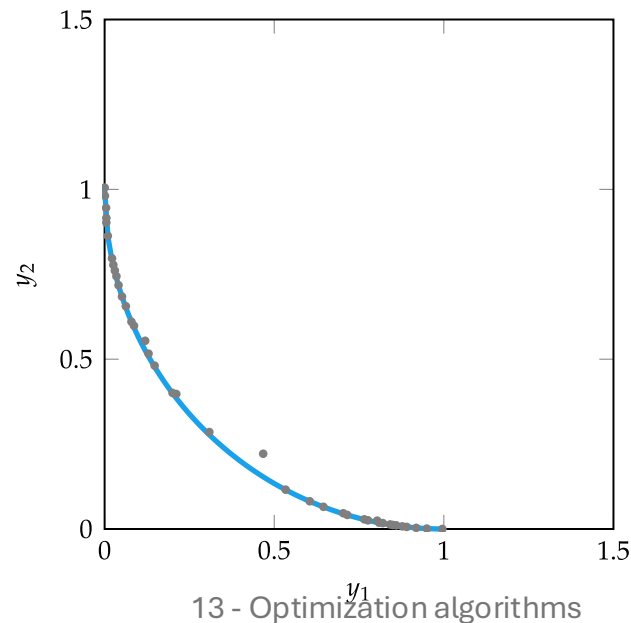
- Nondomination ranking is used to rank individuals in the population
 1. Nondominated individuals (Pareto-frontier)
 2. Nondominated by any except those in (1)
 3. Nondominated by and except those in (1) or (2)
- ...

Multiobjective Population Methods



Multiobjective Population Methods

- In multiobjective population methods, a population that approximates the Pareto frontier is called a Pareto filter
- At every generation, dominant individuals are retained and dominated individuals are removed

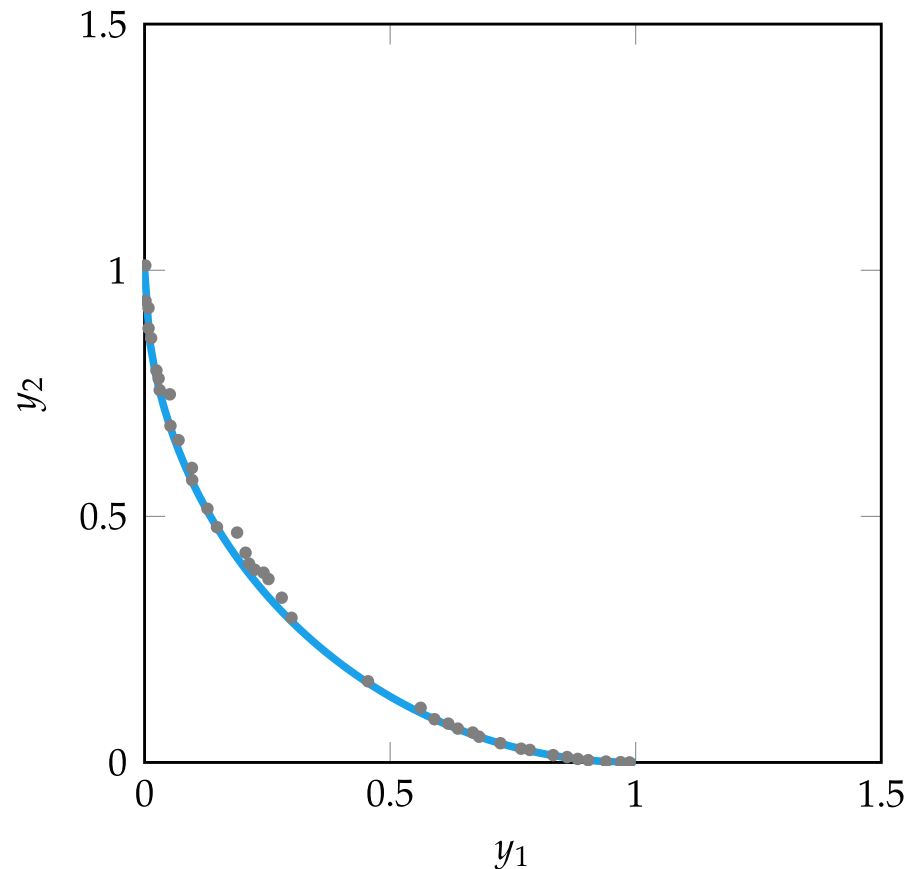


Multiobjective Population Methods

- A niche is a focused cluster of points, typically in the criterion space
- Niche techniques help encourage an even spread of points across the Pareto frontier
- In fitness sharing, the individual objective values are penalized if neighboring points are too close
- In equivalence class sharing, when two individuals are compared, their nondomination ranking is considered first, then fitness sharing is used to as a tie-breaker

Multiobjective Population Methods

- Improved coverage of Pareto frontier using fitness sharing



Summary

- Design problems with multiple objectives often involve trading performance between different objectives
- The Pareto frontier represents the set of potentially optimal solutions
- Vector-valued objective functions can be converted to scalar-valued objective functions using constraint-based or weight-based methods

Expression Optimization

Expression Optimization

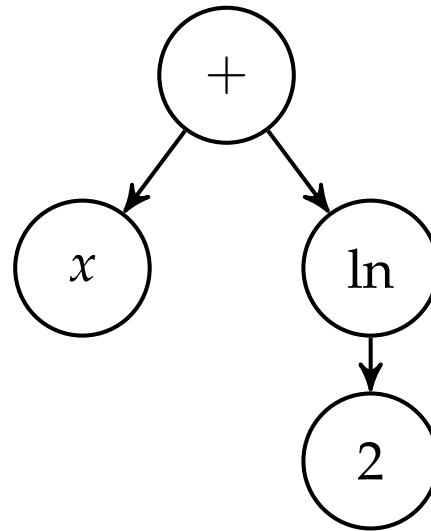
- In some problems, the number of variables is unknown
- Finding optimal solutions of problems with unknown parameters requires optimizing over the set of expressions

Grammars

- Expressions can be represented by a tree of symbols
- Grammars constrain the space of possible expressions
- A grammar is represented by a set of production rules
- Production rules involve symbols and types
- Types are defined sets of expression trees
- Production rules are used to form expression trees by expanding types into new expressions with more types and symbols
- If a type is expanded into just symbols, preventing further expansion, then it is called a terminal

Grammars

Representing the expression $x + \ln 2$ as a tree



Grammars

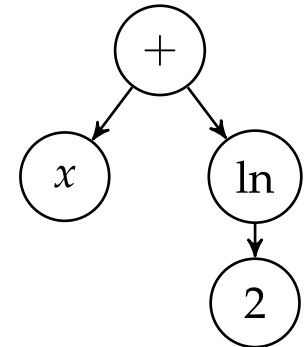
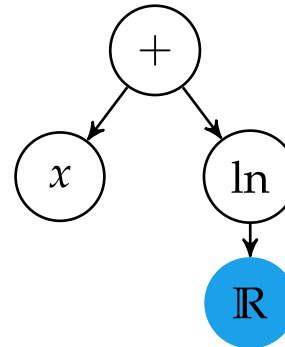
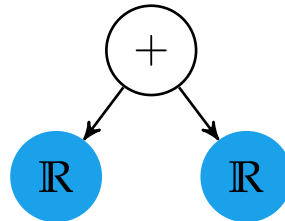
Constructing the expression tree using a grammar

$$\mathbb{R} \mapsto \mathbb{R} + \mathbb{R}$$

$$\mathbb{R} \mapsto x$$

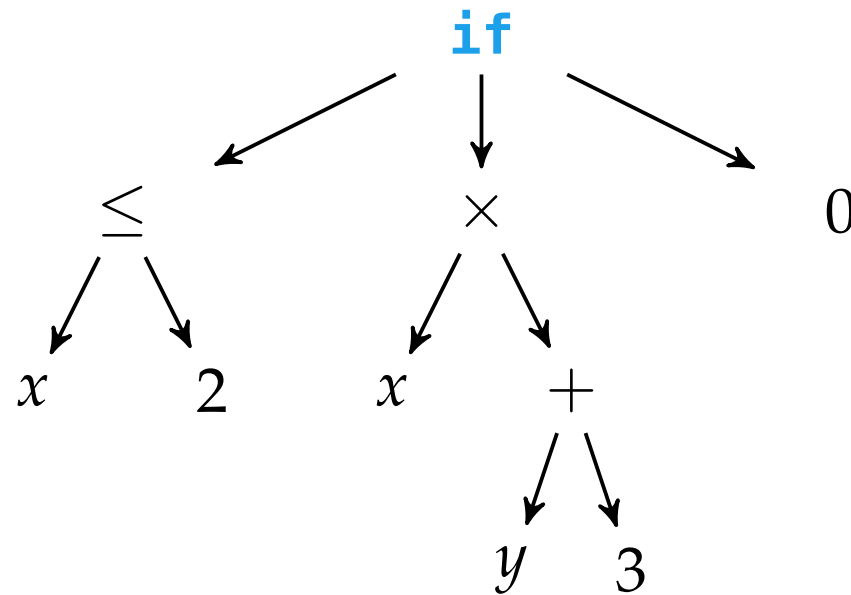
$$\mathbb{R} \mapsto \ln(\mathbb{R})$$

$$\mathbb{R} \mapsto 2$$



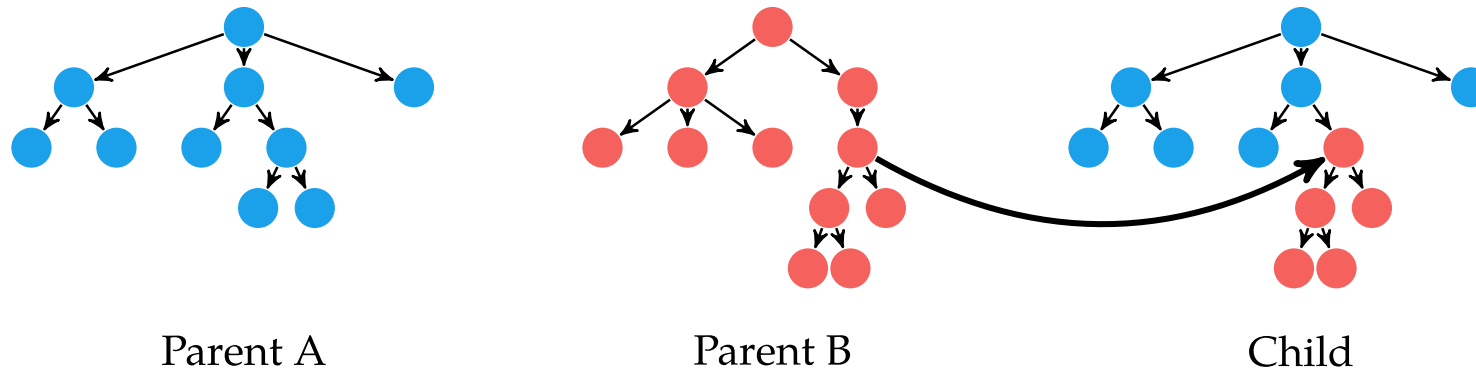
Genetic Programming

- Genetic programming represents individuals as trees and optimizes over them using mutation and tree crossover



Genetic Programming

- Tree crossover combines two parent trees into a child tree



- Over many generations, tree crossover tends to increase tree depth and overall complexity
- It is common to introduce a bias for parsimony, or simplicity in the solution based on the tree's depth

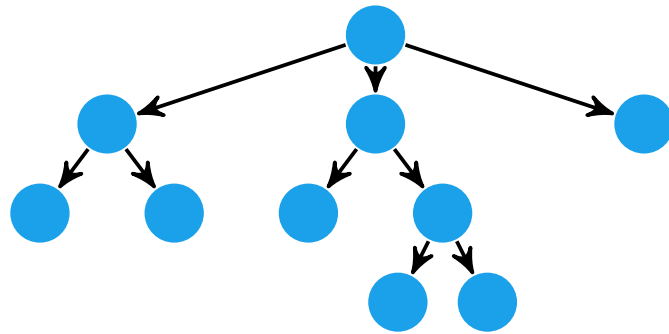
```

struct TreeCrossover <: CrossoverMethod
    grammar
    max_depth
end
function crossover(C::TreeCrossover, a, b)
    child = deepcopy(a)
    crosspoint = sample(b)
    typ = return_type(C.grammar, crosspoint.ind)
    d_subtree = depth(crosspoint)
    d_max = C.max_depth + 1 - d_subtree
    if d_max > 0 && contains_returntype(child, C.grammar, typ, d_max)
        loc = sample(NodeLoc, child, typ, C.grammar, d_max)
        insert!(child, loc, deepcopy(crosspoint))
    end
    child
end

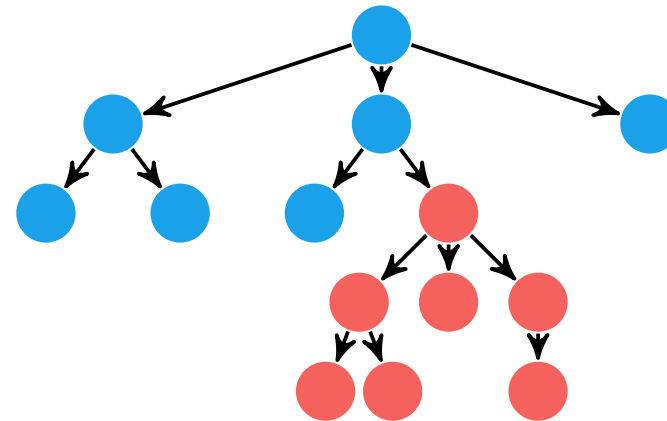
```

Genetic Programming

- Random variation can be added in multiple ways
- Tree mutation is when a random tree is generated and its root replaces a random node in the tree



Before



After

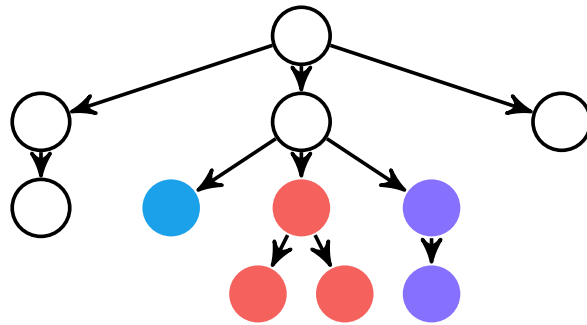

```

struct TreeMutation <: MutationMethod
    grammar
    p
end
function mutate(M::TreeMutation, a)
    child = deepcopy(a)
    if rand() < M.p
        loc = sample(NodeLoc, child)
        typ = return_type(M.grammar, get(child, loc).ind)
        subtree = rand(RuleNode, M.grammar, typ)
        insert!(child, loc, subtree)
    end
    return child
end

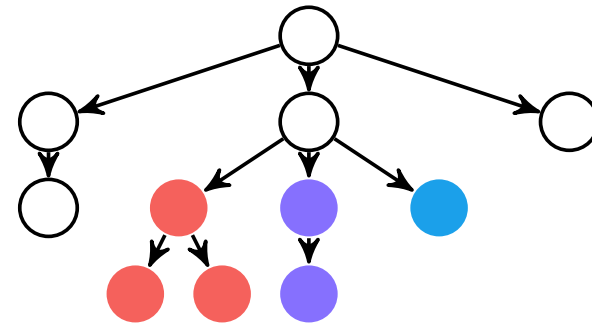
```

Genetic Programming

- Tree permutation is when randomly chosen nodes are permuted
- Often combined with tree mutation



Before



After

```

struct TreePermutation <: MutationMethod
    grammar
    p
end
function mutate(M::TreePermutation, a)
    child = deepcopy(a)
    if rand() < M.p
        node = sample(child)
        n = length(node.children)
        types = child_types(M.grammar, node)
        for i in 1 : n-1
            c = 1
            for k in i+1 : n
                if types[k] == types[i] &&
                    rand() < 1/(c+=1)

                    node.children[i], node.children[k] =
                        node.children[k], node.children[i]

                end
            end
        end
    end
    return child
end

```

Summary

- Optimization is the process of finding the best system design subject to a set of constraints
- Optimization is concerned with finding **global** minima of a function
- Minima can occur where the gradient is zero, but zero-gradient does not imply optimality